

Matr. N INLM03/23286



UNIVERSITÀ CAMPUS BIO-MEDICO DI ROMA

**FACOLTÀ DIPARTIMENTALE DI INGEGNERIA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA DEI SISTEMI
INTELLIGENTI**

**IMPLEMENTAZIONE E
OTTIMIZZAZIONE DELL'ALGORITMO
DI SHOR IN AMBIENTI QUANTISTICI
RUMOROSI MEDIANTE MODELLI DI
INTELLIGENZA ARTIFICIALE**

Relatore

Prof. Paolo Soda

Laureando

Claudio Dragotta

Correlatore

Ing. Floriano Caprio

ANNO ACCADEMICO 2025/2026

*Alla mia famiglia,
per il sostegno incondizionato e l'amore che non è mai venuto meno.
A me stesso, per la determinazione e la perseveranza con cui ho affrontato questo
percorso.*

Abstract

Introduzione

L'algoritmo di Shor [1] fattorizza un intero composto in tempo polinomiale, $O((\log N)^3)$, contro il tempo sub-esponenziale del miglior algoritmo classico noto. È questo divario a minacciare RSA [2], ma è asintotico e i dispositivi *Noisy Intermediate-Scale Quantum* [3] non lo realizzano: il rumore distrugge la struttura periodica su cui l'algoritmo si fonda, le dimostrazioni restano confinate a moduli didattici [4] e RSA-2048 richiederebbe venti milioni di qubit fisici [5]. Fra i due estremi restano due rimedi, la mitigazione a valle e la correzione d'errore durante il calcolo, ai quali un numero crescente di lavori applica modelli appresi dai dati, spesso senza un termine di confronto che isoli il contributo del modello da quello della regola decisionale.

Il lavoro riprende un progetto preesistente — implementazione di Shor in Qiskit e prima campagna rumorosa su $N = 15$ — rigenerato dopo che un audit per truth table ne ha invalidato il moltiplicatore modulare. Chiede quindi *in quale punto della pipeline un modello appreso migliori davvero la resa di Shor in ambiente rumoroso*. Con la stessa tecnologia e lo stesso impianto statistico confronta un classificatore dell'istogramma di output e un decoder addestrato sulle sindromi di un codice correttore. Il contributo è metodologico: disegno appaiato con ablazione, separazione fra metriche QEC e successo di Shor e artefatti riproducibili (seed, manifest e hash del circuito).

Materiali e Metodi

Formalizzazione matematica del problema

Sia N composto, dispari e non potenza di un primo. Scelto a con $1 < a < N$ e $\gcd(a, N) = 1$, la fattorizzazione si riduce alla ricerca dell'*ordine*

$$r = \text{ord}_N(a) = \min\{k \in \mathbb{N}^+ : a^k \equiv 1 \pmod{N}\} : \quad (1)$$

se r è pari e $a^{r/2} \not\equiv -1 \pmod{N}$, allora $\gcd(a^{r/2} \pm 1, N)$ è un fattore non banale, condizione soddisfatta con probabilità almeno $1/2$ su a casuale [6]. La parte quantistica stima r per *phase estimation* dell'unitario $U_a|y\rangle = |ay \bmod N\rangle$: con n qubit di conteggio e $M = 2^n$, dopo la trasformata di Fourier quantistica inversa la misura ha

distribuzione

$$P(y) = \frac{1}{r} \sum_{s=0}^{r-1} F_M\left(\frac{y}{M} - \frac{s}{r}\right), \quad F_M(\delta) = \frac{1}{M^2} \left| \frac{\sin(\pi M \delta)}{\sin(\pi \delta)} \right|^2 \quad (\text{nucleo di Fejér}). \quad (2)$$

Il post-processing Φ è deterministico: frazioni continue di y/M , che restituiscono s/r quando $|y/M - s/r| \leq 1/(2r^2)$ — garantito da $M \geq N^2$ — seguite dai controlli di parità e dal massimo comun divisore. Detto $\mathcal{U} = \{y : \Phi(y) \text{ dà un fattore non banale}\}$, ogni esecuzione è delimitata da due costanti in forma chiusa,

$$P_{\text{id}} = \sum_{y \in \mathcal{U}} P(y) = \frac{3}{4}, \quad P_{\text{unif}} = \frac{|\mathcal{U}|}{M} = \frac{63}{256} \simeq 0,2461, \quad (3)$$

per l'istanza canonica $N = 15$, $a = 7$, $n = 8$, dove $r = 4$ divide M e P è supportata su $\{0, 64, 128, 192\}$. Le due costanti fissano in anticipo la scala di lettura della campagna: P_{id} è la resa per shot della distribuzione ideale; P_{unif} è il *pavimento uniforme*, cioè la resa attesa quando il rumore cancella la struttura dei picchi e rende equiprobabili tutte le M celle. Non è nullo: il post-processing accetta 63 esiti su 256, quindi anche un'estrazione casuale trova un fattore nel 24,6% dei casi. I due valori non sono limiti universali per un rumore arbitrario, ma riferimenti per le distribuzioni ideale e uniforme; raggiungere il pavimento indica la perdita del segnale quantistico utile, non l'impossibilità di fattorizzare.

Modello di rumore e osservabile misurata

Il circuito è compilato deterministicamente (ottimizzazione 2 e seed registrato) nella base hardware $\{r_z, \sqrt{X}, X, CX\}$: $r_z(\theta)$ è una rotazione virtuale attorno a z , $\sqrt{X}$ (SX) è una rotazione di $\pi/2$ attorno a x , X la NOT quantistica che scambia $|0\rangle$ e $|1\rangle$, e CX (CNOT) applica X al bersaglio quando il controllo vale 1. Ogni altra operazione è decomposta in queste porte, sulle quali si modella il rumore. Dopo ogni gate g su q qubit si compone il canale depolarizzante $\mathcal{E}_\lambda(\rho) = (1 - \lambda)\rho + \lambda I/2^q$ con il rilassamento termico: T_1 è il tempo di rilassamento energetico $|1\rangle \rightarrow |0\rangle$ e T_2 il tempo di perdita della coerenza di fase. Poiché tali effetti si accumulano nel tempo, il canale usa la durata di g : r_z è virtuale (durata nulla nel modello), $\sqrt{X}/X$ dura 50 ns e CX 300 ns. La lettura è una matrice di confusione simmetrica R di parametro p_{ro} ; su S shot i conteggi sono $\mathbf{c} \sim \text{Multinomiale}(S, R p_{\text{rum}})$ e l'istogramma normalizzato è $\hat{h} = \mathbf{c}/S$. Lo accompagnano la frazione efficace $f = (m - 4/M)/(1 - 4/M)$, con m massa sui picchi teorici, e il proxy $P_0 = (1 - \frac{15}{16}\lambda_{2q})^{n_{\text{CX}}}$: indicatori dichiarati, non

fedeltà né probabilità di fattorizzazione. I preset sono scenari uniformi illustrativi, non calibrazioni di una QPU.

Politiche decisionali e stimando statistico

Su $\hat{h}$ si definisce l'ordinamento totale $\prec$ per conteggio decrescente, con pareggi risolti dalla priorità deterministica $\tau(y) = \text{SHA-256}(\text{seed} \parallel y)$; la politica π_K prova i primi K candidati secondo $\prec$. Il Metodo 1 è π_1 , l'ablazione è π_4 , il Metodo 2 applica un classificatore $g_\theta(\hat{h}) \in \{0, 1\}$ ed esegue π_4 solo se il gate è positivo. Per ogni replica si osserva

$$M^{(\pi)} = \min\{m \geq 1 : \pi \text{ riesce all'iterazione } m\}, \quad M^{(\pi)} := M_{\max} + 1 \text{ se non riesce,} \quad (4)$$

con $M_{\max} = 50$, sentinella che codifica esplicitamente la censura. Lo stimando dichiarato a priori è $\rho = \bar{M}_1/\bar{M}_2$; l'inferenza usa invece le differenze appaiate $D_i = M_i^{(A)} - M_i^{(B)}$, il test dei ranghi con segno di Wilcoxon (zeri `pratt`), H_0 di simmetria attorno a zero, $\alpha = 0,05$ e correzione di Holm entro famiglia. Poiché in una simulazione Monte Carlo la distribuzione è stimata estraendo pseudocasualmente S shot, anche esecuzioni nominalmente identiche producono istogrammi diversi. A ogni coppia (replica, iterazione) le tre strategie ricevono perciò lo stesso istogramma: D_i confronta le regole decisionali sulla medesima realizzazione e non contiene la componente spuria dovuta a campioni indipendenti, pur restando variabile fra repliche. Il classificatore riceve il vettore di frequenze relative $\hat{h} = (c_0/S, \dots, c_{M-1}/S)$, con somma uno, e l'etichetta binaria $Y_1 = \mathbf{1}[\pi_1 \text{ ha successo}]$, pari a 1 se π_1 trova un fattore non banale e a 0 altrimenti. La variante Y_{16} resta un audit: nel riferimento esattamente uniforme i primi sedici candidati equivalgono a estrazioni senza reimmisione dalle 256 celle, di cui 63 utili, e la probabilità che nessuno dia un fattore è $\binom{193}{16}/\binom{256}{16} = 0,93\%$. Il riferimento uniforme prevede quindi il 99,07% di etichette positive e dimostra il forte sbilanciamento, non l'esito esatto del campione. Nei dati effettivi Y_{16} è risultato positivo in 2000/2000 casi per entrambi gli scenari: questa osservazione empirica ne stabilisce la degenerazione a classe unica.

Correzione d'errore: formalismo, metriche e decodifica appresa

Un codice stabilizzatore è un sottogruppo abeliano $\mathcal{S} \subset \mathcal{P}_n$ con $-I \notin \mathcal{S}$; la sindrome di $E \in \mathcal{P}_n$ è $\sigma(E)_j = 0$ se $[E, s_j] = 0$ e 1 altrimenti, un decoder è una mappa $\mathcal{D} : \mathbb{F}_2^{n-k} \rightarrow \mathcal{P}_n$ e il fallimento logico è l'evento $\mathcal{D}(\sigma(E)) E \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$, la cui

probabilità definisce il *logical error rate* $p_L(p, d)$. Si simulano il codice a ripetizione, la cui forma chiusa $p_L = 3p^2 - 2p^3$ serve da test di correttezza; il codice di Steane [7], con verifica della corrispondenza sindrome→errore sui 21 casi e stima di γ in $p_L \propto p^\gamma$; il surface code ruotato [8] sotto rumore *circuit-level*, decodificato per matching di peso minimo e descritto dalla legge di scala

$$p_L(p, d) = A (p/p_{\text{th}})^{\lfloor (d+1)/2 \rfloor}, \quad (5)$$

adattata ai punti sotto soglia per $d = 3, 5, 7, 9$ così da stimare p_{th} indipendentemente dall'incrocio delle curve. Il decoder appreso è un percettrone multistrato f_θ sugli stessi *detection events*, confrontato con il matching sia come sostituto sia nella forma ibrida $\mathcal{D}_{\text{hyb}}(\sigma) = \mathcal{D}_{\text{MWPM}}(\sigma) \oplus \mathbf{1}[f_\theta(\sigma) > t]$, con soglia t scelta in validazione fra candidate che includono l'astensione totale e significatività valutata con il test esatto di McNemar.

Proxy fenomenologico per la sensibilità di Shor

Dopo ogni gate compilato su q qubit si applica il canale di Pauli $\mathcal{E}_{p_L}(\rho) = (1 - p_L)\rho + \frac{p_L}{4^q - 1} \sum_{P \neq I} P\rho P^\dagger$, ossia $\lambda_1 = \frac{4}{3}p_L$ e $\lambda_2 = \frac{16}{15}p_L$, e si misura il successo per singola misura su una griglia preregistrata di p_L , con probabilità cumulativa $P(k) = 1 - (1 - P_s)^k$ dopo k tentativi indipendenti. Non è una compilazione fault-tolerant: p_L è la probabilità di un Pauli non identità *per gate compilato*, non il tasso per ciclo di sindrome, e la conversione fra le due unità richiederebbe un modello architetturale esplicito.

Strumenti, verifica e validazione

Qiskit/Aer gestisce i circuiti generali, Stim e PyMatching quelli stabilizzatori e scikit-learn i modelli appresi; versioni e ambiente sono registrati negli artefatti. Gli strumenti ad hoc comprendono i moduli sperimentali, un formato JSON versionato (input, seed, vettori appaiati, manifest e hash SHA-256 della netlist), l'aritmetica reversibile di Beauregard per $N = 21/35$ e una demo web che separa esecuzione ideale e rumorosa; codice e artefatti sono pubblici e riutilizzabili. La verifica è *funzionale* (truth table, azzeramento delle ancille e confronto con la phase estimation teorica: distanza di variazione totale $1,38 \times 10^{-9}$ per $N = 21$), *di identità* (profondità, conteggi e hash) e *statistica* (ipotesi e griglie preregistrate, seed separati, intervalli di

Wilson al 95%, Bonferroni per la monotonia, Holm entro famiglia e censura esplicita). L’ablazione sugli stessi dati resta il controllo causale.

Risultati

Fase 1. Sul circuito verificato ($N = 15$, $a = 7$, profondità 412, 224 porte CX, hash registrato) i classificatori raggiungono le prestazioni ipotizzate a priori ($F1 = 0,919$ e $0,923$; $AUC = 0,965$ e $0,958$), ma l’ablazione appaiata mostra che il guadagno non è loro. Su 30 repliche per scenario, con successo 30/30 in ogni braccio, la sola ricerca π_4 porta le iterazioni al minimo ($\bar{M} = 1,00$ contro 1,37 in UC1 e 1,50 in UC2; $p_{\text{Holm}} = 0,0024$ e $0,0015$), mentre il Metodo 2 non migliora la baseline (1,50 e 1,37; $p = 0,7314$ e $0,0874$) e peggiora la propria ablazione ($p_{\text{Holm}} = 0,0033$ e $0,0096$). Nemmeno la zero-noise extrapolation migliora la baseline, e triplica il costo di campionamento; l’etichetta a sedici candidati produce infine 2000/2000 positivi, confermando empiricamente la degenerazione discussa in *Politiche decisionali e stimando statistico*.

Fase 2. La progressione dei codici riproduce le previsioni formulate a priori, con la stima di p_{th} dall’Eq. (5) concorde con l’incrocio delle curve: il codice a ripetizione a $d = 3$ accorda con $p_L = 3p^2 - 2p^3$ entro l’errore statistico; Steane $[[7, 1, 3]]$ dà $\gamma = 1,94$ e pseudo-soglia $p \simeq 0,08$; il surface code per $d = 3-9$ dà $p_{\text{th}} = 0,86\%$ (Z) e $0,81\%$ (X), $A = 3,5 \times 10^{-2}$ e RMS 0,08 in $\ln p_L$; la decodifica appresa non vince mai come sostituto del matching e, come correttore ibrido, riduce p_L del 18–21% a $d = 3$, dell’1–3% a $d = 5$ e per nulla a $d = 7$.

Chiusura quantitativa. Gli estremi della curva di sensibilità coincidono con le costanti dell’Eq. (3): il punto senza rumore, $0,7489$ [$0,7459$; $0,7518$], stima $P_{\text{id}} = 3/4$; il plateau, $0,2459$, coincide con il pavimento uniforme $P_{\text{unif}} = 63/256 = 0,2461$. La curva non descrive un algoritmo che fallisce, ma uno che degrada verso l’estrazione casuale, con entrambi gli asintoti fissati in anticipo dalla teoria.

Discussione e Conclusioni

La domanda di localizzazione ha una risposta a due facce. A valle dell’esecuzione l’ipotesi principale, $\rho \gg 1$, è *smentita* dai suoi stessi dati: il classificatore raggiunge le soglie di $F1$ e AUC fissate a priori e resta inutile, perché scarta istogrammi che π_4 avrebbe risolto. Sulla sindrome, dove il dato conserva una struttura locale

più ricca del solo istogramma finale, la stessa tecnologia guadagna, ma solo nella forma residuale e finché il rumore contiene correlazioni che il decoder analitico non rappresenta. Il criterio unificante è quindi verificabile: un modello appreso aggiunge valore quando sfrutta informazione che il metodo di riferimento lascia inutilizzata, non quando ne replica la regola decisionale.

L’autovalutazione riguarda il metodo: senza il braccio π_4 la riduzione da 1,37 a 1,00 sarebbe stata attribuita al classificatore, un falso positivo del tutto presentabile; l’audit dell’etichetta e la verifica per truth table del moltiplicatore — che ha imposto la rigenerazione di un’intera campagna — sono stati più informativi delle metriche di accuratezza.

I limiti delimitano ogni conclusione: rumore uniforme, indipendente e stazionario, senza topologia, crosstalk, leakage o deriva; $N = 15$ è un’istanza didattica con post-processing permissivo; le stime QEC valgono entro il modello Clifford–Pauli, indicato come potenzialmente ottimistico [9]; l’analisi di sensibilità è un proxy per gate, non una simulazione fault-tolerant. Rispetto al piano, $N = 21$ e $N = 35$ restano validati solo idealmente ed esclusi per costo dalle campagne rumorose, e l’accelerazione GPU prevista non è stata impiegata.

Gli sviluppi futuri discendono dai limiti — mappatura per operazione logica, rumore *channel-first*, decoder ricorrenti o a grafo [10] — ma la ricaduta più immediata è metodologica: ablazione appaiata, contratto interpretativo fra unità diverse e artefatti versionati formano un protocollo riutilizzabile per valutare qualunque mitigazione del rumore quantistico basata su apprendimento automatico.

Indice

Abstract	i
Elenco delle figure	xiv
Elenco delle tabelle	xvi
Elenco degli Acronimi	xx
1 Introduzione	2
1.1 Il Limite del Calcolo Classico	2
1.2 Motivazione e Contributo del Presente Lavoro	3
1.3 Struttura della Tesi	6
2 Obiettivi e Piano Sperimentale	9
2.1 Obiettivi Generali	9
2.2 Obiettivi Specifici	10
2.3 Ipotesi di Lavoro	12
2.4 Contributo Originale	13
2.5 Campagne Rumorose e Validazioni Ideali	15
2.5.1 Parametri dei Use Case	15
2.5.2 Razionale per la Scelta delle Istanze	16
2.6 Metriche di Valutazione	17
2.6.1 Metriche del Metodo 1	17
2.6.2 Metriche del Metodo 2	17
2.6.3 Metrica di Confronto	18
2.6.4 Metriche della Parte QEC	18
2.6.5 Parametri di Error Mitigation Applicati	19
2.7 Approccio Metodologico	19

3	L'Algoritmo di Shor	20
3.1	Richiami di Calcolo Quantistico	21
3.1.1	Qubit, Sovrapposizione e Sfera di Bloch	21
3.1.2	Sistemi Multi-Qubit ed Entanglement	22
3.1.3	Porte, Misura e Interferenza	23
3.2	Il Problema della Fattorizzazione Intera	24
3.2.1	Perché la Fattorizzazione Conta	25
3.3	Riduzione alla Ricerca del Periodo	25
3.4	La Quantum Fourier Transform	26
3.4.1	Definizione	26
3.4.2	Implementazione Efficiente	27
3.4.3	Phase Estimation	27
3.5	L'Algoritmo di Shor: Schema Completo	30
3.5.1	Il Ruolo dell'Entanglement nel Period Finding	31
3.6	Analisi della Complessità	33
3.7	Stato dell'Arte e Confronto con Altri Algoritmi	33
4	Rumore Quantistico nei Sistemi NISQ	34
4.1	Introduzione al Rumore Quantistico	34
4.2	La Macchina Fisica: i Tre Fatti che Contano	36
4.3	Le Quattro Fonti Fisiche di Rumore	36
4.3.1	Decoerenza (T_1 e T_2)	37
4.3.2	Errori di Gate	38
4.3.3	Errori di Misura (Readout Errors)	38
4.3.4	Crosstalk	38
4.4	Dal Fenomeno Fisico al Modello Matematico	39
4.5	I Quattro Canali Matematici del Rumore	40
4.5.1	Il Canale Depolarizzante	40
4.5.2	Gli Altri Tre Canali	41
4.6	Impatto del Rumore sull'Algoritmo di Shor	42
4.6.1	Errori nella Quantum Fourier Transform	43
4.6.2	Errori nella Phase Estimation	43
4.6.3	Errori nella Modular Exponentiation	44
4.7	Sfide Pratiche su Hardware NISQ	45

5	Strategie per la Riduzione del Rumore Quantistico	46
5.1	Le Quattro Strategie: Panoramica e Scelta	46
5.2	Machine Learning Classico per la Robustezza al Rumore	48
5.3	Correzione degli Errori Quantistici e Ruolo del Machine Learning . .	48
5.3.1	Il Problema Fondamentale	48
5.3.2	Il Paradigma in Tre Fasi	49
5.3.3	I Principali Codici Quantistici	50
5.3.4	Machine Learning come Possibile Componente del Decoder QEC	50
5.4	Motivazione Pratica: il Contributo di questa Tesi	51
6	Metodo e Implementazione	53
6.1	Strumenti e Ambiente di Simulazione	53
6.2	Obiettivi della Parte Sperimentale	55
6.3	Architettura del Pipeline Sperimentale	55
6.4	Tecniche di Mitigazione del Rumore: Analisi e Scelte Implementative	56
6.4.1	Transpilazione e Riduzione della Profondità	56
6.4.2	Modello di Rumore Uniforme	56
6.4.3	Livello di Misura: Mitigazione degli Errori di Readout	57
6.4.4	Zero-Noise Extrapolation	57
6.4.5	Analisi di Sensibilità	57
6.4.6	Scelta Adottata: Classificatore come Filtro	58
6.5	Il Metodo 1: Baseline TOP-1	58
6.6	Il Metodo 2 e il Disegno Appaiato	58
6.7	Setup dell'Ambiente di Esecuzione	59
6.7.1	Accelerazione GPU: limitazione dell'ambiente	59
6.8	L'Algoritmo di Shor: Implementazione di Riferimento	59
6.8.1	La Quantum Fourier Transform	59
6.8.2	La Moltiplicazione Modulare Controllata	60
6.8.3	Compilazione del Circuito	60
6.9	Configurazione del Noise Model	60
6.10	Implementazione del Metodo 1	61
6.11	Implementazione del Metodo 2	61
6.11.1	Dataset ed Etichette	61
6.11.2	Addestramento e Compatibilità	61
6.11.3	Inferenza	62
6.12	Infrastruttura di Test e Raccolta Dati	62

6.13	Implementazione della Correzione d'Errore	62
6.13.1	Organizzazione del codice e divisione degli strumenti	63
6.13.2	Il codice a ripetizione: sindrome senza collasso	63
6.13.3	Il codice di Steane	63
6.13.4	Il surface code e l'iniezione del crosstalk	63
6.13.5	Lo Shor logico e il decoder appreso	63
6.13.6	Riproducibilità	64
7	Verifica Sperimentale dell'Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K	65
7.1	Setup Sperimentale	65
7.2	Perimetro Ideale e Rumoroso	66
7.3	La Baseline: Metodo 1 (TOP-1)	67
7.4	L'Ipotesi ML e il Verdetto dell'Ablazione	67
7.5	Robustezza Esplorativa della Strategia TOP-4	69
7.5.1	Frazione Coerente Efficace e Proxy di Nessun Evento	69
7.5.2	Confronto con la Zero-Noise Extrapolation	70
7.6	Bilancio della Verifica e Ponte verso la Correzione d'Errore	71
8	La Correzione d'Errore Quantistico: dal Codice a Ripetizione al Codice di Steane	72
8.1	Dalla Ridondanza Ingenua alla Codifica	72
8.2	Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder	73
8.3	Il Panorama dei Codici e la Scelta Metodologica	74
8.4	Laboratorio I: il Codice a Ripetizione	75
8.4.1	Il codice bit-flip a 3 qubit	75
8.4.2	Risultati del laboratorio repetition	75
8.5	Laboratorio II: il Codice di Steane $[[7, 1, 3]]$	78
8.5.1	Struttura del codice	78
8.5.2	Protocollo di laboratorio	78
8.5.3	Risultati del laboratorio Steane	79
9	Il Surface Code e la Stima dell'Errore Logico	82
9.1	Il Codice su Griglia: Concetto Operativo	83
9.2	La Decodifica: Minimum Weight Perfect Matching	83
9.3	Disegno Sperimentale	84
9.4	Risultati	85

9.5	Oltre il Modello di Pauli: Errori Coerenti e Leakage	88
9.6	La Decodifica Appresa: Sintesi della Campagna	91
10	Analisi di Sensibilità: Shor sotto un Proxy di Errore per Gate	94
10.1	L'architettura a quattro livelli	94
10.2	Definizione operativa del proxy	95
10.3	Il contesto delle stime fault-tolerant	96
10.4	Risultati M8 e M13	96
10.4.1	Discussione	99
11	Sviluppi Futuri	101
11.1	Scalabilità a Istanze di Maggiore Dimensione	101
11.2	Validazione su Hardware Quantistico Reale	102
11.3	Generalizzazione ad Altri Algoritmi Quantistici	102
11.4	Integrazione con la Quantum Error Correction	103
11.5	Evoluzione del Classificatore ML	103
12	Conclusioni	105
12.1	Le Domande di Ricerca	105
12.2	Verifica delle Ipotesi	107
12.3	Contributi del Lavoro	107
12.4	Limiti	108
12.5	Il Criterio che Unifica il Lavoro	108
A	Trattazione Teorica Estesa	110
A.1	Fondamenti del Calcolo Quantistico	110
A.1.1	Dalla Meccanica Classica alla Meccanica Quantistica	110
A.1.2	I Postulati della Meccanica Quantistica	111
A.1.3	Il Qubit	113
A.1.4	Sistemi Multi-Qubit e Prodotto Tensoriale	114
A.1.5	Porte Quantistiche	116
A.1.6	Circuiti Quantistici	119
A.1.7	Misura Quantistica	119
A.1.8	Interferenza Quantistica	119
A.1.9	Modello di Complessità Quantistica	120
A.2	L'Anatomia Fisica di una QPU	121
A.2.1	L'Anatomia Fisica di una QPU	121

A.3	L'Algoritmo di Shor — Approfondimenti e Stato dell'Arte	126
A.3.1	Sviluppi Recenti e Stato dell'Arte	127
A.3.2	Contesto: Confronto con l'Algoritmo di Grover	129
A.4	I Canali di Rumore: Derivazioni Complete	130
B	Strumenti, Ambiente e Codice	132
B.1	Strumenti e Ambiente di Simulazione — Trattazione Estesa	132
B.1.1	Qiskit: il Framework Quantistico di Riferimento	132
B.1.2	Qiskit Aer: il Simulatore	134
B.1.3	Accelerazione GPU con Qiskit Aer	136
B.1.4	scikit-learn: la Libreria per il Classificatore	136
B.1.5	Ambiente di Esecuzione: WSL con Ubuntu	137
B.2	Codice Sorgente dei Componenti Implementati	137
C	Contesto e Motivazione	145
C.1	Contesto Esteso dell'Introduzione	145
C.1.1	Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico	145
C.1.2	La Rivoluzione Algoritmica degli Anni Novanta	149
C.1.3	RSA e le Implicazioni Crittografiche dell'Algoritmo di Shor	150
C.1.4	Crittografia Post-Quantistica: la Risposta dell'Industria	153
C.1.5	Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant	155
C.2	Dettaglio degli Obiettivi e del Piano Sperimentale	156
C.2.1	Motivazione: la Vulnerabilità della Crittografia Attuale	157
C.2.2	Requisiti Funzionali	158
C.2.3	Parametri di Input e Output	159
D	Fase 1: la Campagna Classica nel Dettaglio	161
D.1	Campagna Parametrica e Confronto con la Zero-Noise Extrapolation	161
D.1.1	Variazione di K	162
D.1.2	Variazione di λ_{2q}	162
D.1.3	Variazione degli Shot	163
D.1.4	Analisi Congiunta $K \times \lambda_{2q}$	163
D.1.5	Parametri Secondari	163
D.1.6	Livello di Ottimizzazione	164
D.1.7	Confronto con Zero-Noise Extrapolation	164
D.2	Il Classificatore ML: Architettura, Addestramento e Metriche	165
D.2.1	L'Ipotesi alla Prova: il Classificatore ML	165

E Fase 2: la Campagna QEC nel Dettaglio	170
E.1 La Decodifica Appresa: la Campagna Completa	170
E.1.1 Disegno sperimentale	171
E.1.2 E1 — la sostituzione a parità di modello	171
E.1.3 E2 — quando il rumore esce dal modello del decoder	173
E.1.4 E3 — la rete come validatore della correzione	174
E.1.5 E4 — il decoder ibrido	175
E.1.6 E6 — e se il modello del decoder fosse semplicemente sbagliato?	181
E.1.7 E7 — e se si scegliesse semplicemente un decoder analitico migliore?	183
E.1.8 E8 — le due sorgenti si sommano davvero? E10 — il pavimento comune	186
E.1.9 E5 — il decoder è trasferibile fra dispositivi?	190
E.1.10 Bilancio	191
E.2 Compilazione Consapevole della Struttura: un'Esplorazione	192
E.3 Previsione e selezione non sono la stessa domanda	193
E.4 M11: pilota sui layout	193
E.5 M11b: readout effettivo dopo il routing	195
E.6 M13: TOP-4 sotto il proxy per gate	196
E.7 Conclusione e limiti	197
Bibliografia	198

Elenco delle figure

3.1	La sfera di Bloch	22
3.2	Periodicità di $f(x) = 7^x \bmod 15$	26
3.3	Circuito della QFT su 3 qubit	27
3.4	Circuito QPE per $N = 15$, $a = 2$	28
3.5	Blocco di moltiplicazione modulare U_2 per $N = 15$	29
3.6	Blocco di moltiplicazione modulare U_4 per $N = 15$	29
3.7	Istogramma QPE ideale per $N = 15$	30
3.8	Schema a due registri dell'algoritmo di Shor	31
4.1	Effetto dei canali di rumore sulla sfera di Bloch	42
4.2	Dove colpiscono le quattro fonti di rumore nella pipeline di Shor	43
4.3	Proxy di nessun evento al crescere del circuito	45
6.1	Pipeline sperimentale appaiata	55
7.1	Massima e minima massa sui picchi QPE in UC1	67
7.2	Analisi di ablazione M1, TOP-4 e M2	68
7.3	Frazione efficace e proxy di nessun evento 2Q	70
8.1	Errore logico del codice a ripetizione a 3 qubit	77
8.2	Errore logico del codice di Steane $[[7, 1, 3]]$	81
9.1	Comportamento a soglia del surface code	85
9.2	Errore coerente vs errore di Pauli	89
10.1	Shor sotto un proxy fenomenologico per gate	97
10.2	Probabilità cumulativa dai punti M8	98
A.1	Circuito per lo stato di Bell $ \Phi^+\rangle$	115
A.2	Porte quantistiche a singolo e due qubit	118
A.3	Gerarchia delle classi di complessità quantistica	121

A.4	Anatomia schematica di una QPU superconduttiva a 5 qubit	122
A.5	La QPU reale: sistema integrato e criostati con cablaggio	123
A.6	Chip quantistico superconduttivo a 6 transmon con giunzioni di test .	124
A.7	Il bordo di coerenza: budget di operazioni e rischio di misura	126
D.1	Curve ROC dei classificatori TOP-1 selezionati	166
D.2	Istogramma TOP-1 negativo ma recuperabile con TOP-4	168
D.3	Tetto combinatorio alla classe negativa	169
E.1	Decodifica appresa a parità di modello di rumore	172
E.2	Decodifica appresa in presenza di rumore correlato	174
E.3	Precisione e copertura del flag di validazione	175
E.4	Il decoder ibrido a confronto con matching e rete	177
E.5	Ortogonalità delle due sorgenti di guadagno	185

Elenco delle tabelle

2.1	Preset uniformi illustrativi per le campagne rumorose N15. Non rappresentano la calibrazione di una QPU reale.	16
3.1	Le porte quantistiche essenziali	23
4.1	Riepilogo delle quattro fonti fisiche di rumore nei processori quantistici superconduttori.	39
4.2	Corrispondenza tra fonti fisiche di rumore e canali quantistici.	39
4.3	I quattro canali matematici fondamentali del rumore quantistico [6].	42
5.1	Confronto delle quattro strategie anti-rumore per il contesto di questa tesi.	47
6.1	Organizzazione del codice della parte QEC	63
7.1	Contratto riproducibile della campagna N=15	66
7.2	Perimetro delle validazioni	67
7.3	Confronto appaiato M1, TOP-4 e M2 (30 repliche per scenario)	68
7.4	Sintesi degli otto sweep schema 2 su $N = 15$	69
7.5	Frazione efficace e proxy di nessun evento 2Q	70
7.6	Confronto M1, ZNE e TOP-4 su 30 repliche appaiate	71
8.1	Confronto tra codici correttori quantistici	74
8.2	Verifica della sindrome del codice a ripetizione	76
8.3	Parametri del codice di Steane	78
8.4	Syndrome table del codice di Steane	80
9.1	Disegno sperimentale surface code	84
9.2	Distanza e costo in qubit richiesti dalla legge di scala	87
9.3	Sintesi della campagna sulla decodifica appresa	92

10.1	Architettura a quattro livelli della parte sperimentale	95
10.2	M8: probabilità di ricavare i fattori per singola misura, 20 repliche da 4096 shot per punto.	96
10.3	Contratto tra risultati QEC e curva M8	99
12.1	Esito delle ipotesi	107
A.1	Confronto tra tecnologie di QPU: superconduttori vs ioni intrappolati	125
A.2	Confronto tra l'algoritmo di Shor e l'algoritmo di Grover	129
B.1	Confronto tra i principali framework per la simulazione quantistica con rumore.	134
C.1	Parametri principali del sistema sperimentale. λ segue la convenzione Aer e non coincide con la probabilità aggregata di Pauli non identità.	160
D.1	Sweep del numero di candidati	162
D.2	Sweep del parametro Aer λ_{2q}	162
D.3	Sweep degli shot	163
D.4	Griglia $K \times \lambda_{2q}$	163
D.5	Sweep di T_1/T_2	163
D.6	Sweep di λ_{1q}	164
D.7	Sweep del readout simmetrico	164
D.8	Sweep di <code>optimization_level</code>	164
D.9	Confronto ZNE appaiato	165
D.10	Metriche TOP-1 sul test set indipendente (400 istogrammi per scenario)	166
D.11	Audit delle etichette TOP-1 e TOP-16	167
D.12	Distribuzione della moda su UC1	167
D.13	Tetto combinatorio alla classe negativa in funzione di K	169
E.1	Disegno sperimentale della decodifica appresa	171
E.2	Decodifica in presenza di rumore correlato	173
E.3	La rete come validatore della correzione del matching	175
E.4	Prestazioni del decoder ibrido	176
E.5	Larghezza dell'ingresso contro distanza del codice	179
E.6	Discriminazione e guadagno al crescere della larghezza dell'ingresso .	180
E.7	Effetto di un modello di rumore mis-calibrato sul MWPM	182
E.8	BP+OSD contro MWPM e contro il decoder ibrido	184

E.9	Il residuo appreso costruito sopra BP+OSD	187
E.10	Sei decoder, un solo pavimento	188
E.11	Il pavimento alla prova di modelli più potenti	189
E.12	Trasferibilità del decoder appreso fra profili di rumore	190
E.13	Disegno del pilota sui layout	194
E.14	Stimandi del pilota M11	194
E.15	Disegno osservazionale M11b	195
E.16	Strategie descrittive di M11b	195
E.17	Endpoint preregistrati di M13	196

Elenco degli Acronimi

AUC Area Sotto la Curva ROC (*Area Under the Curve*)

BQP *Bounded-error Quantum Polynomial time*

CCNOT Controlled-Controlled NOT (porta di Toffoli)

CNOT Controlled-NOT

CPTP *Completely Positive Trace-Preserving*

CPU Unità di Elaborazione Centrale (*Central Processing Unit*)

CSS Calderbank-Shor-Steane

DEM *Detector Error Model*

DFT Trasformata Discreta di Fourier (*Discrete Fourier Transform*)

ECDSA *Elliptic Curve Digital Signature Algorithm*

GCD Massimo Comune Divisore (*Greatest Common Divisor*)

GNFS *General Number Field Sieve*

GPU Unità di Elaborazione Grafica (*Graphics Processing Unit*)

HTTPS *HyperText Transfer Protocol Secure*

IBM International Business Machines

ML Apprendimento Automatico (*Machine Learning*)

MLP Percetttrone Multistrato (*Multi-Layer Perceptron*)

MWPM *Minimum Weight Perfect Matching*

NISQ *Noisy Intermediate-Scale Quantum*

NIST *National Institute of Standards and Technology*

NMR *Risonanza Magnetica Nucleare (Nuclear Magnetic Resonance)*

PEC *Probabilistic Error Cancellation*

PQC *Post-Quantum Cryptography*

QEC *Correzione degli Errori Quantistici (Quantum Error Correction)*

QFT *Trasformata di Fourier Quantistica (Quantum Fourier Transform)*

QMA *Quantum Merlin-Arthur*

QML *Quantum Machine Learning*

QPE *Quantum Phase Estimation*

QPU *Unità di Elaborazione Quantistica (Quantum Processing Unit)*

RSA *Rivest-Shamir-Adleman*

SDK *Software Development Kit*

SSH *Secure Shell*

SVM *Support Vector Machine*

WSL *Windows Subsystem for Linux*

ZNE *Zero-Noise Extrapolation*

Capitolo 1

Introduzione

1.1 Il Limite del Calcolo Classico

Per oltre cinquant'anni, lo sviluppo dell'informatica è stato guidato da una legge empirica enunciata nel 1965 da Gordon Moore¹: il numero di transistor per chip raddoppia ogni due anni, e con esso la potenza di calcolo disponibile. Questa crescita esponenziale ha permesso di trasformare calcolatori grandi quanto un'intera stanza nei dispositivi tascabili di oggi. Tuttavia, la Legge di Moore si sta scontrando con i limiti fisici fondamentali della materia: i transistor moderni hanno dimensioni di pochi nanometri, avvicinandosi alla scala atomica oltre la quale gli effetti quantistici – lo stesso fenomeno che si vorrebbe evitare – diventano incontrollabili e inevitabili.

Parallelamente al problema fisico dell'hardware, esiste una barriera di natura puramente matematica: esistono classi di problemi computazionali per i quali nessun algoritmo classico è in grado di fornire una soluzione in tempi ragionevoli, *indipendentemente* dalla potenza di calcolo disponibile. Questi problemi – fattorizzazione di grandi interi, ricerca in spazi combinatoriali enormi, simulazione di sistemi molecolari complessi – resistono a qualunque approccio classico non perché i computer non siano abbastanza veloci, ma perché la complessità intrinseca del problema cresce più rapidamente di qualunque risorsa classica concepibile.

È in questo contesto che nasce il **calcolo quantistico**: la visione radicale che i principi della meccanica quantistica – *sovrapposizione*, *entanglement* e *interferenza* –

¹**Gordon Earle Moore** (1929–2023), cofondatore di Intel, osservò nel 1965 che il numero di transistor integrabili su un chip raddoppiava approssimativamente ogni due anni, con un conseguente aumento esponenziale delle prestazioni computazionali. La Legge di Moore non è una legge fisica, ma una previsione industriale che ha orientato decenni di investimenti tecnologici.

possano essere sfruttati non come ostacoli da evitare, ma come risorse computazionali di straordinaria potenza. L'intuizione originale fu di Richard Feynman², che nel 1982 osservò come la simulazione di sistemi quantistici richiedesse risorse esponenziali su macchine classiche, e propose che un computer basato sugli stessi principi fisici potesse simulare la natura stessa in modo efficiente [11]. David Deutsch formalizzò nel 1985 il primo modello teorico rigoroso di computer quantistico universale [12], aprendo la strada a un nuovo paradigma computazionale.

I principi fisici che rendono possibile questo paradigma — il qubit e la sovrapposizione, la sfera di Bloch, l'entanglement e l'interferenza — insieme alla loro realizzazione sui qubit superconduttori di IBM, sono ripresi nella Sezione C.1 (Sez. C.1.1); la trattazione formale completa è nella Sezione 3.1. La rivoluzione algoritmica degli anni Novanta — da Simon fino all'algoritmo di Shor (1994) e a quello di Grover (1996) — è ripercorsa nella Sez. C.1.2. La portata dirompente di Shor si misura sulla crittografia che minaccia: la struttura del protocollo RSA e il motivo per cui Shor lo rompe (Sez. C.1.3), la risposta della crittografia post-quantistica con gli standard NIST 2024 (Sez. C.1.4) e lo stato dell'arte dell'hardware dall'era NISQ verso i sistemi fault-tolerant (Sez. C.1.5) delineano il contesto strategico in cui si colloca questo lavoro: comprendere i limiti reali dell'algoritmo di Shor su hardware rumoroso.

1.2 Motivazione e Contributo del Presente Lavoro

In questo contesto scientifico e tecnologico si inserisce il presente lavoro di tesi. L'algoritmo di Shor rappresenta uno dei risultati più significativi e dirompenti dell'informatica quantistica: la sua capacità di fattorizzare interi in tempo polinomiale minaccia direttamente le fondamenta della crittografia moderna. Tuttavia, la sua esecuzione su hardware NISQ è ostacolata da requisiti stringenti in termini di profondità del circuito, numero di qubit e fedeltà delle porte — rendendo il rumore quantistico il principale ostacolo alla sua realizzazione pratica.

Questo lavoro affronta tale sfida in due fasi. La **prima fase** è una verifica sperimentale sistematica: eseguire l'algoritmo di Shor su simulatori quantistici con due preset di rumore uniformi e illustrativi e mettere alla prova l'ipotesi che un clas-

²**Richard Phillips Feynman** (1918–1988), fisico teorico statunitense, Premio Nobel per la Fisica nel 1965. Nel suo celebre articolo del 1982 *“Simulating Physics with Computers”* [11], Feynman argomentò che un computer classico non può simulare efficientemente un sistema quantistico senza un overhead esponenziale, e propose che un “computer quantistico” — uno che segua le leggi della meccanica quantistica — potrebbe farlo in modo naturalmente efficiente.

sificatore Apprendimento Automatico (*Machine Learning*) (ML) applicato all’output rumoroso riduca il numero di iterazioni necessarie. I preset non costituiscono la calibrazione di una specifica QPU. La verifica confronta tre configurazioni:

- **Metodo 1** (baseline classico, TOP-1): la misura più frequente dell’istogramma viene usata come candidato. Richiede in media $\bar{M}_1 = 1.37$ esecuzioni nello scenario moderato e 1.50 in quello di stress.
- **Metodo 2** (classificatore ML TOP-1 + TOP-4): una *Support Vector Machine* (SVM), selezionata su validation e valutata su un test set indipendente, filtra gli istogrammi prima della ricerca sui quattro candidati più frequenti. Ottiene $\bar{M}_2 = 1.50$ e 1.37 nei due scenari, senza un miglioramento significativo rispetto a M1.
- **Analisi di ablazione** (M_{TOP4} , TOP-4 senza classificatore): isola il contributo della ricerca multi-candidato. TOP-4 ottiene $\bar{M} = 1.00$ in entrambi gli scenari ed è significativamente migliore della variante con filtro ($p_{\text{Holm}} = 0.0033$ e 0.0096). L’audit delle etichette (Sezione D.2) mostra inoltre che TOP-16 produce 2 000/2 000 positivi: il problema documentato è a classe unica, mentre TOP-1 apprende soltanto la riuscita del candidato dominante.

L’esito della verifica — l’ML applicato all’output, in quel punto e in quel modo, non aggiunge valore alla regola TOP-4 — delimita con precisione la portata del post-processing. L’istogramma conserva candidati utili, come mostra TOP-4, ma nessuna regola a valle può correggere gli errori che hanno già cancellato o spostato fuori dall’insieme osservato la struttura necessaria. Per intervenire su questi ultimi occorre proteggere il calcolo *durante* l’esecuzione. La **seconda fase**, che costituisce il nucleo della tesi, è quindi dedicata alla Correzione degli Errori Quantistici (*Quantum Error Correction*) (QEC): la costruzione e la simulazione dei codici correttori — dal codice a ripetizione al codice di Steane $[[7, 1, 3]]$, fino al surface code con decodifica *Minimum Weight Perfect Matching* (MWPM) — e la misura sperimentale del **logical error rate** p_L in funzione dell’errore fisico p e della distanza di codice d . L’integrazione finale riusa il simbolo p_L per un parametro fenomenologico applicato ai gate compilati inclusi nel proxy e svolge un’analisi di sensibilità di Shor; non identifica direttamente tale parametro con il tasso di fallimento di un memory experiment.

È in questa seconda fase che l’intelligenza artificiale viene ripresa, in un punto che espone direttamente informazione diagnostica: non più sull’istogramma finale, ma sulla **sindrome**. Un decoder appreso viene messo a confronto con il MWPM sugli

stessi campioni, e il risultato precisa il criterio che regge l'intero lavoro. Sostituire il decoder analitico non conviene: a parità di modello di rumore la rete lo pareggia soltanto, e a costo di volumi di dati considerevoli. Affiancarglielo sì: un decoder **ibrido**, in cui la rete apprende unicamente *quando ribaltare* la decisione del matching, riduce l'errore logico fino al 21% nel modello simulato con errori correlati che il grafo del decoder analitico non rappresenta — e ricade su di esso, entro la risoluzione dell'esperimento, quando il rumore è quello previsto. L'apprendimento automatico, dunque, non ottimizza l'esecuzione di Shor ovunque lo si applichi: può farlo dove dispone di informazione strutturale che il metodo di riferimento non sfrutta, e questa tesi ne delimita sperimentalmente un caso d'uso.

L'elaborato si sviluppa dalle fondamenta teoriche del calcolo quantistico, attraverso la verifica sperimentale su simulatori rumorosi, fino alla correzione d'errore quantistica e allo Shor logico. Gli obiettivi specifici e la struttura dettagliata del lavoro sono descritti nel Capitolo 2.

Riproducibilità e materiale supplementare. Tutto il lavoro condotto in questa tesi — i sorgenti `LATEX`, il codice sperimentale e le campagne di simulazione con i relativi risultati (seed e parametri inclusi) — è documentato ed è liberamente consultabile nel repository `GITHUB` pubblico dell'autore: <https://github.com/claudio-dragotta/shor-nisq-qec-thesis>. È inoltre disponibile una **demo interattiva** dell'algoritmo di Shor — circuito e sfere di Bloch passo per passo, con esecuzione statistica su molte iterazioni — consultabile dal vivo all'indirizzo <https://shor-demo-6knp.onrender.com> (codice sorgente: <https://github.com/claudio-dragotta/shor-demo>).

1.3 Struttura della Tesi

L’elaborato è organizzato in dodici capitoli, seguiti da cinque appendici (A–E) che raccolgono il contesto esteso, il materiale di dettaglio richiamato in forma sintetica nel corpo e un’esplorazione condotta oltre il perimetro del lavoro principale. Il Capitolo 2 svolge una funzione trasversale di raccordo tra teoria e pratica; i Capitoli 3–5 costituiscono la parte teorica; i Capitoli 6–10 la parte sperimentale; i Capitoli 11 e 12 chiudono il lavoro.

Il **Capitolo 2** (*Obiettivi e Piano Sperimentale*) motiva la vulnerabilità di RSA all’algoritmo di Shor e fissa il contratto sperimentale: obiettivi, ipotesi di ricerca, contributo, use case e metriche.

Parte teorica (Capitoli 3–5).

Il **Capitolo 3** (*L’Algoritmo di Shor*) si apre con i richiami di calcolo quantistico necessari a seguirlo (Sezione 3.1: qubit, sovrapposizione, porte e circuiti, e l’entanglement come *risorsa fisica fragile* che anticipa la trattazione del rumore), quindi presenta la riduzione al *period finding*, la Quantum Phase Estimation e la complessità polinomiale $O((\log N)^3)$, con le implicazioni per RSA e la transizione post-quantum.

Il **Capitolo 4** (*Rumore Quantistico nei Sistemi NISQ*) analizza le quattro fonti fisiche di errore e i canali di rumore fondamentali, fino a un proxy di nessun evento pari a circa 3,4% nell’esempio illustrativo con lunghezza $n = 15$ bit; tale proxy non coincide con la probabilità di successo di Shor.

Il **Capitolo 5** (*Strategie per la Riduzione del Rumore Quantistico*) confronta le quattro macro-strategie anti-rumore nell’era NISQ e motiva l’approccio adottato.

Parte sperimentale (Capitoli 6–10).

Il **Capitolo 6** (*Metodo e Implementazione*) risponde a tre domande in sequenza: con quali strumenti si lavora (Sezione 6.1: Qiskit, il simulatore Aer su CPU, la valutazione non riuscita del backend GPU, il noise model parametrizzato e la transpilazione), con quale disegno sperimentale — la pipeline a tre stadi, le tecniche di mitigazione e i due metodi confrontati — e come il disegno è stato realizzato in codice, dall’ambiente WSL2 alla QFT inversa, dal circuito di Shor alla pipeline dei due metodi.

Il **Capitolo 7** (*Verifica Sperimentale dell’Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K*) stabilisce la baseline e, tramite analisi di ablazione, dimostra che il guadagno è della strategia TOP-4 e non del classificatore ML — il risultato che motiva il pivot alla QEC.

Il **Capitolo 8** (*La Correzione d'Errore Quantistico: dal Codice a Ripetizione al Codice di Steane*) presenta il ciclo di correzione e lo verifica sul codice a ripetizione e sul codice di Steane [[7, 1, 3]].

Il **Capitolo 9** (*Il Surface Code e la Stima dell'Errore Logico*) estende la correzione al surface code e ne misura sperimentalmente l'errore logico e la soglia con Stim e PyMatching; le due sezioni conclusive mettono alla prova le assunzioni su cui quella stima poggia — il modello di rumore con cui il codice è simulato e quello con cui è decodificato — ed è nella seconda che il confronto fra decodifica appresa e MWPM individua il regime in cui l'apprendimento automatico produce un guadagno misurabile.

Il **Capitolo 10** (*Analisi di Sensibilità: Shor sotto un Proxy di Errore per Gate*) misura la sensibilità di Shor a un proxy Pauli uniforme per gate compilato e spiega perché, senza una compilazione fault-tolerant, tale curva non può essere tradotta direttamente in requisiti hardware.

Il **Capitolo 11** (*Sviluppi Futuri*) delinea le direzioni di ricerca oltre il perimetro sperimentale della tesi.

Il **Capitolo 12** (*Conclusioni*) verifica una per una le sei ipotesi e le sei domande di ricerca fissate nel Capitolo 2, dichiara i contributi e i limiti del lavoro e isola il criterio che unifica le due fasi sperimentali.

Appendici. Il corpo dell'elaborato è tenuto alla lunghezza di un argomento continuo; le cinque appendici raccolgono ciò che quel filo richiama ma non deve interrompere. Sono organizzate per tema, non per capitolo di provenienza, così che chi le consulti trovi in un solo luogo tutto il materiale di una stessa natura.

L'**Appendice A** (*Trattazione Teorica Estesa*) sviluppa per esteso la teoria sintetizzata nei Capitoli 3–4: i postulati della meccanica quantistica e l'insieme completo delle porte (Sez. A.1), l'anatomia fisica di una Unità di Elaborazione Quantistica (*Quantum Processing Unit*) (QPU) e il confronto fra le tecnologie di qubit (Sez. A.2), gli approfondimenti sull'algoritmo di Shor e sullo stato dell'arte della fattorizzazione quantistica (Sez. A.3) e le derivazioni complete dei canali di rumore in forma di Kraus (Sez. A.4).

L'**Appendice B** (*Strumenti, Ambiente e Codice*) motiva in dettaglio le scelte tecnologiche — confronto tra framework, metodi di simulazione di Aer, specifiche GPU e ambiente WSL (Sez. B.1) — e riporta il codice sorgente dei componenti implementati (Sez. B.2), dal circuito di Shor ai codici correttori.

L'**Appendice C** (*Contesto e Motivazione*) sviluppa i principi fisici del calcolo quantistico, la rivoluzione algoritmica degli anni Novanta, il protocollo RSA e la

sua vulnerabilità, la crittografia post-quantistica e lo stato dell’arte dell’hardware (Sez. C.1); raccoglie inoltre la motivazione crittografica estesa, i requisiti funzionali e la specifica completa di input e output del sistema sperimentale (Sez. C.2).

L’**Appendice D** (*Fase 1: la Campagna Classica nel Dettaglio*) riporta integralmente gli otto sweep parametrici su M_{TOP4} , con il confronto previsione–esito per ciascun parametro, e il dettaglio metodologico del confronto con la Zero-Noise Extrapolation (Sez. D.1); documenta poi il classificatore valutato nella prima fase — le tre architetture confrontate, l’addestramento e le metriche — e l’audit delle etichette che ricostruisce quale domanda il modello abbia effettivamente appreso a rispondere (Sez. D.2).

L’**Appendice E** (*Fase 2: la Campagna QEC nel Dettaglio*) espone per intero la campagna sulla decodifica appresa — architettura del decoder, protocollo di addestramento, confronto con MWPM e con BP+OSD (Sez. E.1) — e mette alla prova il criterio conclusivo del lavoro su un caso esterno al perimetro principale, la mappatura del circuito studiata su una snapshot di calibrazione offline e riproducibile (Sez. E.2).

Capitolo 2

Obiettivi e Piano Sperimentale

Il Capitolo 1 ha inquadrato il contesto storico e tecnologico del calcolo quantistico. Il presente capitolo costruisce su quel fondamento e svolge una funzione duplice: da un lato motiva e definisce gli obiettivi scientifici del lavoro; dall'altro stabilisce il contratto sperimentale completo — requisiti, parametri, use case e metriche — che guiderà tutta la parte implementativa. Il lettore che avrà letto questo capitolo avrà una visione completa di *perché* il lavoro esiste, *cosa* si propone di dimostrare e *come* intende farlo.

La motivazione crittografica in forma estesa — il confronto tra la complessità sub-esponenziale del *General Number Field Sieve* (GNFS) e quella polinomiale $O((\log N)^3)$ di Shor, la conseguente compromissione di Rivest-Shamir-Adleman (RSA) e lo scenario *harvest now, decrypt later* — è sviluppata nella Sezione C.2 (Sez. C.2.1). In sintesi: RSA sarebbe vulnerabile a un computer quantistico fault-tolerant sufficientemente grande; oggi profondità, numero di qubit logici, overhead di correzione e rumore separano gli esperimenti dimostrativi da quella scala.

2.1 Obiettivi Generali

Partendo dal contesto delineato, il presente lavoro si propone di analizzare in profondità l'algoritmo di Shor per la fattorizzazione di interi e di affrontare il problema della sua esecuzione su hardware quantistico reale, afflitto da rumore e decoerenza.

Il filo conduttore del lavoro è duplice: da un lato la comprensione del **potenziale computazionale** offerto dall'algoritmo di Shor rispetto ai migliori approcci classici noti; dall'altro lo studio delle **limitazioni pratiche** imposte dal rumore nei dispositivi *Noisy Intermediate-Scale Quantum* (NISQ) attuali, e delle strategie per superarle.

Coerentemente con questo secondo asse, il lavoro sperimentale si articola in **due fasi**. La prima verifica se un’ottimizzazione basata su **modelli di intelligenza artificiale**, applicata a valle dell’esecuzione, riduca il costo dell’algoritmo; la seconda è dedicata alla **Correzione degli Errori Quantistici (QEC)**: costruzione, simulazione e caratterizzazione dei codici correttori (ripetizione, Steane, surface code). Una successiva analisi di sensibilità applica a Shor un proxy Pauli per gate compilato, mantenendolo distinto dalle metriche QEC per ciclo o memoria. Gli esiti quantitativi sono riportati nei capitoli dei risultati e non anticipati nella metodologia.

2.2 Obiettivi Specifici

1. **Fondamenti del calcolo quantistico.** Fornire una trattazione rigorosa dei principi matematici e fisici su cui si basano gli algoritmi quantistici: il modello del qubit, la sovrapposizione, l’entanglement, le porte logiche quantistiche e il meccanismo di interferenza. Questo obiettivo costituisce la base teorica indispensabile per la comprensione dell’algoritmo trattato nei capitoli successivi.
2. **Analisi teorica dell’algoritmo di Shor.** Descrivere in dettaglio il funzionamento dell’algoritmo di Shor per la fattorizzazione di interi in tempo polinomiale, con particolare attenzione alla riduzione al problema del period-finding, alla Quantum Fourier Transform (Trasformata di Fourier Quantistica (*Quantum Fourier Transform*)) (QFT)) e alla tecnica di phase estimation. Analizzare la complessità computazionale e valutare le implicazioni per la crittografia RSA e per la sicurezza informatica post-quantum.
3. **Implementazione su Qiskit e analisi sperimentale.** Implementare l’algoritmo di Shor utilizzando il framework open-source Qiskit, costruendo i circuiti per la QFT, la modular exponentiation e il circuito completo di fattorizzazione. Verificare il comportamento del circuito su simulatore ideale e analizzare le prestazioni su istanze concrete.
4. **Studio del rumore quantistico nell’algoritmo di Shor.** Analizzare le principali fonti di errore che affliggono l’esecuzione dell’algoritmo di Shor su hardware reale – decoerenza, errori di gate, readout errors – con particolare attenzione alle componenti più sensibili al rumore: la QFT e la phase estimation. Studiare le strategie di QEC e di noise mitigation disponibili (*Zero-Noise Extra-*

polation (ZNE), *Probabilistic Error Cancellation* (PEC), readout mitigation), valutandone l'efficacia sull'algoritmo di Shor.

5. **Ottimizzazione mediante modelli di intelligenza artificiale.** Determinare *in quale punto* della pipeline di esecuzione un modello appreso migliori effettivamente la resa dell'algoritmo di Shor in ambiente rumoroso. L'obiettivo non è assumere che l'apprendimento automatico giovi, ma stabilirlo sperimentalmente confrontandone due impieghi distinti, con lo stesso strumento e sotto lo stesso impianto statistico: **(i) a valle dell'esecuzione**, come classificatore che giudica l'istogramma di output, misurandone il contributo tramite analisi di ablazione contro la sola strategia di ricerca multi-candidato; **(ii) durante la correzione d'errore**, come decoder addestrato sulle sindromi del surface code, confrontato con il decoder analitico di riferimento a parità di informazione. Il criterio di valutazione è in entrambi i casi quantitativo — numero medio di iterazioni nel primo, *logical error rate* nel secondo — e l'esito atteso è una delimitazione, non una promozione: individuare il regime in cui l'intelligenza artificiale apporta un guadagno misurabile e quello in cui non lo apporta, con la spiegazione del perché.
6. **Correzione degli errori quantistici e integrazione con Shor.** Costruire e simulare i codici correttori d'errore in ordine di complessità crescente — il codice a ripetizione, il codice di Steane $[[7, 1, 3]]$ e il surface code — caratterizzandone quantitativamente la capacità di protezione: la corrispondenza *sindrome*→*errore*, la soppressione dell'errore logico p_L in funzione dell'errore fisico p e della distanza di codice d , e il comportamento di soglia. Studiare infine la sensibilità di Shor a un residuo Pauli fenomenologico per gate, senza identificare direttamente questo parametro con le metriche aggregate dei benchmark QEC.

Questo secondo blocco di obiettivi si traduce in un insieme di **domande di ricerca** (DR) che la parte sperimentale sulla correzione d'errore affronta esplicitamente:

- DR1.** Come degrada la probabilità di successo di Shor al crescere dell'errore fisico sui gate, e quale fonte di rumore è la più dannosa?
- DR2.** In che modo un codice a ripetizione permette di introdurre il concetto di *sindrome* senza collassare lo stato logico?

- DR3.** Quali vantaggi e limiti offre il codice di Steane $[[7, 1, 3]]$ come primo codice capace di correggere un errore arbitrario su un qubit?
- DR4.** Perché il surface code è più rilevante per scenari scalabili, e come si stima il suo logical error rate tramite decodifica? Esiste un comportamento di soglia osservabile?
- DR5.** Come varia il successo di Shor al crescere di un proxy Pauli uniforme per gate, e quante ripetizioni indipendenti servono per superare una soglia operativa prefissata?
- DR6.** Esiste un regime in cui un decoder *appreso dai dati* supera il decoder analitico standard, e da che cosa dipende? In particolare: il vantaggio dipende dalla qualità della calibrazione del modello di rumore, oppure dalla sua rappresentabilità nella struttura del decoder?

Le prime cinque domande discendono dal piano di lavoro concordato sulla correzione d'errore; la sesta estende l'obiettivo specifico 5 alla seconda fase, chiudendo il confronto fra i due impieghi dell'apprendimento automatico su cui l'intero lavoro è costruito.

2.3 Ipotesi di Lavoro

Il contributo sperimentale della tesi è strutturato attorno a un'ipotesi principale e a due ipotesi secondarie, formulate prima dell'esecuzione degli esperimenti e verificate quantitativamente sui dati raccolti.

Ipotesi principale. L'integrazione di un classificatore ML nella pipeline di analisi riduce significativamente il numero medio di iterazioni necessarie per ottenere un risultato corretto dall'algoritmo di Shor su simulatore rumoroso. In termini formali, il fattore di riduzione

$$\rho = \frac{\bar{M}_1}{\bar{M}_2} \gg 1 \quad (2.1)$$

dove $\bar{M}_1$ e $\bar{M}_2$ sono rispettivamente il numero medio di iterazioni del Metodo 1 (baseline classico) e del Metodo 2 (classificatore ML). Le strategie sono valutate sulle stesse repliche e sugli stessi istogrammi; la verifica inferenziale riguarda la distribuzione delle differenze appaiate, non l'uguaglianza delle medie. Si usa il test dei ranghi con segno di Wilcoxon unilaterale, con gestione degli zeri `zero_method='pratt'` e soglia $\alpha = 0.05$.

Ipotesi secondaria 1. Il classificatore ML raggiunge prestazioni sufficienti a distinguere esecuzioni corrette da errate anche in presenza di rumore variabile: $F1 > 0.85$ e $AUC > 0.90$ sul test set.

Ipotesi secondaria 2. Il vantaggio del Metodo 2, se osservato, si mantiene passando dal preset di riferimento al preset di stress sul medesimo circuito $N = 15$. Le istanze $N = 21$ e $N = 35$ sono riservate alla validazione ideale dell’aritmetica di Beauregard e all’analisi delle risorse; non entrano nelle campagne rumorose né nella verifica inferenziale di questa ipotesi.

La seconda fase del lavoro, sulla correzione d’errore, è retta da un’ulteriore terna di ipotesi, anch’esse formulate a priori e verificate quantitativamente.

Ipotesi QEC 1 (soppressione). La codifica rende il qubit logico più affidabile del qubit fisico al di sotto di una soglia di errore: per un codice di distanza $d = 3$ (ripetizione, Steane) l’errore logico è soppresso quadraticamente, $p_L \propto p^2$, ed esiste un regime in cui $p_L < p$.

Ipotesi QEC 2 (soglia). Il surface code esibisce un comportamento di soglia: al di sotto di un errore fisico critico p_{th} , aumentare la distanza di codice d riduce l’errore logico p_L ; al di sopra, lo aumenta. Le curve $p_L(p)$ a distanze crescenti si incrociano in prossimità di p_{th} .

Ipotesi QEC 3 (sensibilità). Nel proxy fenomenologico M8, la probabilità di successo di Shor è compatibile con una funzione non crescente di p_L , definito per gate compilato incluso nel modello; poche ripetizioni indipendenti possono inoltre superare una soglia cumulativa che il singolo run ideale non raggiunge. L’ipotesi non afferma una conversione diretta dai benchmark Steane o surface code.

2.4 Contributo Originale

Rispetto alla letteratura esistente sull’algoritmo di Shor e sulla mitigazione del rumore nei sistemi NISQ, questo lavoro apporta i seguenti contributi originali.

Progettazione e verifica della strategia TOP-K. La maggioranza degli studi di mitigazione del rumore sull’algoritmo di Shor si concentra su tecniche strutturali applicate al circuito prima della misura. Il presente lavoro valuta invece un approccio di *post-processing*: la ricerca multi-candidato TOP-K applicata direttamente all’istogramma rumoroso. Il protocollo appaiato consente di separarne il contributo da quello del classificatore senza confonderlo con una diversa realizzazione Monte Carlo.

La quantificazione del guadagno proviene dagli artefatti schema 2 della campagna corrente.

Quantificazione empirica del fattore di riduzione ρ . La campagna quantitativa comprende due preset illustrativi sullo stesso circuito $N = 15$, uno di riferimento e uno di stress. Per ogni coppia (replica, iterazione), M1, TOP-4 e M2 ricevono lo stesso istogramma. Le differenze vengono analizzate con Wilcoxon appaiato-Pratt; $N = 21$ e $N = 35$ forniscono soltanto controlli ideali di correttezza e di costo circuitale. I risultati riportati provengono esclusivamente dagli artefatti schema 2 generati dopo la correzione del moltiplicatore $\times 7 \bmod 15$.

Analisi di ablazione che separa TOP-K dal classificatore ML. Il confronto M1-TOP-4-M2 è costruito sul medesimo calendario di seed e sul medesimo istogramma per iterazione. In questo modo l'ablazione misura soltanto la diversa regola decisionale. L'esito quantitativo riportato nel Capitolo 7 usa la rigenerazione schema 2 e non le metriche della campagna precedente.

Caratterizzazione a livelli della correzione d'errore e di Shor. La seconda fase costruisce e simula una progressione di codici correttori — ripetizione, Steane $[[7, 1, 3]]$, surface code con decodifica MWPM — misurandone il logical error rate nei rispettivi protocolli. Separatamente, M8 misura la sensibilità di Shor a un proxy Pauli uniforme per gate. Il contributo metodologico è rendere esplicito il confine fra i due livelli: senza compilazione fault-tolerant non si sovrappongono sulla curva M8 tassi per ciclo o per memoria, e non si presenta un confronto numerico fra codici sotto unità non commensurabili.

Delimitazione sperimentale del regime di utilità dell'apprendimento automatico. Il contributo che unifica le due fasi è la determinazione, sotto un impianto sperimentale esplicito, di *dove* un modello appreso migliori l'esecuzione. A valle di Shor, il confronto appaiato M1-TOP-4-M2 separa il classificatore dalla ricerca multi-candidato usando l'artefatto schema 2 corrente. Nella parte surface code, il modello appreso viene confrontato con il decoder analitico a parità di detection events. Questa struttura consente di formulare il criterio di applicabilità in termini verificabili.

I **requisiti funzionali** del sistema sperimentale e la specifica completa dei **parametri di input e output** sono formalizzati nella Sezione C.2 (Sez. C.2.2 e Sez. C.2.3). Le configurazioni e le metriche che costituiscono il contratto sperimentale versione 2 sono definite qui di seguito.

2.5 Campagne Rumorose e Validazioni Ideali

Il disegno separa due domande che non devono essere confuse:

- **Campagne rumorose N15:** stesso $N = 15$, stessa base $a = 7$, stesso circuito e due preset, *riferimento* e *stress*. I preset sono scenari uniformi illustrativi, non calibrazioni di una QPU reale.
- **Validazioni ideali N21/N35:** i moltiplicatori modulari di Beauregard sono verificati senza rumore, mediante controlli di truth table e di ripristino dei registri ausiliari, e con prove QPE ideali. Queste istanze sono escluse dalle campagne Aer rumorose per il loro costo.

2.5.1 Parametri dei Use Case

La Tabella 2.1 riporta le due sole configurazioni rumorose. I simboli ϵ_{1q} e ϵ_{2q} indicano esattamente il parametro λ passato da Aer a `depolarizing_error`; non sono probabilità Pauli aggregate né dati di calibrazione hardware. Il modello è uniforme su tutti i qubit.

Parametro	Riferimento	Stress
N	15	15
a	7	7
Periodo r atteso	4	4
n_{count} (qubit)	8	8
M_{shots}	1024	1024
$\epsilon_{1q} = \lambda_{1q}$	1×10^{-3}	5×10^{-3}
$\epsilon_{2q} = \lambda_{2q}$	1×10^{-2}	5×10^{-2}
T_1 (ns)	100 000	50 000
T_2 (ns)	80 000	30 000
p_{ro}	2×10^{-2}	5×10^{-2}

Tabella 2.1: Preset uniformi illustrativi per le campagne rumorose N15. Non rappresentano la calibrazione di una QPU reale.

2.5.2 Razionale per la Scelta delle Istanze

N15 — preset di riferimento e stress. La base $a = 7$ ha periodo $r = 4$ e consente di estrarre i fattori 3 e 5. Il moltiplicatore controllato implementa realmente $y \mapsto 7y \bmod 15$: l'operazione elementare viene ripetuta `power` volte, anziché selezionare erroneamente un ramo mediante `power % 4`. Il circuito compilato con Qiskit 2.5, `optimization_level=2`, seed di transpiler fissato e base `rz/sx/x/cx` ha profondità 412 e contiene 224 porte `cx`; questi dati, insieme all'hash SHA-256 della netlist, sono registrati nel manifest.

Le rotazioni `rz` sono cambi di frame virtuali, con durata ed errore nulli; `sx` e `x` hanno durata 50 ns e `cx` durata 300 ns. Depolarizzazione e rilassamento termico sono composti soltanto su `sx/x` e `cx`; il readout è una matrice di confusione simmetrica.

N21 e N35 — validazione ideale. Per $N = 21$, $a = 2$, e $N = 35$, $a = 6$, si

usa l'aritmetica reversibile di Beauregard. La validazione ideale controlla l'azione sui vettori di base validi e il ripristino a zero dei registri ausiliari, oltre al comportamento QPE. Non si attribuiscono a queste istanze risultati rumorosi. Come audit di risorse, il circuito completo $N = 21$, $n_{\text{count}} = 8$, compilato globalmente nella stessa base a livello 2, conta 21 036 **cx** e profondità 23 081. Al valore illustrativo $\lambda = 10^{-3}$, il proxy circuitale registrato è 7.2379×10^{-10} : è un indicatore di accumulo, non la probabilità di successo di Shor.

L'esclusione dal rumore è una scelta di validità e costo: l'aritmetica è validata idealmente, mentre la simulazione rumorosa completa non è scientificamente informativa né sostenibile con le risorse disponibili.

2.6 Metriche di Valutazione

2.6.1 Metriche del Metodo 1

- $\bar{M}_1$: numero medio di esecuzioni necessarie per ottenere il primo risultato corretto, calcolato sulle repliche riuscite e accompagnato dal relativo tasso di successo.
- σ_{M_1} : deviazione standard del numero di esecuzioni, a indicare la variabilità del metodo.
- $P_{\text{success},1}(M_{\text{max}})$: probabilità di successo entro il budget fisso preregistrato $M_{\text{max}} = 50$.

2.6.2 Metriche del Metodo 2

- $\bar{M}_2$: numero medio di esecuzioni prima che il classificatore restituisca la prima etichetta 1 corretta, sulle stesse K ripetizioni usate per il Metodo 1.
- σ_{M_2} : deviazione standard corrispondente.
- Acc_{clf} : accuratezza del classificatore sul test set, definita come il rapporto tra predizioni corrette e numero totale di campioni di test.
- F1_{clf} : F1-score, che bilancia la precisione e il recall nella classe positiva (output corretto). È la metrica preferita rispetto alla sola accuratezza in presenza di classi sbilanciate, un fenomeno frequente quando il tasso di successo dell'algoritmo è basso.

- $P_{\text{success},2}(M_{\text{max}})$: probabilità di successo entro lo stesso budget M_{max} utilizzato per il Metodo 1.

2.6.3 Metrica di Confronto

La metrica principale per il confronto tra i due metodi è il **fattore di riduzione delle iterazioni**:

$$\rho = \frac{\bar{M}_1}{\bar{M}_2} \quad (2.2)$$

Un valore $\rho > 1$ indica una media descrittiva inferiore per il metodo al denominatore. Per l'inferenza, ogni replica genera un unico calendario di istogrammi: M_1 , M_{TOP4} e M_2 analizzano gli stessi conteggi e lo stesso ordinamento. Un fallimento entro M_{max} viene codificato con la sentinella $M_{\text{max}} + 1$, così resta distinto da un successo all'ultima iterazione. Sui vettori appaiati completi si applica Wilcoxon con `zero_method='pratt'`; i confronti direzionali $M_1 > \text{TOP-4}$ e $M_1 > M_2$ sono unilaterali, mentre $\text{TOP-4} - M_2$ è bilaterale. In caso di conteggi uguali, l'ordine dei candidati è determinato da una priorità SHA-256 dipendente dal seed della coppia (replica, iterazione), non dall'ordine del dizionario né dal valore numerico della bitstring.

2.6.4 Metriche della Parte QEC

La fase sulla correzione d'errore adotta un insieme di metriche proprio, centrato sul **logical error rate** anziché sul numero di iterazioni:

- p_L : **logical error rate**, ossia la probabilità che il qubit logico subisca un errore non corretto dopo un ciclo di codifica, rumore, estrazione di sindrome e decodifica. È la metrica centrale della parte QEC, misurata in funzione dell'errore fisico p e della distanza di codice d .
- **Pendenza di soppressione**: l'esponente γ nella relazione $p_L \propto p^\gamma$ a p piccolo; per un codice di distanza $d = 3$ il valore atteso è $\gamma \approx 2$ (correzione di un errore, fallimento a peso due).
- **Break-even**: il regime in cui $p_L < p$, in cui cioè il qubit logico è più affidabile del qubit fisico non protetto.
- **Soglia p_{th}** : per il surface code, il valore di errore fisico a cui le curve $p_L(p)$ a distanze diverse si incrociano, separando il regime in cui aumentare d conviene da quello in cui è controproducente.

- $P_{\text{success}}(p_L)$ e $P(k)$: in M8, p_L è la probabilità totale di un Pauli non identità dopo ciascun gate compilato incluso nel proxy, non il tasso per ciclo dei benchmark QEC; si misurano la probabilità per singola misura e la probabilità cumulativa $P(k) = 1 - (1 - P_s)^k$ dopo k esecuzioni indipendenti.

Ogni stima di p_L è accompagnata dal numero di campioni, dal seed e dall'intervallo di confidenza (deviazione standard binomiale), secondo le stesse regole di riproducibilità della prima fase.

2.6.5 Parametri di Error Mitigation Applicati

Il noise model incorpora il readout error come matrice di confusione simmetrica (parametro p_{ro}) direttamente nel simulatore. Non viene applicata una correzione post-hoc. La sensibilità rispetto a p_{ro} è misurata nell'artefatto parametrico schema 2 corrente (Sezione D.1.5); gli artefatti precedenti alla correzione del circuito non vengono usati.

Il Metodo 1 identifica il candidato per la fattorizzazione selezionando la sola misura più frequente dell'istogramma, senza alcuna sogliatura preliminare: la funzione `extract_factors` è applicata direttamente alla moda della distribuzione. Le varianti multi-candidato (M_{TOP_4} e Metodo 2) estendono il tentativo ai K esiti più frequenti in ordine decrescente, ed è precisamente questa differenza che l'analisi di ablazione isola.

2.7 Approccio Metodologico

Il lavoro integra tre componenti — una parte teorica, una sperimentale e una di analisi comparativa tra strategie di ricerca — che si sviluppano in sequenza lungo l'elaborato. L'architettura dettagliata del sistema sperimentale, le scelte tecnologiche adottate e la struttura dei due metodi sviluppati sono descritte nel Capitolo 6.

Capitolo 3

L'Algoritmo di Shor

Questo capitolo introduce l'algoritmo che dà a questa tesi il proprio oggetto. Si apre con i richiami di calcolo quantistico indispensabili a seguirlo — il qubit, la sovrapposizione, le porte unitarie e il meccanismo di interferenza — di cui l'algoritmo costituisce l'applicazione più celebre.

L'**algoritmo di Shor**, proposto da Peter Shor nel 1994 [1], è un algoritmo quantistico che risolve il problema della **fattorizzazione intera** di un numero N in tempo **polinomiale** nel numero di bit di N :

$$T_{\text{Shor}}(N) = O((\log N)^3) \quad (3.1)$$

Questo risultato costituisce uno **speedup esponenziale** rispetto al miglior algoritmo classico noto — il GNFS — che richiede tempo sub-esponenziale (si veda l'equazione (3.5)), del tutto impraticabile per i numeri di grandi dimensioni usati in crittografia. È considerato il risultato algoritmico più dirompente dell'informatica quantistica, poiché la fattorizzazione è il fondamento matematico del sistema crittografico RSA, che protegge la quasi totalità delle comunicazioni digitali moderne.

L'algoritmo si articola in tre fasi: un *preprocessing* classico che riconduce il problema a trovare il periodo di una funzione modulare; un nucleo quantistico basato sulla QFT e sulla *Quantum Phase Estimation* (QPE), che trova quel periodo in modo esponenzialmente più efficiente di qualunque strategia classica; e una *post-elaborazione* classica che estrae i fattori di N tramite il calcolo del massimo comune divisore. Dopo i richiami si introduce il problema che l'algoritmo risolve e la sua rilevanza crittografica, per poi svilupparne nel dettaglio la struttura.

3.1 Richiami di Calcolo Quantistico

Questa sezione richiama i concetti di meccanica quantistica e di calcolo quantistico necessari a seguire il resto della tesi, riportando per esteso quelli su cui i capitoli successivi si appoggiano operativamente — la sfera di Bloch, le matrici di Pauli e la loro algebra. La trattazione completa, con i quattro postulati, l'insieme esteso delle porte e il modello di complessità, è nella Sezione A.1.

3.1.1 Qubit, Sovrapposizione e Sfera di Bloch

L'unità fondamentale del calcolo quantistico è il **qubit**, descritto da un vettore unitario nello spazio di Hilbert $\mathcal{H} = \mathbb{C}^2$. In notazione di Dirac, lo stato generico è la **sovrapposizione**

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (3.2)$$

dove $|\alpha|^2$ e $|\beta|^2$ sono le probabilità di misurare rispettivamente 0 e 1 (**regola di Born**).

Riparametrizzando le due ampiezze complesse tramite angoli reali — il che è lecito perché la normalizzazione toglie un grado di libertà e la fase globale è inosservabile — ogni stato puro si scrive

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi), \quad (3.3)$$

e si visualizza come un punto sulla superficie della **sfera di Bloch**.¹ I poli sono $|0\rangle$ e $|1\rangle$, l'equatore le sovrapposizioni equiprobabili con fasi diverse.

¹La sfera deve il nome al fisico svizzero-americano Felix Bloch (1905–1983), Premio Nobel per la Fisica nel 1952 per lo sviluppo della spettroscopia a risonanza magnetica nucleare (Risonanza Magnetica Nucleare (*Nuclear Magnetic Resonance*) (NMR)). La stessa notazione T_1 e T_2 per i tempi di decoerenza proviene dal formalismo NMR di Bloch.

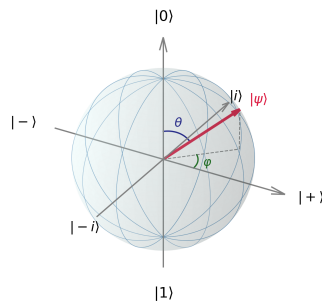


Figura 3.1: La sfera di Bloch: ogni stato puro di un qubit corrisponde a un punto sulla superficie della sfera unitaria, parametrizzato dagli angoli θ e φ della (3.3). I poli nord e sud rappresentano gli stati base $|0\rangle$ e $|1\rangle$; i punti sull'equatore corrispondono a sovrapposizioni equiprobabili. È la rappresentazione a cui si farà riferimento nel Capitolo 4 per descrivere l'azione dei canali di rumore, che deformano la sfera anziché spostare punti su di essa.

3.1.2 Sistemi Multi-Qubit ed Entanglement

Un registro di n qubit vive nello spazio $\mathbb{C}^{2^n}$, ottenuto per **prodotto tensoriale**. La dimensione esponenziale rende possibile rappresentare ampiezze e correlazioni non disponibili a un registro classico di n bit, ma non implica da sola un vantaggio computazionale: alcune famiglie di stati e circuiti quantistici ammettono infatti descrizioni classiche efficienti. Due qubit sono **entangled** quando il loro stato congiunto non è fattorizzabile nel prodotto degli stati individuali; l'esempio canonico è lo stato di Bell $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, per il quale misure nella stessa base producono esiti perfettamente correlati, senza consentire comunicazione istantanea.

L'Entanglement come Risorsa Fisica Fragile

L'entanglement è una risorsa importante per Shor: durante l'esponenziazione modulare il registro di controllo si correla con quello di lavoro, permettendo alla successiva Quantum Fourier Transform di rendere osservabile la periodicità. Un qubit reale, però, non è perfettamente isolato: l'accoppiamento con l'ambiente riduce le coerenze dello stato — la **decoerenza** [13]. Il tempo caratteristico della perdita di fase è T_2 (*dephasing time*); nei dispositivi superconduttori è spesso dello stesso ordine di poche centinaia di durate di una porta a due qubit. Questo confronto segnala un budget di

profondità severo, ma non determina una soglia universale nella dimensione di N : compilazione, parallelismo, tempi di idle, qualità dei gate e architettura fault-tolerant concorrono tutti al limite discusso nel Capitolo 4.

3.1.3 Porte, Misura e Interferenza

Le operazioni sui qubit sono **matrici unitarie** ($U^\dagger U = I$), quindi reversibili. La Tabella 3.1 raccoglie quelle su cui il resto della tesi si appoggia; l'insieme completo, con le derivazioni, è nella Sezione A.1.

Porta	Matrice	Qubit	Effetto
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	1	<i>bit flip</i> : scambia $ 0\rangle$ e $ 1\rangle$
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	1	bit flip e phase flip insieme ($\propto XZ$)
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	1	<i>phase flip</i> : $ 1\rangle \mapsto - 1\rangle$
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	1	genera la sovrapposizione uniforme
R_ϕ	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix}$	1	rotazione di fase; $T = R_{\pi/4}$, $S = R_{\pi/2}$
CNOT	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$	2	nega il <i>target</i> se il <i>control</i> è $ 1\rangle$

Tabella 3.1: Le porte su cui poggia il resto dell'elaborato. Le tre matrici di Pauli descrivono i canali di rumore del Capitolo 4 e gli errori che i codici correttori dei Capitoli 8–9 devono diagnosticare; H e R_ϕ costruiscono la QFT; la Controlled-NOT (CNOT) è la porta a due qubit dominante nel conteggio delle risorse, ed è quella a cui è associato l'errore più severo.

L'identità e le tre matrici di Pauli formano una base dello spazio degli operatori lineari 2×2 : ogni operatore d'errore su un singolo qubit si può quindi decomporre in tale base. Poiché $Y = iXZ$, un codice che soddisfa le condizioni di correzione per gli errori di Pauli singoli corregge, per linearità, anche le loro combinazioni — proprietà

su cui si fonda l'intera parte QEC. Valgono inoltre le relazioni

$$\begin{aligned} X^2 &= Y^2 = Z^2 = I, \\ XY &= iZ, \quad YZ = iX, \quad ZX = iY, \\ \{X, Y\} &= \{Y, Z\} = \{X, Z\} = 0, \end{aligned} \tag{3.4}$$

dove $\{A, B\} = AB + BA$ è l'anticommutatore. Nella teoria degli stabilizzatori, un errore è rilevato quando anticommuta con almeno uno dei generatori misurati; è il vettore completo di questi esiti di commutazione a costituire la sindrome. Nei codici Calderbank-Shor-Steane (CSS), le famiglie di stabilizzatori X e Z permettono di diagnosticare separatamente le due componenti di Pauli, come discusso nel Capitolo 8.

Insieme alle porte a singolo qubit, la CNOT forma un insieme universale: ogni operazione unitaria su n qubit si approssima a piacere componendo elementi di questo insieme. La **misura** in base computazionale collassa lo stato $|\psi\rangle = \sum_x \alpha_x |x\rangle$ sull'esito x con probabilità $|\alpha_x|^2$: è un'operazione non unitaria e irreversibile. Il meccanismo che rende utile tutto ciò è l'**interferenza**: un algoritmo efficiente è costruito perché le ampiezze degli esiti sbagliati si cancellino (interferenza distruttiva) e quelle degli esiti corretti si sommino (costruttiva). È esattamente ciò che la QFT realizza nell'algoritmo di Shor (Capitolo 3) per far emergere il periodo.

3.2 Il Problema della Fattorizzazione Intera

La **fattorizzazione intera** consiste nel trovare i fattori primi di un numero intero N . Per numeri piccoli il problema è banale, ma la sua difficoltà cresce in modo impressionante all'aumentare di N : i migliori algoritmi classici noti, come il GNFS, richiedono un tempo sub-esponenziale ma comunque super-polinomiale:²

$$T_{\text{classico}}(N) = O\left(\exp\left((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3}\right)\right) \tag{3.5}$$

Nella sua formulazione decisionale il problema appartiene a NP, e più precisamente a $\text{NP} \cap \text{co-NP}$: sia l'esistenza sia l'assenza di un fattore in un dato intervallo

²Il GNFS detiene il record mondiale per la fattorizzazione classica: nel 2020, un consorzio internazionale ha fattorizzato RSA-250 (829 bit, 250 cifre decimali) [14], impiegando l'equivalente di circa 2700 anni-core. In precedenza, nel 2009, era stato fattorizzato RSA-768 (768 bit). I numeri RSA usati in produzione sono di 2048 o 4096 bit, rendendo la fattorizzazione classica del tutto impraticabile con qualunque tecnologia convenzionale prevedibile.

ammettono un certificato verificabile in tempo polinomiale. È questa collocazione a rendere improbabile che la fattorizzazione sia NP-completa — lo fosse, ne seguirebbe $NP = co-NP$, contro l'opinione prevalente. Non è però noto se appartenga a P: nessun algoritmo classico polinomiale è *conosciuto*, e si congettura che non esista, ma una dimostrazione di questa impossibilità manca.

3.2.1 Perché la Fattorizzazione Conta

La difficoltà di fattorizzare interi non è una curiosità matematica: è il fondamento su cui poggia RSA, e con esso buona parte della sicurezza delle comunicazioni odierne. Un algoritmo polinomiale per la fattorizzazione compromette RSA in modo irrecuperabile, e non per una debolezza implementativa ma perché ne viola l'assunzione di sicurezza. La trattazione estesa — il confronto fra la complessità sub-esponenziale del GNFS e quella polinomiale di Shor, la risposta della crittografia post-quantistica con gli standard *National Institute of Standards and Technology* (NIST) del 2024, e lo scenario *harvest now, decrypt later* — è nella Sezione C.2 dell'Appendice C. Fra l'algoritmo teorico e un'applicazione crittografica restano scala, compilazione fault-tolerant, overhead QEC e qualità dell'hardware; questa tesi isola soprattutto il ruolo del rumore.

3.3 Riduzione alla Ricerca del Periodo

L'intuizione geniale di Shor consiste nel ricondurre la fattorizzazione a un problema di **period finding**, trattabile quantisticamente in modo efficiente.

Osservazione chiave: Dato N e un intero a con $\gcd(a, N) = 1$ (coprime con N), la funzione

$$f(x) = a^x \bmod N \quad (3.6)$$

è **periodica** con periodo r (detto *ordine* di a modulo N): $f(x + r) = f(x)$ per ogni x .

Una volta trovato il periodo r , se r è pari si può calcolare:

$$\gcd(a^{r/2} - 1, N) \quad \text{e} \quad \gcd(a^{r/2} + 1, N) \quad (3.7)$$

Con alta probabilità, almeno uno di questi due valori è un fattore non banale di N . La prova utilizza proprietà della teoria dei numeri: in particolare, vale $(a^{r/2} - 1)(a^{r/2} + 1) = a^r - 1 \equiv 0 \pmod{N}$.

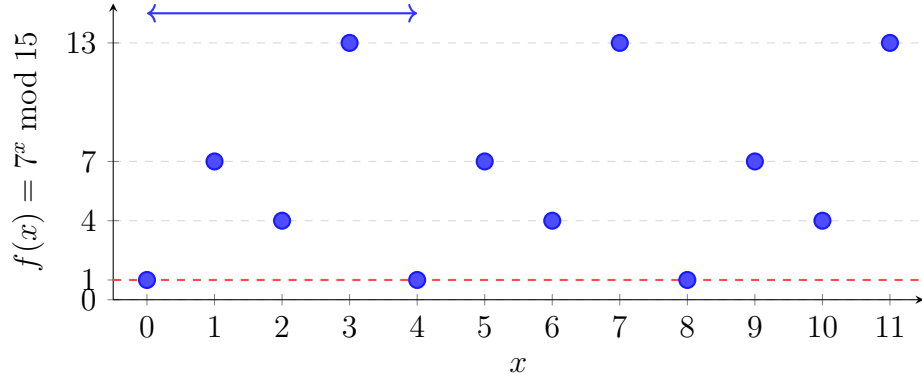


Figura 3.2: La funzione $f(x) = 7^x \bmod 15$ è periodica con periodo $r = 4$. La periodicità è evidente dai valori che si ripetono: $1, 7, 4, 13, 1, 7, 4, 13, \dots$. Trovare r permette di calcolare $\gcd(7^2 - 1, 15) = \gcd(48, 15) = 3$ e $\gcd(7^2 + 1, 15) = \gcd(50, 15) = 5$, ottenendo la fattorizzazione $15 = 3 \times 5$.

3.4 La Quantum Fourier Transform

Il componente quantistico fondamentale dell'algoritmo di Shor è la **Quantum Fourier Transform** (QFT), l'analogo quantistico della Trasformata Discreta di Fourier (Trasformata Discreta di Fourier (*Discrete Fourier Transform*) (DFT)).

3.4.1 Definizione

La QFT su $\mathbb{Z}_{2^n}$ agisce su uno stato base come:

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i j k / 2^n} |k\rangle \quad (3.8)$$

L'azione su uno stato generico $|\psi\rangle = \sum_j \alpha_j |j\rangle$ produce:

$$\text{QFT} |\psi\rangle = \sum_k \hat{\alpha}_k |k\rangle \quad (3.9)$$

dove $\hat{\alpha}_k = \frac{1}{\sqrt{2^n}} \sum_j \alpha_j e^{2\pi i j k / 2^n}$ sono i coefficienti della DFT.

3.4.2 Implementazione Efficiente

La DFT classica su $N = 2^n$ elementi richiede $O(N \log N)$ operazioni (con il Fast Fourier Transform), mentre la QFT si realizza con $O(n^2) = O((\log N)^2)$ porte quantistiche: un conteggio esponenzialmente inferiore.

Questo confronto va però interpretato correttamente, perché la formulazione ingenua — “la QFT calcola la trasformata di Fourier esponenzialmente più in fretta” — è fuorviante. La QFT non restituisce i coefficienti $\hat{\alpha}_k$: li lascia *codificati nelle ampiezze* di uno stato quantistico, e la misura ne estrae un solo valore per esecuzione, con probabilità $|\hat{\alpha}_k|^2$. Non esiste quindi alcun modo di leggere lo spettro completo, e la QFT non sostituisce la FFT come strumento di calcolo generale. Il vantaggio è utilizzabile soltanto quando l’informazione cercata è una *proprietà globale* dello spettro che una singola misura è sufficiente a rivelare — ed è esattamente il caso della periodicità nell’algoritmo di Shor, dove i picchi della distribuzione trasformata cadono sui multipli di $2^n/r$ e una manciata di misure basta a ricostruire r .

Il circuito della QFT è costruito ricorsivamente tramite porte di Hadamard e porte di fase controllate CR_k dove:

$$CR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{2\pi i/2^k} \end{pmatrix} \quad (3.10)$$

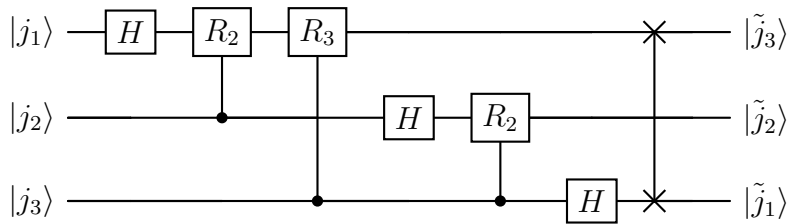


Figura 3.3: Circuito della QFT su $n = 3$ qubit. Ogni qubit riceve una porta di Hadamard H seguita da porte di fase controllate $R_k = \text{diag}(1, e^{2\pi i/2^k})$: la rotazione applicata al qubit j dipende dai qubit successivi. Lo SWAP finale riordina i qubit nell’ordine corretto. La struttura si generalizza a n qubit richiedendo $O(n^2)$ porte.

3.4.3 Phase Estimation

La QFT è impiegata, all’interno dell’algoritmo di Shor, attraverso la subroutine di **Phase Estimation** (QPE). Data una porta U con autovettore $|u\rangle$ e autovalore $e^{2\pi i\theta}$,

la QPE stima θ con precisione arbitraria in tempo $O(n^2)$. Nell'algoritmo di Shor, U è l'operatore di moltiplicazione modulare $U|x\rangle = |ax \bmod N\rangle$, e il suo spettro di fase contiene l'informazione sul periodo r .

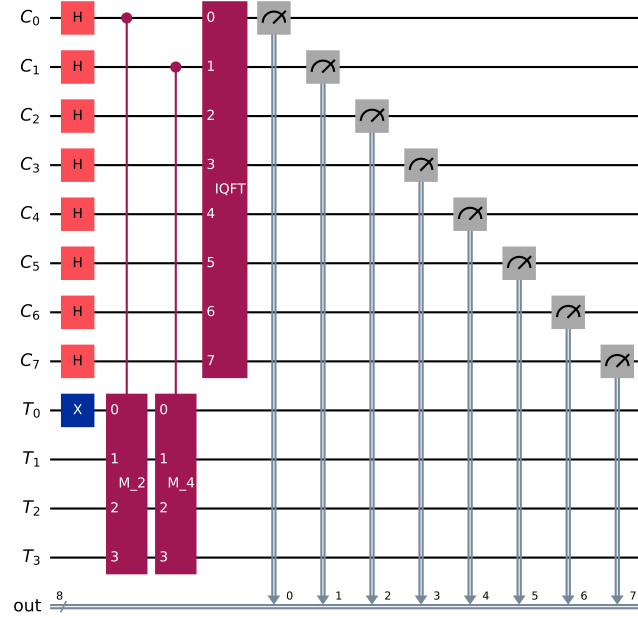


Figura 3.4: Circuito di Quantum Phase Estimation per $N = 15$, $a = 2$, implementato con Qiskit. Il registro di controllo (C , 8 qubit) viene posto in sovrapposizione uniforme, entangolato tramite gli operatori M_2 e M_4 controllati, e trasformato con la QFT^{-1} . La misura del registro C produce una stima della fase $\theta = k/r$, da cui si ricava il periodo r tramite frazioni continue.

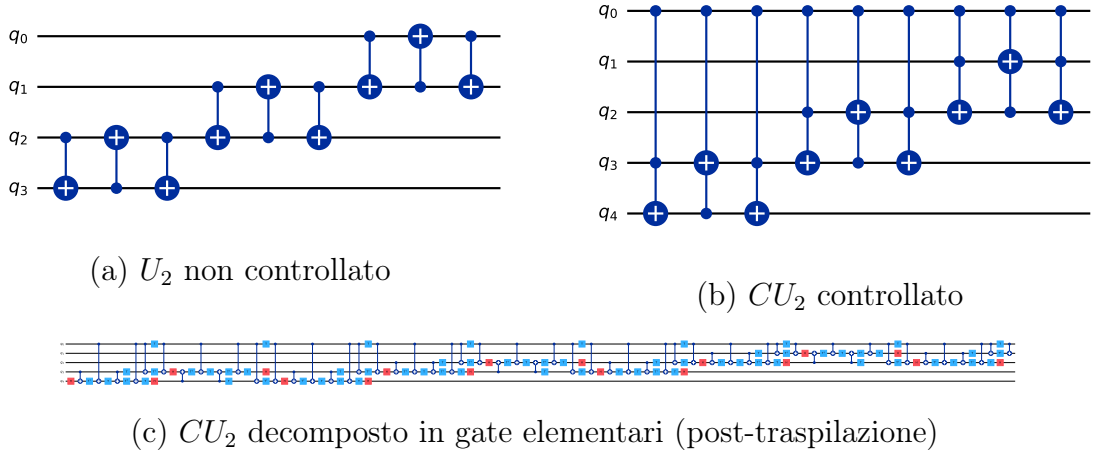


Figura 3.5: Gate di moltiplicazione modulare $U_2 |x\rangle = |2x \bmod 15\rangle$ per $N = 15$, $a = 2$. (a) Versione non controllata su 4 qubit: implementata come sequenza di SWAP. (b) Versione controllata CU_2 su 5 qubit utilizzata nel circuito QPE. (c) Decomposizione in gate elementari dopo la traspilazione: le porte SWAP si espandono in triple di CNOT, rivelando il costo effettivo in gate a due qubit del blocco.

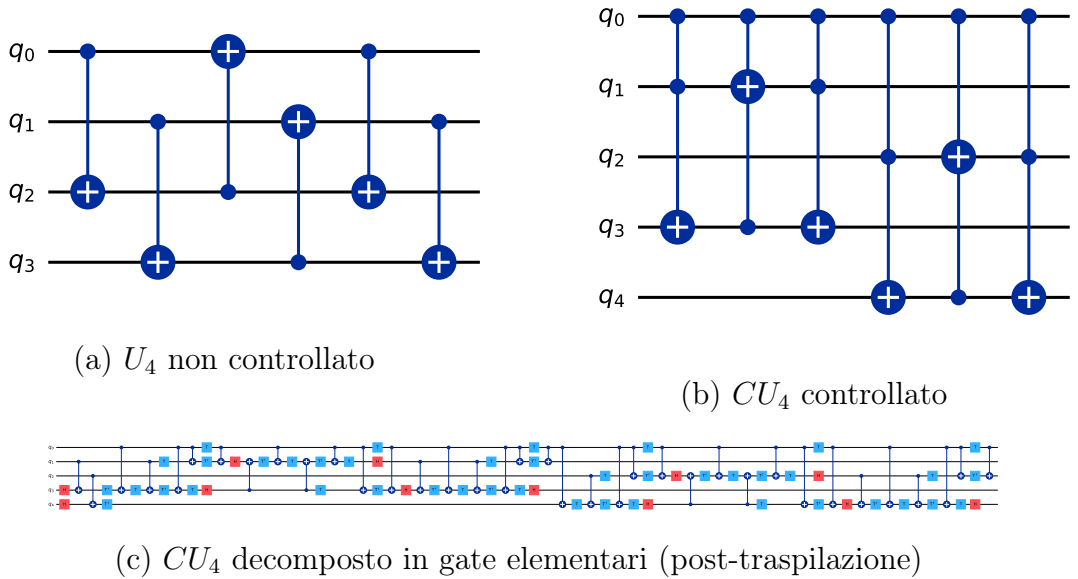


Figura 3.6: Gate di moltiplicazione modulare $U_4 |x\rangle = |4x \bmod 15\rangle$ per $N = 15$, $a = 4 = 2^2$. (a) Versione non controllata: il pattern di SWAP differisce da U_2 poiché la permutazione modulare per $a = 4$ è diversa. (b) Versione controllata CU_4 su 5 qubit, secondo blocco del circuito QPE. (c) Decomposizione in gate elementari: il costo aggiuntivo rispetto a CU_2 riflette la maggiore complessità della permutazione per $a = 4$.

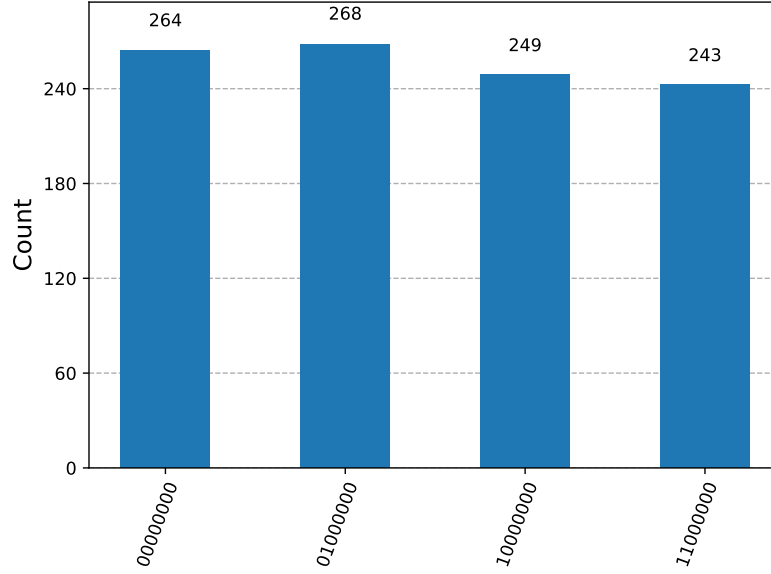


Figura 3.7: Distribuzione ideale campionata del registro di controllo per $N = 15$, $a = 7$ (1024 shot). I quattro stati attesi — $\{0, 64, 128, 192\}$ — sono i multipli di $2^8/r = 64$ per $r = 4$; le oscillazioni dei conteggi fra i picchi sono fluttuazioni di campionamento.

3.5 L'Algoritmo di Shor: Schema Completo

L'algoritmo si articola in tre fasi: un preprocessing classico, l'esecuzione del circuito quantistico per il *period finding*, e una post-elaborazione classica per estrarre i fattori.

Fase 1 — Preprocessing classico

1. Scegliere $a \in \{2, \dots, N-1\}$ uniformemente a caso.
2. Calcolare $g = \gcd(a, N)$: se $g > 1$, restituire g (fattore trovato direttamente).

Fase 2 — Circuito quantistico (period finding)

3. Inizializzare $n = 2\lceil \log_2 N \rceil$ qubit nel registro di controllo: $|0\rangle^{\otimes n} |1\rangle$.
4. Applicare $H^{\otimes n}$ al registro di controllo, creando la sovrapposizione uniforme $\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$.
5. Applicare la moltiplicazione modulare controllata: $|x\rangle |1\rangle \mapsto |x\rangle |a^x \bmod N\rangle$.
6. Misurare il registro di lavoro: il registro di controllo collassa in una sovrapposizione periodica di passo r . Il passo è *concettuale* — serve a rendere evidente

l'origine della periodicità — e nell'implementazione viene omesso, come chiarito nella Sezione 3.5.1.

7. Applicare la QFT^{-1} al registro di controllo.
8. Misurare il registro di controllo: ottenere un intero proporzionale a s/r , con $s \in \{0, \dots, r-1\}$.

Fase 3 — Post-elaborazione classica

9. Applicare l'algoritmo delle frazioni continue al risultato della misura per stimare il periodo r .
10. Se r è dispari o $a^{r/2} \equiv -1 \pmod{N}$: tornare al passo 1 con un nuovo valore di a .
11. Calcolare $p = \gcd(a^{r/2} - 1, N)$ e $q = \gcd(a^{r/2} + 1, N)$.
12. Se p o q è un fattore non banale di N : restituirlo.
Altrimenti: tornare al passo 1.

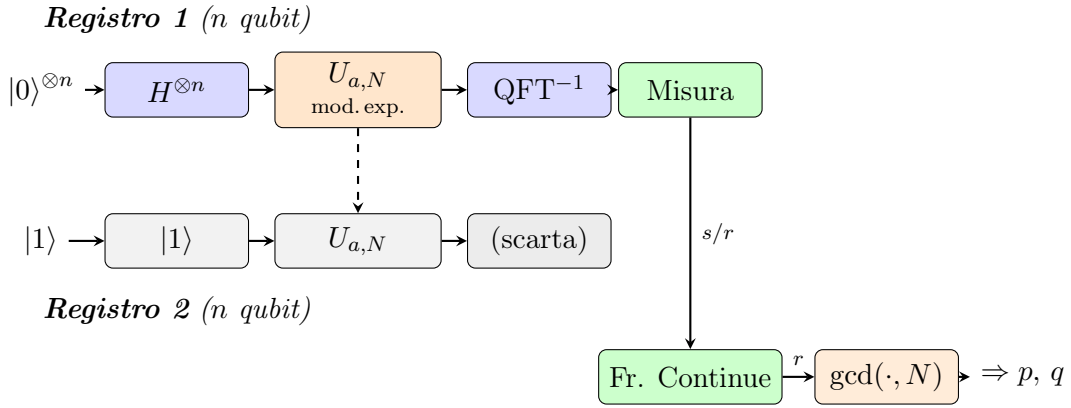


Figura 3.8: Schema a due registri dell'algoritmo di Shor. Il **Registro 1** (di controllo) viene posto in sovrapposizione uniforme con $H^{\otimes n}$, entangolato con il Registro 2 tramite la moltiplicazione modulare $U_{a,N}$, quindi trasformato con la QFT^{-1} . La misura produce un valore s/r elaborato classicamente: le frazioni continue stimano il periodo r , e il calcolo del MCD restituisce i fattori p e q di N .

3.5.1 Il Ruolo dell'Entanglement nel Period Finding

Il collasso descritto al passo 6 dello schema non è un effetto arbitrario: è la conseguenza diretta dell'**entanglement** creato al passo 5. Vale la pena rendere esplicito questo meccanismo, che è il cuore quantistico dell'intero algoritmo.

Dopo l'applicazione della moltiplicazione modulare controllata, i due registri si trovano nello stato entangled:

$$|\Psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |a^x \bmod N\rangle \quad (3.11)$$

Questo stato **non è separabile**: il valore del registro di lavoro è correlato in modo univoco al valore del registro di controllo. I due registri formano un unico sistema entangled, esattamente come accade in uno stato di Bell — dove misurare un qubit determina istantaneamente l'esito dell'altro.

Se a questo punto si misurasse il registro di lavoro e si ottenesse un valore $v = a^{x_0} \bmod N$, il registro di controllo collasserebbe *istantaneamente* nell'unica sovrapposizione compatibile con v : quella di tutti i valori x per cui $a^x \equiv v \pmod{N}$, ovvero

$$|x_0\rangle + |x_0 + r\rangle + |x_0 + 2r\rangle + \dots \quad (3.12)$$

La correlazione tra i due registri è **bidirezionale**: conoscere il valore del registro di lavoro vincola il registro di controllo a una sovrapposizione periodica di passo esattamente r . Questa è la “versione inversa” propria dell'entanglement: misurare uno dei due sottosistemi determina (o in questo caso, restringe) lo stato dell'altro.

In pratica il circuito di Shor non misura esplicitamente il registro di lavoro: la sola creazione dell'entanglement è sufficiente. Applicando la QFT^{-1} al registro di controllo prima della misura, si trasforma la periodicità nascosta dell'equazione (3.12) in picchi netti alle frequenze k/r , con $k \in \{0, \dots, r-1\}$. La misura finale del registro di controllo restituisce allora, con alta probabilità, un valore da cui le frazioni continue estraggono r .

In sintesi: l'entanglement tra i due registri è il meccanismo che *trasferisce* la periodicità della funzione $f(x) = a^x \bmod N$ — che esiste nel registro di lavoro — verso il registro di controllo, dove la QFT la rende misurabile. Senza entanglement, questa trasmissione di informazione tra i due registri sarebbe impossibile, e l'intero speedup esponenziale di Shor verrebbe meno.

3.6 Analisi della Complessità

La complessità dell'algoritmo di Shor è **polinomiale** nel numero di bit di N :

- Il circuito quantistico richiede $O(n^3)$ porte elementari con aritmetica modulare elementare, dove $n = \log N$; ricorrendo alla moltiplicazione veloce il conteggio scende a $O(n^2 \log n \log \log n)$;
- Il numero di qubit necessari è $O(\log N)$; l'implementazione ottimizzata di Beauregard [15] lo riduce a $2n + 3$ tramite addizione modulare in-place;
- Il numero atteso di iterazioni (con basi a diverse) prima del successo è $O(1)$.

Complessivamente, nella versione con aritmetica elementare,

$$T_{\text{Shor}}(N) = O((\log N)^3),$$

a fronte della complessità sub-esponenziale dell'Equazione (3.5) del miglior algoritmo classico generale noto. In funzione della lunghezza dell'input $n = \log N$, il divario è **superpolinomiale**; chiamarlo semplicemente “esponenziale” nasconderebbe il fatto che GNFS è già sub-esponenziale in n .

3.7 Stato dell'Arte e Confronto con Altri Algoritmi

Le realizzazioni sperimentali di Shor su hardware reale, i miglioramenti algoritmici più recenti — fra cui la variante di Regev del 2023 — e la roadmap dei costruttori verso il calcolo fault-tolerant sono discussi nella Sezione A.3 dell'Appendice A, insieme al confronto con l'algoritmo di Grover. Due elementi di quel quadro vanno però anticipati qui, perché delimitano il perimetro della parte sperimentale.

Il primo è che **nessuna implementazione ha finora fattorizzato un numero non banale senza semplificazioni**: le dimostrazioni note sfruttano la conoscenza preventiva del risultato per ridurre il circuito, e questo colloca il presente lavoro — che esegue il circuito completo su simulatore rumoroso — in un regime diverso da quello delle dimostrazioni su hardware. Il secondo è che il divario fra Shor dimostrativo e Shor crittograficamente rilevante non è di scala ma di natura: le stime di risorse per moduli a 2048 bit derivano da analisi architetturali complete, non da estrapolazioni di istanze piccole, e il Capitolo 10 riprende esplicitamente questo punto.

Capitolo 4

Rumore Quantistico nei Sistemi NISQ

4.1 Introduzione al Rumore Quantistico

Il capitolo precedente ha presentato l'algoritmo di Shor nella sua forma ideale: un circuito che, eseguito su un calcolatore quantistico perfetto, fattorizza in tempo polinomiale. Quel calcolatore non esiste. Questo capitolo descrive la distanza fra il modello e la macchina — da dove nasce l'errore, come lo si rappresenta formalmente, e quanto costa all'algoritmo.

I computer quantistici non fault-tolerant operano nel regime NISQ [3]: il solo numero di qubit fisici non determina la capacità utile, perché errori, connettività e assenza di correzione scalabile limitano la profondità dei circuiti eseguibili in modo affidabile.

Il rumore quantistico differisce fundamentalmente dal rumore nei sistemi classici: mentre un bit classico può essere protetto ridondando l'informazione, un qubit non può essere copiato (**teorema del no-cloning**) [16]. Inoltre, gli errori quantistici non sono binari — un qubit può subire rotazioni arbitrarie nello spazio di Hilbert — rendendo la loro caratterizzazione intrinsecamente più ricca e più difficile di quella classica.

Per analizzare il rumore in modo rigoroso è utile distinguere tre livelli di descrizione, che il presente capitolo tratta separatamente:

- **Anatomia della macchina** (Sezione 4.2): com'è fatta fisicamente una QPU e *dove*, nei suoi componenti, gli errori hanno origine;
- **Fonti fisiche** (Sezione 4.3): i quattro meccanismi concreti con cui l'hardware introduce errori;

- **Modelli matematici** (Sezione 4.5): i quattro canali quantistici con cui tali errori vengono formalizzati e simulati.

4.2 La Macchina Fisica: i Tre Fatti che Contano

Il rumore di un processore quantistico non è un disturbo generico ma la conseguenza diretta di come la macchina è costruita. La struttura di una QPU superconduttiva — l'architettura del chip criogenico, il ruolo dell'elettronica di controllo, il confronto fra le tecnologie di qubit disponibili e il meccanismo fisico della lettura — è descritta nella Sezione A.2 dell'Appendice A. Ai fini del seguito bastano tre fatti, che vale la pena isolare perché ricorrono in tutto il lavoro.

Primo: le porte logiche non risiedono nel chip. Aprendo una Unità di Elaborazione Centrale (*Central Processing Unit*) (CPU) classica si trovano registri e porte incise nel silicio; aprendo una QPU si trovano soltanto i qubit e le linee che li collegano. Le operazioni sono impulsi a microonde generati dall'elettronica esterna, e “applicare un gate” significa inviare un'onda sagomata che ruota lo stato dell'angolo voluto. Ne discende che la fedeltà di una porta è la fedeltà di un impulso analogico: non esiste una soglia di rigenerazione del segnale come nell'elettronica digitale, e ogni imprecisione si accumula.

Secondo: il budget di operazioni è finito. Lo stato di un qubit decade con tempi caratteristici T_1 e T_2 , mentre ogni gate richiede un tempo proprio. Il rapporto fra i due fissa quante operazioni si possono eseguire prima che l'informazione si perda, indipendentemente da quanto sia accurata la singola porta. È questo vincolo — non il numero di qubit — a determinare la profondità massima di un circuito utile, ed è la ragione per cui il Capitolo 7 misura la barriera di scalabilità in porte a due qubit anziché in qubit.

Terzo: la topologia decide chi parla con chi. Le QPU differiscono per geometria — catene lineari, griglie *heavy-hex*, connettività completa — e l'entanglement fra qubit non adiacenti richiede di attraversare quelli intermedi, con un costo che si paga in coerenza. La topologia determina quindi sia quali circuiti si mappano efficientemente, sia dove il crosstalk colpirà. È il fondamento architetturale del surface code (Capitolo 9), che richiede interazioni fra soli vicini proprio per questa ragione.

4.3 Le Quattro Fonti Fisiche di Rumore

Sui processori quantistici reali il rumore origina da quattro meccanismi distinti. Ognuno agisce su una scala temporale e spaziale diversa, e richiede strategie di mitigazione differenti.

4.3.1 Decoerenza (T_1 e T_2)

La **decoerenza** è il processo per cui un qubit perde le sue proprietà quantistiche a causa dell'interazione inevitabile con l'ambiente circostante (campi elettromagnetici parassiti, fononi, fluttuazioni termiche). Si manifesta attraverso due tempi caratteristici:

- T_1 – **Relaxation time**: il qubit decade spontaneamente dallo stato eccitato $|1\rangle$ allo stato fondamentale $|0\rangle$ per dissipazione energetica. Fisicamente corrisponde all'emissione di un fotone verso l'ambiente. Su hardware superconduttore tipicamente $T_1 \sim 50\text{--}500\ \mu\text{s}$.¹
- T_2 – **Dephasing time**: la fase relativa tra $|0\rangle$ e $|1\rangle$ si randomizza senza necessariamente modificare le popolazioni, distruggendo la sovrapposizione. T_2 comprende due contributi:

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi} \quad (4.1)$$

dove T_ϕ è il tempo di *pura* perdita di fase. Vale sempre $T_2 \leq 2T_1$; tipicamente $T_2 \sim 50\text{--}200\ \mu\text{s}$.

Un criterio euristico per limitare la decoerenza non corretta lungo il cammino critico è:

$$\tau_{\text{circuito}} \ll \min(T_1, T_2) \quad (4.2)$$

La profondità dell'aritmetica modulare cresce polinomialmente con la dimensione dell'input, ma questa relazione asintotica non fissa da sola una soglia in bit: compilazione, parallelismo, tempi di idle, topologia e correzione d'errore determinano la durata fisica effettiva.

Connessione con l'entanglement. T_2 caratterizza la perdita di coerenza di fase necessaria a sostenere l'entanglement fra registro di controllo e registro di lavoro. Nella phase estimation, ciascun qubit di controllo deve conservare la fase relativa lungo il proprio cammino causale attraverso l'esponenziazione modulare. Confrontare semplicemente T_2/τ_{gate} con il numero totale di gate a due qubit sarebbe però scorretto: il conteggio non è una durata e ignora parallelismo e idle. Per questo la campagna usa durate esplicite nel noise model e riporta separatamente conteggi e profondità della netlist.

¹La notazione T_1 e T_2 è mutuata dalla spettroscopia a NMR, dove Felix Bloch la introdusse nel 1946 per i tempi di rilassamento longitudinale e trasversale della magnetizzazione nucleare.

4.3.2 Errori di Gate

Le porte quantistiche fisiche non implementano perfettamente l'operazione unitaria ideale U , bensì un'operazione perturbata $\tilde{U} = U + \delta E$. L'**errore di gate** è quantificato dalla **gate infidelity**:

$$r(U, \tilde{U}) = 1 - F(U, \tilde{U}), \quad F(U, \tilde{U}) = \frac{1}{d^2} \left| \text{Tr}(U^\dagger \tilde{U}) \right|^2 \quad (4.3)$$

dove d è la dimensione dello spazio di Hilbert e F è la *process fidelity*, pari a 1 quando $\tilde{U} = U$. Le cause principali sono le imprecisioni nel controllo di ampiezza e frequenza dei pulse a microonde, l'accoppiamento residuo *always-on* tra qubit adiacenti, e il *leakage* verso stati non computazionali ($|2\rangle, |3\rangle, \dots$) negli oscillatori di Josephson. Su hardware IBM (2024–2025) i valori tipici sono: $r \approx 0.03\text{--}0.1\%$ per gate a singolo qubit ($F \approx 99.9\text{--}99.97\%$) e $r \approx 0.3\text{--}1\%$ per gate a due qubit come CNOT ed ECR ($F \approx 99\text{--}99.7\%$).

4.3.3 Errori di Misura (Readout Errors)

La misura finale può restituire un risultato errato: il qubit si trova in $|0\rangle$ ma viene letto come 1, o viceversa. Questo errore è modellato da una **matrice di confusione** A :

$$A_{ij} = P(\text{misura } i \mid \text{stato } j), \quad \sum_i A_{ij} = 1 \quad (4.4)$$

Per n qubit indipendenti, $A = \bigotimes_{k=1}^n A_k$ con

$$A_k = \begin{pmatrix} 1 - e_0^{(k)} & e_1^{(k)} \\ e_0^{(k)} & 1 - e_1^{(k)} \end{pmatrix} \quad (4.5)$$

dove $e_0^{(k)}$ è la probabilità di leggere 1 dato $|0\rangle$ per il k -esimo qubit. Valori tipici: $e_0, e_1 \approx 1\text{--}3\%$ su hardware IBM.

4.3.4 Crosstalk

Il **crosstalk** è l'interferenza indesiderata tra qubit adiacenti durante l'esecuzione di operazioni parallele. Origina dall'accoppiamento capacitivo residuo tra qubit vicini nel chip superconduttore: quando si applica un gate su un qubit, le perturbazioni del campo si propagano ai qubit circostanti causando rotazioni parassite non intenzionali. Il crosstalk è particolarmente dannoso nei circuiti profondi con molte operazioni

parallele ed è difficile da modellare analiticamente [17], il che rende i modelli di apprendimento automatico particolarmente adatti a caratterizzarlo. Questa affermazione, qui anticipata, riceve verifica sperimentale nella Sezione 9.6: è precisamente in presenza di errori correlati fra qubit adiacenti — il crosstalk — che un decoder appreso supera quello analitico, perché tali errori non sono rappresentabili nella struttura su cui quest’ultimo si fonda.

Fonte	Causa fisica	Parametro	Valore tipico IBM
Decoerenza	Interazione con l’ambiente	T_1, T_2	50–500 μs
Errori di gate	Imprecisione pulse / leakage	Gate infidelity	0.03–1%
Readout errors	Risposta del rivelatore	e_0, e_1	1–3%
Crosstalk	Accoppiamento residuo	Difficile da parametrizzare	~ 0.1 –1%

Tabella 4.1: Riepilogo delle quattro fonti fisiche di rumore nei processori quantistici superconduttori.

4.4 Dal Fenomeno Fisico al Modello Matematico

I quattro fenomeni fisici descritti nella sezione precedente non sono isolati: ciascuno può essere ricondotto a uno o più canali matematici, che ne forniscono una descrizione quantitativa precisa e simulabile. La tabella seguente stabilisce il collegamento esplicito tra i due livelli di descrizione.

Fonte fisica	Canale matematico	Parametro chiave
Decadimento energetico (T_1)	Amplitude Damping	$\gamma = 1 - e^{-t/T_1}$
Perdita di fase (T_2)	Phase Flip	$p = \frac{1}{2}(1 - e^{-t/T_2})$
Errori di gate (isotropici)	Depolarizzante	$p \approx \text{gate infidelity}$
Errori di gate (asimmetrici)	Bit Flip / Phase Flip	p dal profilo dell’errore
Readout errors	Matrice di confusione A	e_0, e_1 da calibrazione
Crosstalk	Depolarizzante (multi-qubit)	Difficile da parametrizzare

Tabella 4.2: Corrispondenza tra fonti fisiche di rumore e canali quantistici.

In pratica, Qiskit Aer combina rilassamento energetico e dephasing in un **canale di thermal relaxation**, parametrizzato da T_1 , T_2 e dal tempo di gate τ . È un modello Markoviano controllato di queste due componenti, non una riproduzione completa dell’hardware: non include automaticamente leakage, crosstalk, deriva o errori coerenti.

4.5 I Quattro Canali Matematici del Rumore

Per simulare e analizzare il rumore, le fonti fisiche vengono astratte in **canali quantistici**: mappe lineari *Completely Positive Trace-Preserving* (CPTP) che agiscono sulla matrice densità ρ del sistema [6]. Ogni canale CPTP ammette la **rappresentazione di Kraus**:

$$\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger, \quad \sum_k K_k^\dagger K_k = I \quad (4.6)$$

I quattro canali fondamentali, ciascuno dei quali cattura un diverso regime di errore, sono presentati di seguito.

4.5.1 Il Canale Depolarizzante

Fra i quattro, il canale **depolarizzante** merita la trattazione esplicita perché è quello su cui poggia l'intera campagna sperimentale: modella gli errori di gate, ed è il parametro ε_{2q} che la Sezione 7.5 farà variare su tre ordini di grandezza. Con probabilità p viene applicato un errore casuale fra $\{X, Y, Z\}$, ciascuno con probabilità $p/3$:

$$\mathcal{E}_{dep}(\rho) = (1 - p) \rho + \frac{p}{3} (X \rho X + Y \rho Y + Z \rho Z) \quad (4.7)$$

Usando l'identità $X \rho X + Y \rho Y + Z \rho Z = 2I - \rho$, valida per $\text{Tr}(\rho) = 1$, si ottiene la forma compatta

$$\mathcal{E}_{dep}(\rho) = \left(1 - \frac{4p}{3}\right) \rho + \frac{4p}{3} \frac{I}{2}, \quad (4.8)$$

che rende evidente il significato: il canale interpola fra lo stato ρ e quello massimamente misto $I/2$, contraendo *isotropicamente* il vettore di Bloch di un fattore $(1 - 4p/3)$. Per $p = 3/4$ lo stato diventa completamente misto qualunque fosse quello iniziale.

L'isotropia è la ragione per cui questo canale è un modello standard degli errori di gate: non privilegia alcuna direzione e descrive un'imprecisione di controllo di cui non si specifica la natura microscopica. Nel parametro Aer λ , tuttavia, la probabilità aggregata di una componente Pauli non-identità è $3\lambda/4$ per un qubit e $15\lambda/16$ per due qubit. Come mostrerà il Capitolo 7, il proxy di nessun evento su k gate a due qubit è quindi $(1 - 15\lambda_{2q}/16)^k$; non è una probabilità di successo dell'algoritmo.

4.5.2 Gli Altri Tre Canali

Gli altri tre canali si enunciano compattamente; le derivazioni complete — operatori di Kraus espliciti, azione sulla matrice densità e trasformazione del vettore di Bloch — sono nella Sezione A.4 dell’Appendice A.

Il **bit flip** applica X con probabilità p e contrae le componenti r_y, r_z del fattore $(1 - 2p)$, lasciando r_x invariata. Il **phase flip** applica Z con probabilità p : lascia intatte le popolazioni e sopprime le coerenze ρ_{01} dello stesso fattore — è il canale che descrive il decadimento T_2 , poiché ponendo $p(t) = \frac{1}{2}(1 - e^{-t/T_2})$ si ottiene esattamente $\rho_{01}(t) = \rho_{01}(0) e^{-t/T_2}$. L’**amplitude damping**, infine, modella il rilassamento energetico T_1 : il qubit decade da $|1\rangle$ a $|0\rangle$ con probabilità $\gamma = 1 - e^{-t/T_1}$, e a differenza degli altri è *non simmetrico* — contrae la sfera di Bloch verso il polo $|0\rangle$ anziché verso il centro, sicché per $t \rightarrow \infty$ ogni stato converge a $|0\rangle\langle 0|$.

Questi ultimi due sono i canali che la funzione `thermal_relaxation_error` di Qiskit Aer combina per riprodurre la decoerenza. I preset uniformi della prima fase e della demo li impiegano con T_1, T_2 e durate dichiarate; M8 usa invece il solo proxy Pauli per gate, mentre M11 converte l’infedeltà media della snapshot senza aggiungere nuovamente T_1/T_2 , per evitare un doppio conteggio.

Effetto dei quattro canali di rumore sulla sfera di Bloch
(sezione nel piano xz , tratteggio = sfera ideale)

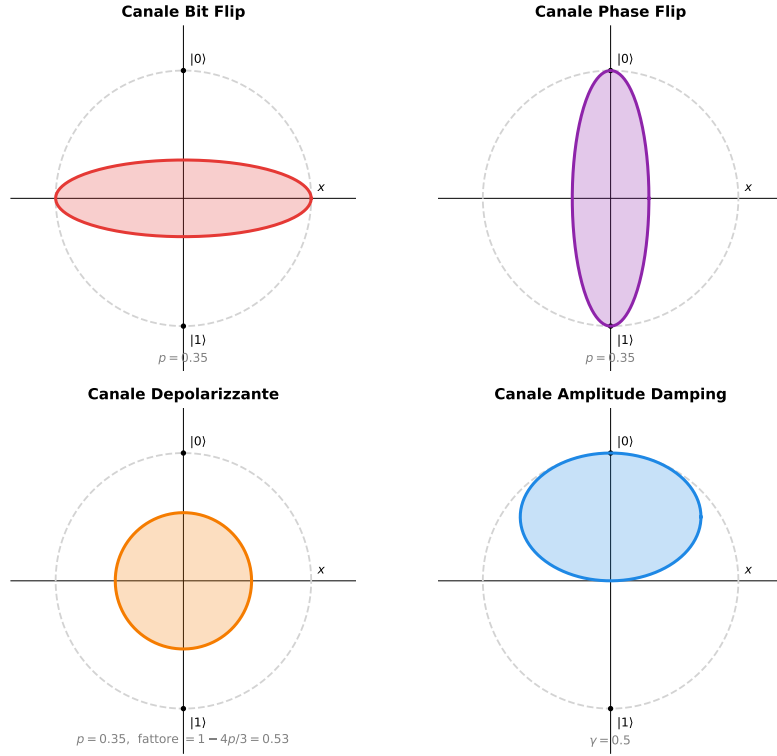


Figura 4.1: Effetto dei quattro canali di rumore sulla sfera di Bloch (sezione nel piano xz). La curva tratteggiata grigia rappresenta la sfera ideale; la superficie colorata è la sfera dopo l'applicazione del canale con il parametro indicato. Il Bit Flip contrae la sfera lungo z ; il Phase Flip lungo x (distrugge le coerenze equatoriali); il canale Depolarizzante contrae isotropicamente verso il centro; l'Amplitude Damping sposta la sfera verso il polo nord $|0\rangle$.

Canale	Parametro	Effetto coerenze	Effetto popolazioni	Modella
Bit Flip	p	Contrae r_y, r_z	Invariate	Errori X (inversione di bit)
Phase Flip	p	Sopprime ρ_{01}	Invariate	Dephasing (T_2)
Depolarizzante	p	Contrae isotropicamente	Verso $I/2$	Errori di gate generici
Amplitude Damping	γ	Sopprime ρ_{01}	Verso $ 0\rangle$	Decadimento T_1

Tabella 4.3: I quattro canali matematici fondamentali del rumore quantistico [6].

4.6 Impatto del Rumore sull'Algoritmo di Shor

L'algoritmo di Shor è particolarmente vulnerabile al rumore a causa della sua elevata profondità circuitale e della dipendenza critica dalla precisione delle fasi. La

Figura 4.2 localizza le quattro fonti fisiche sulle fasi dell’algoritmo: ciascuna colpisce prevalentemente uno stadio diverso della pipeline.

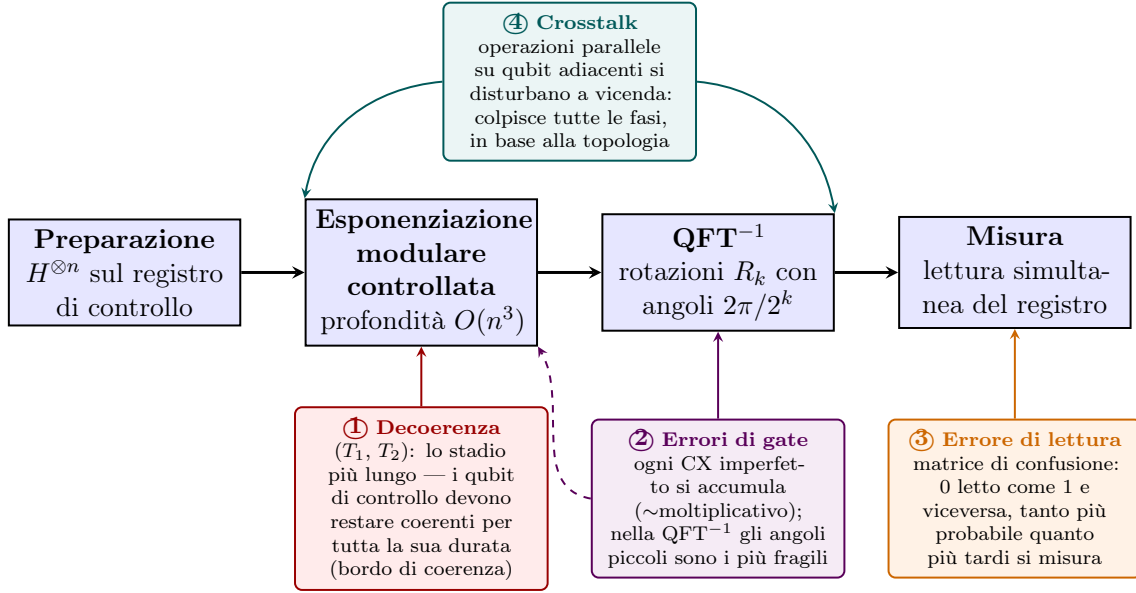


Figura 4.2: Localizzazione delle quattro fonti fisiche di rumore sulla pipeline dell’algoritmo di Shor. La decoerenza ① domina l’esponenziazione modulare (lo stadio più profondo, $O(n^3)$ gate); gli errori di gate ② si accumulano su tutti i CX e sono particolarmente critici sulle rotazioni ad angolo piccolo della QFT^{-1} ; l’errore di lettura ③ colpisce la misura finale, tanto più quanto più il circuito arriva “lungo” al bordo di coerenza; il crosstalk ④ agisce trasversalmente ovunque vi siano operazioni parallele su qubit adiacenti, secondo la topologia del dispositivo (Sezione 4.2).

Le tre componenti più sensibili sono:

4.6.1 Errori nella Quantum Fourier Transform

La QFT richiede porte di rotazione di fase $R_k = \text{diag}(1, e^{2\pi i/2^k})$ con angoli esponenzialmente piccoli all’aumentare di k . Per $k = 10$, l’angolo è $2\pi/1024 \approx 6 \times 10^{-3}$ rad. Errori coerenti di calibrazione possono accumularsi in ampiezza, mentre un canale depolarizzante produce una degradazione stocastica diversa: non è corretto riassumere entrambi come un unico “errore di fase” lineare. La campagna li mantiene quindi come canali separati.

4.6.2 Errori nella Phase Estimation

La phase estimation richiede che i qubit del registro di controllo mantengano la coerenza lungo il cammino critico della modular exponentiation. A livello di ordine

di grandezza si può scrivere:

$$T_2 \gg \tau_{\text{gate}} \cdot O(n^3) \quad (4.9)$$

Questa espressione evidenzia lo scaling, ma non determina una soglia universale in bit: la costante dipende dall'aritmetica e dalla compilazione, e un'architettura fault-tolerant sostituisce il confronto diretto con un budget di errore logico.

4.6.3 Errori nella Modular Exponentiation

La modular exponentiation è la componente con la maggiore profondità. Sotto l'ipotesi puramente illustrativa di eventi indipendenti con probabilità ϵ su L gate, la probabilità che non avvenga alcun evento è:

$$P_{\text{0-eventi}} = (1 - \epsilon)^L, \quad L \sim O(n^3). \quad (4.10)$$

Ponendo soltanto per visualizzare lo scaling $L = n^3$, per $n = 15$ ed $\epsilon = 10^{-3}$ si ottiene $(1 - 10^{-3})^{3375} \approx 0,034$. Questo è un **proxy di nessun evento**, non la probabilità di successo di Shor: alcuni errori possono non cambiare l'esito, mentre rilassamento, readout e struttura del circuito non sono descritti dalla formula. La Figura 4.3 va letta esclusivamente in questo senso.

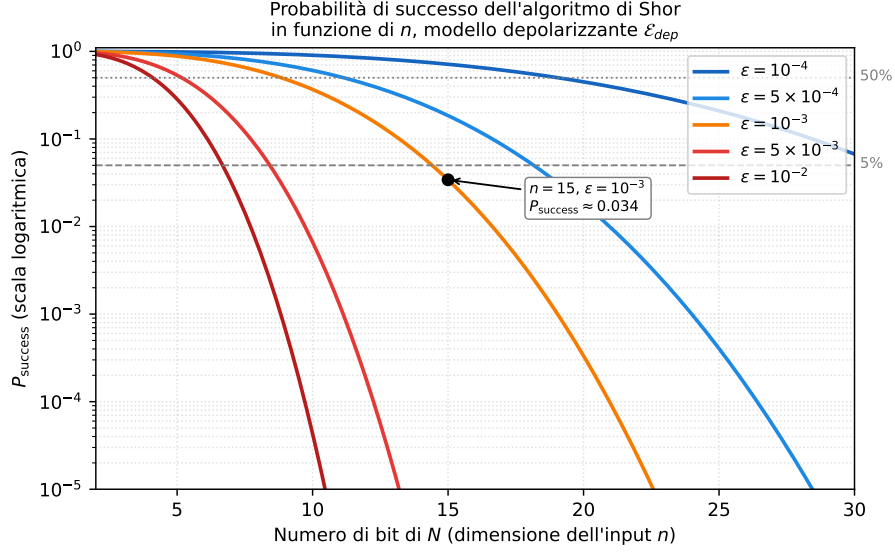


Figura 4.3: Proxy illustrativo $(1 - \epsilon)^{n^3}$ di nessun evento indipendente, per cinque valori di ϵ . Non è una curva di successo di Shor e non deriva dai circuiti sperimentali: mostra soltanto quanto rapidamente un modello moltiplicativo ingenuo accumuli errori al crescere di un conteggio assunto pari a n^3 .

4.7 Sfide Pratiche su Hardware NISQ

- **Requisiti di qubit:** fattorizzare un intero a n bit richiede almeno $2n$ qubit logici per la phase estimation, più ausiliari per la modular exponentiation.
- **Profondità:** il circuito canonico $N = 15$ di questa tesi, compilato nella base astratta rz/sx/x/cx, contiene 224 cx e ha profondità 412; una mappatura hardware aggiungerebbe routing dipendente dalla topologia.
- **Soluzioni parziali:** gli esperimenti su IBM hanno dimostrato la fattorizzazione di $N = 15$ e $N = 21$ solo con circuiti appositamente semplificati, non generalizzabili a N arbitrario [4].

Il problema non è di natura algoritmica ma fisica: l'algoritmo di Shor è teoricamente corretto, ma il substrato hardware introduce errori che ne compromettono l'output. Affrontarlo richiede strategie che operano a diversi livelli, dall'hardware al post-processing classico. Il capitolo seguente tratta separatamente la QEC e i metodi appresi, mettendoli in relazione soltanto nella fase di decodifica della sindrome.

Capitolo 5

Strategie per la Riduzione del Rumore Quantistico

Il capitolo precedente ha mostrato che il rumore quantistico non è un fastidio marginale: circuiti profondi come Shor accumulano errori anche su istanze didattiche. Il problema richiede strategie che agiscano a livelli diversi e che vengano confrontate con metriche esplicite.

In letteratura si distinguono più livelli di intervento: soppressione fisica, mitigazione, correzione/tolleranza ai guasti e metodi appresi. Questi ultimi sono trasversali: possono operare sul post-processing o assistere un decoder, ma non comprendono la **Correzione degli Errori Quantistici (QEC)**, che è una disciplina autonoma.

5.1 Le Quattro Strategie: Panoramica e Scelta

Le quattro macro-strategie per fronteggiare il rumore nei sistemi quantistici sono:

1. **Soppressione del rumore** (*Noise Suppression*): agisce *prima* che gli errori si manifestino, cercando di ridurre fisicamente l'accoppiamento tra il qubit e l'ambiente. Le tecniche principali sono il *Dynamical Decoupling* — sequenze di pulse che mediano a zero l'interazione con l'ambiente — l'ottimizzazione della durata dei gate, e l'isolamento fisico criogenico ed elettromagnetico dell'hardware. Riduce il rumore ma non lo elimina, ed è vincolata alle possibilità dell'hardware disponibile.
2. **Mitigazione degli errori** (*Error Mitigation*): opera *a posteriori*, senza correggere i qubit durante il calcolo. Si eseguono più istanze del circuito

con diversi livelli di rumore o con randomizzazioni, e si combinano i risultati classicamente per stimare l'output ideale. Le tecniche più diffuse sono la *Zero-Noise Extrapolation* (ZNE), il *Probabilistic Error Cancellation* (PEC) e il *Gate Twirling*. Il limite è l'overhead di campionamento, la cui legge dipende dalla tecnica e può diventare esponenziale nell'errore totale del circuito, in particolare per la PEC.

3. **Tolleranza ai guasti** (*Fault-Tolerant Quantum Computing*): è il paradigma architetturale a lungo termine. Codifica ogni qubit *logico* in molti qubit *fisici* e garantisce che, sotto le ipotesi del *threshold theorem* e al di sotto della relativa soglia [18], l'errore logico possa essere ridotto aumentando la distanza e l'overhead. Il costo per qubit logico dipende da codice, qualità fisica, algoritmo e obiettivo di errore e può arrivare a centinaia o migliaia di qubit fisici nelle stime di risorse per algoritmi di grande scala.
4. **Metodi appresi**: usano modelli di ML classico per classificare output, stimare qualità o assistere la decodifica. Non sono *Quantum Machine Learning* (QML) in senso stretto, perché l'addestramento e l'inferenza non usano un modello quantistico; non sostituiscono per definizione né la mitigazione né la QEC.

Strategia	Quando agisce	Overhead	Era	Adottata
Noise Suppression	Prima del calcolo	Basso (hardware)	NISQ / FT	No
Error Mitigation	Dopo il calcolo	Esponenziale	NISQ	No*
Fault-Tolerant QC	Architetturale	Molto alto (qubit)	Futuro	No
ML classico / metodi appresi	Post-processing o decoder	Classico, dipendente dai dati	NISQ / FT	Valutato

Tabella 5.1: Confronto delle quattro strategie anti-rumore per il contesto di questa tesi.

*La ZNE non è adottata come strategia principale, ma viene confrontata *a posteriori* con M_{TOP4} nel Capitolo 7: ZNE-2 e ZNE-3 non riducono significativamente le iterazioni rispetto alla baseline, a fronte di un costo da due a tre volte superiore in shot.

Motivazione per l'approccio appreso. Le prime tre strategie presentano limitazioni nel contesto di questa tesi. La soppressione del rumore è vincolata all'hardware disponibile; la mitigazione richiede esecuzioni aggiuntive; la tolleranza ai guasti richiede molte più risorse fisiche. Un modello appreso può invece essere applicato a valle senza modificare il circuito. Che ciò riduca davvero il numero di esecuzioni è però un'*ipotesi sperimentale*, non un vantaggio assunto: il confronto con TOP-1 e con l'ablazione TOP-4 è necessario proprio per misurarne il contributo marginale.

5.2 Machine Learning Classico per la Robustezza al Rumore

In questa tesi il termine corretto è **machine learning classico applicato a dati quantistici**: Random Forest, SVM e Percettrone Multistrato (*Multi-Layer Perceptron*) (MLP) sono modelli eseguiti interamente su hardware classico. Il QML, in senso stretto, usa invece circuiti o modelli quantistici nell'apprendimento e non viene implementato qui. L'idea sperimentata è lasciare che un modello classico apprenda dai conteggi rumorosi o dalle sindromi, mantenendo esplicita la natura classica della pipeline.

Tre possibili impieghi di metodi appresi nella robustezza quantistica sono:

- **Caratterizzazione del rumore**: un modello di regressione (Random Forest, rete neurale) apprende la relazione tra i parametri hardware (T_1 , T_2 , profondità del circuito) e la fedeltà dell'output, permettendo di predire la qualità del risultato prima di eseguire il circuito.
- **Ottimizzazione variazionale**: i parametri del circuito (decomposizione della modular exponentiation, routing dei qubit) vengono ottimizzati tramite agenti di Reinforcement Learning che massimizzano la probabilità di successo sul simulatore rumoroso, adattando il circuito al profilo di errore specifico del dispositivo.
- **Decodifica appresa per la QEC**: una rete neurale o un classificatore usa le sindromi per proporre una correzione o per intervenire residualmente sulla decisione di un decoder analitico. È un'applicazione del machine learning *dentro una fase* della pipeline QEC, non una ridefinizione della QEC come sottocategoria del machine learning.

5.3 Correzione degli Errori Quantistici e Ruolo del Machine Learning

5.3.1 Il Problema Fondamentale

Come si è visto nel capitolo precedente, il teorema del no-cloning impedisce di proteggere un qubit semplicemente duplicandolo, come si farebbe con un bit classico. Eppure la correzione degli errori quantistici è possibile — e questa fu una delle

scoperte più sorprendenti degli anni Novanta. Il principio è che si può *distribuire* l'informazione di un qubit logico su più qubit fisici in modo tale che, misurando opportune quantità collettive (le **sindromi**), sia possibile rilevare e correggere gli errori senza misurare direttamente — e quindi distruggere — lo stato logico.

5.3.2 Il Paradigma in Tre Fasi

Qualunque codice di correzione degli errori quantistici funziona secondo lo stesso schema:

1. **Codifica:** lo stato logico $|\psi_L\rangle$ viene distribuito su n qubit fisici tramite un circuito di codifica. L'informazione è ridondante: nessun singolo qubit fisico contiene da solo l'informazione logica.
2. **Misura di sindrome:** si misurano operatori stabilizzatori — operatori che commutano con tutti i codeword ma anticommutano con gli operatori di errore. Questi operatori forniscono informazione sull'errore avvenuto senza rivelare il valore del qubit logico. Il risultato della misura è la **sindrome**, un vettore binario che identifica quale errore (o classe di errori) si è verificato.
3. **Decodifica e correzione:** dato il vettore di sindrome, un algoritmo di decodifica determina l'operazione di recupero. Un modello appreso può sostituire o affiancare il decoder analitico, ma un vantaggio non è automatico: va misurato sullo stesso insieme di sindromi e dipende dalla struttura del rumore rappresentabile dai due metodi.

5.3.3 I Principali Codici Quantistici

Nel corso degli anni sono stati sviluppati numerosi codici di correzione. I tre più rilevanti per questa tesi sono:

Codice di Shor (1995). Il primo codice di correzione degli errori quantistici [19], proposto dallo stesso Peter Shor dell'algoritmo di fattorizzazione, codifica 1 qubit logico in 9 qubit fisici ed è in grado di correggere qualsiasi errore singolo (bit flip, phase flip o entrambi). Lo stato logico è:

$$|0_L\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |111\rangle)^{\otimes 3}, \quad |1_L\rangle = \frac{1}{2\sqrt{2}}(|000\rangle - |111\rangle)^{\otimes 3} \quad (5.1)$$

La struttura è a due livelli: tre copie del qubit (ciascuna nella forma $|000\rangle \pm |111\rangle$) proteggono da phase flip tramite interferenza; la ripetizione delle tre copie protegge da bit flip tramite ridondanza. Questo risultato dimostrò per la prima volta che il teorema del no-cloning non impedisce la correzione degli errori quantistici.

Codice di Steane (1996). Il codice di Steane [7] riduce l'overhead a 7 qubit fisici per 1 qubit logico, sfruttando la struttura del codice classico di Hamming [7, 4, 3]. Corregge qualsiasi errore singolo ed è il primo esempio di codice CSS (Calderbank–Shor–Steane) [20], una famiglia sistematica di costruzioni che collegano codici classici lineari a codici quantistici.

Surface Code. Il **surface code** [8] organizza i qubit fisici in una griglia 2D con interazioni locali, una struttura compatibile con varie architetture a qubit superconduttori. La sua proprietà chiave è il **threshold**: sotto una soglia che dipende dal circuito, dal modello di rumore e dal decoder, aumentare la distanza d sopprime l'errore logico; sopra soglia lo amplifica. La campagna di questa tesi stima tale comportamento nel proprio modello invece di importare una soglia universale.

5.3.4 Machine Learning come Possibile Componente del Decoder QEC

La QEC e il machine learning sono discipline distinte. Il loro punto di contatto è la decodifica: un modello appreso può ricavare dai campioni di sindrome una decisione di recupero o correggere residualmente un decoder analitico [21, 10]. La sindrome è un punto strutturalmente favorevole perché conserva informazione sull'errore; resta comunque necessario confrontare il modello appreso con un decoder analitico adeguato, a parità di dati e di calendario sperimentale.

Questa affermazione non resta, in questo lavoro, un richiamo alla letteratura: è verificata sperimentalmente nella Sezione 9.6, che confronta un decoder appreso con il decoder analitico standard sugli stessi campioni di sindrome. L'esito va formulato con precisione: *sostituire* il decoder analitico con una rete non conviene nel protocollo studiato, mentre *affiancarlo* produce un guadagno misurabile nel modello simulato con correlazioni assenti dal grafo del matching. La QEC non diventa per questo una specializzazione del machine learning: il modello appreso è soltanto una possibile componente classica del decoder, la cui utilità va confrontata con riferimenti analitici adeguati.

5.4 Motivazione Pratica: il Contributo di questa Tesi

Chiarito il quadro teorico, è importante spiegare concretamente cosa porta il modello appreso rispetto all'alternativa più semplice.

Senza ML — approccio statistico classico. Ogni iterazione sperimentale esegue il circuito per un numero fissato di shot, costruisce un istogramma e applica il post-processing ai candidati ordinati per frequenza. TOP-1 prova la sola moda; TOP-4 prova i primi quattro esiti. Il numero di *iterazioni* della campagna non va confuso con gli shot contenuti in ciascun istogramma: nei risultati correnti la media è compresa fra 1 e 1.5 iterazioni, non decine o centinaia.

Con ML — l'ipotesi che questa tesi mette alla prova. Il classificatore riceve le feature di un istogramma già campionato e decide se attivare la ricerca TOP-4; non classifica un singolo shot e non elimina il costo necessario a costruire la distribuzione. L'ipotesi preregistrata era che questo filtro riducesse il numero di iterazioni rispetto a TOP-1. L'ablazione separata TOP-4 verifica se l'eventuale beneficio provenga dal modello o dalla sola ricerca multi-candidato.

La prima fase sperimentale verifica questa ipotesi con un confronto appaiato. La riduzione delle iterazioni si ottiene davvero (da $\bar{M}_1 = 1.37$ a $\bar{M} = 1.00$ nello scenario moderato), ma l'ablazione attribuisce il beneficio alla ricerca multi-candidato TOP-K: aggiungere il classificatore aumenta il costo rispetto a TOP-4 in entrambi gli scenari. Il risultato negativo delimita il solo filtro ML applicato a questo output e a queste etichette; non è una conclusione generale sull'apprendimento per la mitigazione o la decodifica.

È questo esito a determinare la struttura del resto dell'elaborato. Il Capitolo 6 descrive gli strumenti, l'architettura e l'implementazione del sistema sperimentale, e il Capitolo 7 riporta la verifica con la relativa ablazione. L'esperimento mostra che l'istogramma contiene candidati utili, sfruttati da TOP-4, ma non che un classificatore su quelle etichette aggiunga informazione alla regola semplice. Per intervenire sugli errori che il post-processing non può recuperare, il **nucleo del lavoro** si sposta durante il calcolo e occupa i tre capitoli successivi: la costruzione e la caratterizzazione dei codici correttori, dal codice a ripetizione al codice di Steane (Capitolo 8); il surface code, la stima sperimentale dell'errore logico e il confronto fra decodifica analitica e appresa (Capitolo 9); infine, una sensibilità fenomenologica di Shor mantenuta esplicitamente distinta dai tassi QEC per ciclo (Capitolo 10). In questa seconda parte i metodi appresi e la QEC vengono confrontati entro i rispettivi contratti, senza identificarli come un'unica disciplina.

Capitolo 6

Metodo e Implementazione

Il capitolo precedente ha concluso la parte teorica della tesi con un'ipotesi precisa da mettere alla prova: che un classificatore di ML classico addestrato per la rilevazione degli errori, riduca in misura significativa il numero medio di esecuzioni necessarie ($\rho = \bar{M}_1/\bar{M}_2 \gg 1$), portandolo a circa tre. Questo capitolo definisce il sistema sperimentale costruito per verificarla, e risponde in sequenza a tre domande: con quali strumenti si lavora, con quale disegno, e come quel disegno è stato realizzato in codice.

Si tratta dell'architettura della *prima fase* del lavoro: l'esito della verifica — che il guadagno osservato è reale ma attribuibile alla ricerca multi-candidato e non al classificatore (Capitolo 7) — motiva la seconda fase, che adotta un'architettura diversa perché diversa ne è la natura. Il protocollo sperimentale di ciascun codice correttore, essendo specifico del codice, è descritto nel capitolo che lo introduce (Capitoli 8–10); l'infrastruttura comune — organizzazione del codice, divisione degli strumenti fra Qiskit, Stim e PyMatching, e disciplina di riproducibilità — è nella Sezione 6.13.

6.1 Strumenti e Ambiente di Simulazione

Questa sezione riassume gli strumenti software e l'ambiente usati nella parte sperimentale. La motivazione estesa di ciascuna scelta — il confronto tra i framework, i metodi di simulazione di Aer, le specifiche GPU e la configurazione WSL — è riportata nella Sezione B.1.

Framework: Qiskit + Qiskit Aer. L'intera pipeline usa **Qiskit** [22], l'SDK di IBM Quantum, e il simulatore **Qiskit Aer**. La scelta è motivata dal supporto

esplicito ai canali depolarizzanti, termici e di readout, dalla compilazione controllabile in un gate set dichiarato e dall'allineamento con l'implementazione di riferimento dell'algoritmo di Shor [4, 23]. Il noise model della prima fase è uniforme e illustrativo: non è una calibrazione di hardware IBM.

Metodo di simulazione e limite di scala. La campagna rumorosa principale usa `AerSimulator(method='statevector')` sul circuito didattico a 12 qubit per $N = 15$. Il costo dello stato cresce come 2^n e quello delle traiettorie rumorose cresce anche con gli shot; le istanze $N = 21/35$ sono quindi riservate ai controlli ideali di correttezza e costo circuitale. La simulazione di uno Shor interamente codificato resta impraticabile e impone l'uso di Stim per i soli circuiti *stabilizer* della QEC (Capitoli 9 e 10).

Strumenti della parte QEC. La stima del *logical error rate* del surface code richiede milioni di esecuzioni di circuiti stabilizer, fuori portata per un simulatore statevector. Si adottano quindi due strumenti specializzati: **Stim** [24] (versione 1.16.0), che campiona *detection events* di circuiti Clifford sfruttando il teorema di Gottesman–Knill, e **PyMatching** [25] (versione 2.4.0), che implementa la decodifica MWPM a partire dal modello di errore del circuito. La divisione dei compiti — Qiskit Aer per i circuiti generali di piccola taglia, Stim e PyMatching per la QEC stabilizer — è discussa nel Capitolo 10.

Modello di rumore. Il modulo `qiskit_aer.noise` compone errore **depolarizzante**, **rilassamento termico** T_1/T_2 e **readout error** (matrice di confusione). I due preset sono uniformi e illustrativi, non calibrati su una macchina nominata. Il circuito è compilato in `rz/sx/x/cx` con `optimization_level=2`: le `rz` sono virtuali, mentre i canali fisici agiscono su `sx/x` e `cx` con durate dichiarate.

Machine learning e ambiente. Il classificatore del Metodo 2 è addestrato con **scikit-learn** [26], scelto per l'interfaccia unificata `fit/predict` che consente di confrontare più modelli sullo stesso dataset. Le simulazioni canoniche girano in **WSL2**, distro Ubuntu 24.04, Python 3.12, su CPU; interprete e dipendenze sono fissati nel manifest e nel file `requirements.txt`. L'accelerazione Unità di Elaborazione Grafica (*Graphics Processing Unit*) (GPU) prevista dal piano iniziale non viene usata.

Stabiliti gli strumenti, questo capitolo definisce il disegno sperimentale della prima fase. Il contratto separa la correttezza ideale dell'algoritmo, il modello illustrativo di rumore e le regole del confronto statistico. Gli esiti numerici non fanno parte della metodologia: le tabelle con $\bar{M}$, ρ e p-value nel Capitolo 7 sono estratte esclusivamente dagli artefatti schema 2 prodotti con il circuito N15 corretto.

6.2 Obiettivi della Parte Sperimentale

Il primo obiettivo è costruire una **baseline riproducibile**. Il Metodo 1 esamina la misura più frequente dell'istogramma (TOP-1), applica frazioni continue e Massimo Comune Divisore (*Greatest Common Divisor*) (GCD) e ripete il campionamento fino al successo o al budget massimo. Il secondo obiettivo è confrontare, sullo stesso dato, due estensioni: TOP-4 senza apprendimento e il Metodo 2, nel quale un classificatore decide se attivare la ricerca TOP-4. Questa terna permette di distinguere il contributo della ricerca multi-candidato da quello del modello appreso.

La campagna rumorosa è limitata a $N = 15$, $a = 7$, con due preset uniformi illustrativi: riferimento e stress. I moltiplicatori di Beauregard per $N = 21$ e $N = 35$ sono invece validati idealmente e sottoposti ad audit di risorse, ma esclusi dalle simulazioni rumorose complete. La separazione evita di trasformare un limite computazionale del simulatore in un risultato sulla probabilità di successo dell'algoritmo.

6.3 Architettura del Pipeline Sperimentale

Il sistema è strutturato come una pipeline a tre stadi, illustrata in Figura 6.1.

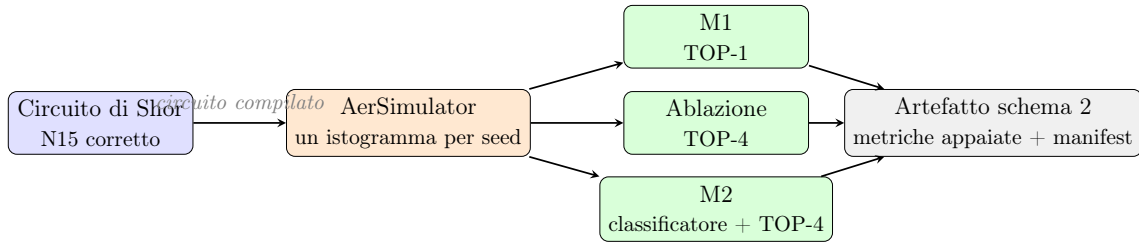


Figura 6.1: Pipeline versione 2. Per ogni coppia (replica, iterazione) Aer genera un solo istogramma, poi riusato dalle tre strategie. Il confronto non introduce quindi differenti realizzazioni Monte Carlo.

Stadio 1 — circuito. Per $N = 15$ il moltiplicatore controllato implementa $y \mapsto 7y \bmod 15$ ripetendo l'operazione elementare esattamente **power** volte. Il circuito viene compilato con Qiskit 2.5, `optimization_level=2`, seed del transpiler fissato e base nativa esplicita `rz/sx/x/cx`. Il manifest della configurazione di riferimento registra profondità 412, 224 porte `cx`, conteggi completi delle istruzioni e hash SHA-256 della netlist. La simulazione ideale valida il periodo e l'estrazione dei fattori in termini probabilistici; non si assume che ogni singolo shot produca necessariamente un fattore.

Stadio 2 — rumore. Il circuito compilato viene eseguito con un noise model uniforme e controllabile. I due preset servono a studiare il comportamento del metodo, non a emulare o calibrare una specifica QPU. Ogni artefatto conserva sia i parametri sia il manifest del circuito, così risultati ottenuti con revisioni differenti non possono essere combinati inavvertitamente.

Stadio 3 — analisi. M1, TOP-4 e M2 ricevono il medesimo istogramma. Un ordinamento deterministico risolve i pareggi di frequenza mediante una priorità SHA-256 costruita da seed e bitstring: l'ordine non dipende dall'inserimento nel dizionario e non favorisce sistematicamente bitstring alte o basse.

6.4 Tecniche di Mitigazione del Rumore: Analisi e Scelte Implementative

6.4.1 Transpilazione e Riduzione della Profondità

Il gate set è fissato a **rz/sx/x/cx** prima dell'esecuzione. Questa scelta rende esplicite le sole istruzioni a cui il noise model può essere associato ed evita che porte composite sopravvivano fuori dal contratto. Per $N = 15$ la compilazione globale di riferimento produce 224 **cx** e profondità 412; questi valori sono proprietà della coppia (circuito, versione e opzioni del transpiler), perciò vengono verificati e registrati anziché assunti dal sorgente astratto.

Per $N = 21$ e $N = 35$ si usa la costruzione aritmetica reversibile di Beauregard. I test ideali coprono le truth table sui sottospazi validi, la pulizia di registro ausiliario e ancilla e il comportamento QPE. La compilazione completa $N = 21$, $a = 2$, $n_{\text{count}} = 8$, livello 2, conta 21 036 **cx** e ha profondità 23 081. Il proxy registrato a $\lambda = 10^{-3}$ è 7.2379×10^{-10} ; è un indicatore circuitale di accumulo, non una stima di $P(\text{successo})$. Il costo giustifica l'esclusione di N21/N35 dalle campagne rumorose, pur lasciandone valida la verifica ideale.

6.4.2 Modello di Rumore Uniforme

Le rotazioni **rz** sono trattate come cambi di frame virtuali, con durata ed errore nulli. I gate **sx** e **x** hanno durata 50 ns; **cx** ha durata 300 ns. Su **sx/x** si compongono il canale depolarizzante a un qubit e il rilassamento termico della durata 1Q; su **cx** si compongono i corrispondenti canali a due qubit e il rilassamento per 300 ns. Il

readout è modellato da una matrice di confusione simmetrica. Non vengono associati errori a `rz`, né a istruzioni fuori dalla base.

Nel costruttore Aer, il parametro passato a `depolarizing_error` è λ nella mappa

$$\mathcal{E}(\rho) = (1 - \lambda)\rho + \lambda \frac{I}{2^n}. \quad (6.1)$$

Per un gate a due qubit, la probabilità aggregata della componente di Pauli non identità nella rappresentazione equivalente è $15\lambda/16$. Di conseguenza, per i 224 `cx` del circuito N15 si usa esclusivamente il proxy

$$P_{\text{no-Pauli-2Q}} = \left(1 - \frac{15\lambda}{16}\right)^{224}. \quad (6.2)$$

Questa quantità non comprende rilassamento, errori 1Q o readout e, soprattutto, non è la probabilità di successo dell'algoritmo di Shor.

6.4.3 Livello di Misura: Mitigazione degli Errori di Readout

Il contratto base non applica un'inversione post-hoc della matrice di assegnazione. Il parametro p_{ro} viene invece variato nella campagna parametrica, mantenendo il resto del preset costante. Le conclusioni quantitative sull'impatto del readout provengono dall'artefatto schema 2 corrente; le campagne antecedenti alla correzione del circuito non vengono riutilizzate.

6.4.4 Zero-Noise Extrapolation

La ZNE resta un termine di confronto metodologico. I fattori di scala moltiplicano i parametri del noise model, mentre il costo in shot e le trasformazioni dell'istogramma vengono registrati separatamente. Il suo effetto su $\bar{M}$ e la significatività del confronto sono valutati sotto lo stesso circuito e lo stesso schema statistico della baseline; l'esito è riportato nell'Appendice D.1.

6.4.5 Analisi di Sensibilità

La campagna parametrica adotta una variazione singola rispetto al preset di riferimento e comprende K , λ_{2q} , shots, T_1/T_2 , λ_{1q} e p_{ro} , oltre alle griglie congiunte dichiarate nel manifest. La base circuitale resta invariata. Le tabelle correnti provengono dalla

rigenerazione schema 2; gli artefatti storici precedenti alla correzione di `c_amod15` non sostengono conclusioni scientifiche.

6.4.6 Scelta Adottata: Classificatore come Filtro

Il classificatore riceve il vettore normalizzato delle $2^{n_{\text{count}}}$ frequenze. La generazione del dataset calcola nello stesso passaggio sia l’etichetta TOP-1 sia l’etichetta TOP-16, così l’effetto della definizione di target è auditabile senza cambiare i campioni. Il modello ammesso nella pipeline M2 è quello con `label_top_k=1` e manifest compatibile; TOP-16 resta un’analisi di sensibilità dell’etichetta. Dopo l’accettazione del classificatore, M2 tenta i primi quattro candidati dello stesso ordinamento usato dall’ablazione TOP-4.

6.5 Il Metodo 1: Baseline TOP-1

Per ogni iterazione il Metodo 1:

1. riceve l’istogramma comune della replica;
2. seleziona il primo candidato nell’ordinamento frequenza–SHA-256;
3. stima il periodo con frazioni continue e verifica i fattori non banali;
4. in caso di fallimento procede all’istogramma successivo, fino a M_{max} .

Il seed Aer è separato fra iterazioni di più del numero di shot, secondo la formula dichiarata nell’implementazione. Questo evita la sovrapposizione quasi completa dei flussi pseudo-casuali prodotta da semi consecutivi.

6.6 Il Metodo 2 e il Disegno Appaiato

TOP-4 prova, in ordine, fino a quattro candidati dello stesso istogramma. M2 applica prima il classificatore e, soltanto se il gate è positivo, esegue la medesima ricerca TOP-4. Per ciascuna replica si registra la prima iterazione di successo di ogni strategia. Un mancato successo entro M_{max} è codificato con la sentinella $M_{\text{max}} + 1$, mantenendolo distinto da un successo all’ultima iterazione.

Il calendario di seed e gli istogrammi sono comuni a tutte le strategie. I confronti $M1 > \text{TOP-4}$ e $M1 > M2$ usano il Wilcoxon appaiato unilaterale con gestione degli

zeri secondo Pratt; il confronto TOP-4–M2 è bilaterale. Medie, deviazioni e tassi di successo sono statistiche descrittive; il vettore completo con le sentinelle è usato per l’inferenza. L’output JSON schema 2 include input, seed, vettori appaiati, test, versione del software e manifest/hash della compilazione.

Il Capitolo 2 ha formalizzato il contratto sperimentale versione 2: due preset rumorosi per N15, validazioni ideali separate per N21/N35 e confronto appaiato. Le sezioni seguenti descrivono il setup, l’integrazione di Shor con il simulatore, i metodi di analisi della prima fase e, nella Sezione 6.13, i codici correttori della seconda. Gli esiti quantitativi sono demandati agli artefatti schema 2 e ai capitoli dei risultati.

6.7 Setup dell’Ambiente di Esecuzione

L’ambiente Python è isolato in un virtual environment su *Windows Subsystem for Linux* (WSL); versioni di interprete, Qiskit, Aer, NumPy, SciPy e scikit-learn vengono registrate nel manifest di ogni artefatto. La campagna di riferimento usa Qiskit 2.5 e simulazione CPU. Il Listato B.1 riporta le dipendenze necessarie.

6.7.1 Accelerazione GPU: limitazione dell’ambiente

L’accelerazione con `qiskit-aer-gpu` non è parte del contratto riproducibile: nell’ambiente WSL di sviluppo il backend ha prodotto l’errore riportato nel Listato B.2. Le campagne vengono pertanto orchestrate su CPU e il backend effettivo è registrato nei metadati.

6.8 L’Algoritmo di Shor: Implementazione di Riferimento

L’implementazione costruisce la QPE con un registro di conteggio, un registro di lavoro e la QFT inversa. Il post-processing stima il periodo tramite frazioni continue e verifica i fattori non banali con il GCD (Listato B.5).

6.8.1 La Quantum Fourier Transform

La QFT inversa viene espressa mediante Hadamard, rotazioni di fase controllate e scambi (Listato B.3); la successiva compilazione riduce tutte le operazioni alla base nativa dichiarata.

6.8.2 La Moltiplicazione Modulare Controllata

Il blocco di *modular exponentiation* implementa l'azione controllata

$$U_a^x |y\rangle = |a^x y \bmod N\rangle.$$

Per $N = 15$, $a = 7$, la rete textbook esegue realmente l'operazione elementare $\times 7 \bmod 15$ `power` volte (Listato B.4). Questo dettaglio corregge la precedente selezione mediante `power % 4`, che non rappresentava U^{power} . Per $N = 21$ e $N = 35$ si usa l'aritmetica reversibile di Beauregard, validata idealmente mediante truth table, pulizia degli ausiliari e prove QPE.

6.8.3 Compilazione del Circuito

La compilazione è centralizzata in una funzione comune a training, baseline e sweep. Il contratto fissa il livello di ottimizzazione a 2, il seed del transpiler e la base nativa `rz/sx/x/cx`. Nel manifest Qiskit 2.5 il circuito $N = 15$, $a = 7$, $n_{\text{count}} = 8$ ha profondità 412 e contiene 224 porte `cx`; dimensione, conteggi completi e SHA-256 della netlist sono salvati insieme ai risultati.

Per $N = 21$, $a = 2$, $n_{\text{count}} = 8$, la compilazione globale Beauregard a livello 2 produce 21 036 `cx` e profondità 23 081. Il proxy circuitale registrato a $\lambda = 10^{-3}$ è 7.2379×10^{-10} : documenta il costo della netlist, non la probabilità di successo dell'algoritmo. N21 e N35 sono quindi esclusi dalle campagne rumorose complete, ma non dalla validazione ideale.

6.9 Configurazione del Noise Model

Il noise model è costruito con `qiskit_aer.noise` come modello **uniforme illustrativo**, non come calibrazione di una QPU reale (Listato B.6). Le rotazioni `rz` sono cambi di frame virtuali, con durata ed errore nulli; `sx/x` durano 50 ns e `cx` 300 ns. Depolarizzazione e rilassamento termico sono composti soltanto su `sx/x` e `cx`; il readout è una matrice di confusione simmetrica separata. I preset di riferimento e stress definiscono punti controllati nello spazio dei parametri e non rappresentano dati di calibrazione hardware.

I simboli ϵ_{1q} e ϵ_{2q} usati dagli script sono il parametro Aer λ di `depolarizing_error`. Per un canale a due qubit, la probabilità aggregata della componente equivalente di Pauli non identità è $15\lambda/16$. Per i 224 `cx` del circuito N15 il proxy senza Pauli 2Q

non identità è pertanto

$$\left(1 - \frac{15\lambda}{16}\right)^{224}. \quad (6.3)$$

Non è $P(\text{successo})$ e non comprende rilassamento, errori 1Q o readout.

6.10 Implementazione del Metodo 1

Il Metodo 1 adotta TOP-1: per ogni istogramma seleziona la prima misura nell'ordinamento per frequenza, tenta il post-processing e procede all'iterazione successiva in caso di fallimento. I pareggi sono risolti da una priorità SHA-256 calcolata a partire da seed e bitstring, così il risultato non dipende dall'ordine del dizionario o dal valore numerico del candidato (Listato B.7).

6.11 Implementazione del Metodo 2

Il Metodo 2 comprende la generazione del dataset e l'addestramento offline, seguiti dal filtro del classificatore e dalla ricerca TOP-4 online.

6.11.1 Dataset ed Etichette

La stessa netlist compilata viene riusata per tutti i campioni e cambia soltanto il noise model. Da ciascun istogramma vengono calcolate nello stesso passaggio le etichette TOP-1 e TOP-16: in questo modo l'effetto della definizione del target è auditabile senza introdurre dataset diversi (Listato B.8). La pipeline M2 accetta soltanto un modello schema 2 con `label_top_k=1` e manifest compatibile; TOP-16 resta un controllo di sensibilità.

6.11.2 Addestramento e Compatibilità

L'addestramento confronta Random Forest, SVM con kernel RBF e MLP con split stratificato e seed registrato. Le metriche e la selezione finale provengono dalla rigenerazione sul circuito corretto; i valori della campagna precedente non vengono riutilizzati. Il file del modello include schema, revisione e hash del circuito, preset, seed e definizione dell'etichetta. L'inferenza rifiuta un classificatore incompatibile. L'audit delle etichette è descritto nella Sezione D.2 dell'Appendice D.

6.11.3 Inferenza

Il classificatore agisce da gate di qualità. Se accetta l'istogramma, M2 tenta fino a quattro candidati nello stesso ordinamento usato da TOP-4. M1, TOP-4 e M2 ricevono quindi gli stessi conteggi, lo stesso seed di tie-break e lo stesso post-processing (Listato B.9).

6.12 Infrastruttura di Test e Raccolta Dati

Le campagne N15 usano $K = 30$ repliche e un'output directory esplicita. Per ogni iterazione,

$$\text{seed_simulator} = \text{rep} \cdot 1\,000\,000 + \text{iteration} \cdot 10\,000, \quad (6.4)$$

una distanza superiore al numero di shot che evita la sovrapposizione quasi completa degli stream Aer prodotta da semi consecutivi. Una sola simulazione genera l'istogramma comune alle tre strategie.

La prima iterazione di successo viene registrata separatamente per M1, TOP-4 e M2. Un fallimento entro `max_iter` è codificato, per il test inferenziale, con la sentinella `max_iter + 1`. I confronti usano il Wilcoxon appaiato con `zero_method='pratt'`: unilaterale per $M1 > \text{TOP-4}$ e $M1 > M2$, bilaterale per $\text{TOP-4} - M2$. L'output JSON schema 2 include input completi, vettori appaiati, seed, test, versioni software, hash e manifest della compilazione (Listato B.10).

6.13 Implementazione della Correzione d'Errore

La seconda fase richiede strumenti differenti: non più un singolo circuito profondo di cui interessa l'istogramma finale, ma circuiti di memoria di cui interessa la *sindrome*.

6.13.1 Organizzazione del codice e divisione degli strumenti

Cartella	Oggetto	Strumento	Costo dominante
M5	codice a ripetizione	Qiskit Aer	trascurabile
M6	Steane $[[7, 1, 3]]$	Qiskit Aer	Monte Carlo Pauli
M7	surface code	Stim + PyMatching	decodifica
M8	proxy per gate su Shor	Qiskit Aer	matrix product state
M10	decodifica appresa	Stim + scikit-learn	addestramento

Tabella 6.1: Struttura del codice sperimentale della seconda fase. I circuiti stabilizer vengono trattati con Stim dove la scala lo richiede; lo Shor logico resta su Qiskit Aer.

6.13.2 Il codice a ripetizione: sindrome senza collasso

Gli stabilizzatori sono misurati indirettamente, copiandone la parità su qubit ancillari invece di misurare i dati. Porte identità poste nel punto di iniezione del rumore costituiscono l'aggancio a cui Aer associa il canale, senza alterare lo stato ideale (Listato B.11).

6.13.3 Il codice di Steane

Lo stato $|0_L\rangle$ è preparato individuando tre qubit “seme”, ponendoli in sovrapposizione e propagandone il supporto con CNOT (Listato B.12). La stima di p_L usa il formalismo di Pauli e una lookup table a peso minimo; il limite dichiarato è che l'estrazione di sindrome così modellata è ideale e non costituisce un protocollo fault-tolerant.

6.13.4 Il surface code e l'iniezione del crosstalk

Stim genera il circuito e PyMatching esegue la decodifica. Per lo studio del rumore correlato, il canale fra dati adiacenti viene inserito ricorsivamente anche nei blocchi REPEAT; i qubit di misura sono identificati dal primo MR, non dalla misura finale che include anche i dati (Listato B.13).

6.13.5 Lo Shor logico e il decoder appreso

Nel modello a livelli, p_L è definito come probabilità totale del Pauli non identità dopo ogni gate compilato modellato. Prima di chiamare Aer viene convertito nel parametro depolarizzante: $\lambda_1 = 4p_L/3$ per un gate 1Q e $\lambda_2 = 16p_L/15$ per un gate

2Q. Le **rz** restano virtuali, mentre il canale viene associato a **sx/x** e **cx**. Questo sweep è un modello illustrativo per gate e non identifica direttamente il logical error rate per round o memoria dei codici M6/M7 senza un mapping separato. Il decoder appreso e MWPM ricevono gli stessi *detection events*.

6.13.6 Riproducibilità

Ogni script propaga il seed alle sorgenti di casualità e lo registra nel JSON insieme a campioni, parametri e versioni. Le figure vengono generate leggendo gli artefatti, senza trascrivere manualmente i valori.

Capitolo 7

Verifica Sperimentale dell’Ipotesi Iniziale: dal Classificatore ML alla Strategia TOP-K

Il Capitolo 6 ha descritto M1 (TOP-1), M2 (classificatore TOP-1 seguito da TOP-4) e l’ablazione M_{TOP4} . Questo capitolo presenta la campagna rigenerata dopo la correzione dell’operatore $\times 7 \bmod 15$ e dei contratti statistici. Il risultato principale è che la ricerca multi-candidato riduce le iterazioni, mentre il classificatore a valle non aggiunge beneficio rispetto alla stessa ricerca priva di filtro.

7.1 Setup Sperimentale

Il percorso rumoroso è limitato a $N = 15$, $a = 7$ e otto qubit di controllo. Il circuito è compilato nella base `rz/sx/x/cx` con `optimization_level=2` e seed del transpiler 20260819. I gate RZ sono virtuali; SX/X hanno durata illustrativa di 50 ns e CX di 300 ns. Il modello uniforme combina depolarizzazione a uno e due qubit, rilassamento/dephasing e readout; non riproduce una calibrazione o la topologia di una QPU specifica.

Ogni iterazione usa 1 024 shot, il budget massimo è 50 iterazioni e ogni confronto principale comprende 30 repliche. Le strategie della stessa replica ricevono il medesimo istogramma e il tie-break fra esiti equiprobabili è deterministico mediante SHA-256. Un fallimento al budget è codificato con la sentinella 51; $\bar{M}$ è calcolata sui successi e il budget medio include la censura. Le differenze appaiate sono valutate con Wilcoxon–Pratt e correzione di Holm entro ciascuna famiglia.

Una campagna storica usava semi consecutivi che condividevano quasi tutti gli shot derivati da Aer. Quei valori non sono riutilizzati. La campagna schema 2 separa replica e iterazione di più del budget di shot e registra nel JSON seed, manifest, revisione del rumore e revisione del post-processing.

Tabella 7.1: Invarianti del circuito compilato impiegato da training, baseline, sweep e ZNE.

Grandezza	Valore
Qubit totali	12
Profondità / size	412/604
RZ / SX / CX	302/70/224
Misure	8
SHA-256 circuito	c7f4cf3bc1735dce9d5b56d8e09686e...
Qiskit / Aer	2.5.0/0.17.2

7.2 Perimetro Ideale e Rumoroso

L'aritmetica Beauregard per $N = 21$ e $N = 35$ è stata verificata separatamente nel caso ideale mediante truth table esaustive dei moltiplicatori controllati, pulizia delle ancille e confronto della legge QPE con quella teorica. La distanza di variazione totale è 1.38×10^{-9} per $N = 21$, $a = 2$, $r = 6$ e 2.50×10^{-13} per $N = 35$, $a = 6$, $r = 2$.

Questa validazione non rende però praticabile la stessa campagna rumorosa. Per $N = 21$, otto controlli e compilazione congiunta nella base esecutiva, il circuito ha 21 036 CX e profondità 23 081. A titolo puramente diagnostico, con il canale depolarizzante Aer a $\lambda_{2q} = 10^{-3}$, la probabilità che nessuna delle CX applichi un Pauli non-identità è

$$\left(1 - \frac{15}{16}\lambda_{2q}\right)^{21036} = 2.70 \times 10^{-9}.$$

Questa quantità non è una fedeltà né una probabilità di fattorizzazione. Le istanze $N = 21/35$ restano quindi nella verifica ideale e non vengono mescolate ai risultati rumorosi $N=15$.

Tabella 7.2: Separazione fra validazione ideale e campagna rumorosa.

Istanza	Ideale	Rumorosa
$N = 15, a = 7$	truth table e QPE	campagna completa
$N = 21, a = 2$	truth table e QPE	esclusa per costo
$N = 35, a = 6$	truth table e QPE	esclusa per costo

7.3 La Baseline: Metodo 1 (TOP-1)

La Figura 7.1 mostra, fra le 30 repliche nominali UC1, gli istogrammi con massa massima e minima sui quattro picchi teorici $\{0, 64, 128, 192\}$. La selezione è dichiarata nel file di provenienza: replica 14 (0.628) e replica 16 (0.570). Entrambi conservano la struttura periodica; non sono presentati come esempi arbitrari di “successo” e “fallimento”.

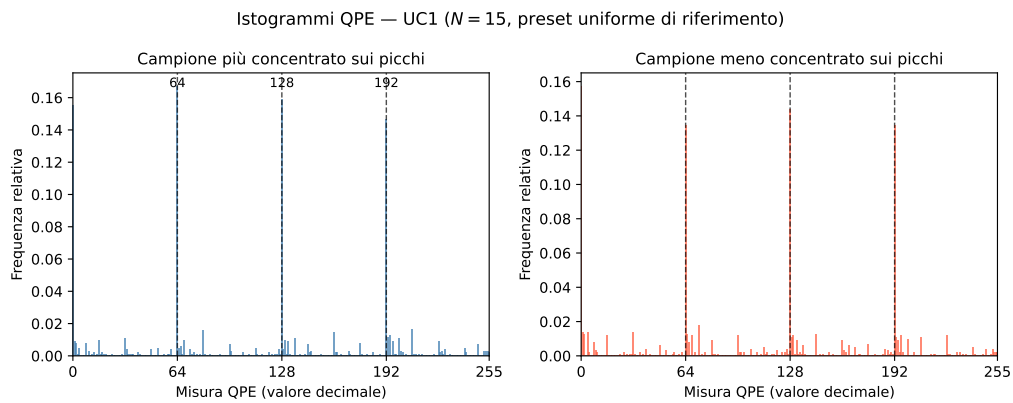


Figura 7.1: Istogrammi nominali UC1 con massima e minima massa sui quattro picchi teorici fra un insieme prefissato di 30 repliche. La selezione e i seed sono salvati nell’artefatto di provenienza schema 2.

La diagnostica indipendente di 60 repliche trova la moda $y = 0$ in 30/60 casi e una moda utile negli altri 30/60, mentre TOP-4 trova sempre almeno un candidato utile. L’instabilità di TOP-1 deriva quindi da quale dei quattro picchi risulta dominante, non dalla completa scomparsa del segnale.

7.4 L’Ipotesi ML e il Verdetto dell’Ablazione

I classificatori TOP-1 sono stati rigenerati su 2 000 istogrammi per scenario; TOP-16 ha prodotto 2000/2000 positivi in entrambi gli scenari e viene registrato come

audit a classe unica. Le metriche e il protocollo train/validation/test sono riportati nella Sezione D.2.1. La Tabella 7.3 riporta invece la misura operativa, calcolata sui medesimi istogrammi per tutte le strategie.

Tabella 7.3: Confronto appaiato M1, TOP-4 e M2 (30 repliche per scenario)

Scenario	Strategia	Successo	$\bar{M}$	ρ vs M1	p_{Holm} vs M1
UC1	M1	30/30	1.37	—	—
UC1	TOP-4	30/30	1.00	1.367	0.0024
UC1	M2	30/30	1.50	0.911	0.7314
UC2	M1	30/30	1.50	—	—
UC2	TOP-4	30/30	1.00	1.500	0.0015
UC2	M2	30/30	1.37	1.098	0.0874

TOP-4 converge al primo tentativo in tutte le repliche. M2 non migliora significativamente M1 e, rispetto alla propria ablazione, richiede più iterazioni: Wilcoxon bilaterale con correzione di Holm $p = 0.0033$ (UC1) e $p = 0.0096$ (UC2). Il filtro scar- ta dunque istogrammi che la ricerca TOP-4 avrebbe risolto; il guadagno appartiene alla ricerca multi-candidato, non al classificatore.

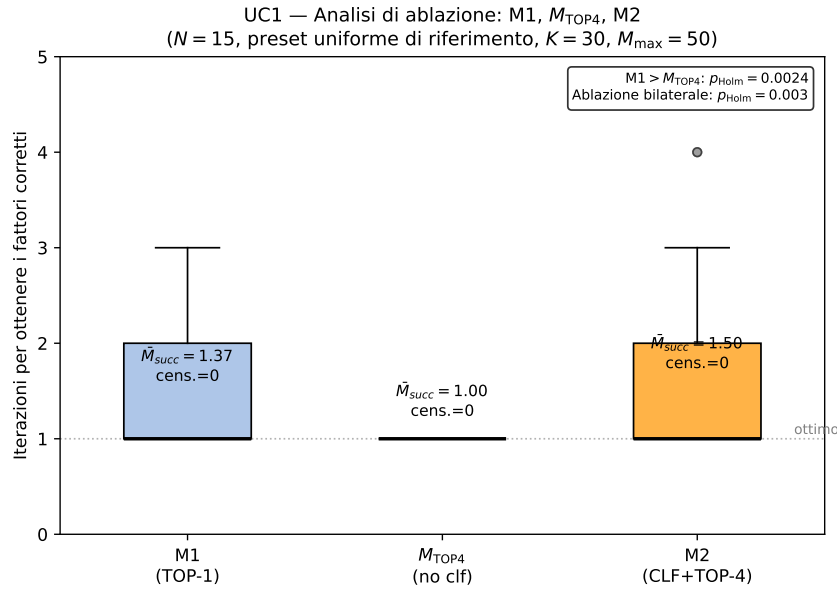


Figura 7.2: Distribuzione delle iterazioni nelle 30 repliche appaiate. I fallimenti, se presenti, sarebbero mostrati separatamente con sentinella 51; in questa baseline tutte le repliche hanno successo.

7.5 Robustezza Esplorativa della Strategia TOP-4

Otto famiglie di sweep variano un parametro per volta. I test sono appaiati all'interno del singolo punto, ma i p non ricevono una correzione globale fra le otto famiglie: i risultati vanno quindi letti come esplorativi. La Tabella 7.4 evita di trasformare l'assenza di variazione osservata in una pretesa universale di insensibilità al rumore.

Tabella 7.4: Sintesi degli otto sweep schema 2 su $N = 15$

Famiglia	Intervallo	Esito TOP-4
K	1, 2, 3, 4, 6, 8	soglia osservata $K = 2$
λ_{2q}	10^{-3} – 10^{-1}	$\bar{M} = 1.00$ – 1.07 , successo 100%
shot	128–2 048	$\bar{M} = 1.00$
$K \times \lambda_{2q}$	$K = 2$ – 8 , 5×10^{-3} – 5×10^{-2}	$\bar{M} = 1.00$
T_1/T_2	20/16–500/400 μs	$\bar{M} = 1.00$
λ_{1q}	10^{-4} – 2×10^{-2}	$\bar{M} = 1.00$
readout simmetrico	0–20%	$\bar{M} = 1.00$
ottimizzazione	livelli 0–3	$\bar{M} = 1.00$

Il livello di ottimizzazione modifica il circuito: livello 0 produce 242 CX e profondità 427, livello 1 produce 236/421, mentre i livelli 2 e 3 coincidono a 224/412. Nonostante ciò TOP-4 resta a una iterazione nei quattro punti. Le oscillazioni di M1 negli sweep non sono monotone e, con 30 repliche, non sono interpretate come leggi causali.

7.5.1 Frazione Coerente Efficace e Proxy di Nessun Evento

La massa media m sui quattro picchi può essere convertita in un indice descrittivo

$$f = \frac{m - 4/256}{1 - 4/256}, \quad (7.1)$$

che vale zero per il fondo uniforme e uno per una distribuzione concentrata sui picchi. Non è una fedeltà. La Tabella 7.5 la confronta con il proxy che nessuna delle 224 CX applichi un Pauli non-identità, $P_0 = (1 - 15\lambda_{2q}/16)^{224}$.

Tabella 7.5: Media su 20 repliche per punto. Nessuna delle due colonne è una probabilità di fattorizzazione.

λ_{2q}	f efficace	P_0 proxy
10^{-3}	0.8036	8.11×10^{-1}
5×10^{-3}	0.7039	3.49×10^{-1}
10^{-2}	0.5938	1.21×10^{-1}
2×10^{-2}	0.4286	1.44×10^{-2}
5×10^{-2}	0.1688	2.14×10^{-5}
10^{-1}	0.0421	2.65×10^{-10}
2×10^{-1}	0.0047	6.32×10^{-21}
3×10^{-1}	0.0015	7.47×10^{-33}

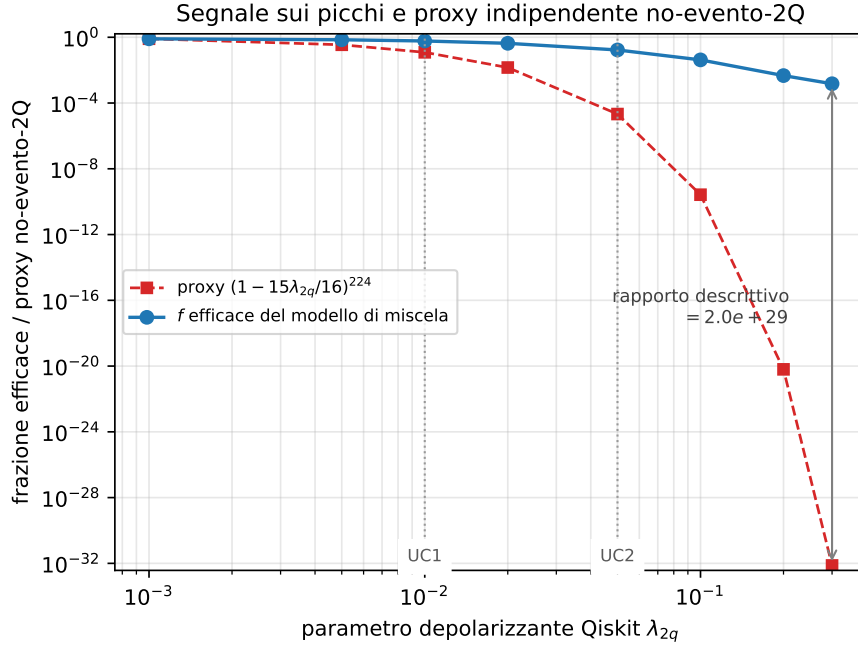


Figura 7.3: Il proxy moltiplicativo decade più rapidamente della massa strutturata osservata. La divergenza mostra che “almeno un evento Pauli” non equivale a “algoritmo fallito”.

7.5.2 Confronto con la Zero-Noise Extrapolation

La ZNE digitale scala soltanto λ_{1q} e λ_{2q} ; tempi di coerenza e readout restano fissi. Non è gate folding hardware. ZNE-2 usa i livelli (1,2) e ZNE-3 i livelli (1,2,3), condivisi fra strategie nella stessa replica.

Tabella 7.6: Confronto M1, ZNE e TOP-4 su 30 repliche appaiate

Strategia	Successo	$\bar{M}$	shot/iter.	shot medi totali	p_{Holm}
M1	100%	1.37	1 024	1 399	—
ZNE-2	100%	1.50	2 048	3 072	1.000
ZNE-3	100%	1.57	3 072	4 813	1.000
TOP-4	100%	1.00	1 024	1 024	0.0024

La ZNE non migliora M1 e aumenta il costo di campionamento; TOP-4 è l'unica strategia che riduce significativamente le iterazioni in questo confronto. Il risultato riguarda il modello uniforme e questa specifica istanza: non è una valutazione generale della ZNE su hardware calibrato.

7.6 Bilancio della Verifica e Ponte verso la Correzione d'Errore

La campagna consente quattro conclusioni delimitate dai dati:

1. l'operatore $\times 7 \bmod 15$ e il circuito compilato sono fissati da truth table, manifest e hash;
2. TOP-4 riduce $\bar{M}$ da 1.37/1.50 a 1.00 nei due scenari;
3. il classificatore TOP-1 non migliora M1 e peggiora la propria ablazione TOP-4;
4. la robustezza osservata negli sweep è esplorativa e non rende il modello uniforme equivalente a una QPU reale.

Il post-processing recupera informazione ancora presente nell'istogramma, ma non può correggere uno stato già corrotto durante il calcolo. Questo limite motiva il passaggio ai codici di correzione e alla decodifica della sindrome nei capitoli successivi.

Capitolo 8

La Correzione d'Errore Quantistico: dal Codice a Ripetizione al Codice di Steane

Il Capitolo 7 ha chiuso la verifica dell'ipotesi iniziale con un esito netto e una lezione precisa: il filtro appreso sull'output rumoroso non aggiunge nulla che la ricerca multi-candidato ottenga già con una regola deterministica più semplice. TOP-4 può sfruttare picchi utili ancora presenti nell'istogramma, ma il post-processing non può ripristinare la coerenza o la struttura periodica persa durante l'evoluzione. Per prevenire questa seconda classe di danno occorre intervenire *durante* il calcolo: è la prospettiva della QEC, introdotta nel quadro teorico del Capitolo 5 e resa operativa qui. Questo capitolo costruisce gli strumenti concettuali e sperimentali della correzione d'errore — stabilizzatori, sindrome, decodifica — e li applica in due laboratori a complessità crescente: il codice a ripetizione a 3 qubit, che introduce il meccanismo della sindrome nel caso più semplice, e il codice di Steane $[[7, 1, 3]]$, il primo codice pienamente quantistico della tesi, capace di correggere un errore arbitrario su un singolo qubit. Il capitolo successivo estende il percorso al surface code e alla stima quantitativa dell'errore logico.

8.1 Dalla Ridondanza Ingenua alla Codifica

L'idea di proteggere l'informazione replicandola è classica: si trasmettono tre copie di un bit e si decide a maggioranza. Sul piano quantistico questa strada è sbarrata due volte. Il teorema di no-cloning [16] (Sezione 3.1) impedisce di copiare uno

stato ignoto; e la misura diretta di un qubit, necessaria per un ipotetico “voto di maggioranza”, distrugge la sovrapposizione che si voleva proteggere. Per questo la pratica NISQ più diffusa si limita a una ridondanza *a livello di esecuzione* — eseguire istanze gemelle dello stesso circuito su regioni diverse del dispositivo e usare l’esito della copia sopravvissuta — che paga un raddoppio di qubit fisici senza agire sulla causa degli errori.

La QEC risolve il problema in modo strutturalmente diverso [19, 7, 27]: l’informazione di un **qubit logico** (*logical qubit*) viene *codificata* — non copiata — nelle correlazioni di entanglement di più **qubit fisici** (*physical qubits*), e gli errori vengono diagnosticati misurando operatori ausiliari che rivelano *che cosa è andato storto* senza rivelare — e quindi senza distruggere — *che cosa è codificato*. È questa distinzione, non la semplice ridondanza, a rendere possibile la correzione durante il calcolo.

8.2 Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder

Il funzionamento di ogni codice correttore quantistico segue lo stesso ciclo concettuale:

1. **Codifica**: lo stato logico $|\psi_L\rangle$ viene preparato su n qubit fisici;
2. **Rumore**: i canali fisici del Capitolo 4 agiscono sui qubit fisici;
3. **Estrazione della sindrome**: qubit ausiliari (*ancilla*) misurano gli stabilizzatori del codice; l’esito — una stringa di bit classica detta **sindrome** (*syndrome*) — identifica la classe di errore senza toccare l’informazione logica;
4. **Decodifica**: un algoritmo classico, il **decoder**, interpreta la sindrome e propone la correzione più probabile;
5. **Correzione**: la correzione viene applicata fisicamente, oppure registrata in un registro classico — il **Pauli frame** — che tiene memoria delle correzioni da interpretare sulle misure successive, riducendo le operazioni fisiche e la propagazione di errori.

Stabilizzatori. Uno **stabilizzatore** (*stabilizer*) [28] è un operatore S che lascia invariati tutti gli stati validi del codice: $S|\psi_L\rangle = |\psi_L\rangle$. Finché lo stato resta nello

spazio del codice, ogni misura di S restituisce $+1$; un esito -1 segnala che un errore ha portato lo stato fuori dal sottospazio consentito. La collezione degli esiti di tutti i generatori del gruppo stabilizzatore costituisce la sindrome. Il punto cruciale è che gli stabilizzatori commutano con l'informazione logica: la loro misura proietta lo stato in un autospazio dell'errore, ma non distingue — e quindi non collassa — i coefficienti della sovrapposizione logica.

Distanza del codice. Un codice si denota $[[n, k, d]]$: n qubit fisici, k qubit logici, distanza d . La distanza è il peso minimo di un errore capace di trasformare uno stato logico valido in un altro senza essere rilevato; un codice di distanza d corregge fino a

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \quad (8.1)$$

errori arbitrari. Lo Steane code ($d = 3$) corregge un errore; un surface code di distanza 5 ne corregge due, di distanza 7 tre. La soglia oltre la quale aggiungere distanza *conviene* — il regime sotto-soglia in cui l'errore logico decresce all'aumentare di d — è l'oggetto quantitativo del Capitolo 9, e la sua esistenza è garantita in linea di principio dal teorema della soglia per il calcolo fault-tolerant [18].

8.3 Il Panorama dei Codici e la Scelta Metodologica

Codice	Parametri	Corregge	Pro / Contro	Uso in questa tesi
Repetition (bit-flip)	$[[3, 1, 1]]$ su X	solo bit-flip	semplice / non corregge phase-flip	primo laboratorio (sindrome)
Repetition (phase-flip)	3 qubit, base H	solo phase-flip	mostra dualità X/Z / parziale	secondo laboratorio
Shor code	$[[9, 1, 3]]$	errore arbitrario singolo	storico, intuitivo / più pesante dello Steane	approfondimento didattico [19]
Steane code	$[[7, 1, 3]]$	errore arbitrario singolo	CSS, compatto / distanza bassa	codice centrale [7]
Five-qubit	$[[5, 1, 3]]$	errore arbitrario singolo	minimo possibile / non CSS	confronto teorico
Surface code	d^2 dati + $O(d^2)$ ancille	errori locali su griglia 2D	prospettiva scalabile / overhead elevato	Capitolo 9 [8]

Tabella 8.1: Confronto tra i principali codici correttori e loro ruolo nella tesi. La progressione metodologica — repetition per introdurre la sindrome, Steane per la correzione di errori arbitrari, surface code per la prospettiva scalabile — segue il documento di indirizzo del lavoro.

La scelta metodologica è dunque una progressione a tre stadi: il *repetition code* introduce l'estrazione di sindrome nel caso più leggibile; lo *Steane code* è il codice centrale, primo capace di correggere un errore arbitrario; il *surface code* porta il discorso sul terreno scalabile e permette la stima del **logical error rate** p_L con un decoder reale.

8.4 Laboratorio I: il Codice a Ripetizione

8.4.1 Il codice bit-flip a 3 qubit

Il codice a ripetizione contro errori X codifica $|0_L\rangle = |000\rangle$ e $|1_L\rangle = |111\rangle$: uno stato arbitrario $\alpha|0\rangle + \beta|1\rangle$ diventa $\alpha|000\rangle + \beta|111\rangle$ — uno stato entangled, non tre copie. Gli stabilizzatori sono Z_1Z_2 e Z_2Z_3 : misurano la *parità* tra coppie di qubit senza misurare i singoli qubit. Un bit-flip su un qubit altera una o entrambe le parità, e la sindrome a 2 bit identifica univocamente quale dei tre qubit ha subito l'errore — che viene corretto riapplicando X. La misura di parità avviene tramite ancilla: due CNOT copiano la parità sull'ancilla, che viene misurata al posto dei qubit dati.

Il limite è strutturale: il codice non rileva gli errori Z (phase-flip), che commutano con tutti gli stabilizzatori. Il duale — codificare nella base di Hadamard, $|\pm_L\rangle = |\pm\pm\pm\rangle$, con stabilizzatori X_1X_2 e X_2X_3 — protegge dai phase-flip ma non dai bit-flip. La dualità X/Z è la ragione per cui la correzione quantistica è più difficile di quella classica, e la concatenazione delle due protezioni è esattamente la costruzione del codice di Shor a 9 qubit [19].

8.4.2 Risultati del laboratorio repetition

Il laboratorio è stato implementato in Qiskit con estrazione di sindrome tramite due qubit ancilla, secondo il ciclo della Sezione 8.2. Il primo test verifica che la sindrome identifichi correttamente il qubit colpito: si inietta un errore deterministico su ciascuno dei tre qubit dati (un X per il codice bit-flip, un Z per il duale phase-flip) e si legge la sindrome. La Tabella 8.2 riporta l'esito: in tutti i casi la sindrome osservata coincide con la previsione della lookup table, e la posizione dell'errore è ricostruita senza ambiguità.

Errore iniettato	Sindrome (a_0, a_1)	Qubit dedotto	Esito
nessuno	$(0, 0)$	—	✓
su q_0	$(1, 0)$	q_0	✓
su q_1	$(1, 1)$	q_1	✓
su q_2	$(0, 1)$	q_2	✓

Tabella 8.2: Verifica della sindrome: iniezione deterministica di un errore su ciascun qubit e sindrome osservata (senza rumore). Vale identicamente per il codice bit-flip (errore X , stabilizzatori Z_0Z_1, Z_1Z_2) e per il duale phase-flip (errore Z , stabilizzatori X_0X_1, X_1X_2): il duale si ottiene dal primo per coniugazione di Hadamard, e a parità di seed le due simulazioni producono esiti identici cifra per cifra. La coincidenza non è quindi una conferma indipendente della dualità — che è un fatto algebrico — ma la verifica che l’implementazione la rispetti. L’ancilla a_0 misura la parità dei qubit $\{0, 1\}$, l’ancilla a_1 quella di $\{1, 2\}$.

Il secondo test misura il beneficio della correzione. Si prepara lo stato logico, si applica a ciascun qubit dato un errore indipendente con probabilità p (canale X per il bit-flip, Z per il phase-flip), si estrae la sindrome, si applica la correzione e si decodifica: la frazione di esiti logici errati su 20 000 ripetizioni (seed = 42) stima l’errore logico p_L . La Figura 8.1 confronta i punti Monte Carlo con la previsione analitica del voto di maggioranza: il codice sbaglia quando almeno due dei tre qubit subiscono un errore, cioè

$$p_L = 3p^2(1 - p) + p^3 = 3p^2 - 2p^3. \quad (8.2)$$

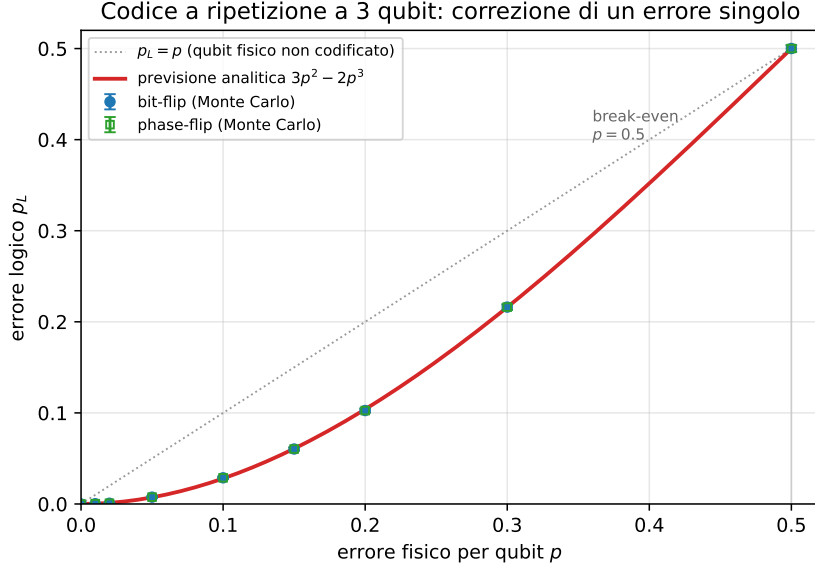


Figura 8.1: Errore logico p_L in funzione dell'errore fisico p per il codice a ripetizione a 3 qubit. I punti Monte Carlo del codice bit-flip e del duale phase-flip (20 000 ripetizioni per punto) coincidono con la previsione analitica $3p^2 - 2p^3$ (Eq. 8.2) entro l'errore statistico. Nella regione $p < 0.5$ la curva giace sotto la bisettrice $p_L = p$: la codifica rende il qubit logico più affidabile del qubit fisico non protetto. Il punto di *break-even* è a $p = 0.5$, dove correzione e assenza di correzione si equivalgono; oltre, la ridondanza amplifica gli errori anziché sopprimerli.

L'accordo con l'Eq. 8.2 è quantitativo su tutto l'intervallo: ad esempio, per $p = 0.10$ si osserva $p_L = 0.0288 \pm 0.0012$ contro il valore teorico 0.0280, e per $p = 0.20$ si ha $p_L = 0.1026 \pm 0.0021$ contro 0.1040. Il messaggio operativo è quello che l'intera parte QEC svilupperà su codici più potenti: sotto una soglia di errore fisico, la codifica *sopprime* l'errore — qui da p a $\mathcal{O}(p^2)$ — mentre sopra la soglia lo *amplifica*. Il codice a ripetizione paga però un limite strutturale già discusso: protegge da un solo tipo di errore per volta. Un errore Z sul codice bit-flip commuta con entrambi gli stabilizzatori, non produce sindrome e resta invisibile — ragione per cui il passo successivo richiede un codice, come quello di Steane, capace di correggere *qualunque* errore su un qubit.

8.5 Laboratorio II: il Codice di Steane $[[7, 1, 3]]$

8.5.1 Struttura del codice

Lo Steane code [7] codifica un qubit logico in sette qubit fisici, ha distanza 3 e corregge un errore arbitrario (X, Z o Y) su un singolo qubit. È un codice CSS, costruito a partire dal codice classico di Hamming $[7, 4, 3]$: la stessa matrice di parità classica genera sia gli stabilizzatori di tipo X sia quelli di tipo Z, il che permette di trattare separatamente le due famiglie di errori.

Parametro	Valore	Significato
n	7	qubit fisici
k	1	qubit logici
d	3	distanza del codice
t	1	errori arbitrari correggibili (Eq. 8.1)
Tipo	CSS	sindromi X e Z separate

Tabella 8.3: Parametri del codice di Steane $[[7, 1, 3]]$.

I generatori degli stabilizzatori, nella convenzione adottata, sono:

$$\begin{aligned} \text{tipo X: } & IIIXXXX, \quad IXXIIXX, \quad XIXIXIX \\ \text{tipo Z: } & IIIZZZZ, \quad IZZIIZZ, \quad ZIZIZIZ \end{aligned} \tag{8.3}$$

Gli stabilizzatori di tipo Z rilevano gli errori X; quelli di tipo X rilevano gli errori Z; un errore Y, essendo proporzionale a XZ , produce entrambe le sindromi. La struttura di Hamming rende la decodifica trasparente: i tre bit di sindrome di ciascuna famiglia, letti come numero binario, indicano direttamente la *posizione* del qubit colpito (sindrome 011 $\rightarrow$ qubit 3, sindrome 110 $\rightarrow$ qubit 6, e così via), e una lookup table $\text{sindrome} \rightarrow \text{correzione}$ completa il decoder.

8.5.2 Protocollo di laboratorio

Il laboratorio segue il protocollo del documento di indirizzo: (i) preparazione di uno stato logico $|0_L\rangle$ o $|+_L\rangle$ secondo una costruzione documentata; (ii) iniezione manuale di un errore X su ciascuno dei sette qubit e misura della sindrome; (iii) ripetizione con errori Z e Y; (iv) costruzione della lookup table; (v) valutazione della probabilità di correzione riuscita al crescere dell'errore fisico p ; (vi) distinzione tra errori sui qubit dati ed errori sulle ancilla durante l'estrazione della sindrome.

Su quest’ultimo punto va dichiarata fin d’ora un’assunzione metodologica: lo Steane code corregge un errore arbitrario *solo se* il modello assume al più un errore rilevante nel blocco, o se il protocollo fault-tolerant impedisce che un singolo guasto si propaghi in più errori non correggibili. La versione implementata in questa tesi usa un’estrazione di sindrome non fault-tolerant, e i risultati vanno letti entro questa assunzione — distinguere ciò che il codice garantisce da ciò che l’implementazione didattica mostra è parte del rigore richiesto al lavoro.

8.5.3 Risultati del laboratorio Steane

Il laboratorio è stato implementato in Qiskit. Lo stato logico $|0_L\rangle$ si prepara come sovrapposizione uniforme degli otto *codeword* del codice: partendo da tre qubit “seme” (quelli che compaiono in un solo stabilizzatore di tipo X) posti in sovrapposizione con una porta di Hadamard, una rete di CNOT propaga la struttura degli stabilizzatori sugli altri quattro qubit. La correttezza della codifica è il primo controllo: misurando i sei stabilizzatori sullo stato preparato si ottiene sindrome nulla 000000 su tutti i 2000 campioni — lo stato è effettivamente nel *code space*.

La syndrome table è biiettiva. Il secondo controllo è il cuore del codice: iniettando un errore su ciascuno dei sette qubit e leggendo la sindrome, la posizione del qubit colpito dev’essere ricostruita senza ambiguità. La Tabella 8.4 riporta l’esito per gli errori X (letti dagli stabilizzatori Z): le sette sindromi sono tutte distinte e non nulle, e coincidono con la rappresentazione binaria della posizione, esattamente come prescrive la struttura di Hamming. Per dualità CSS, gli errori Z producono la stessa tabella sugli stabilizzatori X (sindrome Z nulla), e un errore Y accende *entrambe* le famiglie con la medesima sindrome — confermando che il codice gestisce l’errore arbitrario, non solo bit-flip o phase-flip. La biiettività è verificata per tutti e 21 i casi (X, Z, Y su 7 qubit).

Qubit	Sindrome	Valore	Correzione
q_0	001	1	applica X (o Z) su q_0
q_1	010	2	... su q_1
q_2	011	3	... su q_2
q_3	100	4	... su q_3
q_4	101	5	... su q_4
q_5	110	6	... su q_5
q_6	111	7	... su q_6

Tabella 8.4: Tabella sindrome→qubit→correzione, verificata sperimentalmente. La sindrome (tre bit) letta come numero binario indica direttamente la posizione del qubit colpito. Vale per gli errori X (via stabilizzatori Z) e, per dualità, per gli errori Z (via stabilizzatori X); un errore Y accende entrambe le famiglie con la stessa sindrome.

Il qubit logico batte il fisico: soppressione quadratica. Il terzo controllo misura il beneficio della correzione. Sotto un canale depolarizzante di intensità p per qubit, si stima la probabilità di errore logico p_L con una simulazione Monte Carlo nel formalismo di Pauli (200 000 campioni per punto, seed = 42): per ogni campione si estrae la sindrome, si applica la correzione a peso minimo dalla Tabella 8.4 e si verifica se il residuo contiene un operatore logico non banale. La Figura 8.2 mostra il risultato in scala doppio-logaritmica: i punti seguono un andamento $p_L \propto p^2$, con pendenza log-log misurata 1.94 — la firma di un codice di distanza $d = 3$, che corregge ogni errore singolo e fallisce solo a partire da due errori. Poiché gli eventi di peso due, il cui numero di coppie di qubit scala come $\binom{7}{2}$, forniscono il contributo dominante al fallimento, l'errore logico è soppresso da $\mathcal{O}(p)$ a $\mathcal{O}(p^2)$.

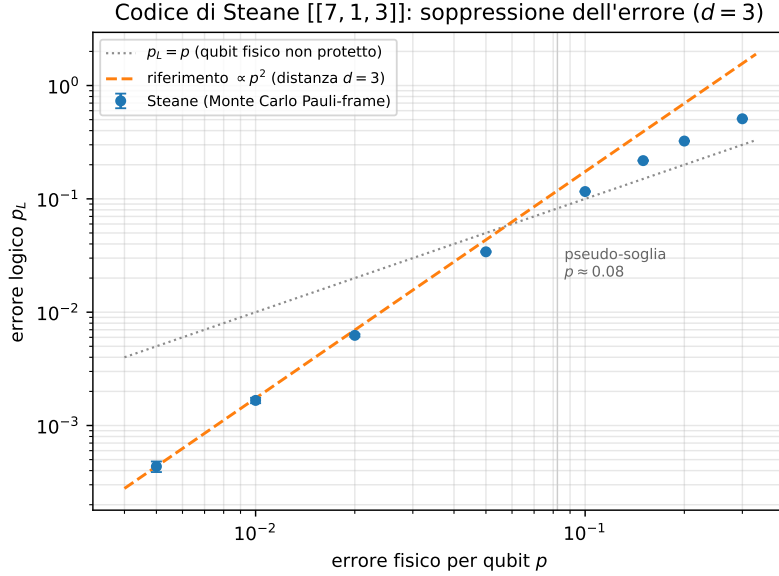


Figura 8.2: Errore logico p_L in funzione dell'errore fisico p (canale depolarizzante), scala doppio-logaritmica. I punti Monte Carlo seguono il riferimento $\propto p^2$ (pendenza $d = 3$); la bisettrice $p_L = p$ rappresenta il qubit fisico non protetto. L'intersezione individua la **pseudo-soglia** $p \approx 0.08$: sotto di essa il qubit logico è più affidabile del fisico, sopra la ridondanza amplifica l'errore. Curva rigenerabile da `figure_src/gen_qec_steane.py`.

Il regime di convenienza è netto: per $p = 0.01$ si ha $p_L = 1.67 \times 10^{-3}$ (già un ordine di grandezza sotto p), per $p = 0.05$ si ha $p_L = 3.4 \times 10^{-2} < p$, mentre a $p = 0.10$ si ha $p_L = 0.116 > p$. La pseudo-soglia — l'intersezione tra la curva e la bisettrice $p_L = p$ — cade attorno a $p \approx 0.08$: al di sotto, codificare conviene.

Confronto con il repetition e assunzioni. Rispetto al repetition code (Sezione 8.4), che pure esibisce una soppressione $\propto p^2$, lo Steane paga sette qubit fisici invece di tre ma ottiene un vantaggio qualitativo decisivo: protegge simultaneamente da errori X , Z e Y , mentre il repetition è cieco a un intero tipo di errore. Va infine ribadita l'assunzione dichiarata nella Sezione 8.5: l'estrazione di sindrome qui implementata *non è fault-tolerant*. Il modello Monte Carlo inietta errori solo sui qubit dati e assume una lettura di sindrome ideale; un errore sulle ancilla durante l'estrazione può corrompere la sindrome e indurre una correzione errata. La quantificazione di questo effetto — e le tecniche fault-tolerant che lo mitigano — appartiene al terreno del surface code, oggetto del Capitolo 9, dove il decoder opera su cicli ripetuti di misura proprio per distinguere gli errori di dato da quelli di misura.

Capitolo 9

Il Surface Code e la Stima dell'Errore Logico

Il Capitolo 8 ha mostrato che un codice di distanza 3 può correggere un errore arbitrario su un singolo qubit — ma anche che questa protezione, da sola, non basta: con sette qubit fisici per qubit logico e distanza fissa, lo Steane code non offre alcuna via per *ridurre a piacere* l'errore residuo. La proprietà che manca è la scalabilità della protezione: un parametro che, aumentato, faccia diminuire l'errore logico. È esattamente ciò che offre il **surface code** [29, 8], oggi lo standard di riferimento industriale per il calcolo fault-tolerant: un codice definito su una griglia bidimensionale di qubit con sole interazioni locali, la cui distanza d cresce con la taglia della griglia. Questo capitolo ne descrive il funzionamento operativo, introduce il problema della decodifica e la sua soluzione standard — il *matching* di peso minimo — e presenta l'esperimento centrale della parte QEC della tesi: la stima del **logical error rate** p_L in funzione dell'errore fisico p e della distanza d , condotta con gli strumenti specializzati Stim e PyMatching. Le due sezioni conclusive mettono poi alla prova le assunzioni su cui quella stima poggia: prima il modello di rumore con cui il codice viene *simulato* (Sezione 9.5), poi quello con cui viene *decodificato* (Sezione 9.6) — ed è in quest'ultima che l'apprendimento automatico, giudicato superfluo nel Capitolo 7 quando applicato all'output, trova il punto della pipeline in cui produce un guadagno misurabile.

9.1 Il Codice su Griglia: Concetto Operativo

Il surface code dispone su una griglia bidimensionale due popolazioni di qubit: i **qubit dati** (*data qubits*), che portano l'informazione logica, e i **qubit di misura** (*measure qubits*), che estraggono ciclicamente stabilizzatori *locali* — ciascuno coinvolge solo i pochi qubit dati adiacenti sulla griglia. Questa località è il vantaggio architetturale decisivo: ogni operazione richiesta dal codice è un'interazione tra vicini fisici, compatibile con le topologie reali delle QPU superconduttive descritte nel Capitolo 4.

Un patch planare ruotato di distanza d usa d^2 qubit dati per codificare un qubit logico, oltre a un numero dello stesso ordine di ancille di misura: il costo fisico totale cresce quindi come $O(d^2)$. Una distanza maggiore richiede una griglia più grande ma corregge più errori (Eq. 8.1). L'estrazione degli stabilizzatori viene ripetuta per più cicli (*rounds*) consecutivi, perché anche le misure di sindrome sono rumorose: la diagnosi non può basarsi su un singolo ciclo. Una *variazione* dell'esito di uno stabilizzatore tra due cicli successivi costituisce un **detection event**: è il segnale grezzo da cui il decoder deve ricostruire cosa è accaduto.

9.2 La Decodifica: Minimum Weight Perfect Matching

Nel bulk gli errori fisici sul surface code producono tipicamente detection events *in coppie*; vicino a un bordo, invece, un estremo della catena può terminare sul bordo stesso. In generale le catene di errore sono osservate attraverso i loro estremi nello spazio-tempo. Il problema del decoder è quindi: dati gli eventi osservati, inferire la configurazione di errori *più probabile* che li ha generati. La formulazione standard è un problema di accoppiamento su grafo, includendo i nodi di bordo: il MWPM associa gli eventi minimizzando un peso legato alla probabilità degli errori, e la libreria PyMatching [25] lo implementa in modo efficiente a partire dal modello di errore del circuito.

Se la catena di correzione proposta dal decoder e la catena di errori reale differiscono per un ciclo che attraversa la griglia da parte a parte, la correzione fallisce *silenziosamente*: l'informazione logica è stata alterata senza che alcuna sindrome lo riveli. La frequenza di questi fallimenti è il logical error rate p_L — la metrica centrale di tutta la parte QEC.

Perché servono strumenti dedicati. La stima di p_L richiede milioni di esecuzioni di circuiti con centinaia di qubit: fuori portata per un simulatore statevector, il cui costo cresce come 2^n (Sezione 6.1). I circuiti del surface code, però, sono interamente *stabilizer* (composti di sole operazioni Clifford e misure), e per questa classe il teorema di Gottesman-Knill garantisce la simulabilità classica efficiente. Stim [24] sfrutta esattamente questa proprietà: campiona *detection events* di circuiti stabilizer a velocità di milioni di shot al secondo. Il rovescio della medaglia — il fatto che Shor *non* sia un circuito stabilizer — è il punto di partenza del Capitolo 10.

La struttura del codice che realizza questa stima — generazione parametrica del circuito, campionamento dei *detection events*, costruzione del *Detector Error Model* (DEM) e decodifica — è riportata nel Listato B.14 (Sezione B.2).

9.3 Disegno Sperimentale

L'esperimento segue il protocollo del documento di indirizzo:

Variabile	Valori	Osservazione attesa
distanza d	3, 5, 7, 9	p_L diminuisce con d se p è sotto soglia
rounds	d e $2d$	cicli di memoria quantistica
p fisico	$10^{-4} \dots 2 \times 10^{-2}$	curva p vs p_L
shots	$\geq 10^4$ per punto	stima stabile di p_L con intervalli di confidenza
base	memory X e memory Z	confronto protezione errori X/Z

Tabella 9.1: Griglia sperimentale prescritta per la stima del logical error rate. La campagna effettivamente eseguita ne copre un sottoinsieme: tutte e quattro le distanze ed entrambe le basi, con rounds = d e $p \in [2 \times 10^{-3}, 10^{-2}]$ — l'intervallo in cui cade l'incrocio delle curve, cioè la grandezza che l'esperimento deve misurare. Lo scaling a rounds = $2d$ non è stato eseguito (Sezione 9.4).

Il risultato scientifico atteso è una famiglia di curve p vs p_L , una per distanza. Se, in una regione di p sufficientemente bassa, le curve si ordinano in modo che d maggiore dia p_L minore — mentre sopra una certa soglia l'ordine si inverte — l'esperimento avrà osservato il comportamento *threshold-like* che il teorema della soglia [18] prevede: sotto soglia, aggiungere ridondanza conviene; sopra soglia, il codice amplifica i propri stessi errori. Il valore di p a cui le curve si incrociano, confrontato con i tassi d'errore fisici realistici del Capitolo 4, dice quanto margine separa l'hardware attuale dal regime in cui la correzione scalabile funziona.

9.4 Risultati

L'esperimento è stato realizzato con **Stim** per il campionamento dei circuiti stabilizzatori e **PyMatching** per la decodifica MWPM, secondo lo schema della Sezione 9.3: memory experiment a rounds = d , modello di rumore *circuit-level* (depolarizzazione dopo ogni Clifford, flip di reset e di misura tutti pari a p), campionamento dimensionato punto per punto in modo da raccogliere circa duecento eventi logici — da 2×10^5 a 4×10^7 campioni secondo la distanza e il livello di rumore, poiché a $d = 9$ e $p = 2 \times 10^{-3}$ il tasso logico è dell'ordine di 10^{-5} e una numerosità fissa produrrebbe una manciata di eventi. La Figura 9.1 riporta le curve p vs p_L per $d = 3, 5, 7, 9$ in base Z .

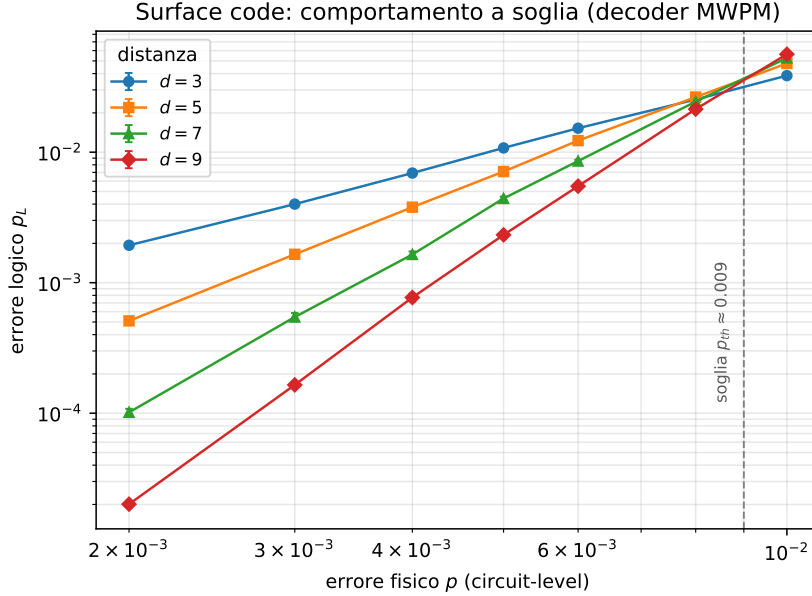


Figura 9.1: Errore logico p_L in funzione dell'errore fisico p (*circuit-level*) per il surface code ruotato a distanza $d = 3, 5, 7, 9$, decoder MWPM, scala doppio-logaritmica. A p basso le curve *divergono* — distanza maggiore dà errore logico molto minore — e si *incrociano* attorno alla soglia $p_{th} \approx 0.9\%$. È la firma del comportamento *threshold-like*: sotto soglia la ridondanza sopprime l'errore, sopra soglia lo amplifica. Barre d'errore statistiche (spesso più piccole del marcatore); curva rigenerabile da `figure_src/gen_gec_surface.py`.

La soglia è osservata. Il risultato centrale è la presenza di un incrocio netto. Nel regime sotto-soglia la soppressione è marcata e cresce con la distanza: a $p = 2 \times 10^{-3}$ si passa da $p_L = 1.94 \times 10^{-3}$ ($d = 3$) a 5.10×10^{-4} ($d = 5$), a 1.02×10^{-4} ($d = 7$), a 2.01×10^{-5} ($d = 9$) — con fattori di soppressione 3.8, 5.0 e 5.1 per incremento di

distanza. La vicinanza degli ultimi due fattori è compatibile con una soppressione approssimativamente geometrica nell'intervallo $d = 5-9$; quattro sole distanze non bastano per affermare che sia stato raggiunto il regime asintotico. L'ordine si mantiene fino a circa $p = 6 \times 10^{-3}$; oltre, le curve si accavallano e a $p = 10^{-2}$ è *invertito* ($p_L = 3.9\%$ per $d = 3$ contro 5.6% per $d = 9$): sopra soglia aumentare la distanza peggiora l'errore. La soglia osservata, $p_{th} \approx 0.9\%$ in base Z e $\approx 0.7\%$ in base X, è una stima specifica del circuito, del decoder e del modello di rumore adottati. Il comportamento qualitativo comune alle due basi costituisce un controllo di coerenza, non una prova di universalità della soglia.

Dalla constatazione alla legge di scala. Con quattro distanze anziché tre la soglia non va più soltanto letta dall'incrocio: può essere *stimata*. La teoria prevede che, sotto soglia, l'errore logico segua

$$p_L(d, p) = A \left(\frac{p}{p_{th}} \right)^{\lfloor (d+1)/2 \rfloor}, \quad (9.1)$$

dove l'esponente è il numero di errori fisici necessari a produrre un errore logico non rilevabile. Adattando (9.1) ai punti con $p \leq 6 \times 10^{-3}$ per tutte e quattro le distanze si ottiene $A = 3.5 \times 10^{-2}$ e $p_{th} = 0.86\%$ in base Z ($A = 3.4 \times 10^{-2}$, $p_{th} = 0.81\%$ in base X), con residuo quadratico medio di 0.08 in $\ln p_L$ — ossia un accordo entro l'8% su cinque ordini di grandezza di errore logico. La stima della soglia ricavata dal fit e quella letta dall'incrocio concordano, il che è un controllo non banale: la prima usa il comportamento sotto soglia, la seconda quello al confine.

Il valore di questa formulazione è che rende il risultato *predittivo* anziché descrittivo. Invertendo la (9.1) si ottiene la distanza minima necessaria a raggiungere un dato errore logico, e con essa il costo in qubit fisici, che per il surface code ruotato è $2d^2 - 1$ per qubit logico.

Tabella 9.2: Distanza minima e costo in qubit fisici per qubit logico, ricavati invertendo la (9.1) con i parametri stimati in base Z. Il confronto fra le due colonne quantifica quanto la fedeltà dei gate valga in risorse: dimezzare l'errore fisico da 2×10^{-3} a 10^{-3} riduce di oltre tre volte il numero di qubit necessari a raggiungere $p_L = 10^{-6}$.

p_L richiesto	$p = 2 \times 10^{-3}$		$p = 10^{-3}$	
	d	qubit fisici	d	qubit fisici
10^{-4}	9	161	5	49
10^{-6}	15	449	9	161
10^{-9}	23	1 057	17	577
10^{-12}	33	2 177	23	1 057

Distanza dall'hardware attuale. Il confronto fra la soglia e i preset uniformi del Capitolo 4 è soltanto qualitativo. Il parametro p del memory experiment circuit-level non coincide con λ_{2q} del circuito di Shor, e il logical error rate dell'osservabile di memoria non coincide automaticamente con una probabilità per gate logico. Nel modello simulato, ridurre p sotto la regione di incrocio rende vantaggioso aumentare la distanza; sopra tale regione la ridondanza amplifica il rumore. Il Capitolo 10 esplora invece una griglia esogena e dichiarata di errore Pauli per gate compilato: non trasferisce questi valori uno-a-uno. Per una previsione end-to-end servirebbe una mappatura per operazione logica e relativo circuito fault-tolerant.

La legge di scala permette però di andare oltre il singolo punto e di rispondere in modo diretto alla domanda di risorse. L'istanza $N = 15$ richiede dodici qubit logici (otto di controllo e quattro di lavoro); combinando questo dato con la Tabella 9.2 si ottiene che una versione codificata al livello $p_L = 10^{-4}$ — quello che il Capitolo 10 individua come sufficiente — costerebbe **circa 1 900 qubit fisici** a $p = 2 \times 10^{-3}$, e ne basterebbero **poco meno di 600** se la fedeltà dei gate migliorasse di un fattore due. È il modo più concreto di enunciare il costo della correzione: non un fattore moltiplicativo astratto, ma due ordini di grandezza di qubit per eseguire in modo protetto la fattorizzazione di un numero a quattro bit.

Riproducibilità. Ogni punto è stimato con seed del campionatore fissato in modo deterministico per ciascuna coppia (errore fisico, distanza), e gli intervalli mostrati sono le corrispondenti deviazioni standard binomiali: la curva è rigenerabile bit per bit dallo script versionato. La numerosità è scelta in due passate — una prima stima a 2×10^5 campioni fissa l'ordine di grandezza di p_L , la seconda dimensiona il campione per raccogliere circa duecento eventi logici — ed è riportata nel JSON dei risultati punto per punto. Il criterio è necessario alle distanze grandi: a $d = 9$

e $p = 2 \times 10^{-3}$ una numerosità fissa di 2×10^5 campioni produce due soli eventi, con incertezza relativa del 70%, mentre il campionamento adattivo porta lo stesso punto al 3.5%. È questa precisione a rendere possibile l'adattamento della legge di scala (9.1): con la statistica precedente i fattori di soppressione a $d = 7$ e $d = 9$ non sarebbero stati distinguibili. Una ripetizione indipendente della campagna, condotta con seed diverso, ha riprodotto ogni punto entro l'errore statistico e ha restituito gli stessi valori di soglia. Lo scaling rounds = $2d$ e distanze ancora maggiori sono estensioni naturali; l'implementazione con Stim rende il campionamento a queste distanze poco oneroso, e il vincolo pratico è la memoria richiesta dal campionamento a blocchi più che il tempo di calcolo.

9.5 Oltre il Modello di Pauli: Errori Coerenti e Leakage

La stima del logical error rate appena presentata poggia su un'assunzione che va resa esplicita, perché ne delimita la validità: Stim è un simulatore a **stabilizzatori**, efficiente proprio perché i gate sono Clifford e gli errori sono Pauli casuali (X , Y , Z) con probabilità assegnate. È il modello standard delle grandi campagne di soglia, ma l'hardware reale non commette necessariamente errori di Pauli. Un lavoro recente [9] argomenta in modo sistematico che questa semplificazione può rendere le stime *ottimistiche*, e che una parte della fisica rilevante — errori coerenti, leakage, correlazioni temporali — va conservata nella simulazione per non certificare un'astrazione al posto del dispositivo.

Due sono le sorgenti di errore non-Pauli più importanti. La prima è l'**errore coerente**: un'imperfezione di calibrazione produce una piccola sovrarotazione sistematica $U = e^{-i\epsilon H}$, uguale a ogni ripetizione del gate. A differenza di un errore stocastico, le sovrarotazioni si sommano *in ampiezza*, non in probabilità. La seconda è il **leakage**: un transmon non è un sistema a due livelli e può popolare lo stato $|2\rangle$, un grado di libertà che un modello Pauli su due livelli non può nemmeno rappresentare.

Una verifica diretta su scala trattabile. Il meccanismo alla base del divario si illustra con un esperimento elementare, alla portata della simulazione *full-state* (Figura 9.2): un singolo qubit soggetto a N operazioni rumorose ripetute, con la stessa probabilità di errore per operazione $p = 10^{-2}$, modellata in due modi — come errore di Pauli (X con probabilità p) e come errore coerente (sovrarotazione $RX(\theta)$)

con $\sin^2(\theta/2) = p$). Nel regime iniziale l'errore stocastico cresce approssimativamente come Np , mentre quello coerente cresce come N^2p , perché gli angoli si sommano; le espressioni esatte restano limitate e sono rispettivamente $[1 - (1 - 2p)^N]/2$ e $\sin^2(N\theta/2)$. La simulazione (Qiskit Aer) coincide con queste previsioni e mostra un divario netto: a $N = 8$ operazioni l'errore coerente è circa 7 volte quello stocastico in questo esempio. Il fattore non è universale: dipende da p , N e dalla coerenza della sovrarotazione.

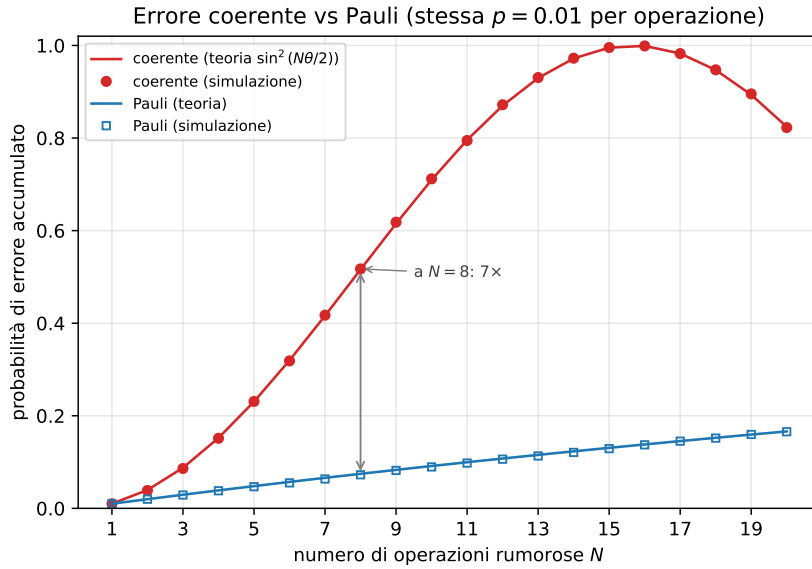


Figura 9.2: Accumulo dell'errore su un singolo qubit al crescere del numero N di operazioni rumorose, a parità di probabilità di errore per operazione ($p = 10^{-2}$). Nel tratto iniziale la sovrarotazione coerente (rosso) cresce quadraticamente, mentre l'errore stocastico di Pauli (blu) cresce approssimativamente in modo lineare; entrambe le curve seguono poi le rispettive espressioni limitate. La simulazione *full-state* (marcatori) coincide con la teoria (linee). A $N = 8$ il divario è di circa un fattore 7 per i parametri mostrati. Rigenerabile da `figure_src/gen_coherent.py`.

Implicazioni per la stima di soglia. Su un codice completo il divario si amplifica. Lo studio citato riporta, su esperimenti didattici controllati, che il modello Clifford-Pauli stima un errore logico dell'1.7% contro il 23.5% della simulazione completa in presenza di leakage, e del 18.1% contro il 38.7% in presenza di errori coerenti; su un modello di gate CZ fra transmon con surface code a distanza 9, l'errore logico reale può risultare da 25 a 56 volte superiore alla stima Clifford-Pauli, con la soglia sovrastimata di circa il 20% [9]. La direzione metodologica che ne emerge è l'approccio *channel-first*: descrivere il rumore fisico una sola volta come canale quantistico completo (operatori di Kraus, equazione di Lindblad) e scegliere di volta

in volta la rappresentazione di simulazione più adatta, conservando gli effetti non-Pauli dove contano. La soglia cessa così di essere un singolo numero e diventa una *superficie* nello spazio dei parametri fisici del dispositivo (T_1 , T_2 , durata di gate, leakage, crosstalk), che indica quale imperfezione limiti davvero la macchina.

Quanto pesa davvero, dentro un ciclo di correzione. I numeri citati sopra si riferiscono a regimi e distanze diversi da quelli qui studiati, e vale la pena misurare l'entità del divario sul proprio sistema anziché ereditarla. La verifica è stata condotta su un codice di distanza 3 simulato in *full-state* — Stim, essendo un simulatore a stabilizzatori, non può rappresentare una sovrarotazione coerente — iniettando alternativamente errori di Pauli e sovrarotazioni R_X di *pari probabilità marginale* ($\sin^2(\theta/2) = p$), su quattro cicli di estrazione della sindrome.

Il risultato è più contenuto di quanto l'esempio a singolo qubit lasci temere. A $p = 3 \times 10^{-2}$, il rumore coerente produce *meno* flip logici grezzi del corrispondente rumore di Pauli (0.237 contro 0.262, ossia circa il 9.5% in meno), ma dopo la decodifica MWPM l'errore logico risulta *maggiore* (0.0789 contro 0.0767). L'aumento relativo della metrica decodificata è circa il 2.9%: il segno mostra che i due canali non sono intercambiabili, mentre l'entità resta molto inferiore al divario dell'esempio a singolo qubit.

Una spiegazione compatibile con il risultato è nel protocollo stesso. L'estrazione della sindrome entangla ripetutamente i qubit dati con le ancille e proietta settori di sindrome a ogni ciclo, interrompendo parte dell'accumulo coerente osservato sul qubit isolato. Questo effetto è talvolta descritto come una *stochasticizzazione* parziale, ma non equivale in generale a un twirling Pauli completo. I dati mostrano che, in questo circuito, l'approssimazione di Pauli è meno distante dal canale coerente di quanto suggerisca l'esempio senza misure; non identificano da soli un meccanismo universale.

Verifica sul decoder appreso. Se le misure di sindrome riducono gran parte della differenza osservabile fra i due canali, ci si attende che il vantaggio del modello appreso non cambi molto passando dal rumore di Pauli a quello coerente. Il decoder ibrido della Sezione 9.6 è stato eseguito su entrambi i tipi di rumore: il guadagno rispetto al MWPM è 1.019, 1.036 e 1.041 sotto errori di Pauli contro 1.029, 1.048 e 1.052 sotto errori coerenti, ai tre livelli $p = 1, 2, 3 \times 10^{-2}$.

Lo scarto è di circa un punto percentuale, dell'ordine di una deviazione standard sulla stima di p_L ($\sigma \approx 9.6 \times 10^{-4}$ su 7.5×10^4 campioni di test): **non significativo**.

Va però registrato che il segno è lo stesso in tutti e tre i punti, il che è compatibile con un effetto reale ma di entità pari a circa un decimo di quello che l'intuizione “il rumore coerente è più lontano dal modello di Pauli, quindi la rete deve renderci di più” lascerebbe attendere. La cecità del matching che l'apprendimento sfrutta non è dunque genericamente il “non essere Pauli”: una sovrarotazione R_X su un singolo qubit dato, quando produce un flip, accende gli stessi stabilizzatori di un errore X ed è rappresentabile come arco del grafo esattamente come quello. Ciò che il matching non può rappresentare, e che l'apprendimento recupera, è la *non decomponibilità* — un meccanismo che accende più di due detector, come il crosstalk fra qubit dati adiacenti della Sezione 9.6. È una precisazione del criterio, non un'eccezione ad esso.

Per questa tesi la conseguenza è triplice e va dichiarata: i valori di soglia e di p_L qui riportati sono una **baseline entro il modello Pauli**, non una previsione calibrata di hardware; nel controllo coerente a distanza 3 lo scarto relativo sulla metrica decodificata è circa il 2.9%, ma un singolo punto non autorizza a fissarne segno o entità ad altre distanze; il raffinamento verso canali fisici più ricchi è quindi una naturale continuazione del lavoro. Infine, nei punti esaminati la decodifica appresa non mostra un vantaggio statisticamente risolto specifico per l'errore coerente.

9.6 La Decodifica Appresa: Sintesi della Campagna

La sezione precedente ha mostrato che il modello di rumore usato per *simulare* il codice è un'idealizzazione. Esiste però una seconda idealizzazione, indipendente dalla prima e altrettanto conseguente, che riguarda il modello usato per *decodificare*. Il MWPM non è un algoritmo generico: presuppone che ogni meccanismo d'errore accenda al più due detection events e sia quindi rappresentabile come un arco del grafo. Un errore correlato che colpisca due qubit dati adiacenti ne accende fino a quattro e non ammette alcun arco che lo rappresenti — non viola l'ipotesi di Pauli, che resta soddisfatta, ma quella di **decomponibilità**.

Da qui la domanda che chiude il cerchio metodologico della tesi: *esiste un regime in cui un decoder appreso dai dati, che non riceve un grafo esplicito degli errori ma stima la regola decisionale dai campioni, fa meglio di quello analitico?* Il Capitolo 7 ha stabilito che l'apprendimento applicato *a valle* dell'esecuzione non aggiunge nulla; qui il medesimo strumento — un percettrone multistrato di **scikit-learn** — viene applicato alla **sindrome**, che conserva relazioni spaziali e temporali non rappresentate dal solo istogramma finale. Cambia il punto di applicazione, non la tecnologia: è ciò che permette di attribuire l'esito al *dove* e non al *come*.

La campagna consta di sette esperimenti, riportati per esteso nella Sezione E.1 dell'Appendice E. Se ne riassumono qui gli esiti e il criterio che ne discende.

	domanda	esito
E1	sostituire il matching con una rete?	No : nessuna vittoria su 14 configurazioni; pareggio a $d = 3$, sconfitta a $d = 5$
E1b	quanti dati servirebbero?	10^5 – 10^6 campioni solo per pareggiare a $d = 3$
E2	e se il rumore esce dal modello del decoder?	la rete vince a $d = 3$ (1.14 – $1.19\times$)
E3	sa prevedere <i>quando</i> il matching sbaglia?	sì, $AUC = 0.941$
E4	affiancarlo invece di sostituirlo?	$-18/ - 21\%$ a $d = 3$, $-1/ - 3\%$ a $d = 5$, nullo a $d = 7$
E5	è trasferibile fra profili di rumore?	sì, 12 trasferimenti su 12 battono il matching
E6	un modello <i>mal calibrato</i> apre un margine?	No : 0 su 12; il matching perde solo l'1–3%
E7	un decoder analitico migliore lo cattura già?	a $d = 5$ sì, e meglio della rete

Tabella 9.3: Sintesi dei sette esperimenti sulla decodifica appresa. Il dettaglio, con tabelle, figure e test di significatività, è nella Sezione E.1.

Che cosa se ne ricava. Il risultato centrale è che **sostituire** il decoder analitico non conviene, mentre **affiancarlo** può convenire nel perimetro testato. Il decoder ibrido, in cui la rete apprende unicamente *quando ribaltare* la decisione del matching, riduce l'errore logico fino al 21% in presenza di errori correlati e, entro la risoluzione degli esperimenti, non lo peggiora quando il rumore è quello previsto.

Due controlli successivi ne delimitano però la portata, e vanno enunciati senza attenuazioni. Il primo (E6) esclude che il vantaggio nasca dal fornire al matching un modello esatto che l'hardware non offre: perturbando il modello fino a un ordine di grandezza l'errore logico varia dell'1–3%, e la rete non vince mai. Il secondo (E7) confronta con BP+OSD, decoder analitico strettamente più espressivo: a $d = 5$ esso guadagna il 17–27% dove l'ibrido si ferma all'1–3%.

Il quadro che ne esce non è però una sconfitta dell'apprendimento, bensì una precisazione su *quale* informazione esso sfrutti. Il guadagno di BP+OSD **decresce** al crescere del rumore correlato, quello dell'ibrido **cresce**: i due metodi agiscono su assi ortogonali, perché il primo sfrutta meglio il modello che riceve e il secondo cattura ciò che al modello sfugge.

Sarebbe naturale inferirne che i due guadagni si *sommino*, e che costruire il residuo appreso sopra BP+OSD anziché sopra il matching li cumuli. Due ulteriori esperimenti mostrano che l’inferenza è sbagliata, e insieme spiegano perché. Il primo (E8) realizza la costruzione: il decoder combinato eguaglia sempre il migliore dei due singoli e mai il loro prodotto, e sopra BP+OSD il residuo migliora in modo significativo in una sola configurazione su dieci. Il secondo (E10) ne dà la ragione, misurando sei decoder sui medesimi campioni: in presenza di rumore correlato il matching col residuo, BP+OSD col residuo e la rete usata *da sola*, senza alcun decoder analitico sotto, atterrano tutti entro l’1–2% sullo stesso errore logico, mentre i decoder analitici da cui provengono restano il 16–18% sopra; in assenza di rumore correlato quel pavimento comune non esiste e i metodi si ordinano per qualità intrinseca. Un terzo controllo (E11) esclude che si tratti del tetto dell’architettura scelta: una rete con quaranta volte i parametri atterra sullo stesso valore entro lo 0.8%, e un decoder che memorizzi le sindromi anziché stimarne un modello fa peggio. **Il limite non appartiene dunque a nessuno dei due metodi**, e impilarli non porta al di sotto di esso.

Il criterio, nella sua forma definitiva. Ne discende l’enunciato che regge l’intero lavoro, ora verificato su due impieghi opposti dello stesso strumento: *un modello appreso produce valore quando la classe di errore non è rappresentabile nella struttura del metodo analitico — non quando è rappresentata con parametri imprecisi*. La cecità che conta è strutturale, non di calibrazione. E poiché il vantaggio si annulla a $d = 7$, dove la discriminazione della rete densa sui 336 detector resta troppo bassa perché ribaltare la predizione del matching risulti in attivo — e ciò accade anche quando gli esempi di addestramento abbondano — la nicchia in cui l’apprendimento è la scelta giusta risulta reale ma stretta: il primo controllo da eseguire, prima di introdurre un modello appreso, è verificare di avere già adottato il miglior metodo analitico disponibile.

Capitolo 10

Analisi di Sensibilità: Shor sotto un Proxy di Errore per Gate

I Capitoli 8 e 9 studiano la correzione d'errore su circuiti dedicati, mentre il Capitolo 7 misura la degradazione di Shor sotto un modello di rumore fisico. Collegare direttamente i due insiemi di risultati richiederebbe una compilazione fault-tolerant completa: codifica dei registri, estrazione ripetuta della sindrome, decoder, operazioni logiche e distillazione delle risorse non-Clifford. Questa tesi non simula tale architettura. Il presente capitolo svolge invece un esperimento di **sensibilità fenomenologica**: applica al circuito compilato un errore Pauli indipendente per gate, indicato con p_L , e misura come varia la probabilità di fattorizzazione. Il pedice “L” identifica il ruolo concettuale del parametro, ma non autorizza a sostituirvi direttamente le metriche logiche aggregate dei capitoli precedenti.

10.1 L'architettura a quattro livelli

La separazione adottata evita di attribuire a una simulazione ridotta più informazione di quella che contiene.

Livello	Oggetto	Strumento	Domanda
1	Shor ideale	Qiskit Aer	limite senza rumore
2	Shor rumoroso	Aer + noise model	sensibilità agli errori fisici
3	QEC isolata	Qiskit; Stim/PyMatching	soppressione dell'errore
4	proxy per gate	Aer + canale Pauli	sensibilità a un residuo uniforme

Tabella 10.1: I quattro livelli isolano domande diverse. Il quarto non è una simulazione fault-tolerant e non riceve automaticamente come input i tassi aggregati misurati al terzo livello.

Una simulazione esplicita incontra due barriere. La prima è dimensionale: sostituire ciascuno dei 12 qubit dell'istanza $N = 15$ con un blocco di codice e includere ancilla e cicli di sindrome porta rapidamente a centinaia di qubit. La seconda è strutturale: Stim rimane efficiente per circuiti stabilizer, mentre l'aritmetica modulare e le rotazioni della QFT rendono Shor non-Clifford. Aer può simulare il circuito generale piccolo; Stim/PyMatching può caratterizzare circuiti QEC stabilizer. Nessuno dei due risultati, da solo, equivale all'esecuzione fault-tolerant completa.

10.2 Definizione operativa del proxy

In M8, p_L è definito come la **probabilità totale di applicare un Pauli non identità dopo ciascun gate compilato incluso nel modello**. Per un gate su q qubit il canale distribuisce p_L uniformemente sui $4^q - 1$ Pauli non identità. Poiché Qiskit Aer parametrizza il canale depolarizzante con λ_q , lo script usa

$$\lambda_1 = \frac{4}{3}p_L, \quad \lambda_2 = \frac{16}{15}p_L. \quad (10.1)$$

Il rumore è applicato a **sx/x** e **cx**; le rotazioni **rz** sono considerate virtuali. Si tratta quindi di un proxy uniforme *per gate nella base compilata*, non di una probabilità per ciclo di correzione, per round di sindrome o per intero blocco logico.

Il circuito è quello canonico $N = 15$, $a = 7$, con 8 qubit di conteggio e 4 di lavoro. La transpilazione deterministica usa la base **rz/sx/x/cx**, livello di ottimizzazione 2 e seed 20260819; il manifest registra 224 **cx**, 302 **rz**, 70 **sx**, 8 misure e lo SHA-256 **c7f4cf3b...a50f4ba2c**. Ogni punto M8 aggrega 20 repliche indipendenti da 4096 shot, per un totale di 81 920 misure. Il successo per misura vale uno se le frazioni continue e i controlli con il massimo comun divisore restituiscono una coppia di fattori non banali.

Questa definizione rende riproducibile la domanda sperimentale:

come cambia il successo di questa specifica implementazione di Shor al crescere di un errore Pauli indipendente e uniforme per gate?

Non consente invece di rispondere, senza un ulteriore modello architetturale, a quale distanza di surface code corrisponda un dato punto della curva.

10.3 Il contesto delle stime fault-tolerant

Le stime per istanze crittografiche comprendono routing, cicli di sindrome, fabbriche di stati magici e schedulazione. Per esempio, Gidney ed Ekerå stimano circa 20 milioni di qubit fisici rumorosi e otto ore per RSA-2048 sotto un insieme esplicito di assunzioni architetturali [5]. Questi numeri non si ricavano estrapolando $N = 15$: il circuito didattico e una stima di risorse fault-tolerant descrivono oggetti differenti. La curva seguente va dunque letta come analisi locale di sensibilità, non come previsione di risorse hardware.

10.4 Risultati M8 e M13

M8: successo per singola misura

La Tabella seguente riporta l'intera griglia preregistrata. Gli intervalli sono Wilson al 95% sui conteggi aggregati; il controllo di monotonia usa inoltre il massimo tra l'errore standard binomiale e quello tra repliche e applica una copertura simultanea di Bonferroni ai confronti adiacenti.

p_L	P_{success}	IC 95%	p_L	P_{success}	IC 95%
0	0,7489	[0,7459; 0,7518]	0,005	0,6450	[0,6417; 0,6483]
0,0001	0,7469	[0,7439; 0,7498]	0,01	0,5651	[0,5617; 0,5685]
0,0005	0,7373	[0,7343; 0,7403]	0,02	0,4518	[0,4484; 0,4552]
0,001	0,7250	[0,7219; 0,7280]	0,05	0,2953	[0,2921; 0,2984]
0,0017	0,7083	[0,7051; 0,7114]	0,1	0,2503	[0,2474; 0,2533]
0,002	0,6992	[0,6961; 0,7024]	0,2	0,2441	[0,2412; 0,2471]
			0,5	0,2459	[0,2430; 0,2489]

Tabella 10.2: M8: probabilità di ricavare i fattori per singola misura, 20 repliche da 4096 shot per punto.

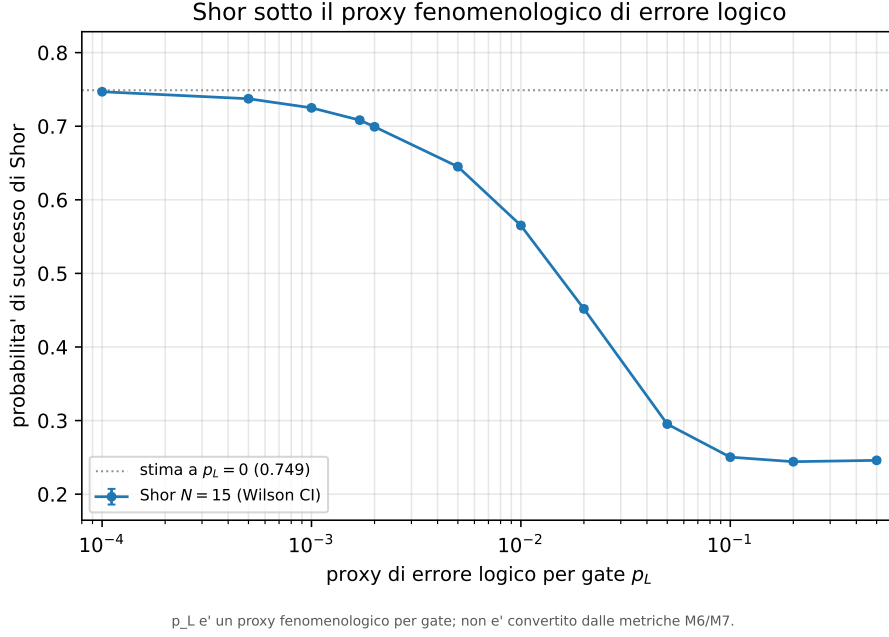


Figura 10.1: Probabilità di successo di Shor ($N = 15$, $a = 7$) in funzione di p_L . I punti positivi sono mostrati in scala logaritmica con IC Wilson al 95%; la linea orizzontale riporta la stima separata a $p_L = 0$. Non sono sovrapposti marcatori Steane o surface code, perché le rispettive metriche aggregate non sono direttamente convertibili nel proxy per gate M8. Figura rigenerata dal JSON schema 2 con `figure_src/gen_shor_logico.py`.

La curva è compatibile con una funzione non crescente al livello simultaneo prefissato. Non tutte le diminuzioni adiacenti sono però individualmente confermate: la differenza tra $p_L = 0$ e 10^{-4} è piccola e, sul plateau, le stime a 0,2 e 0,5 si sovrappongono. L'affermazione supportata è quindi “nessun aumento significativo”, non “ogni passo produce una diminuzione significativa”. Tra $p_L = 0$ e 0,02 il successo passa da 0,7489 a 0,4518; oltre $p_L = 0,1$ raggiunge un plateau empirico vicino a 0,245. Tale valore nasce dalla struttura degli esiti che il post-processing di $N = 15$ riconosce come utili e non va assunto a priori come universale.

Ripetizioni indipendenti

Il valore 0,7489 a $p_L = 0$ è una probabilità per singola misura, non un limite alla probabilità di concludere la fattorizzazione. Se ogni tentativo è indipendente e ha probabilità P_s , dopo k tentativi vale

$$P(k) = 1 - (1 - P_s)^k. \quad (10.2)$$

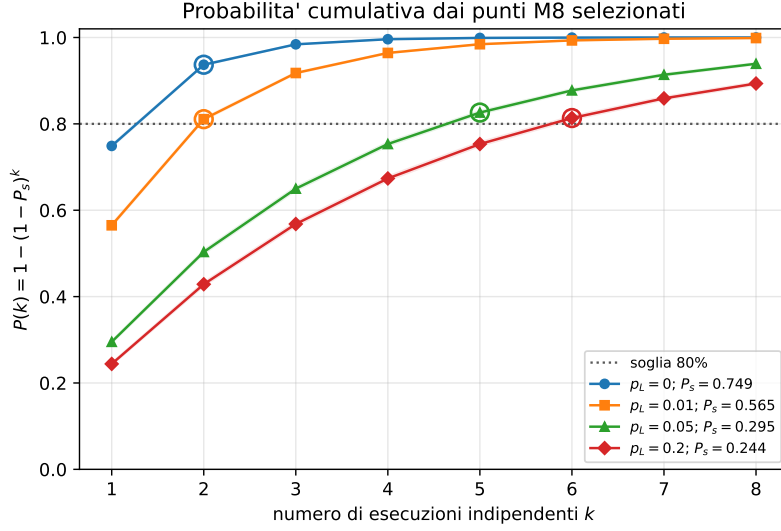


Figura 10.2: Probabilità cumulativa ottenuta esclusivamente dai punti M8 simulati $p_L \in \{0; 0,01; 0,05; 0,2\}$. La soglia dell'80% richiede rispettivamente 2, 2, 5 e 6 tentativi indipendenti. Le bande propagano gli estremi degli IC Wilson.

In particolare, due tentativi portano la stima a $1 - (1 - 0,7489)^2 \simeq 0,937$ nel caso ideale e a circa 0,811 per $p_L = 0,01$. La soglia dell'80% non può essere usata come requisito sul singolo run di questa istanza, poiché nemmeno il punto senza rumore la raggiunge; ha invece un significato operativo come soglia cumulativa.

M13: TOP-4 con pareggi espliciti

M13 completa la lettura di M8 separando tre quantità sullo stesso istogramma: successo per misura, TOP-1 e TOP-4. Per ogni valore di p_L sono state eseguite 200 repliche indipendenti da 1024 shot. Il TOP-4 rimane pari a 1,000 dal caso ideale fino a $p_L = 0,05$, mentre la probabilità per singola misura scende nello stesso intervallo da 0,7512 a 0,2979. Questo conferma che, sull'istanza piccola $N = 15$, selezionare quattro candidati può recuperare un fattore anche quando la massa utile per singolo shot è già molto degradata.

La saturazione non si estende però all'intera griglia. A $p_L = 0,1$ il TOP-4 seeded scende a 0,915, con intervallo identificato $[0,855; 0,940]$ dovuto ai pareggi; a $p_L = 0,2$ scende a 0,670, con intervallo $[0,535; 0,860]$. Il contrasto primario preregistrato fra $p_L = 0$ e $p_L = 0,2$ produce una diminuzione stimata di 0,330 con IC Newcombe 95% $[0,266; 0,398]$. Poiché il limite inferiore supera il margine assoluto 0,10, l'esito primario è una diminuzione discriminante, non equivalenza. L'inviluppo simultaneo

robusto ai pareggi $[0, 059; 0, 553]$ conferma inoltre che non si può sostenere equivalenza per tutte le risoluzioni ammissibili dei pareggi.

Barriera di scalabilità

M8 e M13 sono limitati a $N = 15$. La variante Beauregard validata produce per $N = 21$ un circuito ideale corretto, ma la compilazione nella stessa base raggiunge 21 036 gate cx e profondità 23 081 su 22 qubit. L'esecuzione rumorosa generale non è quindi commensurabile con la campagna M8 su hardware classico disponibile. Il risultato non dimostra un limite dell'algoritmo quantistico: documenta il limite della simulazione classica adottata.

Perché le metriche QEC non sono proiettate sulla curva

La precedente versione del capitolo interpolava sulla curva M8 valori provenienti da Steane, surface code e decoder appreso. Quell'operazione confonde unità statistiche diverse. La Tabella 10.3 rende esplicito il contratto corretto.

Grandezza	Unità/contesto	Inseribile direttamente in M8?
p_L di M8	Pauli non identità dopo ogni sx/x/cx compilato	sì, per definizione
errore Steane	fallimento del protocollo protetto sotto il modello dedicato	no
errore surface code	fallimento logico per memoria/ciclo e distanza	no
guadagno del decoder	differenza osservata sotto rumore correlato del benchmark	no

Tabella 10.3: Le metriche QEC richiedono un modello di conversione che specifichi circuito logico, numero e durata dei cicli, operazioni fault-tolerant e correlazioni residue. In sua assenza, sovrapporre i valori sulla curva produrrebbe una precisione non giustificata.

10.4.1 Discussione

M8 stabilisce una relazione quantitativa robusta all'interno del proprio modello: ridurre la probabilità Pauli per gate riporta il successo verso il limite ideale dell'istanza. Non stabilisce quale codice raggiunga uno specifico p_L per gate, né confronta Steane e surface code a parità di hardware. Per compiere quel passo servirebbe almeno

una compilazione in un gate set logico, il conteggio dei cicli di correzione associati a ciascuna operazione, un decoder con latenze e correlazioni esplicite e un modello delle operazioni non-Clifford.

Il risultato chiarisce anche il ruolo del post-processing. Il successo per misura degrada in modo netto lungo la curva; una strategia TOP-K può invece saturare su questa piccola istanza perché quattro candidati coprono rapidamente almeno un esito utile. Una percentuale TOP-4 elevata non prova dunque che l'informazione quantistica sia rimasta intatta: deve essere letta insieme alla massa utile per misura e ai limiti dovuti ai pareggi.

Infine, $N = 15$ resta un'istanza didattica. Le conclusioni supportate riguardano la sensibilità del circuito corretto e riproducibile a canali d'errore controllati. La fattibilità di Shor su scala crittografica rimane una domanda architetturale distinta, dominata dall'overhead fault-tolerant e non risolta da questa simulazione.

Capitolo 11

Sviluppi Futuri

I risultati delimitano con chiarezza ciò che la demo e la campagna dimostrano: l'esecuzione ideale è verificata anche per $N = 21/35$, mentre il confronto controllato fra canali di rumore è sostenibile su $N = 15$. Le estensioni seguenti mirano a colmare questa distanza senza estrapolare i risultati oltre il loro perimetro.

11.1 Scalabilità a Istanze di Maggiore Dimensione

La priorità è ridurre il costo dell'aritmetica modulare. La costruzione Beauregard implementata nella tesi è stata validata idealmente mediante truth table e distribuzione QPE, ma per $N = 21$, $a = 2$ e otto controlli il circuito esecutivo compilato nella base `rz/sx/x/cx` contiene 21 036 CX e ha profondità 23 081. Questi sono conteggi della compilazione congiunta, non somme di blocchi né stime su un gate set più astratto.

Le direzioni immediate sono l'approximate QFT con soglia d'angolo esplicita, addizionatori alternativi, sintesi mirata al gate set del backend e riduzione delle operazioni controllate prima della traspilazione. Ogni variante dovrebbe superare gli stessi controlli già usati qui: verità funzionale della moltiplicazione, ancille pulite, coerenza di fase e distanza della QPE ideale dalla legge teorica. Il numero di gate, da solo, non è una prova di correttezza.

Anche la scelta delle istanze deve essere governata dall'ordine moltiplicativo r , non soltanto dal modulo N . L'istanza $N = 35$, $a = 6$ ha ordine 2 e non costituisce una progressione monotona rispetto a $N = 21$, $a = 2$, che ha ordine 6. Una campagna di scalabilità dovrebbe quindi stratificare sia la larghezza sia l'ordine e dichiarare a priori quale delle due variabili intende studiare.

11.2 Validazione su Hardware Quantistico Reale

Il modello rumoroso della tesi è uniforme e illustrativo: non include coupling map, routing, crosstalk, leakage o deriva della calibrazione. Il passo successivo è ripetere il protocollo $N = 15$ su una QPU, registrando backend, timestamp, proprietà dei qubit, layout iniziale e finale, conteggi post-routing e job identifier. Il confronto deve usare lo stesso budget di shot e una politica di seed dichiarata, senza presentare il preset uniforme come replica della macchina.

La demo è già predisposta per questo confronto metodologico: distingue rumore disattivato, scenario moderato, scenario di stress e canali singoli. Un'estensione hardware dovrebbe affiancare una quinta modalità “calibrazione acquisita”, costruita dallo snapshot reale e marcata con la sua data di validità. In questo modo diventerebbe misurabile il *simulation gap* fra il modello controllato e il dispositivo.

Per $N = 21/35$ l'hardware non elimina il problema della profondità: evita il costo esponenziale dello statevector classico, ma non le migliaia di operazioni rumorose e i vincoli di connettività. Prima di eseguire queste istanze servono quindi una sintesi più corta e un budget d'errore basato sul circuito effettivamente instradato.

11.3 Generalizzazione ad Altri Algoritmi Quantistici

TOP-K è utile soltanto quando più candidati possono essere verificati classicamente a costo contenuto. Questa proprietà vale per Shor, perché ogni candidato al periodo produce o meno divisori verificabili, e può valere per problemi di ricerca con un oracolo di verifica. Non segue automaticamente che la stessa strategia migliori VQE o QAOA, dove l'output centrale è una stima di valore atteso e non una lista di soluzioni equivalenti.

Una generalizzazione corretta dovrebbe quindi definire prima:

1. una funzione classica di validazione del candidato;
2. il costo aggiuntivo delle K verifiche;
3. una baseline TOP-1 appaiata sugli stessi campioni;
4. un criterio che impedisca a K di rendere il successo quasi certo per puro effetto combinatorio.

Grover su problemi con verifica economica è un primo banco di prova; per gli algoritmi variazionali è invece più naturale studiare allocazione adattiva degli shot o mitigazione del bias dell'osservabile.

11.4 Integrazione con la Quantum Error Correction

La campagna di Shor logico usa p_L come probabilità fenomenologica di Pauli non-identità per gate compilato incluso nel proxy. Non identifica questo parametro con il logical error rate di un memory experiment surface-code: le due grandezze hanno circuiti, osservabili e granularità differenti. Il passo successivo scientificamente significativo è una mappatura esplicita fra un set di operazioni logiche fault-tolerant e i canali residui stimati per ciascuna operazione.

Solo dopo tale mappatura sarebbe lecito combinare surface code e Shor in una previsione end-to-end. Servirebbero almeno: conteggio delle operazioni logiche per tipo, costo di magic-state distillation per le non-Clifford, decoder e latenza di feed-forward, errori correlati e intervalli di confidenza propagati. Il modello fenomenologico della tesi resta utile come analisi di sensibilità, non come stima di risorse hardware.

Per la decodifica appresa, le direzioni più promettenti restano modelli a grafo o convoluzionali che rispettino la località del reticolo, uso della storia temporale dei cicli e soft information analogica della misura. Queste estensioni aggiungono informazione o struttura; non si limitano a ingrandire una rete densa sullo stesso vettore di sindrome.

11.5 Evoluzione del Classificatore ML

La campagna aggiornata mostra che TOP-1 è addestrabile ma operativo solo come predittore della riuscita del candidato dominante; TOP-16 produce invece 2000/2000 positivi in entrambi gli scenari. Inoltre M2 è peggiore della propria ablazione TOP-4 nei due confronti appaiati. Non è quindi giustificato migliorare soltanto l'architettura mantenendo la stessa etichetta.

Un nuovo problema supervisionato dovrebbe prevedere un obiettivo legato al costo, ad esempio il valore atteso degli shot risparmiati dopo aver considerato il costo di uno scarto errato. Training e test dovrebbero essere separati anche per giorno di calibrazione o backend, non solo per seed, e la soglia decisionale scelta su validation.

Prima di addestrare si deve infine verificare il bilanciamento della classe e confrontare il modello con regole semplici come massa sui picchi, entropia e TOP-K.

Il contributo più credibile dell'apprendimento resta comunque a monte della misura finale: sulla sindrome, dove può sfruttare correlazioni che un decoder analitico non modella, anziché tentare di recuperare informazione già persa nell'output.

Capitolo 12

Conclusioni

Questo lavoro è partito da una domanda operativa: quanto dell'output rumoroso di Shor può essere recuperato a valle, e quando diventa invece necessario correggere l'errore durante il calcolo? La risposta è una delimitazione, non una promessa generale. Su $N = 15$ una semplice ricerca multi-candidato usa meglio l'informazione già presente; un classificatore sullo stesso output non aggiunge valore. Per andare oltre occorre spostarsi sulla sindrome e sulla correzione d'errore.

12.1 Le Domande di Ricerca

DR1 — Come cambia Shor al variare dei canali di rumore? Nel modello uniforme $N=15$ la struttura QPE si attenua all'aumentare di λ_{2q} : la frazione efficace sui quattro picchi passa da 0.804 a $\lambda_{2q} = 10^{-3}$ a 0.042 a 10^{-1} . Il proxy di nessun evento Pauli sulle 224 CX decade molto più rapidamente, da 0.811 a 2.65×10^{-10} , e non predice il successo dell'algoritmo. Gli sweep di λ_{1q} , T_1/T_2 , readout e shot non mostrano un andamento monotono di M1 con 30 repliche; non è quindi lecito ordinarne causalmente l'importanza da questi soli dati. TOP-4 resta a una iterazione in tutti i punti salvo $\lambda_{2q} = 0.1$, dove sale a 1.07.

DR2 — Come si estrae una sindrome senza misurare lo stato logico? Il codice a ripetizione misura parità, non i singoli qubit. Gli stabilizzatori Z_1Z_2 e Z_2Z_3 rivelano quale bit è stato alterato senza rivelare l'ampiezza logica. La curva Monte Carlo coincide con $p_L = 3p^2 - 2p^3$ entro l'incertezza; il limite resta la protezione di un solo tipo di errore per volta.

DR3 — Quali vantaggi e limiti offre Steane $[[7, 1, 3]]$? La struttura CSS corregge un errore Pauli arbitrario su un singolo qubit e la corrispondenza sindrome-errore è stata verificata sui 21 casi. La pendenza log-log misurata, 1.94, è compatibile con la soppressione quadratica attesa da un codice di distanza 3. La distanza fissa e l'estrazione di sindrome non fault-tolerant impediscono però di interpretarlo come soluzione scalabile.

DR4 — Il surface code mostra un comportamento di soglia? Sì, nel memory experiment e nel modello circuit-level adottati. Le curve per $d = 3, 5, 7, 9$ si incrociano attorno allo 0.9% in base Z e allo 0.7% in base X; sotto la regione di incrocio aumentare la distanza sopprime il logical error rate, sopra la amplifica. Il parametro fisico p di questa simulazione non coincide con λ_{2q} della campagna Shor, quindi il confronto fra i due resta qualitativo.

DR5 — Quale errore logico è compatibile con Shor? Il modello fenomenologico mostra come la probabilità per singola misura decada al crescere della probabilità di Pauli non-identità per gate logico. A $p_L = 0$ il limite dell'istanza è circa 0.75, perché una parte degli esiti QPE è intrinsecamente non utile; ripetere l'algoritmo aumenta la probabilità cumulativa. La curva è un'analisi di sensibilità, non una previsione end-to-end: il logical error rate di un memory experiment non può essere assegnato direttamente a ogni gate di Shor senza specificare le operazioni fault-tolerant e i rispettivi canali residui.

DR6 — Quando un decoder appreso può superare quello analitico? Nei dati della tesi il vantaggio compare quando il rumore contiene correlazioni assenti dal grafo del matching. Il decoder ibrido riduce l'errore rispetto a MWPM a distanza 3, ma il beneficio si restringe con la distanza e scompare a distanza 7 per la rete densa adottata. Il risultato non sostiene la sostituzione generalizzata dei decoder analitici: indica una nicchia in cui un modello residuale può sfruttare informazione strutturale non rappresentata dal metodo base.

12.2 Verifica delle Ipotesi

Tabella 12.1: Esito dopo la rigenerazione schema 2.

Ipotesi	Esito	Evidenza
Il classificatore riduce le iterazioni	smentita	M2 non migliora M1; TOP-4 è migliore di M2 in UC1 e UC2 ($p_{\text{Holm}} = 0.0033/0.0096$).
$F1 > 0.85$ e $AUC > 0.90$	confermata, ma insufficiente	SVM: F1 0.919/0.923, AUC 0.965/0.958 sul test; la metrica misura TOP-1, non l'utilità operativa.
Il vantaggio si mantiene crescendo N	non verificata	$N = 21/35$ sono validati idealmente, ma esclusi dal rumore per costo; l'ordine r deve essere controllato insieme a N .
Soppressione quadratica nei codici di distanza 3	confermata nel modello	ripetizione coerente con $3p^2 - 2p^3$; Steane con pendenza 1.94.
Comportamento di soglia del surface code	confermato nel modello	incrocio delle curve $d = 3, 5, 7, 9$ e inversione sopra soglia.
Trasferimento diretto di p_L a Shor	non dimostrato	la curva logica è fenomenologica; manca la mappatura per operazione fault-tolerant.

L'audit del classificatore è metodologicamente centrale. TOP-1 produce classi addestrabili, ma chiede soltanto se il candidato dominante porta ai fattori. TOP-16 produce invece 2000/2000 positivi in entrambi gli scenari. Il conto combinatorio spiega il fenomeno: 63 dei 256 esiti sono utili e la probabilità di non includerne alcuno scegliendo sedici celle uniformi è appena $\binom{193}{16}/\binom{256}{16} = 0.9275\%$. Una metrica elevata non garantisce dunque che l'etichetta rappresenti un problema utile.

12.3 Contributi del Lavoro

Una demo scientificamente separata in ideale e rumoroso. Il percorso ideale espone circuito, stato e QPE senza rumore; il percorso rumoroso rende attivabili separatamente depolarizzazione 1Q/2Q, T_1/T_2 , readout e perturbazione coerente. $N=21/35$ sono validati idealmente, mentre l'esperimento pubblico rumoroso è limitato a $N=15$.

Un'ablazione causalmente interpretabile. M1, TOP-4 e M2 ricevono lo stesso istogramma. TOP-4 riduce $\bar{M}$ da 1.37 a 1.00 nello scenario moderato e da

1.50 a 1.00 nello stress; il classificatore peggiora la propria ablazione. Seed, tie-break, sentinelle, revisioni e manifest sono salvati negli artefatti.

Una validazione funzionale dell'aritmetica. Il moltiplicatore $\times 7 \bmod 15$ è coperto da truth table e il percorso Beauregard da proprietà esaustive e confronti QPE per $N=21/35$. Questo controllo ha mostrato perché i soli picchi o il solo ordine non bastano a validare l'operatore implementato.

Una progressione QEC con limiti dichiarati. Ripetizione, Steane, surface code e decodifica appresa sono collegati da metriche esplicite, ma i rispettivi modelli di rumore non vengono presentati come direttamente commensurabili. Lo Shor logico chiude la progressione come analisi fenomenologica di sensibilità.

12.4 Limiti

La campagna Shor usa un simulatore e un rumore uniforme, indipendente e stazionario; non include topologia, routing, crosstalk, leakage o deriva di calibrazione. $N=15$ è un'istanza didattica e TOP-K beneficia di una post-elaborazione particolarmente permissiva. Gli sweep hanno 30 repliche e i test non sono corretti globalmente fra le otto famiglie. $N=21/35$ non sono simulati sotto rumore.

Nella parte QEC, Steane non usa un'estrazione fault-tolerant; i memory experiment e il modello per-gate di Shor non condividono la stessa definizione di p_L ; il decoder appreso è una rete densa e il vantaggio non persiste alle distanze maggiori testate. Le stime di risorse crittografiche devono quindi provenire da architetture fault-tolerant complete, non da un'estrapolazione delle curve $N=15$.

12.5 Il Criterio che Unifica il Lavoro

In entrambe le fasi, il complemento semplice è utile quando sfrutta informazione che il metodo base lascia inutilizzata senza modificare il processo che la genera. TOP-4 prova altri candidati già presenti nell'istogramma; il decoder residuale interviene soltanto sui casi in cui il matching ha una debolezza rappresentazionale. Un modello più complesso non produce valore se replica una regola semplice, se le etichette sono degeneri o se il costo introdotto compensa il beneficio.

Applicato a Shor, questo criterio separa nettamente post-processing e correzione: TOP-K può recuperare un picco utile già misurato, ma non può ricostruire informazione fisicamente distrutta. Rendere Shor scalabile richiede operazioni logiche

fault-tolerant; l'apprendimento può contribuire alla decodifica solo dove dispone di una struttura che il decoder analitico non rappresenta.

Appendice A

Trattazione Teorica Estesa

A.1 Fondamenti del Calcolo Quantistico

A.1.1 Dalla Meccanica Classica alla Meccanica Quantistica

La meccanica quantistica descrive il comportamento della materia e dell'energia alle scale atomiche e subatomiche, dove i principi della fisica classica cessano di essere validi. I suoi postulati, formulati nella prima metà del Novecento da Heisenberg, Schrödinger, Dirac e Bohr, introducono concetti radicalmente diversi da quelli classici: lo stato di un sistema fisico non è determinato in modo assoluto, ma è descritto da una **funzione d'onda** la cui evoluzione è governata dall'equazione di Schrödinger.

Il calcolo quantistico traduce questi principi fisici in risorse computazionali. La **sovrapposizione** consente a un bit quantistico di trovarsi in una combinazione lineare di 0 e 1 simultaneamente; l'**entanglement** crea correlazioni non classiche tra qubit distinti; l'**interferenza** permette di amplificare le soluzioni corrette e cancellare quelle errate.

A.1.2 I Postulati della Meccanica Quantistica

Il formalismo matematico della meccanica quantistica poggia su quattro postulati fondamentali, che costituiscono l'assiomatica della teoria e il punto di partenza rigoroso per la descrizione di qualsiasi sistema quantistico [6].

1. **Postulato dello spazio degli stati.** Ad ogni sistema fisico isolato è associato uno **spazio di Hilbert** $\mathcal{H}$, detto *spazio degli stati* del sistema. Lo stato del sistema è completamente descritto da un vettore unitario $|\psi\rangle \in \mathcal{H}$, detto *vettore di stato* o *ket*. Per un sistema a due livelli (qubit), $\mathcal{H} = \mathbb{C}^2$ e lo stato generale è $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ con $|\alpha|^2 + |\beta|^2 = 1$.
2. **Postulato dell'evoluzione.** L'evoluzione di un sistema quantistico chiuso è descritta da una **trasformazione unitaria**: se lo stato del sistema all'istante t_1 è $|\psi(t_1)\rangle$, allora all'istante t_2 esso si trova nello stato

$$|\psi(t_2)\rangle = U(t_1, t_2) |\psi(t_1)\rangle \quad (\text{A.1})$$

dove $U(t_1, t_2)$ è un operatore unitario ($U^\dagger U = I$) che dipende dagli istanti t_1 e t_2 . In forma differenziale, l'evoluzione è governata dall'**equazione di Schrödinger**:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle \quad (\text{A.2})$$

dove $\hat{H}$ è l'**Hamiltoniano** del sistema — un operatore hermitiano ($\hat{H} = \hat{H}^\dagger$) che rappresenta l'energia totale — e $\hbar = h/(2\pi)$ è la costante di Planck ridotta.¹

3. **Postulato della misura.** Le misure quantistiche sono descritte da un insieme $\{M_m\}$ di **operatori di misura**, indicizzati dagli esiti m , soggetti alla relazione di completezza:

$$\sum_m M_m^\dagger M_m = I \quad (\text{A.3})$$

Se il sistema si trova nello stato $|\psi\rangle$ prima della misura, la probabilità di ottenere l'esito m è:

$$p(m) = \langle \psi | M_m^\dagger M_m | \psi \rangle \quad (\text{A.4})$$

¹Nel calcolo quantistico a porte, ogni porta è la realizzazione discreta di questo postulato: la matrice unitaria U descrive l'evoluzione del registro di qubit in un passo computazionale. L'Hamiltoniano fisico che implementa una porta specifica dipende dall'architettura hardware (impulsi a microonde per qubit superconduttori, impulsi laser per ioni intrappolati, ecc.).

e lo stato immediatamente dopo la misura con esito m è il vettore normalizzato:

$$|\psi'\rangle = \frac{M_m |\psi\rangle}{\sqrt{p(m)}} \quad (\text{A.5})$$

Nel caso della **misura proiettiva in base computazionale** — il caso standard nel calcolo quantistico — gli operatori sono i proiettori $M_m = |m\rangle \langle m|$, e la probabilità di ottenere m si riduce alla **regola di Born**: $p(m) = |\langle m|\psi\rangle|^2$. La misura è un'operazione *non unitaria* e *irreversibile*: il collasso dello stato distrugge irrecuperabilmente le ampiezze che non corrispondono all'esito osservato.

4. **Postulato dei sistemi composti.** Lo spazio degli stati di un sistema fisico composto da n sottosistemi con spazi $\mathcal{H}_1, \dots, \mathcal{H}_n$ è il **prodotto tensoriale**:

$$\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \dots \otimes \mathcal{H}_n \quad (\text{A.6})$$

Se ciascun sottosistema i si trova nello stato $|\psi_i\rangle$, lo stato del sistema composto è $|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle$. Esistono tuttavia stati del sistema composto — gli stati **entangled** — che non possono essere scritti in questa forma fattorizzata per nessuna scelta degli stati individuali: la misura di un sottosistema modifica istantaneamente le probabilità degli esiti di misura sull'altro, indipendentemente dalla distanza fisica tra i due.

Questi quattro postulati forniscono il quadro assiomatico completo entro cui si collocano tutti i concetti che seguono: il qubit (Postulato 1), le porte quantistiche come trasformazioni unitarie (Postulato 2), la misura e il collasso dello stato (Postulato 3), i registri multi-qubit e l'entanglement (Postulato 4).

A.1.3 Il Qubit

Definizione Matematica

L'unità fondamentale del calcolo quantistico è il **qubit** (*quantum bit*). Mentre un bit classico assume uno dei due valori $\{0, 1\}$, un qubit è descritto da un vettore unitario nello spazio di Hilbert bidimensionale $\mathcal{H} = \mathbb{C}^2$.²

Introducendo la **notazione di Dirac**³, i due stati base computazionali sono:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (\text{A.7})$$

Lo stato generico di un qubit è la **sovrapposizione**:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (\text{A.8})$$

dove $|\alpha|^2$ e $|\beta|^2$ rappresentano le probabilità di ottenere rispettivamente 0 e 1 al momento della misura. Il vincolo di normalizzazione $|\alpha|^2 + |\beta|^2 = 1$ garantisce che le probabilità sommino a 1.

La Sfera di Bloch

Ogni stato puro di un qubit può essere visualizzato come un punto sulla superficie della **sfera di Bloch**, una sfera unitaria in $\mathbb{R}^3$.⁴ Parametrizzando le ampiezze complesse tramite angoli reali $\theta \in [0, \pi]$ e $\phi \in [0, 2\pi]$:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle \quad (\text{A.9})$$

I poli nord e sud corrispondono agli stati $|0\rangle$ e $|1\rangle$, mentre i punti sull'equatore rappresentano sovrapposizioni equiprobabili con fasi diverse.

²Lo **spazio di Hilbert** prende il nome dal matematico tedesco David Hilbert (1862–1943), uno dei più influenti matematici del XX secolo. In meccanica quantistica indica uno spazio vettoriale complesso dotato di prodotto interno, che fornisce il formalismo matematico per descrivere gli stati di un sistema fisico.

³La **notazione bra-ket** fu introdotta dal fisico britannico Paul Adrien Maurice Dirac (1902–1984), Premio Nobel per la Fisica nel 1933. Il simbolo $|\psi\rangle$ è detto *ket*, mentre $\langle\psi|$ è detto *bra*; il prodotto interno $\langle\phi|\psi\rangle$ forma il *bracket*, da cui il nome.

⁴La **sfera di Bloch** deve il suo nome al fisico svizzero-americano Felix Bloch (1905–1983), Premio Nobel per la Fisica nel 1952 (insieme a Edward Purcell) per lo sviluppo della spettroscopia a risonanza magnetica nucleare (NMR). La stessa notazione T_1 e T_2 per i tempi di decoerenza proviene dal formalismo NMR di Bloch.

La figura corrispondente è riportata nel corpo (Figura 3.1).

A.1.4 Sistemi Multi-Qubit e Prodotto Tensoriale

Un sistema di n qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H}^{\otimes n} = \mathbb{C}^{2^n}$, ottenuto come prodotto tensoriale degli spazi individuali. Uno stato generico di n qubit è:

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1 \quad (\text{A.10})$$

La dimensione esponenziale 2^n dello spazio di Hilbert è la fonte del potere computazionale dei sistemi quantistici: con $n = 300$ qubit, lo spazio degli stati ha dimensione 2^{300} , superiore al numero di atomi nell'universo osservabile.

Entanglement

Due qubit sono detti **entangled** quando il loro stato congiunto non può essere scritto come prodotto tensoriale degli stati individuali. Gli stati di Bell costituiscono la base canonica degli stati entangled a due qubit:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (\text{A.11})$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \quad (\text{A.12})$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \quad (\text{A.13})$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \quad (\text{A.14})$$

Lo stato $|\Phi^+\rangle$ non può essere fattorizzato come $|\psi_A\rangle \otimes |\psi_B\rangle$ per nessuna scelta di $|\psi_A\rangle$ e $|\psi_B\rangle$: misurare il primo qubit come 0 garantisce che il secondo sia 0, indipendentemente dalla distanza fisica tra i due sistemi.

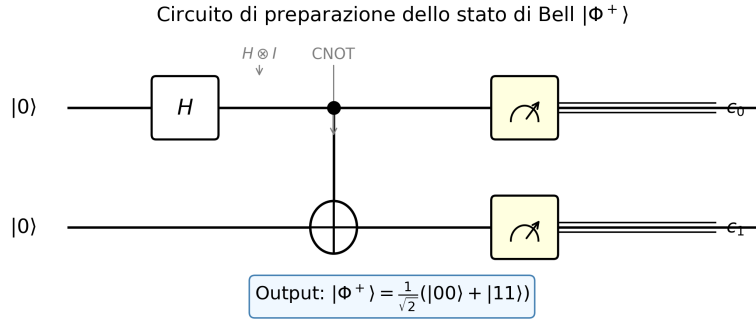


Figura A.1: Circuito quantistico per la preparazione dello stato di Bell $|\Phi^+\rangle$. Una porta di Hadamard H posta sul qubit di controllo crea la sovrapposizione $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$; la successiva porta CNOT introduce le correlazioni di entanglement, producendo lo stato $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

L'Entanglement come Risorsa Fisica Fragile

L'entanglement non è soltanto una proprietà astratta degli stati quantistici: è la **risorsa fisica** che gli algoritmi quantistici consumano per ottenere il loro vantaggio computazionale. Nell'algoritmo di Shor, il registro di controllo e il registro di lavoro devono restare entangled per tutta la durata della phase estimation affinché la Quantum Fourier Transform possa estrarre il periodo corretto. La perdita anche parziale di questa correlazione è sufficiente a degradare il risultato.

La fragilità dell'entanglement ha un'origine fisica precisa. Un sistema quantistico reale non è mai perfettamente isolato dall'ambiente che lo circonda: campi elettromagnetici residui, vibrazioni termiche del reticolo cristallino e fluttuazioni di carica si accoppiano continuamente ai qubit. Quando questo accoppiamento avviene, il sistema entra in uno stato entangled *con l'ambiente*, trasferendo ad esso parte dell'informazione di fase. Dal punto di vista del solo sistema, le ampiezze di interferenza si riducono: il qubit non è più in una sovrapposizione pura $\alpha|0\rangle + \beta|1\rangle$, ma in un **miscuglio statistico** in cui la fase relativa tra $|0\rangle$ e $|1\rangle$ è persa. Questo processo prende il nome di **decoerenza** [13].

Il tempo caratteristico entro cui questa perdita di fase diventa apprezzabile è T_2 , il **tempo di coerenza di fase** (*dephasing time*). Esso misura per quanto tempo un qubit riesce a mantenere la propria sovrapposizione coerente prima che l'interazione con l'ambiente la degradi. La crescita della profondità di Shor rende questo budget critico, ma il solo ordine asintotico $O(n^3)$ non determina una soglia universale in bit: contano compilazione, parallelismo, idle e architettura fault-tolerant. La trattazione quantitativa e i limiti del confronto diretto con T_2 sono nel Capitolo 4.

A.1.5 Porte Quantistiche

Le operazioni su qubit sono rappresentate da **matrici unitarie**: $U^\dagger U = I$. La condizione di unitarietà garantisce la reversibilità del calcolo e la conservazione della norma (delle probabilità).

Porte a Singolo Qubit

Le principali porte a singolo qubit sono:

Le tre **matrici di Pauli** X , Y , Z costituiscono, insieme all'identità I , una base per lo spazio delle matrici 2×2 hermitiane a traccia nulla, e giocano un ruolo centrale sia nella descrizione degli stati del qubit sia nella caratterizzazione dei canali di rumore.

Porta di Pauli-X (NOT quantistico / Bit Flip):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle \quad (\text{A.15})$$

È l'analogo quantistico del NOT classico: inverte i valori di $|0\rangle$ e $|1\rangle$.

Porta di Pauli-Y:

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle \quad (\text{A.16})$$

Combina un bit flip e un phase flip: è equivalente (a meno di una fase globale) alla composizione iXZ . Compare naturalmente nella decomposizione di rotazioni generali su qubit e nella descrizione del canale di *depolarizzazione*.

Porta di Pauli-Z (Phase Flip):

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle \quad (\text{A.17})$$

Lascia invariato $|0\rangle$ e aggiunge una fase -1 a $|1\rangle$: è un *phase flip*.

Le tre matrici di Pauli soddisfano le seguenti relazioni algebriche fondamentali:

$$X^2 = Y^2 = Z^2 = I \quad (\text{A.18})$$

$$XY = iZ, \quad YZ = iX, \quad ZX = iY \quad (\text{A.19})$$

$$\{X, Y\} = \{Y, Z\} = \{X, Z\} = 0 \quad (\text{A.20})$$

dove $\{A, B\} = AB + BA$ è l'**anticommutatore**. Le prime due relazioni mostrano che le matrici di Pauli formano l'algebra di Lie $\mathfrak{su}(2)$; la terza — l'anticommutazione — è alla base della struttura dei codici di correzione degli errori quantistici, dove X e Z rappresentano rispettivamente errori di bit flip e phase flip indipendenti.

Porta di Hadamard:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \equiv |+\rangle, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \equiv |-\rangle \quad (\text{A.21})$$

La porta di Hadamard è fondamentale: applicata a n qubit inizializzati in $|0\rangle^{\otimes n}$, genera la **sovrapposizione uniforme** di tutti i 2^n stati base:

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \quad (\text{A.22})$$

Porta di fase R_ϕ :

$$R_\phi = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix} \quad (\text{A.23})$$

Casi particolari: $T = R_{\pi/4}$ (T-gate) e $S = R_{\pi/2}$ (S-gate o Phase gate).

Porte a Due Qubit

Porta CNOT:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (\text{A.24})$$

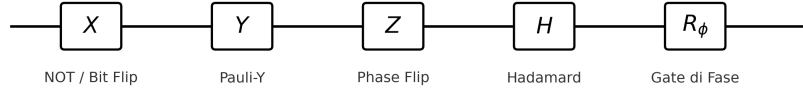
Applica una NOT sul qubit *target* se e solo se il qubit *control* è $|1\rangle$. È la porta a due qubit più comune e, insieme alle porte a singolo qubit, forma un insieme universale.

Porta Toffoli (Controlled-Controlled NOT (porta di Toffoli) (CCNOT)):

Generalizzazione a tre qubit; applica NOT sul terzo qubit solo se entrambi i qubit di controllo sono $|1\rangle$.

Principali porte quantistiche: simboli circuitali

Porte a singolo qubit:



NOT / Bit Flip

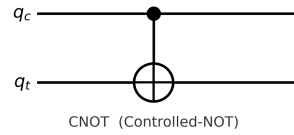
Pauli-Y

Phase Flip

Hadamard

Gate di Fase

Porta a due qubit:



CNOT (Controlled-NOT)

Figura A.2: Simboli circuitali delle principali porte quantistiche a singolo qubit (X , Y , Z , H , R_ϕ) e della porta CNOT a due qubit. Nella porta CNOT, il punto pieno indica il qubit di controllo q_c e il cerchio con croce indica il qubit target q_t .

A.1.6 Circuiti Quantistici

Un **circuito quantistico** è una sequenza ordinata di porte quantistiche applicata a un registro di qubit. I circuiti si leggono da sinistra a destra, con ogni filo orizzontale che rappresenta l'evoluzione temporale di un qubit. La composizione di porte corrisponde al prodotto matriciale degli operatori unitari associati.

L'equivalente quantistico del teorema di universalità classico afferma che qualunque trasformazione unitaria su n qubit può essere **approssimata con precisione arbitraria** da una sequenza finita di porte prese da un insieme universale, tipicamente $\{H, T, \text{CNOT}\}$. La distinzione fra approssimazione e decomposizione esatta non è pedanteria: le sequenze finite su un insieme di porte fissato sono numerabili, mentre le trasformazioni unitarie non lo sono, sicché una decomposizione esatta è impossibile in generale. Il teorema di Solovay–Kitaev garantisce che l'approssimazione a precisione ε richiede $O(\log^c(1/\varepsilon))$ porte, con c costante piccola: il costo dell'approssimazione cresce dunque solo polilogaritmicamente, ed è questa la ragione per cui l'universalità resta praticamente utile.

A.1.7 Misura Quantistica

La **misura** in base computazionale di uno stato $|\psi\rangle = \sum_x \alpha_x |x\rangle$ produce il risultato x con probabilità $|\alpha_x|^2$, e collassa irrevocabilmente lo stato in $|x\rangle$. La misura è una operazione *non unitaria* e *non reversibile*: una volta effettuata, l'informazione sulle ampiezze è irreversibilmente persa.

Questa caratteristica impone che gli algoritmi quantistici siano progettati per amplificare selettivamente l'ampiezza della risposta corretta prima della misura, in modo da ottenere il risultato voluto con alta probabilità.

A.1.8 Interferenza Quantistica

L'**interferenza** è il meccanismo fondamentale attraverso cui gli algoritmi quantistici realizzano il loro vantaggio computazionale. Le ampiezze quantistiche sono numeri complessi e possono combinarsi costruttivamente (quando hanno lo stesso segno/fase) o distruttivamente (quando hanno fase opposta).

Un algoritmo quantistico efficiente è costruito in modo che:

- Le ampiezze degli stati **sbagliati** si cancellino per **interferenza distruttiva**;
- Le ampiezze degli stati **corretti** si sommino per **interferenza costruttiva**.

Questo meccanismo è alla base sia dell'algoritmo di Grover sia dell'algoritmo di Shor (Capitolo 3), dove la Quantum Fourier Transform realizza l'interferenza necessaria al ritrovamento del periodo.

A.1.9 Modello di Complessità Quantistica

La complessità di un algoritmo quantistico è misurata in termini di numero di porte (equivalente delle operazioni elementari classiche) e di profondità del circuito (numero di livelli di porte parallele). Si definiscono le classi di complessità quantistiche più rilevanti:

- *Bounded-error Quantum Polynomial time* (BQP): classe dei problemi risolvibili in tempo polinomiale con probabilità di errore al più $1/3$ da un computer quantistico. È la classe centrale del calcolo quantistico.
- *Quantum Merlin-Arthur* (QMA): analogo quantistico di NP; problemi la cui soluzione può essere verificata efficientemente da un computer quantistico.

Le inclusioni $\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP} \subseteq \mathbf{PSPACE}$ sono *dimostrate*; ciò che resta aperto è se siano **strette**. Non è noto se BQP contenga propriamente P o BPP — cioè se esista davvero un problema che un computer quantistico risolva efficientemente e uno classico no — né se sia propriamente contenuto in PSPACE. Una dimostrazione di separazione stretta fra BQP e BPP implicherebbe $\mathbf{P} \neq \mathbf{PSPACE}$, un risultato che la teoria della complessità non ha ancora raggiunto: è questa la ragione per cui il vantaggio quantistico, per Shor come per gli altri algoritmi, resta una congettura ampiamente creduta e non un teorema.

Gerarchia delle classi di complessità nel calcolo quantistico

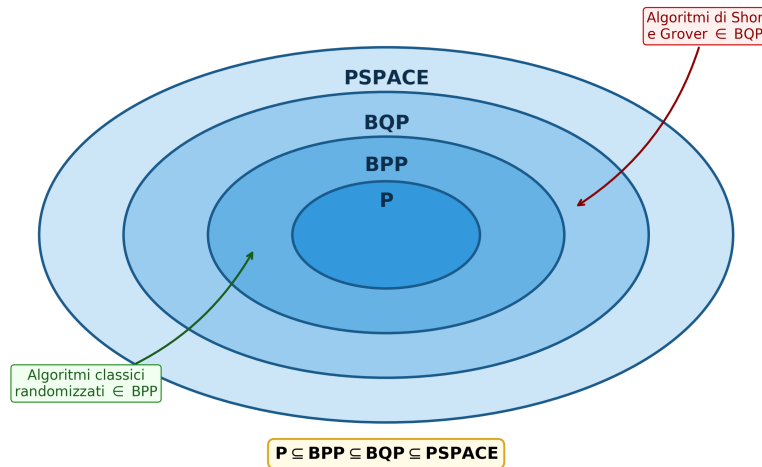


Figura A.3: Gerarchia delle classi di complessità nel calcolo quantistico. La classe BQP cattura i problemi efficientemente risolvibili da un computer quantistico; le inclusioni mostrate ($P \subseteq BPP \subseteq BQP \subseteq PSPACE$) sono dimostrate, mentre è congetturata — e non dimostrata — la loro *strettezza*. Gli algoritmi di Shor e Grover risiedono in BQP; che la fattorizzazione non appartenga a P è ritenuto probabile ma non provato.

A.2 L'Anatomia Fisica di una QPU

A.2.1 L'Anatomia Fisica di una QPU

Per capire dove nascono gli errori bisogna prima capire com'è fatta la macchina. Una QPU superconduttiva è un chip criogenico che contiene *esclusivamente* i qubit e le linee fisiche che li collegano: contrariamente all'intuizione, **le porte logiche non risiedono nel chip**. Aprendo una CPU classica si trovano registri e porte logiche incise nel silicio; aprendo una QPU si trovano solo i qubit. Le operazioni sono realizzate dall'**elettronica di controllo esterna**, che invia impulsi a microonde convogliati ai qubit tramite guide d'onda: “far passare un qubit in un gate” significa in realtà inviargli un'onda sagomata che ne ruota lo spin dell'angolo desiderato (90° , 30° , 10° , ...). La Figura A.4 schematizza questa architettura per una QPU a 5 qubit con topologia a catena lineare.

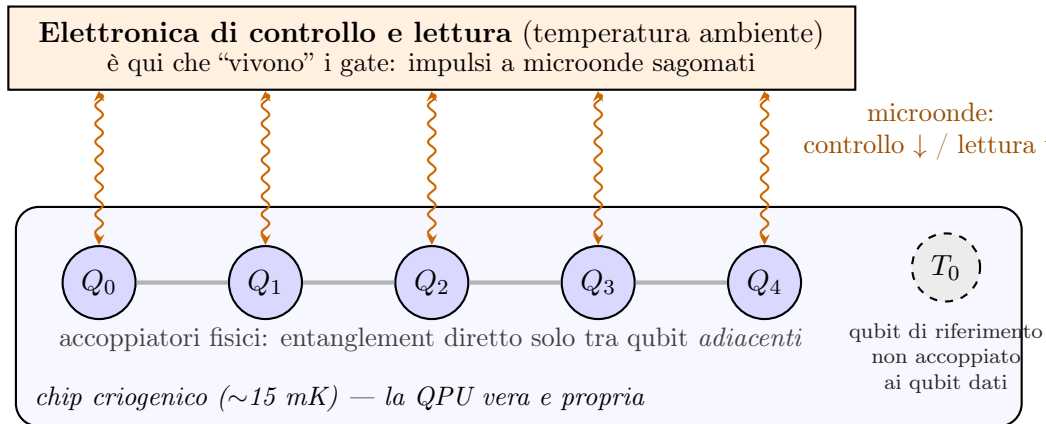


Figura A.4: Schema didattico di una QPU superconduttiva a 5 qubit con topologia a catena. Nel chip criogenico risiedono qubit, risonatori e accoppiatori; le operazioni sono pilotate dall’elettronica esterna mediante linee di controllo e lettura. L’interazione a due qubit diretta è disponibile solo lungo gli accoppiamenti disegnati (Q_1 – Q_2 sì, Q_0 – Q_2 mediante routing). T_0 rappresenta un qubit di riferimento non accoppiato alla catena, ma ancora dotato delle linee necessarie a preparazione e misura: non fornisce una misura “pura” di T_1 e non elimina perdite, Purcell effect o altre sorgenti di decoerenza.

La Figura A.5 mostra come questa architettura si presenta nella realtà: all’esterno il sistema integrato (criostato, elettronica e schermature racchiusi nella teca), all’interno della sala macchine i cilindri criogenici sospesi con il fascio di tubi e cavi coassiali che convogliano le microonde di controllo e lettura — le “guide d’onda” dello schema precedente.



(a) IBM Quantum System One (Ehningen, Germania)



(b) Criostati IBM Q con il cablaggio di controllo

Figura A.5: La QPU reale. (a) Il sistema integrato IBM Quantum System One: il cilindro nero al centro è il criostato che contiene il chip, circondato dall'elettronica di controllo. (b) Criostati IBM Q in sala macchine: dall'alto arrivano i tubi per la refrigerazione e i cavi coassiali che convogliano le microonde di controllo e lettura verso il chip, sospeso sul fondo del cilindro a ~ 15 mK. *Crediti: (a) IBM Research, licenza CC BY 2.0; (b) Connie Zhou per IBM, pubblico dominio; entrambe via Wikimedia Commons.*

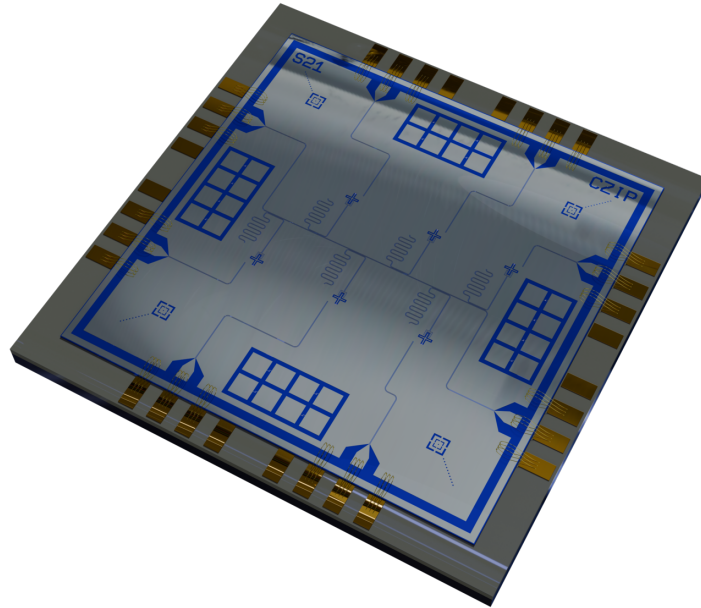


Figura A.6: Resa tridimensionale di un chip quantistico superconduttivo con 6 qubit transmon e 12 giunzioni di test. Si riconoscono gli elementi dello schema di Figura A.4: i qubit con i loro risonatori, le linee di accoppiamento, le piazzole dorate per il collegamento alle guide d'onda e — ai quattro angoli — le strutture di test isolate, l'analogo reale del qubit T_0 . *Credito: Onri Jay Benally, licenza CC BY 4.0, via Wikimedia Commons.*

Questa architettura ha tre conseguenze dirette per gli errori.

La lettura distrugge lo stato — fisicamente. Anche la misura avviene tramite microonde: per leggere un qubit lo si “bombarda” con un’onda di sonda e si osserva come essa torna indietro — un modo di risposta corrisponde a $|0\rangle$, un altro a $|1\rangle$. L’atto stesso di sondare blocca la dinamica del qubit: è per questo che la misura di uno stato quantistico lo distrugge *realmente*, non per convenzione matematica. Ed è per questo che nei circuiti le letture stanno tutte alla fine, in parallelo: misurare un solo qubit lasciando gli altri in sovrapposizione significa inviare un’onda che, per quanto focalizzata, perturba anche i vicini e può farli precipitare.

Il budget di operazioni: coerenza vs velocità dei gate. Le prestazioni di una QPU dipendono dal rapporto tra due tempi: quanto a lungo il qubit resta coerente e quanto dura una singola operazione. Con una finestra di coerenza di $100\ \mu\text{s}$ e gate da $\sim 25\ \text{ns}$, si eseguono $\sim 4\,000$ operazioni; una macchina con coerenza superiore ma gate

più lenti può eseguirne *meno*. È il rapporto che conta, non il singolo parametro — ed è questo rapporto a decidere se il circuito di Shor per una data istanza è fisicamente eseguibile. La Tabella A.1 confronta gli ordini di grandezza delle due principali tecnologie.

	Superconduttori (IBM, IQM, ...)	Ioni intrappolati (IonQ, Quantinuum, ...)
Tempo di coerenza	50–300 μ s	da secondi a minuti
Durata gate (1 qubit)	~ 15 –40 ns	~ 1 –100 μ s
Durata gate (2 qubit)	~ 60 –500 ns	~ 0.1 –1 ms
Operazioni per finestra	$\sim 10^3$ – 10^4	$\sim 10^3$
Lettura	più esposta a errori e interferenze	più stabile e affidabile
Crosstalk	presente (accoppiamento capacitivo)	ridotto
Velocità di esecuzione	molto alta	bassa (gate lenti)

Tabella A.1: Ordini di grandezza indicativi delle due principali tecnologie di QPU. La coerenza lunghissima degli ioni intrappolati non si traduce in più operazioni utili, perché i gate sono da 10^3 a 10^4 volte più lenti: il budget di operazioni per finestra di coerenza è simile, ma i superconduttori lo spendono molto più in fretta. Gli ioni sono invece più robusti su lettura e crosstalk.

Il bordo di coerenza. Gli errori non sono distribuiti uniformemente nel tempo: finché il calcolo occupa una piccola frazione della finestra di coerenza, la probabilità di decadimento è bassa; quando il circuito si spinge *verso la fine* della finestra — il “bordo di coerenza” — la probabilità che il qubit sia già decaduto al momento della misura cresce rapidamente. La Figura A.7 illustra il concetto: è la ragione per cui circuiti profondi come quello di Shor, che consumano gran parte del budget, producono letture inaffidabili anche quando “tecnicamente” rientrano nella finestra.

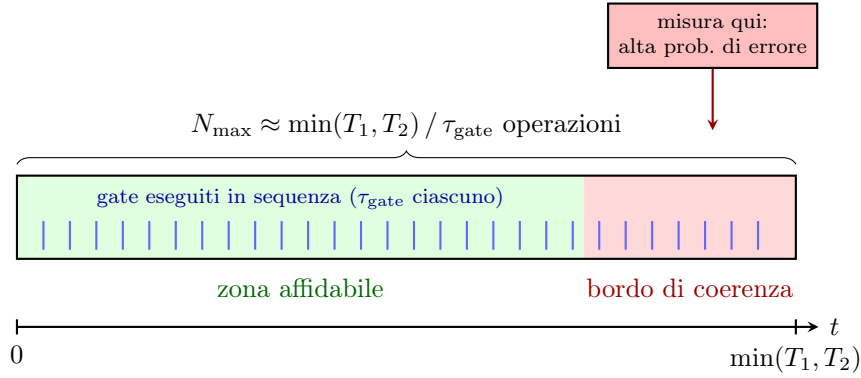


Figura A.7: Il “budget” di operazioni di un qubit: la finestra di coerenza $\min(T_1, T_2)$ divisa per la durata del singolo gate dà il numero massimo di operazioni eseguibili $N_{\max}$. Un circuito che consuma solo la prima parte della finestra opera in zona affidabile; un circuito profondo come quello di Shor spinge la misura verso il *bordo di coerenza*, dove la probabilità che il qubit sia già decaduto — e che la lettura restituisca un valore errato — è massima.

La topologia decide chi parla con chi. Le QPU differiscono anche per geometria: catene lineari, griglie (heavy-hex di IBM), connettività completa. Su una catena, l’entanglement tra qubit non adiacenti richiede un “cross-entanglement” attraverso i qubit intermedi: ogni passaggio disperde energia e accorcia la vita dello stato condiviso — più cross si fanno, prima lo stato decade. La topologia determina quindi sia *quali* circuiti si possono mappare in modo efficiente, sia *dove* il crosstalk colpirà. Questo aspetto — insieme al fatto che i parametri di coerenza variano da qubit a qubit dello stesso chip — è ripreso nel Capitolo 9, dove la località delle interazioni sulla griglia è il fondamento architetturale del surface code.

A.3 L’Algoritmo di Shor — Approfondimenti e Stato dell’Arte

Questa sezione raccoglie il materiale di contesto e di approfondimento dell’algoritmo di Shor: lo stato delle realizzazioni sperimentali, i miglioramenti algoritmici recenti, la roadmap verso il calcolo fault-tolerant e il confronto con l’algoritmo di Grover. Il Capitolo 3 ne riporta la sintesi.

A.3.1 Sviluppi Recenti e Stato dell'Arte

Implementazioni su Hardware Reale

Nonostante la solidità teorica, l'esecuzione di Shor su hardware reale rimane limitata a istanze molto piccole a causa dei requisiti di profondità circuitale. I principali risultati sperimentali sono:

- **2001 – NMR (Vandersypen et al.):** prima dimostrazione sperimentale di Shor su un sistema a 7 spin nucleari; fattorizzazione di $N = 15$ in 3×5 [30].
- **2016 – Ioni intrappolati (Monz et al.):** implementazione scalabile su 5 qubit ionici; fattorizzazione di $N = 15$ con un'architettura modulare [31].
- **2021 – Qubit superconduttori International Business Machines (IBM):** dimostrazione compilata della fattorizzazione di $N = 21$ su processori IBM Quantum utilizzando 5 qubit [4].

Queste istanze evidenziano quanto la distanza dalle chiavi crittograficamente rilevanti sia ancora enorme. Una specifica stima di risorse per fattorizzare un numero a L bit riporta circa $5L+1$ qubit logici e $72L^3$ gate [32]; costanti e overhead dipendono però dall'architettura aritmetica e dal protocollo fault-tolerant. In ogni caso, i valori per RSA-2048 restano fuori portata per i processori NISQ attuali.

Miglioramenti Algoritmici: Regev 2023

Un risultato teorico rilevante è stato pubblicato nel 2023 da Oded Regev [33]. La proposta fattorizza un intero di n bit eseguendo indipendentemente circa $\sqrt{n} + 4$ circuiti, ciascuno con $\tilde{O}(n^{3/2})$ gate, seguiti da post-processing classico polinomiale. Il confronto con Shor non può quindi essere ridotto al solo conteggio di gate di una singola esecuzione: aumentano il numero di run e, nella costruzione originale, anche lo spazio quantistico; inoltre l'articolo dichiara non chiaro l'effettivo vantaggio implementativo. Il risultato è un miglioramento asintotico importante, non una dimostrazione di praticabilità su dispositivi NISQ.

Roadmap IBM verso il Quantum Fault-Tolerant

IBM ha delineato una roadmap concreta verso il calcolo quantistico fault-tolerant [34]:

- **2025:** processori Loon e Nighthawk (120 qubit con 218 coupler sintonizzabili), con dimostrazione dei componenti chiave per la fault-tolerance tramite codici qLDPC;
- **2026:** target di vantaggio quantistico su problemi pratici;
- **2029:** sistema Quantum Starling con 200 qubit logici e oltre 10^8 operazioni quantistiche.

Questi traguardi sono obiettivi industriali dichiarati da IBM, soggetti a revisione, e non stabiliscono quando una macchina avrà le risorse logiche necessarie a rompere RSA-2048. Rafforzano la necessità di preparare la migrazione, ma non forniscono una previsione crittografica.

Le sfide hardware principali per l'esecuzione di Shor su istanze crittograficamente rilevanti sono analizzate in dettaglio nel Capitolo 4, mentre l'implementazione pratica con Qiskit e le tecniche di ottimizzazione basate su intelligenza artificiale sono presentate nel Capitolo 6.

A.3.2 Contesto: Confronto con l'Algoritmo di Grover

Per collocare correttamente l'algoritmo di Shor nel panorama dell'informatica quantistica, è utile confrontarlo con l'altro grande risultato algoritmico degli anni Novanta: l'algoritmo di Grover per la ricerca non strutturata [35].

Il Problema di Grover

Dato un insieme di N elementi e una funzione oracolo $f : \{0, \dots, N-1\} \rightarrow \{0, 1\}$, l'obiettivo è trovare x^* tale che $f(x^*) = 1$. Classicamente, nel caso peggiore, sono necessarie $O(N)$ interrogazioni. Grover (1996) dimostrò che un algoritmo quantistico, sfruttando il meccanismo di *amplitude amplification*, può trovare la soluzione in sole $O(\sqrt{N})$ interrogazioni – uno **speedup quadratico** che è stato dimostrato ottimale: nessun algoritmo quantistico può fare meglio [36, 37].

Differenze Fondamentali

I due algoritmi rappresentano due regimi distinti di vantaggio quantistico:

Caratteristica	Shor	Grover
Problema target	Fattorizzazione intera / period finding	Ricerca non strutturata
Speedup	Esponenziale rispetto al miglior classico	Quadratico: $O(\sqrt{N})$ vs $O(N)$
Tecnica quantistica core	QFT + Phase Estimation	Amplitude Amplification
Ottimalità	Non dimostrata in assoluto	Dimostrata ottimale [36]
Impatto crittografico	Rompe RSA, DH, ECDSA (chiave pubblica)	Dimezza sicurezza AES/SHA (simmetrica)
Requisiti hardware	$O(\log N)$ qubit, alta profondità	$O(\log N)$ qubit, bassa profondità

Tabella A.2: Confronto tra l'algoritmo di Shor e l'algoritmo di Grover

L'impatto sulla crittografia simmetrica di Grover è significativo ma gestibile: raddoppiare la lunghezza delle chiavi (da AES-128 a AES-256) è sufficiente a ripristinare il livello di sicurezza. Al contrario, non esiste un analogo rimedio per RSA: l'unica risposta è la migrazione verso gli standard post-quantum discussi nella sezione precedente. È per questa ragione che l'algoritmo di Shor rappresenta la minaccia quantistica più urgente e motivante per la ricerca sul calcolo quantistico applicato.

Tutta l'analisi svolta in questo capitolo ha però assunto un modello di calcolo *ideale*: porte unitarie perfette, coerenza illimitata, nessuna interazione con l'ambiente.

Su hardware reale questi presupposti non reggono, e la distanza tra il modello teorico e l'esecuzione fisica è precisamente il nodo critico che rende ancora irraggiungibile la fattorizzazione di numeri crittograficamente rilevanti. Comprendere le origini e gli effetti del rumore quantistico è quindi il passo necessario per valutare le reali prospettive dell'algoritmo di Shor.

A.4 I Canali di Rumore: Derivazioni Complete

I quattro canali quantistici che modellano il rumore sono mappe *Completely Positive Trace-Preserving* (CPTP) in rappresentazione di Kraus, secondo l'Eq. 4.6. Il Capitolo 4 ne riporta il canale depolarizzante per esteso, essendo quello su cui poggia la campagna sperimentale, e l'enunciato compatto degli altri tre; qui se ne danno le derivazioni complete.

Canale Bit Flip

Il canale **bit flip** modella un errore di tipo X discreto: con probabilità p il qubit viene invertito ($|0\rangle \leftrightarrow |1\rangle$), con probabilità $1 - p$ rimane invariato. Gli operatori di Kraus sono $K_0 = \sqrt{1-p} I$ e $K_1 = \sqrt{p} X$, e il canale agisce come:

$$\mathcal{E}_{BF}(\rho) = (1-p)\rho + pX\rho X \quad (\text{A.25})$$

Sulla sfera di Bloch, le componenti r_y e r_z vengono contratte di un fattore $(1 - 2p)$ mentre la componente r_x rimane invariata. Per $p = 1/2$ il canale annulla le componenti y e z , ma conserva r_x : equivale a una completa dephasing nella base degli autostati di X , non alla distruzione completa dello stato.

Canale Phase Flip

Il canale **phase flip** modella la perdita di informazione di fase senza cambiamento di popolazione: la sovrapposizione decade senza che l'energia venga dissipata. Con operatori di Kraus $K_0 = \sqrt{1-p} I$ e $K_1 = \sqrt{p} Z$, il canale è:

$$\mathcal{E}_{PF}(\rho) = (1-p)\rho + pZ\rho Z \quad (\text{A.26})$$

L'effetto sulla matrice densità è immediato:

$$\mathcal{E}_{PF} \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} = \begin{pmatrix} \rho_{00} & (1-2p) \rho_{01} \\ (1-2p) \rho_{10} & \rho_{11} \end{pmatrix} \quad (\text{A.27})$$

Le popolazioni ρ_{00} e ρ_{11} rimangono invariate, mentre le **coerenze** ρ_{01} , ρ_{10} vengono sopprese di un fattore $(1-2p)$. Questo è il canale che descrive il decadimento T_2 : parametrizzando $p(t) = \frac{1}{2}(1 - e^{-t/T_2})$, si ottiene $\rho_{01}(t) = \rho_{01}(0) e^{-t/T_2}$.

Canale di Amplitude Damping

Il canale di **amplitude damping** modella il rilassamento energetico T_1 : il qubit decade spontaneamente dallo stato eccitato $|1\rangle$ allo stato fondamentale $|0\rangle$ con probabilità $\gamma = 1 - e^{-t/T_1}$. Gli operatori di Kraus sono:

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix} \quad (\text{A.28})$$

Il canale agisce sulla matrice densità come:

$$\mathcal{E}_{AD}(\rho) = \begin{pmatrix} \rho_{00} + \gamma \rho_{11} & \sqrt{1-\gamma} \rho_{01} \\ \sqrt{1-\gamma} \rho_{10} & (1-\gamma) \rho_{11} \end{pmatrix} \quad (\text{A.29})$$

La popolazione ρ_{11} decade verso ρ_{00} con tasso γ , mentre le coerenze vengono sopprese di $\sqrt{1-\gamma}$. A differenza dei canali precedenti, questo canale è **non simmetrico**: sulla sfera di Bloch contrae la sfera verso il polo nord $|0\rangle$, non verso il centro:

$$r_x \rightarrow \sqrt{1-\gamma} r_x, \quad r_y \rightarrow \sqrt{1-\gamma} r_y, \quad r_z \rightarrow (1-\gamma) r_z + \gamma \quad (\text{A.30})$$

Per $t \rightarrow \infty$ ($\gamma \rightarrow 1$), qualsiasi stato converge a $|0\rangle \langle 0|$, indipendentemente dallo stato iniziale.

Appendice B

Strumenti, Ambiente e Codice

B.1 Strumenti e Ambiente di Simulazione — Trattamento Esteso

La scelta degli strumenti di simulazione non è neutrale. In informatica quantistica, il framework adottato determina quali modelli di rumore sono disponibili, verso quale hardware è possibile traslare i circuiti e con quali risultati della letteratura è possibile confrontarsi direttamente. Questo capitolo descrive gli strumenti utilizzati nella parte sperimentale di questa tesi, motivando ciascuna scelta e collocandola rispetto alle alternative disponibili.

B.1.1 Qiskit: il Framework Quantistico di Riferimento

Qiskit (*Quantum Information Science Kit*) è un *Software Development Kit* (SDK) open-source sviluppato da IBM Quantum per la programmazione, la simulazione e l'esecuzione di algoritmi quantistici [22]. Rilasciato pubblicamente nel 2017, offre integrazione diretta con l'hardware IBM ed è uno dei framework ampiamente usati nella ricerca su circuiti quantistici a corto termine [4, 38].

Architettura del Framework

Qiskit è organizzato in tre componenti principali, ciascuno con un ruolo distinto nel ciclo di vita di un esperimento quantistico:

- **Qiskit (core)** — il nucleo del framework, già noto come Qiskit Terra. Fornisce le primitive per costruire i circuiti, compilarli verso un backend target

tramite `transpile` e accedere alle interfacce `Sampler` ed `Estimator`. Gestisce la pipeline dalla descrizione astratta fino all'invio al simulatore o all'hardware reale.

- **Qiskit Aer** — il modulo di simulazione classica ad alte prestazioni. Implementa diversi metodi di simulazione esatta e approssimata, con supporto nativo per modelli di rumore realistici. È il componente centrale di questa tesi e viene descritto in dettaglio nella Sezione B.1.2.
- **Qiskit IBM Runtime** — l'interfaccia per l'esecuzione su processori IBM Quantum reali tramite cloud (`qiskit-ibm-runtime`). Fornisce le primitive `Sampler` e `Estimator` ottimizzate per l'hardware reale, con supporto per tecniche di soppressione del rumore come il *Dynamical Decoupling* e il *Gate Twirling* [39].

Perché Qiskit e non un'alternativa

La scelta di Qiskit è stata formalizzata nell'incontro con il relatore del 18 marzo 2026, in seguito all'analisi del tutorial ufficiale IBM per l'algoritmo di Shor [39]. La decisione è motivata da quattro ragioni convergenti.

Prima ragione: controllo esplicito del modello. Qiskit Aer permette di comporre canali depolarizzanti, termici e di readout e di legarli a un gate set dichiarato. Questa flessibilità consente esperimenti controllati e riproducibili; non rende però i preset uniformi della tesi direttamente equivalenti alla calibrazione di una QPU reale.

Seconda ragione: copertura delle primitive necessarie. Qiskit Aer espone nello stesso ambiente i canali depolarizzanti, termici, di readout e coerenti richiesti dal disegno. Altri framework, fra cui Cirq [40] e PennyLane [41], offrono anch'essi simulazione rumorosa; la scelta non pretende di stabilire una graduatoria generale, ma riduce le conversioni rispetto al codice Qiskit adottato.

Terza ragione: disponibilità dell'implementazione di riferimento. Il circuito dell'algoritmo di Shor utilizzato in questa tesi è tratto da un'implementazione open-source in Qiskit [23]. Questo punto di partenza, validato dalla letteratura [4], ha reso naturale mantenere Qiskit come framework unico lungo l'intero pipeline sperimentale.

Quarta ragione: allineamento con le fonti implementative. Diversi lavori usati come riferimento adottano Qiskit [4, 38, 42]. Condividere il framework facilita

il confronto delle API e dei circuiti, ma non rende direttamente comparabili risultati ottenuti con gate set, versioni, transpiler o noise model differenti.

Tabella B.1: Confronto tra i principali framework per la simulazione quantistica con rumore.

Framework	Sviluppatore	Simulatore con rumore	Target hardware primario
Qiskit + Aer	IBM Quantum	Completo (T1, T2, depolarz., readout)	IBM superconduttori
Cirq	Google Quantum AI	Parziale	Google superconduttori
PennyLane	Xanadu	Limitato	Vari (plug-in)
Forest / PyQuil	Rigetti	Parziale	Rigetti superconduttori

B.1.2 Qiskit Aer: il Simulatore

Qiskit Aer è il modulo di simulazione classica di Qiskit. Il suo componente principale è `AerSimulator`, un simulatore ad alte prestazioni che supporta più metodi di simulazione selezionabili a seconda della dimensione del circuito e del tipo di analisi richiesta.

Metodi di Simulazione

`AerSimulator` espone i seguenti metodi principali tramite il parametro `method`:

- **statevector** — simulazione del vettore di stato completo. Richiede 2^n ampiezze complesse in memoria ed è il metodo usato dalla campagna rumorosa principale su $N = 15$.
- **density_matrix** — evolve la matrice densità ρ anziché il vettore di stato, permettendo di modellare canali di rumore non unitari in modo esatto. Richiede 4^n elementi e riduce il limite pratico a circa 25 qubit.
- **stabilizer** — simulazione efficiente di circuiti Clifford puri (senza porte non-Clifford come T o R_z arbitrarie). Scala a migliaia di qubit ma non è applicabile all'algoritmo di Shor, che utilizza rotazioni di fase arbitrarie.
- **matrix_product_state** — rappresenta lo stato come rete di tensori; senza troncamenti imposti resta una rappresentazione controllata, ma il costo cresce

con l'entanglement. È usato in M8/M13 per contenere il costo delle molte repliche.

La scelta è quindi per campagna, non globale: `statevector` per la prima fase N15 e `matrix_product_state` per la sensibilità M8/M13. Le istanze N21/N35 sono validate idealmente ma escluse dalle campagne rumorose generali.

Modelli di Rumore

Il modulo `qiskit_aer.noise` fornisce le primitive per costruire modelli di rumore compositi. Ogni errore è rappresentato come un canale quantistico che viene applicato ai gate specificati, con la possibilità di assegnare errori diversi a qubit diversi (rumore eterogeneo) o un errore uniforme a tutti i qubit (*all-qubit error*). I tipi di errore disponibili includono:

- `depolarizing_error` — errore depolarizzante su gate a 1 o 2 qubit, parametrizzato da λ nella convenzione Aer; la probabilità Pauli non-identità è $3\lambda/4$ o $15\lambda/16$.
- `thermal_relaxation_error` — modella il rilassamento verso lo stato fondamentale (T_1) e la perdita di coerenza di fase (T_2), con un tempo di gate configurabile.
- `ReadoutError` — matrice di confusione che descrive la probabilità di misurare $|0\rangle$ come $|1\rangle$ e viceversa.
- Errori coerenti — applicazione di una rotazione indesiderata su ogni gate, modellabile tramite operatori unitari arbitrari.

Il modello di rumore usato nella prima fase combina errori depolarizzanti, rilassamento termico e readout error con parametri uniformi. È un modello illustrativo per isolare i meccanismi, non uno snapshot di `ibm_marrakesh`: non include coupling map, routing, errori per qubit, crosstalk o deriva temporale. La configurazione dettagliata è riportata nel Capitolo 2.

Transpilazione

Prima dell'esecuzione, la funzione `transpile` riscrive il circuito nella base esplicita `rz/sx/x/cx` e applica ottimizzazioni strutturali. La campagna usa il livello di ottimizzazione 2 e il seed del transpiler 20260819; questa coppia, insieme all'hash

della sequenza compilata, è registrata in ogni artefatto. La base esplicita impedisce che porte composte restino opache e sfuggano al canale di rumore previsto.

B.1.3 Accelerazione GPU con Qiskit Aer

La libreria `qiskit-aer-gpu` estende `AerSimulator` con il supporto per l'accelerazione tramite GPU NVIDIA attraverso CUDA. Il backend `statevector` è interamente eseguito sulla GPU: il vettore di stato viene allocato nella memoria VRAM e le operazioni di gate sono eseguite come moltiplicazioni matrice-vettore parallelizzate su migliaia di core CUDA.

Motivazione pratica. L'accelerazione era stata valutata per la generazione dei dataset e per le campagne replicate. Non vengono però riportati fattori di accelerazione ipotetici come risultati sperimentali, poiché l'ambiente canonico finale usa la CPU.

Specifiche hardware. La GPU prevista è una NVIDIA GeForce RTX 4070 con 12 GB di memoria VRAM e 5888 core CUDA (architettura Ada Lovelace). La sola allocazione di uno statevector `complex128` avrebbe un tetto teorico di 29 qubit, inferiore nella pratica per memoria di lavoro e overhead. Questo limite dimensionale non garantisce inoltre la fattibilità: circuiti molto profondi e traiettorie rumorose moltiplicano il costo temporale, ed è proprio ciò che esclude N21/N35 dalla campagna rumorosa.¹

Esito: l'accelerazione non è stata utilizzabile. Quanto sopra descrive il piano iniziale. In sede di implementazione il backend GPU si è rivelato inutilizzabile nell'ambiente WSL adottato — il driver CUDA riconosce la scheda, ma il backend C++ di Aer solleva un errore a runtime sull'esecuzione dei circuiti — e **tutti gli esperimenti riportati in questa tesi sono stati eseguiti su CPU**. La diagnosi completa e le conseguenze sui tempi di esecuzione sono nella Sezione 6.7.1. Il vincolo di memoria resta quello indicato, riferito però alla memoria di sistema anziché alla VRAM.

B.1.4 scikit-learn: la Libreria per il Classificatore

Il Metodo 2 richiede l'addestramento di un classificatore binario. La libreria scelta è scikit-learn [26], una delle librerie di machine learning più consolidate nell'ecosistema

¹La memoria del solo vettore è $2^n \times 16$ byte. Con 12 GB decimali, $\log_2(12 \times 10^9/16) \simeq 29.48$, quindi il massimo intero teorico è $n = 29$.

Python scientifico.

La motivazione principale è pragmatica: scikit-learn espone un'interfaccia unificata (`fit` / `predict`) per un ampio catalogo di classificatori — Random Forest, SVM, MLP, e altri — permettendo di confrontarli sullo stesso dataset con minime modifiche al codice. Questo facilita la selezione sperimentale del modello migliore senza dover introdurre dipendenze da framework più pesanti (TensorFlow, PyTorch) per un classificatore che, per il problema in esame, si prevede non richieda architetture profonde.

B.1.5 Ambiente di Esecuzione: WSL con Ubuntu

Le simulazioni vengono eseguite in ambiente Linux tramite WSL (versione 2) su Windows 11. La scelta è motivata dalla necessità di mantenere la compatibilità con l'ecosistema Linux di Qiskit e delle librerie scientifiche Python, garantendo al contempo l'integrazione con i driver CUDA per WSL forniti da NVIDIA.

WSL 2 esegue un kernel Linux completo in una macchina virtuale leggera. L'ambiente Python è isolato in un virtual environment dedicato, garantendo la riproducibilità delle dipendenze e la compatibilità delle versioni tra le librerie usate; le versioni esatte sono versionate nel file `requirements.txt` della repository. È in questo stesso ambiente che l'accesso alla GPU si è rivelato non praticabile (Sezione 6.7.1).

Con l'ambiente così definito, il Capitolo 6 descrive come Qiskit, Aer e scikit-learn si integrano nell'architettura del pipeline sperimentale: dalla costruzione del circuito di Shor alla generazione del dataset di addestramento, fino all'inferenza del classificatore.

B.2 Codice Sorgente dei Componenti Implementati

Questa sezione raccoglie gli estratti necessari a verificare le scelte implementative del Capitolo 6. Gli output numerici non sono codificati nei listati: ogni campagna li salva in un artefatto JSON schema 2 corredato di manifest.

```
python3 -m venv ~/quantum-env
source ~/quantum-env/bin/activate
python -m pip install -r requirements.txt
```

```
python -c "import qiskit, qiskit_aer; print(qiskit.__version__)"
```

Listing B.1: Installazione dell'ambiente Python e delle librerie necessarie

```
RuntimeError: Simulation device "GPU" is not supported on this system
```

Listing B.2: Errore GPU osservato su WSL durante i test

```
def inverse_qft(n_qubits):
    qc = QuantumCircuit(n_qubits, name='QFT_dagger')
    for q in range(n_qubits // 2):
        qc.swap(q, n_qubits - q - 1)
    for j in range(n_qubits):
        for m in range(j):
            qc.cp(-np.pi / 2 ** (j - m), m, j)
        qc.h(j)
    return qc
```

Listing B.3: Implementazione della Quantum Fourier Transform inversa

```
def c_amod15(a, power):
    if a not in [2, 7, 8, 11, 13]:
        raise ValueError("base non supportata per N=15")
    U = QuantumCircuit(4)
    for _ in range(power):
        if a in [2, 13]:
            U.swap(0, 1); U.swap(1, 2); U.swap(2, 3)
        if a in [7, 8]:
            U.swap(2, 3); U.swap(1, 2); U.swap(0, 1)
        if a == 11:
            U.swap(1, 3); U.swap(0, 2)
        if a in [7, 11, 13]:
            U.x(range(4))
    return U.control(annotated=False)
```

Listing B.4: Moltiplicazione modulare N15 corretta: l'operazione $\times a$ viene ripetuta $power$ volte

```
def shor_circuit(N, a, n_count):
    if N != 15:
```

```

        return shor_circuit_beauregard(N, a, n_count)
n_work = 4
qc = QuantumCircuit(n_count + n_work, n_count)
qc.h(range(n_count))
qc.x(n_count + 3)    #  $|1\rangle$  nella convenzione del registro
                    textbook
for j in range(n_count):
    power = 2 ** j
    if power % 4 != 0:    #  $U^4 = I$  per  $a=7 \bmod 15$ 
        qc.append(c_amod15(a, power),
                    [j] + list(range(n_count, n_count +
                                    n_work)))

qc.barrier()
qc.append(inverse_qft(n_count), range(n_count))
qc.measure(range(n_count), range(n_count))
return qc

def extract_factors(measured_value, n_count, N, a):
    if measured_value == 0:
        return None, None
    phase = measured_value / 2 ** n_count
    r = Fraction(phase).limit_denominator(N).denominator
    if r % 2:
        return None, None
    for candidate in (gcd(a ** (r // 2) - 1, N),
                      gcd(a ** (r // 2) + 1, N)):
        if 1 < candidate < N:
            return candidate, N // candidate
    return None, None

```

Listing B.5: Circuito N15 e post-processing classico

```

BASIS_GATES = ('rz', 'sx', 'x', 'cx')
GATE_TIME_1Q_NS = 50.0
GATE_TIME_2Q_NS = 300.0

def compose_errors(errors):
    out = None
    for error in errors:

```

```

        if error is not None:
            out = error if out is None else out.compose(error)
    return out

def build_noise_model(eps_1q, eps_2q, t1_ns, t2_ns, p_ro=0.02)
:
    # eps_1q ed eps_2q sono i lambda di depolarizing_error.
    thermal_1q = thermal_relaxation_error(
        t1_ns, t2_ns, GATE_TIME_1Q_NS)
    thermal_one_2q = thermal_relaxation_error(
        t1_ns, t2_ns, GATE_TIME_2Q_NS)
    thermal_2q = thermal_one_2q.tensor(thermal_one_2q)
    error_1q = compose_errors([
        depolarizing_error(eps_1q, 1) if eps_1q else None,
        thermal_1q,
    ])
    error_2q = compose_errors([
        depolarizing_error(eps_2q, 2) if eps_2q else None,
        thermal_2q,
    ])
    nm = NoiseModel()
    nm.add_all_qubit_quantum_error(error_1q, ['sx', 'x'])
    nm.add_all_qubit_quantum_error(error_2q, ['cx'])
    if p_ro:
        nm.add_all_qubit_readout_error(
            ReadoutError([[1-p_ro, p_ro], [p_ro, 1-p_ro]]))
    return nm

```

Listing B.6: Noise model uniforme illustrativo: RZ virtuale, durate 1Q/2Q separate

```

def compile_shor_circuit(N, a, n_count):
    return transpile(
        shor_circuit(N, a, n_count),
        basis_gates=['rz', 'sx', 'x', 'cx'],
        optimization_level=2,
        seed_transpiler=20260819,
    )

def rank_measurements(counts, tie_seed):

```

```

def priority(bitstring):
    payload = f'{{int(tie_seed)}}:{{bitstring}}'.encode('ascii')
    return hashlib.sha256(payload).digest()
return sorted(counts.items(),
              key=lambda item: (-item[1], priority(item
              [0])))

def method1_from_counts(counts, tie_seed, n_count, N, a):
    bitstring = rank_measurements(counts, tie_seed)[0][0]
    return extract_factors(int(bitstring, 2), n_count, N, a)

```

Listing B.7: Compilazione deterministica, tie-break SHA-256 e baseline TOP-1

```

LABEL_TOP_KS = (1, 16)
labels = {top_k: [] for top_k in LABEL_TOP_KS}

for sample_index in range(n_samples):
    sample_seed = seed * 1_000_000 + sample_index * 10_000
    counts = simulator.run(
        compiled, noise_model=sample_noise, shots=shots,
        seed_simulator=sample_seed,
    ).result().get_counts()
    ranked = rank_measurements(counts, tie_seed=sample_seed)
    for top_k in LABEL_TOP_KS:
        useful = any(
            extract_factors(int(bits, 2), n_count, N, a)[0] is
                not None
            for bits, _ in ranked[:top_k]
        )
        labels[top_k].append(int(useful))
    X.append(normalized_histogram(counts, n_count, shots))

# Ogni modello salva schema_version=2.0, label_top_k e
manifest.

```

Listing B.8: Dataset condiviso per l'audit delle etichette TOP-1 e TOP-16

```

def evaluate_same_histogram(counts, simulation_seed,
    classifier,

```



```

        n_count, N, a, shots):
ranked = rank_measurements(counts, simulation_seed)
valid = [
    extract_factors(int(bits, 2), n_count, N, a)[0] is not
        None
    for bits, _ in ranked[:4]
]
m1_success = valid[0]
top4_success = any(valid)
feature = normalized_histogram(counts, n_count, shots)
m2_success = bool(classifier.predict([feature])[0]) and
    top4_success
return m1_success, top4_success, m2_success

```

Listing B.9: Decisione M2 sullo stesso istogramma usato da M1 e TOP-4

```

PRESETS = {
    'reference': dict(N=15, a=7, n_count=8,
        eps_1q=1e-3, eps_2q=1e-2,
        t1_ns=100_000, t2_ns=80_000, p_ro=0.02),
    'stress': dict(N=15, a=7, n_count=8,
        eps_1q=5e-3, eps_2q=5e-2,
        t1_ns=50_000, t2_ns=30_000, p_ro=0.05),
}

K_REPETITIONS, SHOTS, MAX_ITER = 30, 1024, 50

for rep in range(K_REPETITIONS):
    first = {'m1': None, 'top4': None, 'm2': None}
    for iteration in range(1, MAX_ITER + 1):
        simulation_seed = rep * 1_000_000 + iteration * 10_000
        counts = simulator.run(compiled, shots=SHOTS,
            seed_simulator=simulation_seed).result().
            get_counts()
        flags = evaluate_same_histogram(counts,
            simulation_seed, ...)
        update_first_success(first, flags, iteration)
    encoded = {name: value if value is not None else MAX_ITER
        + 1
        for name, value in first.items()}

```

```
# Inferenza: wilcoxon(paired_a, paired_b,
#                               zero_method='pratt', alternative=...)
# Output: schema_version='2.0', input espliciti e manifest/
# hash.
```

Listing B.10: Orchestrazione appaiata delle sole campagne rumorose N15

```
if logical_value == 1:
    qc.x(0)
qc.cx(0, 1); qc.cx(0, 2)
if basis == 'X':
    qc.h(DATA)
for q in DATA:
    qc.id(q) # punto di iniezione del
             rumore
if basis == 'Z':
    qc.cx(0, 3); qc.cx(1, 3)
    qc.cx(1, 4); qc.cx(2, 4)
else:
    qc.h(3); qc.cx(3, 0); qc.cx(3, 1); qc.h(3)
    qc.h(4); qc.cx(4, 1); qc.cx(4, 2); qc.h(4)
qc.measure(ANC[0], 0); qc.measure(ANC[1], 1)
```

Listing B.11: Codifica ed estrazione di sindrome del codice a ripetizione.

```
def encode_zero_L(qc):
    for seed in SEED:
        qc.h(seed)
    for seed, support in SEED.items():
        for data in support:
            if data != seed:
                qc.cx(seed, data)

def measure_syndrome(qc):
    for k, support in enumerate(GZ):
        for data in support:
            qc.cx(data, ANC_Z[k])
    for k, support in enumerate(GX):
        qc.h(ANC_X[k])
```

```

    for data in support:
        qc.cx(ANC_X[k], data)
    qc.h(ANC_X[k])

```

Listing B.12: Preparazione di $|0_L\rangle$ e misura degli stabilizzatori di Steane.

```

def inject(block, pairs, p_ct):
    out = stim.Circuit()
    first = True
    for inst in block:
        if isinstance(inst, stim.CircuitRepeatBlock):
            out += inject(inst.body_copy(), pairs, p_ct) *
                inst.repeat_count
            continue
        out.append(inst)
        if inst.name == 'DEPOLARIZE1':
            if first:
                out.append('DEPOLARIZE2', pairs, p_ct)
                first = not first
    return out

```

Listing B.13: Iniezione ricorsiva del crosstalk nel circuito del surface code.

```

circuit = stim.Circuit.generated(
    'surface_code:rotated_memory_x', distance=d, rounds=rounds
    ,
    after_clifford_depolarization=p,
    after_reset_flip_probability=p,
    before_measure_flip_probability=p,
)
sampler = circuit.compile_detector_sampler()
events, observables = sampler.sample(shots,
    separate_observables=True)
model = circuit.detector_error_model(decompose_errors=True)
matching = pymatching.Matching.from_detector_error_model(model
    )
predictions = matching.decode_batch(events)
p_L = (predictions != observables).any(axis=1).mean()

```

Listing B.14: Stima di p_L con Stim e PyMatching.

Appendice C

Contesto e Motivazione

C.1 Contesto Esteso dell'Introduzione

Questa appendice raccoglie in forma estesa il contesto richiamato sinteticamente nel Capitolo 1: i principi fisici del calcolo quantistico e la loro implementazione hardware, la rivoluzione algoritmica degli anni Novanta, il protocollo RSA e la sua vulnerabilità all'algoritmo di Shor, la crittografia post-quantistica e lo stato dell'arte dell'hardware quantistico.

C.1.1 Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico

Per comprendere appieno le potenzialità e i limiti del calcolo quantistico, è necessario esaminare le differenze fisiche fondamentali tra l'unità di informazione classica e quella quantistica.

Il Qubit e la Sovrapposizione

Un computer classico elabora l'informazione attraverso **bit** – unità fisiche che possono trovarsi in uno di due stati discreti, convenzionalmente 0 e 1, realizzati come livelli di tensione in circuiti elettronici. L'unità fondamentale del calcolo quantistico è invece il **qubit** (*quantum bit*): un sistema fisico a due livelli che obbedisce alle leggi della meccanica quantistica.

Matematicamente, lo stato di un qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H} \cong \mathbb{C}^2$:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (\text{C.1})$$

dove $|0\rangle$ e $|1\rangle$ sono i due stati della base computazionale, analoghi ai valori 0 e 1 del bit classico. La differenza fondamentale è che α e β possono essere simultaneamente diversi da zero: il qubit si trova in una **sovrapposizione** dei due stati base, con probabilità $|\alpha|^2$ di essere misurato in $|0\rangle$ e probabilità $|\beta|^2$ di essere misurato in $|1\rangle$.

È cruciale sottolineare che la sovrapposizione non equivale a ignoranza sullo stato del sistema: il qubit non è “segretamente” in uno stato definito che non conosciamo. Si tratta di una proprietà fisica genuina, verificata sperimentalmente con precisione straordinaria, che non ha analogo nella fisica classica.

La Sfera di Bloch

Poiché la norma è unitaria e la fase globale è fisicamente irrilevante, lo stato di un qubit puro può essere parametrizzato da soli due angoli reali:

$$|\psi\rangle = \cos\frac{\theta}{2} |0\rangle + e^{i\phi} \sin\frac{\theta}{2} |1\rangle, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi) \quad (\text{C.2})$$

Questo definisce un punto sulla superficie di una sfera unitaria – la **sfera di Bloch** – dove il polo nord corrisponde a $|0\rangle$, il polo sud a $|1\rangle$, e i punti equatoriali a sovrapposizioni equiprobabili con fasi diverse. Questa rappresentazione geometrica è di grande utilità pratica: le operazioni unitarie su un singolo qubit corrispondono a rotazioni, mentre i canali rumorosi trasformano in generale la sfera in un ellissoide traslato. Un canale unital può contrarre verso il centro; l’amplitude damping trasla invece gli stati verso $|0\rangle$.

L’Entanglement

Quando due o più qubit interagiscono tramite porte quantistiche opportune, possono formare stati **entangled**: stati del sistema composto che non possono essere scritti come prodotto tensoriale di stati individuali. Il prototipo è lo stato di Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (\text{C.3})$$

In questo stato, se la misura del primo qubit produce 0, una misura del secondo nella stessa base produce anch'essa 0, e analogamente per 1, indipendentemente dalla distanza fisica; la correlazione non consente però di trasmettere informazione più velocemente della luce. Con opportune scelte delle basi di misura, gli stati entangled violano disuguaglianze di Bell e producono correlazioni incompatibili con modelli a variabili nascoste locali [43, 44]. Un registro di n qubit descrive ampiezze in uno spazio $\mathbb{C}^{2^n}$, ma questa dimensione non basta da sola a garantire un vantaggio: alcune famiglie entangled, come gli stati stabilizer, restano simulabili classicamente in modo efficiente. Il vantaggio algoritmico nasce dalla struttura con cui preparazione, entanglement e interferenza trasformano tali ampiezze.

L'Interferenza Quantistica

Il terzo pilastro del calcolo quantistico è l'**interferenza**: la capacità di far sommare costruttivamente o annullare distruttivamente le ampiezze di probabilità di percorsi computazionali diversi. Le ampiezze α e β in (C.1) sono numeri complessi, e possono quindi avere segni opposti o sfasamenti arbitrari: quando si sommano, possono amplificarsi o cancellarsi esattamente come onde fisiche.

Un algoritmo quantistico ben progettato sfrutta questo meccanismo per modificare la distribuzione degli esiti: Grover amplifica l'ampiezza degli elementi marcati, mentre in Shor la QFT concentra la probabilità vicino alle frequenze legate al periodo. Non tutti gli esiti prodotti sono quindi “corretti”, e resta necessario un post-processing classico probabilistico.

Implementazione Fisica: i Qubit Superconduttori di IBM

Un qubit può essere realizzato con qualunque sistema fisico a due livelli: la polarizzazione di un fotone, lo spin di un elettrone, i livelli energetici di un atomo intrappolato. I **qubit superconduttori** utilizzati da IBM sono circuiti elettrici macroscopici che operano nel regime quantistico grazie al raffreddamento a temperature criogeniche estreme. In questa tesi costituiscono l'architettura di riferimento per la snapshot FakeSherbrooke e per la discussione hardware; gli esperimenti principali restano simulazioni classiche.

Il componente chiave è la **giunzione di Josephson** [45]: una sottile barriera isolante (spessa $\sim 1\text{--}2\text{ nm}$) interposta tra due materiali superconduttori. Questo elemento introduce una non-linearità nell'energia del circuito che rompe l'equidistanza dei livelli energetici, permettendo di indirizzare selettivamente i due stati

computazionali $|0\rangle$ e $|1\rangle$. Il risultato è il **transmon** [46] – il tipo di qubit superconduttore adottato da IBM – un oscillatore anarmonico con frequenza tipica 4–6 GHz, controllato da impulsi a microonde.

Per operare nel regime quantistico, questi circuiti devono essere raffreddati a temperature dell'ordine di 15 mK (circa -273.135°C), molto inferiori ai 2.7 K della radiazione cosmica di fondo, tramite refrigeratori a diluizione. Il raffreddamento abilita il regime superconduttivo e sopprime fortemente le eccitazioni termiche, ma non elimina la dissipazione né le altre sorgenti di decoerenza.

I parametri operativi tipici dei processori IBM (serie Eagle, Heron) sono:

- **Temperatura operativa:** ~ 15 mK
- **Tempi di coerenza:** $T_1, T_2 \sim 50\text{--}500\ \mu\text{s}$
- **Durata gate a singolo qubit:** ~ 50 ns
- **Durata gate a due qubit (ECR/CNOT):** $\sim 300\text{--}500$ ns
- **Fedeltà gate a singolo qubit:** $\sim 99.9\text{--}99.97\%$
- **Fedeltà gate a due qubit:** $\sim 99\text{--}99.7\%$

Il rapporto tra il tempo di coerenza e la durata di un gate contribuisce a determinare quante operazioni utili possano essere eseguite prima che il rumore domini; insieme agli errori di controllo, al crosstalk, al readout e alla connettività, è uno dei principali colli di bottiglia per algoritmi profondi come Shor sui processori attuali.

C.1.2 La Rivoluzione Algoritmica degli Anni Novanta

Negli anni successivi alla proposta di Deutsch, la comunità scientifica si interrogò sulla effettiva esistenza di problemi che un computer quantistico potesse risolvere in modo *probabilmente* più efficiente di qualunque macchina classica. La risposta arrivò in forma dirompente nel corso di pochi anni.

Nel 1994, Daniel Simon¹ identificò un problema oracle per il quale un algoritmo quantistico richiede esponenzialmente meno interrogazioni di qualunque algoritmo classico probabilistico. Nello stesso anno Peter Shor pubblicò algoritmi quantistici polinomiali per fattorizzazione e logaritmo discreto; nella formulazione textbook della fattorizzazione, il costo è spesso riassunto come $O(n^3)$ [1]. Le implicazioni furono immediate: RSA dipende dalla difficoltà della fattorizzazione, mentre Diffie–Hellman ed *Elliptic Curve Digital Signature Algorithm* (ECDSA) dipendono da varianti del logaritmo discreto. Una macchina quantistica sufficientemente grande comprometterebbe quindi molte delle primitive a chiave pubblica oggi impiegate, pur senza rendere insicura ogni componente simmetrica dei protocolli.

¹L'**algoritmo di Simon**, presentato nel 1994 e pubblicato in forma estesa nel 1997, dimostrò nel modello oracle una separazione esponenziale rispetto agli algoritmi classici probabilistici. Shor riconobbe che la sua tecnica si ispirava direttamente alle idee di Simon.

Due anni dopo, nel **1996**, Lov Grover presentò un algoritmo di tutt'altra natura, ma di eguale importanza [35]. Mentre Shor aveva risolto un problema specifico della teoria dei numeri, Grover affrontò una sfida universale: la **ricerca non strutturata**. Dato un insieme di N elementi senza alcuna struttura o ordinamento, trovare quello che soddisfa una certa proprietà richiede, nel caso classico, un numero di interrogazioni proporzionale a N . Grover dimostrò che un algoritmo quantistico può fare lo stesso lavoro in sole $O(\sqrt{N})$ interrogazioni – uno *speedup quadratico* – e che questo risultato è *ottimale*: nessun algoritmo quantistico può fare di meglio [37]. Sebbene meno spettacolare dello speedup esponenziale di Shor, il vantaggio di Grover è universale, applicabile a qualunque problema di ricerca, e porta con sé implicazioni profonde per la crittografia simmetrica, l'ottimizzazione combinatoria e la ricerca in grandi basi di dati.

Questi due algoritmi – Shor e Grover – rappresentano ancora oggi i pilastri dell'informatica algoritmica quantistica: il primo come dimostrazione del potere *esponenziale* del calcolo quantistico su problemi strutturati, il secondo come strumento *universale* di accelerazione quadratica applicabile al problema computazionale più fondamentale.

C.1.3 RSA e le Implicazioni Crittografiche dell'Algoritmo di Shor

Per comprendere appieno la portata dell'algoritmo di Shor è necessario esaminare la struttura matematica del sistema crittografico che esso minaccia. Il sistema **RSA**, proposto nel 1977 da Rivest, Shamir e Adleman e pubblicato nel 1978 [2], è uno degli schemi storicamente centrali della crittografia a chiave pubblica. Continua a essere impiegato in certificati e firme, ma i protocolli moderni possono adottare anche primitive diverse, in particolare su curve ellittiche.

Struttura del Protocollo RSA

La sicurezza di RSA si basa sulla **asimmetria computazionale** tra due operazioni inverse: moltiplicare due numeri primi grandi p e q per ottenere $N = p \cdot q$ è un'operazione elementare (complessità $O(n^2)$ per numeri a n bit), mentre fattorizzare N per recuperare p e q è computazionalmente intrattabile con gli algoritmi classici noti. La generazione delle chiavi segue i passi seguenti:

1. Scegliere due primi grandi p e q (tipicamente a 1024–2048 bit ciascuno) e calcolare $N = p \cdot q$;
2. Calcolare la funzione di Eulero $\phi(N) = (p-1)(q-1)$;
3. Scegliere e coprimo con $\phi(N)$ (convenzionalmente $e = 65537 = 2^{16} + 1$, primo di Fermat);
4. Calcolare $d \equiv e^{-1} \pmod{\phi(N)}$ tramite l'algoritmo di Euclide esteso.

La **chiave pubblica** è la coppia (N, e) ; la **chiave privata** è d . Cifrare un messaggio $m < N$ produce il crittogramma $c = m^e \bmod N$; decifrarlo richiede $m = c^d \bmod N$. La correttezza si fonda sul **teorema di Eulero** — la generalizzazione del piccolo teorema di Fermat a moduli composti, secondo cui $m^{\phi(N)} \equiv 1 \pmod{N}$ per m coprimo con N : essendo $ed \equiv 1 \pmod{\phi(N)}$, si ha $ed = 1 + k\phi(N)$ e quindi $(m^e)^d = m^{1+k\phi(N)} \equiv m \pmod{N}$. Tutta la sicurezza risiede nell'impossibilità di calcolare d da (N, e) senza conoscere la fattorizzazione di N .

Perché l'Algoritmo di Shor Rompe RSA

Il miglior algoritmo classico per la fattorizzazione di un intero a n bit è il *General Number Field Sieve* (GNFS) [47], con complessità sub-esponenziale:

$$T_{\text{GNFS}}(N) = O(\exp[c \cdot (\log N)^{1/3} (\log \log N)^{2/3}]) \quad (\text{C.4})$$

Per N a 2048 bit, questa espressione implica circa 2^{112} operazioni classiche — un numero astronomico, ben oltre la capacità di qualunque infrastruttura computazionale esistente o prevedibile in ambito classico. I migliori risultati pratici al 2020 hanno raggiunto la fattorizzazione di RSA-250 (829 bit) [14], impiegando circa 2700 anni-CPU di calcolo distribuito.

L'algoritmo di Shor riduce questa complessità a $O(n^3)$ — un salto da sub-esponenziale a **polinomiale** — rendendo la fattorizzazione di chiavi RSA-2048 un problema risolvibile in ore su un computer quantistico fault-tolerant con qualche migliaio di qubit logici. La struttura dell'algoritmo, descritta in dettaglio nel Capitolo 3, sfrutta la Quantum Fourier Transform per trovare il periodo di una funzione modulare, problema che si riduce efficientemente alla fattorizzazione.

Implicazioni Pratiche per la Sicurezza Informatica

Le conseguenze sono sistemiche e riguardano più in generale la crittografia a chiave pubblica vulnerabile a Shor: RSA, Diffie–Hellman e le primitive su curve ellittiche. Questi algoritmi compaiono, in combinazioni diverse, in:

- **HTTPS e TLS:** per autenticazione e scambio delle chiavi di sessione, a seconda della versione e della configurazione;
- **Certificati X.509:** nell’infrastruttura a chiave pubblica usata per autenticare servizi e soggetti;
- **SSH:** il protocollo standard per l’accesso remoto sicuro ai server;
- **Firme digitali:** documenti legali, transazioni finanziarie, aggiornamenti software firmati.

Un computer quantistico crittograficamente rilevante renderebbe vulnerabili anche comunicazioni registrate in precedenza quando la riservatezza della chiave di sessione dipende da una primitiva a chiave pubblica attaccabile da Shor. È il rischio *“harvest now, decrypt later”*: acquisire oggi dati cifrati ancora sensibili e conservarli per una futura decifratura. Il NIST sottolinea però che non esiste una data affidabile per la comparsa di una macchina di questo tipo; le stime spaziano da pochi anni a decenni. Le scadenze 2030–2035 presenti nelle roadmap istituzionali sono obiettivi di migrazione e dismissione degli algoritmi vulnerabili, non previsioni della data in cui RSA-2048 verrà violato [48, 49].

C.1.4 Crittografia Post-Quantistica: la Risposta dell’Industria

La consapevolezza della minaccia quantistica ha spinto il NIST a lanciare nel 2016 un processo di standardizzazione internazionale per la **crittografia post-quantistica** (*Post-Quantum Cryptography* (PQC)) – algoritmi crittografici resistenti agli attacchi di computer quantistici, progettati per essere eseguiti su hardware classico convenzionale e quindi immediatamente distribuibili.

Gli Standard NIST 2024

Dopo otto anni di valutazione pubblica, crittoanalisi internazionale e tre successivi turni di selezione tra 69 candidati iniziali, nell’agosto 2024 il NIST ha pubblicato i primi tre standard definitivi [50]:

- **ML-KEM** (FIPS 203, ex CRYSTALS-Kyber): meccanismo di incapsulamento di chiavi basato sul problema *Module Learning With Errors* (MLWE) su reticoli. È il sostituto designato per lo scambio di chiavi RSA e Diffie-Hellman nei protocolli TLS;
- **ML-DSA** (FIPS 204, ex CRYSTALS-Dilithium): schema di firma digitale basato sullo stesso fondamento matematico di ML-KEM. Sostituisce RSA e ECDSA per le firme digitali;
- **SLH-DSA** (FIPS 205, ex SPHINCS+): schema di firma basato esclusivamente su funzioni hash, senza assunzioni algebriche. Più conservativo per sicurezza, adottato come backup contro potenziali attacchi futuri ai reticoli.

La sicurezza di questi schemi si fonda su problemi matematici – trovare vettori corti in reticoli ad alta dimensione, o invertire funzioni hash – per i quali non sono noti algoritmi quantistici con speedup esponenziale. In particolare, l’algoritmo di Grover fornisce al più uno speedup quadratico sui problemi basati su hash, motivando l’aumento dei parametri di sicurezza (es. passaggio da 128 a 256 bit di sicurezza classica equivalente).

La Sfida della Migrazione

La transizione verso la crittografia post-quantistica è un’operazione di ingegneria su scala globale, paragonabile per complessità alla migrazione da IPv4 a IPv6, ma con urgenza maggiore. Le principali sfide includono:

- **Eterogeneità dei sistemi:** miliardi di dispositivi embedded, sensori industriali e infrastrutture legacy con cicli di vita decennali che non riceveranno aggiornamenti software;
- **Overhead di banda:** il materiale crittografico post-quantistico è sensibilmente più voluminoso. Al livello di sicurezza più basso (ML-KEM-512), la chiave di incapsulamento occupa 800 byte e il crittogramma 768 byte, contro i circa 256 byte di una chiave pubblica RSA-2048: un fattore di crescita che pesa sui protocolli a bassa latenza e sui dispositivi con banda limitata;
- **Migrazione a doppio binario:** nella fase transitoria, molte implementazioni adottano schemi *ibridi* (classico + post-quantistico in parallelo) per mantenere compatibilità con sistemi non aggiornati.

Aziende come Google (Chrome 131), Apple (iMessage PQ3) [51] e Cloudflare hanno già avviato implementazioni ibride nei loro protocolli di produzione. La coesistenza di questa transizione con la ricerca attiva su algoritmi quantistici come Shor definisce il contesto strategico in cui si inserisce il presente lavoro: comprendere i limiti reali di Shor su hardware NISQ è rilevante non solo teoricamente, ma anche per calibrare con maggiore precisione le tempistiche e le priorità della transizione crittografica globale.

C.1.5 Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant

Nonostante la solidità teorica di questi risultati, la realizzazione pratica di computer quantistici di scala sufficiente ad eseguirli su istanze realmente significative incontra ostacoli formidabili. I sistemi quantistici fisici sono intrinsecamente fragili: qualunque interazione non controllata con l'ambiente esterno – vibrazioni termiche, campi elettromagnetici, imperfezioni nei materiali – provoca fenomeni di **decoerenza** – ossia la perdita progressiva della coerenza quantistica causata dall'entanglement indesiderato tra il qubit e i gradi di libertà ambientali, che sopprime le interferenze quantistiche riducendo il sistema a una miscela statistica classica – degradando lo stato quantistico in modo irreversibile prima che il calcolo possa completarsi. A differenza di un bit classico, che può essere protetto dalla ridondanza, un qubit non può essere copiato (teorema del no-cloning [16]) né letto senza distruggerlo: questo rende la correzione degli errori quantistici un problema radicalmente diverso e molto più complesso di quella classica.

Il panorama tecnologico attuale è dominato da un insieme variegato di piattaforme hardware in competizione, ciascuna con vantaggi e svantaggi distinti:

- **Qubit superconduttori** (IBM, Google, Rigetti): alta velocità dei gate ($\sim 50\text{--}500\text{ ns}$), scalabilità su chip in silicio, ma tempi di coerenza limitati a $\sim 100\text{--}500\text{ }\mu\text{s}$ e necessità di raffreddamento a 15 mK ;
- **Ioni intrappolati** (IonQ, Quantinuum): tempi di coerenza eccellenti ($> 1\text{ s}$), fedeltà dei gate superiore al 99.9% anche su due qubit, ma gate lenti ($\sim 10\text{--}100\text{ }\mu\text{s}$) e scalabilità più complessa;
- **Fotoni** (PsiQuantum, Xanadu): operano a temperatura ambiente, naturalmente adatti alla comunicazione quantistica, ma la generazione deterministica di fotoni entangled rimane una sfida aperta;
- **Spin in silicio** (Intel, UNSW): compatibili con i processi di fabbricazione CMOS esistenti, potenzialmente scalabili a milioni di qubit, ma con tecnologia di controllo ancora in fase di sviluppo.

Un momento simbolico di questa corsa fu raggiunto nel **2019**, quando il team di Google rivendicò il raggiungimento della cosiddetta *quantum supremacy*² con il processore Sycamore a 53 qubit [52]: un calcolo specifico fu completato in 200 secondi, mentre le stime attribuivano al più potente supercomputer classico un tempo di circa 10.000 anni per lo stesso compito. Nel 2023, IBM ha presentato il processore *Condor* a 1.121 qubit e ha delineato una roadmap [34] verso sistemi con architettura fault-tolerant entro la fine del decennio, con il target *Quantum Starling* previsto per il 2029.

Tuttavia, la distanza tra i dispositivi attuali e un computer quantistico *fault-tolerant* — in grado di eseguire Shor su istanze RSA reali — è ancora enorme. I processori odierni sono comunemente descritti nel regime NISQ, termine introdotto da John Preskill nel 2018 [3]: il numero di qubit fisici, da solo, non li rende fault-tolerant, perché errori, connettività e assenza di correzione scalabile limitano la profondità affidabile. In questo scenario, la ricerca si concentra su due direzioni parallele e complementari:

- **QEC:** tecniche per codificare qubit logici in qubit fisici ridondanti, in modo da rilevare e correggere gli errori durante il calcolo, a costo di un overhead fisico significativo (centinaia o migliaia di qubit fisici per qubit logico);
- **Noise mitigation:** tecniche che, senza richiedere overhead hardware, riducono l'effetto del rumore a livello di post-processing classico, estendendo la portata dei circuiti eseguibili su hardware NISQ.

C.2 Dettaglio degli Obiettivi e del Piano Sperimentale

Questa appendice raccoglie il materiale di dettaglio del Capitolo 2: la motivazione crittografica estesa, i requisiti funzionali del sistema sperimentale e la specifica completa dei parametri di input e di output.

²Il termine *quantum supremacy*, coniato da John Preskill nel 2012, indica il punto in cui un computer quantistico esegue un calcolo specifico in un tempo che nessun supercomputer classico potrebbe eguagliare in modo pratico. La rivendicazione di Google con il processore Sycamore è rimasta controversa: IBM sostenne che il calcolo avrebbe potuto essere eseguito classicamente in 2.5 giorni anziché 10.000 anni, con un algoritmo migliorato. Questo dibattito evidenzia la difficoltà di definire e misurare il vantaggio quantistico.

C.2.1 Motivazione: la Vulnerabilità della Crittografia Attuale

La quasi totalità delle comunicazioni digitali sicure — dalle transazioni bancarie alle connessioni *HyperText Transfer Protocol Secure* (HTTPS), dai protocolli *Secure Shell* (SSH) ai certificati digitali — è protetta da sistemi crittografici la cui sicurezza si fonda su un presupposto computazionale: la difficoltà di fattorizzare il prodotto di due grandi numeri primi. Il sistema RSA, ad esempio, costruisce la propria chiave pubblica come $N = p \cdot q$, dove p e q sono primi di centinaia o migliaia di bit. Finché non esiste un algoritmo classico efficiente per recuperare p e q a partire da N , la chiave privata rimane al sicuro.

Per comprendere perché i sistemi crittografici attuali siano strutturalmente vulnerabili all'algoritmo di Shor, è necessario mettere a confronto le complessità computazionali in gioco. Il miglior algoritmo classico per la fattorizzazione, il GNFS, richiede un tempo:

$$T_{\text{classico}}(N) = O(\exp((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3})) \quad (\text{C.5})$$

che, pur crescendo più lentamente di un esponenziale puro, rimane del tutto impraticabile per i valori di N usati in crittografia. A titolo di esempio, fattorizzare un numero RSA a 2048 bit con algoritmi classici richiederebbe un tempo stimato superiore all'età dell'universo, anche impiegando l'intera potenza di calcolo disponibile sul pianeta.

Nel 1994, Peter Shor dimostrò che un computer quantistico è in grado di eseguire la stessa operazione in tempo **polinomiale** [1]:

$$T_{\text{Shor}}(N) = O((\log N)^3) \quad (\text{C.6})$$

Lo scarto tra le due espressioni non è una differenza di grado, ma una differenza di *natura*: il tempo classico cresce in modo super-polinomiale con la lunghezza della chiave, mentre il tempo quantistico cresce polinomialmente. **Aumentare la lunghezza della chiave RSA non è quindi una contromisura strutturale** contro Shor: raddoppiare n porta il costo asintotico textbook da $O(n^3)$ a $8O(n^3)$. Sul piano concreto questo fattore può richiedere molte più risorse fault-tolerant e ritardare un attacco, ma non ripristina un problema computazionalmente difficile per un avversario quantistico scalabile. La stessa vulnerabilità asintotica si estende ai sistemi basati sul logaritmo discreto (Diffie–Hellman, ECDSA).

La minaccia ha già una conseguenza operativa: nel modello *harvest now, decrypt*

later, dati cifrati oggi e destinati a restare sensibili vengono conservati in vista di una futura macchina crittograficamente rilevante. Non esiste però una previsione affidabile della sua data di arrivo. Le roadmap dei produttori descrivono obiettivi ingegneristici e il NIST sottolinea che le stime spaziano da pochi anni a decenni; proprio l'incertezza, unita ai tempi lunghi di migrazione, motiva l'adozione anticipata della PQC [48, 34].

Gli ostacoli pratici sono la scala, la profondità fault-tolerant e il **rumore**. I processori attuali operano in regime NISQ: dispongono di molti qubit fisici, ma decoerenza, errori di gate, readout, connettività e overhead di correzione impediscono di sostenere il numero di operazioni logiche richiesto da Shor su chiavi crittografiche. Comprendere questi limiti e stabilire quali strategie li riducano — e in quale punto della pipeline i modelli di machine learning contribuiscano davvero — è la ragione d'essere del presente lavoro.

C.2.2 Requisiti Funzionali

Definiti gli obiettivi e il contributo originale, è possibile formalizzare cosa il sistema sperimentale deve concretamente fare. I requisiti seguenti traducono le ipotesi di ricerca in vincoli tecnici verificabili, e costituiscono il riferimento rispetto al quale l'implementazione descritta nel Capitolo 6 verrà valutata.

Il sistema sperimentale deve soddisfare i seguenti requisiti funzionali:

1. **Esecuzione configurabile entro il perimetro validato.** La campagna rumorosa e la demo pubblica usano l'istanza canonica $N = 15$, $a = 7$ e un numero configurabile di shot entro limiti dichiarati. Le implementazioni Beauregard per $N = 21$ e $N = 35$ sono sottoposte a test ideali e audit di risorse, ma non sono esposte come campagne rumorose equivalenti.
2. **Modello di rumore parametrizzabile.** Il sistema permette di configurare separatamente i parametri Aer λ_{1q} e λ_{2q} , i tempi T_1/T_2 , il readout asimmetrico della demo e l'eventuale sovrarotazione coerente. I preset della campagna restano uniformi e illustrativi; non riproducono una calibrazione hardware nominata.
3. **Raccolta e archiviazione degli output.** Il sistema registra la distribuzione di misura del registro di controllo per ogni esecuzione e la archivia in formato strutturato. La riproducibilità usa un seed base e calendari derivati separati per replica, iterazione, simulatore e risoluzione dei pareggi.

4. **Analisi statistica del Metodo 1.** Il sistema implementa la pipeline del Metodo 1: selezione della misura più frequente dell'istogramma (strategia TOP-1, senza sogliatura preliminare), stima del periodo r tramite frazioni continue, verifica della correttezza dei fattori estratti. Misura e registra il numero di iterazioni necessarie per raggiungere il primo risultato corretto.
5. **Training, validazione e inferenza del Metodo 2.** Il sistema genera il dataset dal simulatore rumoroso, addestra il classificatore con divisione training/validation/test e ne valuta le metriche. Il modello corrente apprende l'etichetta TOP-1: decide se il candidato dominante conduce ai fattori; se accetta l'istogramma, M2 applica poi la ricerca TOP-4. L'audit TOP-16 è a classe singola e non produce un classificatore.
6. **Confronto sistematico sui use case previsti.** M1, TOP-4 e M2 vengono eseguiti sugli stessi istogrammi nei due preset $N = 15$, producendo metriche appaiate secondo la Sezione 2.6. Per $N = 21$ e $N = 35$ si riportano correttezza ideale e costo compilato; non si afferma che la probabilità rumorosa sia zero, perché la campagna rumorosa non viene eseguita.
7. **Riproducibilità.** Tutti gli esperimenti sono riproducibili: seed fisso, parametri esplicitati, codice disponibile nel repository del progetto. Questo requisito è condizione necessaria per la validazione accademica dei risultati [38].

C.2.3 Parametri di Input e Output

Parametri di Input

La Tabella C.1 elenca i parametri di configurazione del sistema. I loro valori vengono fissati per ciascun use case prima dell'esecuzione e rimangono costanti per tutta la durata dell'esperimento corrispondente.

Parametro	Unità	Descrizione
N	intero	Numero da fattorizzare
a	intero	Base della moltiplicazione modulare ($\gcd(a, N) = 1$)
n_{count}	intero	Qubit del registro di controllo
M_{shots}	intero	Shot per esecuzione del simulatore
λ_{1q}	adimensionale	Parametro Aer del canale depolarizzante 1Q
λ_{2q}	adimensionale	Parametro Aer del canale depolarizzante 2Q
T_1	ns	Tempo di rilassamento (amplitude damping)
T_2	ns	Tempo di dephasing
p_{ro}	adimensionale	Probabilità di readout error

Tabella C.1: Parametri principali del sistema sperimentale. λ segue la convenzione Aer e non coincide con la probabilità aggregata di Pauli non identità.

Output del Sistema

Per ogni use case e per ciascuno dei due metodi, il sistema produce le seguenti quantità:

- I fattori trovati p e q tali che $p \cdot q = N$, con verifica esplicita della correttezza ($p \neq 1, q \neq 1, p \cdot q = N$);
- Il numero di iterazioni necessarie per il primo risultato corretto;
- La distribuzione di frequenza completa degli output del registro di controllo su M_{shots} esecuzioni;
- Le metriche di confronto tra i due metodi definite nella Sezione 2.6.

Per il solo Metodo 2, vengono prodotte in aggiunta:

- L'accuratezza del classificatore e il punteggio F1 sul test set;
- La curva ROC con l'area sottostante (AUC).

Appendice D

Fase 1: la Campagna Classica nel Dettaglio

D.1 Campagna Parametrica e Confronto con la Zero-Noise Extrapolation

Questa appendice riporta la campagna parametrica schema 2 sulla strategia TOP-K. Il protocollo usa $N = 15$, $a = 7$, $n_{\text{count}} = 8$, 30 repliche, budget massimo 50 e, salvo diversa indicazione, 1 024 shot. Il preset di riferimento è $\lambda_{1q} = 10^{-3}$, $\lambda_{2q} = 10^{-2}$, $T_1/T_2 = 100/80 \mu\text{s}$ e readout simmetrico 2%.

M1 e TOP-K condividono lo stesso istogramma per ogni coppia replica-iterazione. I fallimenti sono codificati con sentinella 51 e i pareggi con una priorità SHA-256 dipendente dal seed. I confronti usano Wilcoxon appaiato-Pratt. Poiché non è applicata una correzione globale fra le otto famiglie, i p di questa appendice hanno finalità esplorativa.

D.1.1 Variazione di K

Tabella D.1: M1 e TOP-K sui medesimi 30 vettori di iterazioni. $K = 1$ coincide con il controllo M1.

K	$\bar{M}_1$	$\bar{M}_{\text{TOP}K}$	successo	ρ	p
1	1.37	1.37	100%	1.000	1.000
2	1.37	1.00	100%	1.367	< 0.001
3	1.37	1.00	100%	1.367	< 0.001
4	1.37	1.00	100%	1.367	< 0.001
6	1.37	1.00	100%	1.367	< 0.001
8	1.37	1.00	100%	1.367	< 0.001

La soglia osservata è $K = 2$, non $K = r = 4$. Questo risultato dipende dalla banda di esiti che la post-elaborazione mappa su un periodo utile e non costituisce una regola generale per altre istanze.

D.1.2 Variazione di λ_{2q}

Per il canale Aer a due qubit, $\mathcal{E}(\rho) = (1 - \lambda)\rho + \lambda I/4$, la probabilità aggregata della componente Pauli non-identità è $15\lambda/16$. Sul circuito da 224 CX il proxy di nessun evento è quindi $(1 - 15\lambda_{2q}/16)^{224}$; non è una probabilità di successo.

Tabella D.2: Il tasso di successo è 100% per tutte le 30 repliche di ciascun punto.

λ_{2q}	proxy	$\bar{M}_1$	$\bar{M}_{\text{TOP}4}$	ρ	p
10^{-3}	8.11×10^{-1}	1.93	1.00	1.933	< 0.001
5×10^{-3}	3.49×10^{-1}	1.90	1.00	1.900	< 0.001
10^{-2}	1.21×10^{-1}	1.37	1.00	1.367	< 0.001
2×10^{-2}	1.44×10^{-2}	1.33	1.00	1.333	0.001
5×10^{-2}	2.14×10^{-5}	1.33	1.00	1.333	< 0.001
10^{-1}	2.65×10^{-10}	1.57	1.07	1.469	0.001

Al punto estremo TOP-4 richiede in media 1.07 iterazioni: la robustezza non è assoluta, pur restando il successo 30/30 entro il budget.

D.1.3 Variazione degli Shot

Tabella D.3: Risultati su 30 repliche per punto.

shot	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p
128	1.37	1.00	1.367	0.001
256	1.57	1.00	1.567	< 0.001
512	1.57	1.00	1.567	0.001
1 024	1.37	1.00	1.367	< 0.001
2 048	1.70	1.00	1.700	< 0.001

TOP-4 è stabile già a 128 shot; M1 oscilla senza andamento monotono, coerentemente con una competizione campionaria fra quattro picchi.

D.1.4 Analisi Congiunta $K \times \lambda_{2q}$

Tabella D.4: Ogni cella è $\bar{M}_{\text{TOP}K}$ su 30 repliche e ha successo 100%.

K	5×10^{-3}	10^{-2}	2×10^{-2}	5×10^{-2}
2	1.00	1.00	1.00	1.00
4	1.00	1.00	1.00	1.00
6	1.00	1.00	1.00	1.00
8	1.00	1.00	1.00	1.00

D.1.5 Parametri Secondari

RZ è virtuale (0 ns), SX/X dura 50 ns e CX 300 ns. T_2 viene variato insieme a T_1 mantenendo $T_2 = 0.8T_1$.

Tabella D.5: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

T_1/T_2 (μs)	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p
20/16	1.47	1.00	1.467	< 0.001
50/40	2.20	1.00	2.200	< 0.001
100/80	1.37	1.00	1.367	< 0.001
200/160	1.60	1.00	1.600	0.001
500/400	1.80	1.00	1.800	< 0.001

Tabella D.6: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

λ_{1q}	$\bar{M}_1$	ρ	p
10^{-4}	1.60	1.600	< 0.001
5×10^{-4}	1.43	1.433	< 0.001
10^{-3}	1.37	1.367	< 0.001
5×10^{-3}	1.67	1.667	< 0.001
10^{-2}	1.57	1.567	< 0.001
2×10^{-2}	1.47	1.467	< 0.001

Tabella D.7: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

p_{ro}	$\bar{M}_1$	ρ	p
0%	1.33	1.333	0.004
1%	1.50	1.500	< 0.001
2%	1.37	1.367	< 0.001
5%	1.47	1.467	< 0.001
10%	1.30	1.300	0.002
20%	1.47	1.467	< 0.001

Le oscillazioni non monotone di M1 sconsigliano inferenze causali sui singoli parametri con questa numerosità. Il risultato supportato è più limitato: TOP-4 non varia nei punti osservati.

D.1.6 Livello di Ottimizzazione

Tabella D.8: Conteggi della compilazione effettiva e metriche su 30 repliche.

livello	CX	profondità	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ
0	242	427	1.47	1.00	1.467
1	236	421	1.50	1.00	1.500
2	224	412	1.37	1.00	1.367
3	224	412	1.37	1.00	1.367

D.1.7 Confronto con Zero-Noise Extrapolation

La ZNE digitale scala λ_{1q} e λ_{2q} , mantenendo fissi coerenza e readout. ZNE-2 usa due livelli e ZNE-3 tre livelli condivisi; non è gate folding su hardware.

Tabella D.9: I p sono corretti con Holm nella famiglia M1 contro ZNE-2, ZNE-3 e TOP-4.

strategia	successo	$\bar{M}$	shot/iter.	shot medi totali	p_{Holm}
M1	100%	1.37	1 024	1 399	—
ZNE-2	100%	1.50	2 048	3 072	1.000
ZNE-3	100%	1.57	3 072	4 813	1.000
TOP-4	100%	1.00	1 024	1 024	0.0024

Validazione fuori dalla campagna rumorosa. I moltiplicatori Beauregard per $N = 21/35$ sono validati idealmente e non entrano negli sweep. Per $N = 21$, $a = 2$ e otto controlli, la compilazione globale registra 21 036 CX e profondità 23 081; il proxy di nessun evento 2Q a $\lambda = 10^{-3}$ è 7.2379×10^{-10} , non una probabilità di successo.

D.2 Il Classificatore ML: Architettura, Addestramento e Metriche

Questa appendice documenta il classificatore valutato nella prima fase sperimentale (Capitolo 7) e, soprattutto, separa due domande che nella prima versione della campagna erano state confuse: *il candidato più frequente conduce ai fattori?* (TOP-1) e *esiste un candidato utile fra i primi sedici?* (TOP-16). Tutti i numeri riportati provengono dagli artefatti schema 2 rigenerati con il circuito $N = 15$, $a = 7$ corretto, il medesimo manifest del circuito e seed deterministici.

D.2.1 L’Ipotesi alla Prova: il Classificatore ML

Il Metodo 2 usa un classificatore come *gate di qualità*: riceve la distribuzione normalizzata sui $2^8 = 256$ esiti e, se approva l’istogramma, tenta l’estrazione dei fattori dai quattro candidati più frequenti. L’ablazione M_{TOP4} applica la stessa ricerca senza filtro e consente quindi di distinguere l’effetto del classificatore da quello della ricerca multi-candidato.

Per ciascuno dei due scenari sono stati generati 2 000 istogrammi da 1 024 shot. I cinque parametri del modello uniforme (λ_{1q} , λ_{2q} , T_1 , T_2 e readout) sono stati campionati indipendentemente in un intervallo relativo del $\pm 50\%$ attorno al preset. Il dataset è stato separato in train, validation e test con proporzioni 60/20/20: la

scelta dell'architettura usa soltanto la validation; il test viene valutato una sola volta dopo il refit su train+validation.

La regola TOP-1 produce 1 262 positivi e 738 negativi in UC1 (63.1%/36.9%), e 1 453 positivi e 547 negativi in UC2 (72.7%/27.3%). Random Forest, SVM RBF calibrata e MLP (256, 128) sono state confrontate mediante F1 sulla validation. La Tabella D.10 riporta la valutazione indipendente del modello selezionato.

Tabella D.10: Metriche TOP-1 sul test set indipendente (400 istogrammi per scenario)

Scenario	Modello selezionato	F1	Accuratezza	AUC-ROC
UC1	SVM	0.919	0.895	0.965
UC2	SVM	0.923	0.888	0.958

Sulla validation l'SVM ottiene F1 pari a 0.908 in UC1 e 0.929 in UC2, contro 0.811/0.842 della Random Forest e 0.874/0.906 dell'MLP. Queste metriche mostrano che il modello riproduce bene la regola TOP-1; non dimostrano però che il filtro riduca il costo operativo. Quel confronto richiede gli stessi istogrammi per M1, TOP-4 e M2 ed è riportato nella Sezione 7.4.

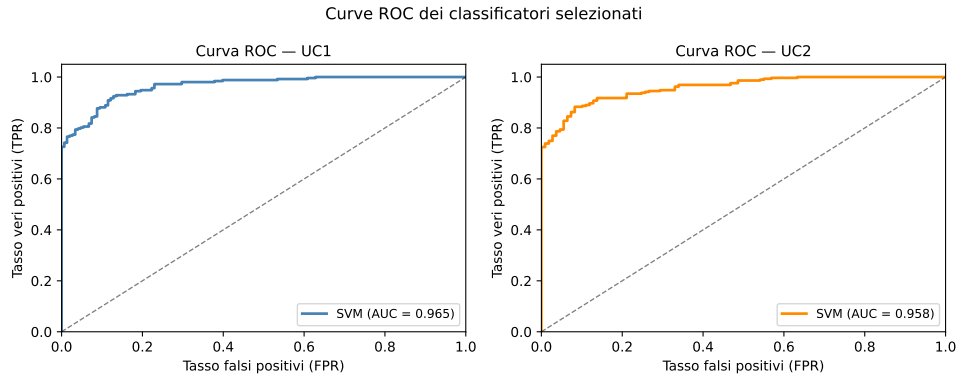


Figura D.1: Curve ROC sul test set indipendente dei due classificatori SVM selezionati mediante validation. Le aree sono 0.965 (UC1) e 0.958 (UC2); la diagonale rappresenta un classificatore casuale.

D.2.2 Audit delle Etichette: TOP-1 e TOP-16 Sono Problemi Diversi

La campagna rigenerata calcola entrambe le etichette sullo *stesso* istogramma, eliminando l'ambiguità del dataset storico. L'esito è riassunto nella Tabella D.11.

Tabella D.11: Bilanciamento delle due regole sul dataset completo. TOP-16 è a classe unica e non consente un addestramento discriminante.

Regola	UC1: positivi/negativi	UC2: positivi/negativi
TOP-1	1 262/738	1 453/547
TOP-16	2 000/0	2 000/0

Il runner non forza un modello sul dataset TOP-16: registra esplicitamente `single-class-dataset` e interrompe l'addestramento per quella regola. M2 usa perciò soltanto i modelli TOP-1 e conserva nel payload `label_top_k=1`. Questo controllo evita di attribuire capacità predittiva a un classificatore costante.

Una diagnostica separata su 60 esecuzioni nominali UC1 chiarisce la struttura della classe TOP-1. La moda è $y = 0$ in 30/60 casi e conduce ai fattori negli altri 30/60; i primi quattro candidati contengono invece almeno un esito utile in 60/60 casi. La Tabella D.12 riporta la distribuzione della moda.

Tabella D.12: Distribuzione della misura più frequente su 60 istogrammi nominali indipendenti.

Valore della moda	Occorrenze	Frazione
0	30	50.0%
64	5	8.3%
128	15	25.0%
192	10	16.7%

L'esito $y = 0$ rappresenta la fase nulla e non fornisce il periodo, ma non implica che l'istogramma sia privo di segnale. Nel campione rappresentativo della Figura D.2, la moda $y = 0$ raccoglie 176 conteggi, mentre $y = 64$, 192 e 128 ne raccolgono rispettivamente 160, 141 e 131: tutti e tre conducono ai fattori. La massa complessiva sui quattro picchi è 0.594.

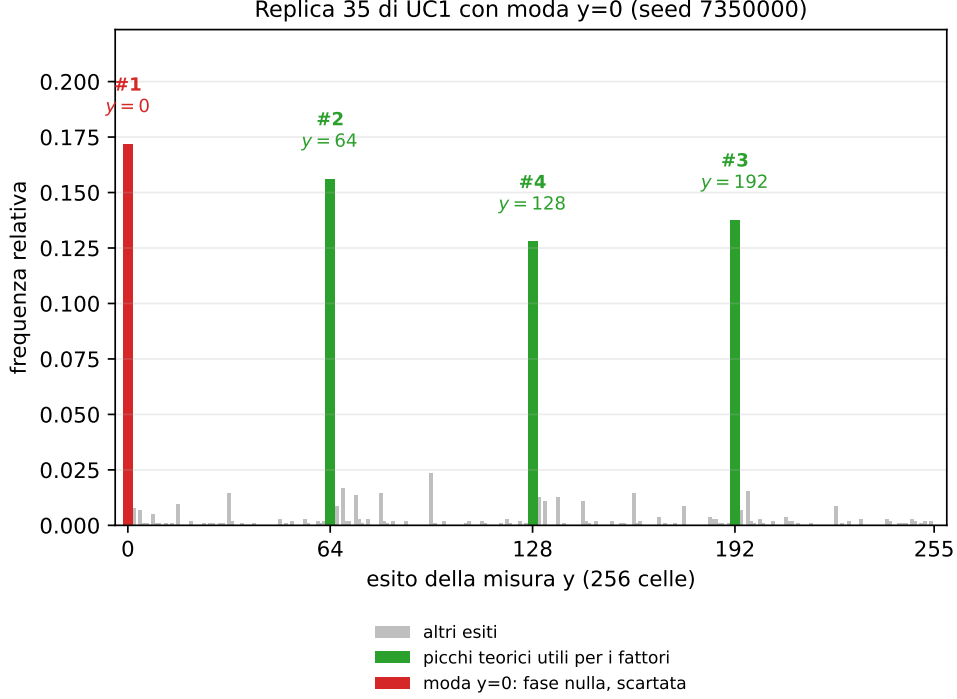


Figura D.2: Esempio nominale UC1 in cui TOP-1 fallisce perché la moda è $y = 0$, mentre i successivi tre picchi conducono ai fattori. Il classificatore TOP-1 apprende quindi una regola sul candidato dominante, non una misura generale della presenza di segnale quantistico.

Perché TOP-16 Produce una Classe Quasi Sempre Positiva

Per $N = 15$, $a = 7$ e otto bit di controllo, 63 dei 256 esiti conducono ai fattori tramite la procedura di frazioni continue adottata. Anche per un istogramma uniforme, la probabilità che nessuno dei primi K candidati sia utile è quindi

$$P_{\text{neg}}(K) = \frac{\binom{256-63}{K}}{\binom{256}{K}}. \quad (\text{D.1})$$

Per $K = 16$ essa vale soltanto 0.9275%: la regola assegna dunque un'etichetta positiva a oltre il 99% degli istogrammi uniformi per puro effetto combinatorio. Il risultato 2000/0 osservato nei due dataset è coerente con questo limite e non autorizza a concludere che il segnale sia perfetto.

Tabella D.13: Probabilità di non includere alcuno dei 63 esiti utili scegliendo K celle da 256 senza informazione sulla distribuzione.

Regola	$P_{\text{neg}}(K)$
TOP-1	75.39%
TOP-2	56.76%
TOP-4	32.06%
TOP-16	0.93%

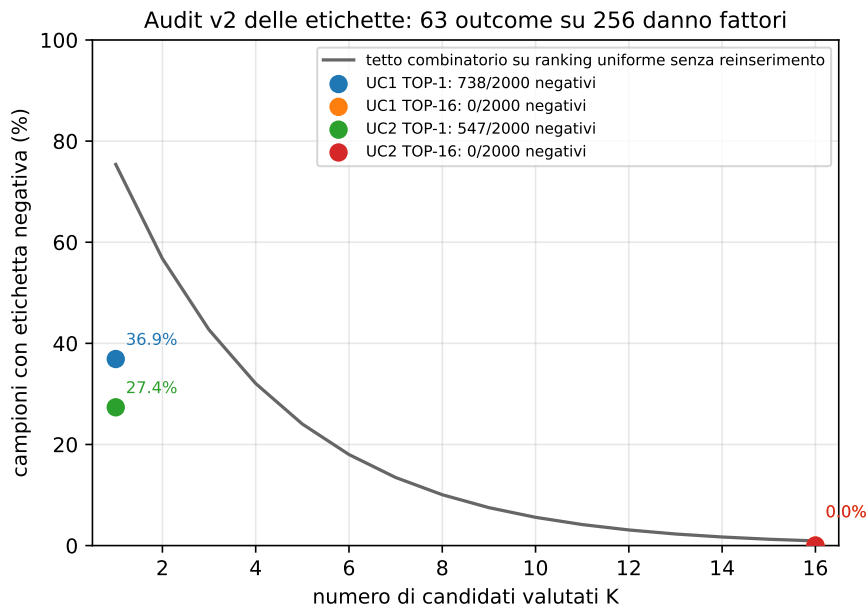


Figura D.3: Decadimento di $P_{\text{neg}}(K)$ al crescere del numero di candidati. La figura è rigenerata dai dati schema 2 e rende visibile perché TOP-16 non definisce un problema di classificazione bilanciato su questa istanza.

Il verdetto ha un perimetro preciso: non mostra che l'apprendimento automatico sia inutile in generale, ma che un filtro a valle addestrato su TOP-1 replica una decisione già superata da TOP-4, mentre la regola TOP-16 è quasi sempre positiva per costruzione. Questo motiva lo spostamento dell'apprendimento dalla distribuzione finale alla sindrome di errore nei capitoli dedicati alla QEC.

Appendice E

Fase 2: la Campagna QEC nel Dettaglio

E.1 La Decodifica Appresa: la Campagna Completa

La sezione precedente ha mostrato che il modello di rumore usato per *simulare* il codice è un'idealizzazione. Esiste però una seconda idealizzazione, indipendente dalla prima e altrettanto conseguente, che riguarda il modello di rumore usato per *decodificare*. Il MWPM non è un algoritmo generico: presuppone che gli errori si organizzino in un grafo, ossia che ogni meccanismo d'errore accenda al più due detection events e sia quindi rappresentabile come un arco. È questa proprietà — la **decomponibilità in archi** — a rendere il problema della decodifica un accoppiamento di peso minimo, ed è ciò che la libreria richiede esplicitamente quando si costruisce il modello d'errore con `decompose_errors=True`. Un errore correlato che colpisca simultaneamente due qubit dati adiacenti accende invece fino a quattro detector e non ammette alcun arco che lo rappresenti: non viola l'ipotesi di Pauli, che resta soddisfatta, ma viola quella di decomponibilità. Le due idealizzazioni sono distinte e vanno tenute separate.

Questa distinzione individua con precisione la domanda a cui la presente sezione risponde: *esiste un regime in cui un decoder **appreso dai dati** — che non riceve un grafo esplicito degli errori, ma stima la regola decisionale dai campioni — fa meglio del decoder analitico?* La domanda chiude anche il cerchio metodologico dell'intera tesi. Il Capitolo 7 ha stabilito che l'apprendimento automatico applicato *a valle* dell'esecuzione non aggiunge nulla; qui lo stesso strumento — un percettrone multistrato di `scikit-learn`, la medesima tecnologia del classificatore giudicato superfluo — viene applicato alla **sindrome**, cioè a un dato che conserva relazioni

spaziali e temporali non rappresentate dal solo istogramma finale. Cambia il punto di applicazione, non la tecnologia: è esattamente il confronto che permette di attribuire l'esito al *dove*, e non al *come*.

E.1.1 Disegno sperimentale

Il dato di addestramento non richiede alcuna infrastruttura aggiuntiva: è già ciò che Stim produce per stimare p_L . Il campionatore restituisce la matrice dei detection events e il corrispondente esito dell'osservabile logico, che sono rispettivamente le variabili indipendenti e l'etichetta del problema di apprendimento supervisionato:

```
det, obs = sampler.sample(shots, separate_observables=True)
# X = det (detection events)      -> sindrome misurata
# y = obs (logical observable)    -> flip logico da predire
```

Listing E.1: Il dataset del decoder appreso coincide con l'output del campionatore di Stim: `det` è la sindrome osservata, `obs` l'esito logico da predire. Il decoder analitico e quello appreso ricevono quindi esattamente la stessa informazione.

Il modello è un MLP con due strati nascosti (128, 64), attivazione ReLU, ottimizzatore Adam e arresto anticipato su una frazione di validazione interna; il decoder di riferimento è PyMatching sullo stesso insieme di test. Tutte le stime adottano il codice ruotato in base Z con $\text{rounds} = d$ e rumore *circuit-level*, come nella campagna della Sezione 9.4. La Tabella E.1 riassume i quattro esperimenti.

	Domanda	Condizione	Campioni
E1	la rete sostituisce il matching?	rumore nominale, modello esatto	4×10^5 per punto
E1b	quanto costa in dati il pareggio?	$d = 3$, $p = 3 \times 10^{-3}$	fino a 8×10^5
E2	e se il rumore esce dal modello?	crosstalk correlato	4×10^5 per punto
E4	e se la si affianca invece di sostituirla?	crosstalk correlato	8×10^5 per punto

Tabella E.1: I quattro esperimenti sulla decodifica appresa. E3, discusso nella Sezione E.1.4, riusa la configurazione di E2 senza nuovi campionamenti.

E.1.2 E1 — la sostituzione a parità di modello

Il primo esperimento pone il confronto nelle condizioni più favorevoli al decoder analitico: rumore uniforme e modello d'errore esatto, cioè esattamente ciò per cui il

MWPM è costruito. Su quattordici configurazioni ($d = 3, 5$; sette valori di p) la rete **non supera mai** il matching con significatività statistica. A distanza 3 lo affianca, con un rapporto $p_L^{\text{MWPM}}/p_L^{\text{rete}}$ che sale da 0.83 a $p = 2 \times 10^{-3}$ fino a 1.01 a $p = 10^{-2}$; a distanza 5 resta indietro di un ordine di grandezza (da 0.05 a 0.29).

L'andamento monotono suggerisce la spiegazione, che il pannello (b) della Figura E.1 verifica direttamente: la rete è limitata dal **volume di dati**, non dalla capacità del modello. Fissando $d = 3$ e $p = 3 \times 10^{-3}$ e facendo variare il solo insieme di addestramento, il rapporto cresce in modo regolare da 0.50 (2.5×10^4 campioni) a 1.02 (8×10^5 campioni). L'ultimo punto misura $p_L = 3.98 \times 10^{-3}$ contro 4.04×10^{-3} del matching: un sorpasso nominale che, su un insieme di test di 10^5 campioni, vale 0.2 deviazioni standard e **non è statisticamente significativo**. Il risultato di E1 è dunque un pareggio, e il suo contenuto informativo è il prezzo di quel pareggio: dell'ordine di 10^5 – 10^6 campioni a distanza 3, con un fabbisogno che cresce rapidamente con d — coerente con il fatto che i decoder neurali allo stato dell'arte ricorrono ad architetture ricorrenti e a volumi di dati di diversi ordini di grandezza superiori [10, 53].

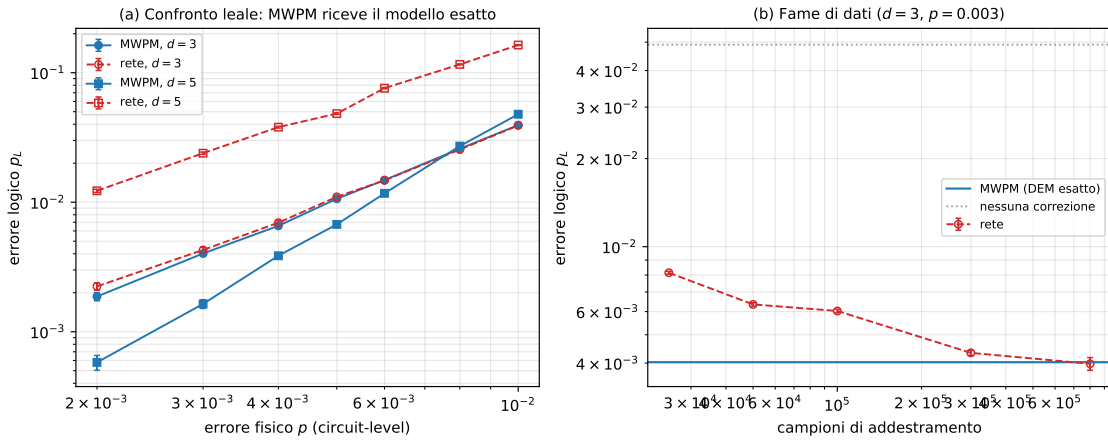


Figura E.1: (a) Errore logico del decoder appreso e del MWPM in funzione dell'errore fisico, per $d = 3$ e $d = 5$, con modello d'errore esatto fornito al matching. La rete affianca il matching a distanza 3 e resta al di sopra a distanza 5. (b) Errore logico della rete al crescere dell'insieme di addestramento ($d = 3$, $p = 3 \times 10^{-3}$): l'avvicinamento al matching è monotono e il pareggio si raggiunge fra 3×10^5 e 8×10^5 campioni. Rigenerabile da `figure_src/gen_qec_neural.py`.

E.1.3 E2 — quando il rumore esce dal modello del decoder

Il secondo esperimento rimuove l'ipotesi di decomponibilità. Al circuito viene aggiunto un canale correlato a due qubit fra i **qubit dati fisicamente adiacenti** sulla griglia, una volta per ciclo di sindrome e con intensità p_{ct} : è il modello elementare del **crosstalk**, la quarta fonte fisica di rumore discussa nel Capitolo 4, che l'accoppiamento capacitivo residuo produce fra qubit vicini. L'errore fisico resta fissato a $p = 3 \times 10^{-3}$, sotto la soglia misurata nella Sezione 9.4. Si confrontano tre bracci: il matching con il modello *nominale* (rumore uniforme — ciò che la calibrazione del dispositivo dichiara), il matching con il modello del circuito *reale* (che include il crosstalk), e la rete addestrata sui campioni effettivamente osservati.

Il crosstalk degrada la decodifica in modo severo: a $p_{ct} = 5 \times 10^{-3}$ l'errore logico a $d = 3$ passa da 3.8×10^{-3} a 3.1×10^{-2} , quasi un ordine di grandezza, pur restando l'errore fisico invariato. In questo regime la rete supera il matching su tutte e quattro le intensità testate a distanza 3, con guadagni fra 1.14 e 1.19 (Tabella E.2). A distanza 5 il vantaggio non si manifesta: la rete perde, per la stessa ragione di E1 — lo spazio delle sindromi cresce più rapidamente della sua capacità di generalizzazione.

d	p_{ct}	nessuna corr.	MWPM nominale	MWPM reale	rete
3	5×10^{-3}	0.0845	0.03097	0.05656	0.02707
	1×10^{-2}	0.1163	0.05966	0.08975	0.05012
	2×10^{-2}	0.1746	0.11184	0.15107	0.09547
	4×10^{-2}	0.2652	0.20806	0.25481	0.17568
5	5×10^{-3}	0.2007	0.03521	0.02796	0.08266
	1×10^{-2}	0.2648	0.08655	0.06526	0.13957
	2×10^{-2}	0.3565	0.19933	0.15662	0.28906
	4×10^{-2}	0.4459	0.37458	0.32515	0.44634

Tabella E.2: Errore logico p_L in presenza di crosstalk fra qubit dati adiacenti, a errore fisico costante $p = 3 \times 10^{-3}$. “Nessuna correzione” è la frequenza con cui l'osservabile logico risulta alterato in assenza di decodifica. A distanza 3 la rete domina entrambe le varianti del matching; a distanza 5 il quadro si inverte.

Un'osservazione inattesa: conoscere il rumore non basta. Il braccio con il modello d'errore *reale* — quello che conosce il crosstalk — si comporta in modo opposto alle due distanze, ed è il risultato più istruttivo dell'esperimento. A $d = 3$ è il **peggiore** dei tre (5.7×10^{-2} contro 3.1×10^{-2} del modello nominale a $p_{ct} = 5 \times 10^{-3}$): fornire al matching l'informazione corretta lo peggiora. La ragione è strutturale e non contraddittoria. Poiché l'errore correlato non è rappresentabile come arco, la

libreria è costretta a scomporlo in archi equivalenti, e su un grafo piccolo come quello di distanza 3 gli archi spurî introdotti dalla scomposizione sviano l'accoppiamento più di quanto l'informazione aggiuntiva lo guidi. A $d = 5$, dove il grafo è più ricco e la scomposizione pesa relativamente meno, il bilancio si inverte e il modello reale torna a essere il migliore.

Ne discende un enunciato che vale la pena isolare, perché delimita esattamente ciò che un decoder analitico può e non può fare: *quando la classe di errori non è esprimibile nella struttura del decoder, il limite non è la calibrazione ma la struttura stessa* — e migliorare la stima del rumore può non produrre alcun beneficio, o risultare controproducente.

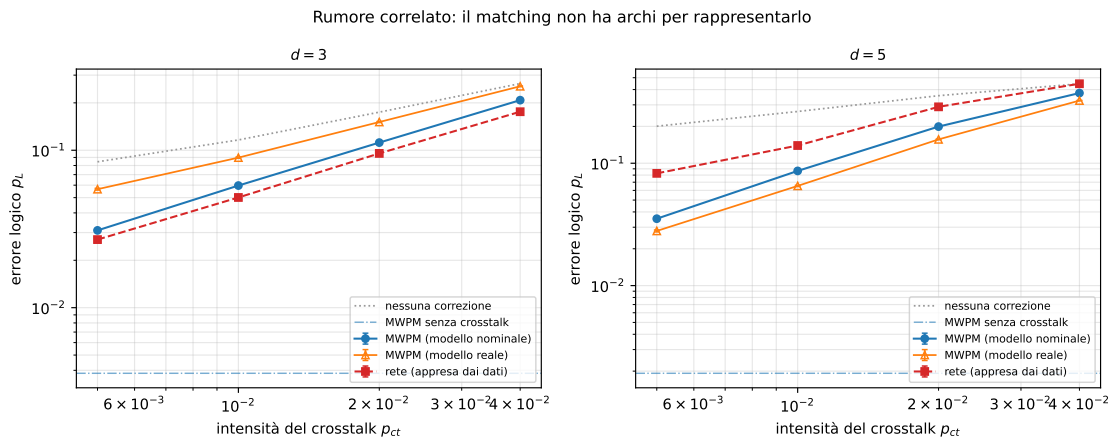


Figura E.2: Errore logico in funzione dell'intensità del crosstalk, a errore fisico costante $p = 3 \times 10^{-3}$. A distanza 3 la rete (rosso) domina il matching sia con modello nominale (blu) sia con modello reale (arancio), che risulta il peggiore; a distanza 5 l'ordine si inverte. La linea tratteggiata inferiore indica l'errore logico del matching in assenza di crosstalk, cioè il livello che la decodifica raggiungerebbe se il modello fosse corretto.

E.1.4 E3 — la rete come validatore della correzione

I due esperimenti precedenti valutano la rete come *sostituto* del decoder. Una domanda diversa, e operativamente più modesta, è se la rete sappia riconoscere gli shot in cui il matching ha sbagliato — cioè se possa **confermare o negare** la correzione proposta, senza doverla rimpiazzare. Si definisce allora la confidenza che la rete assegna alla risposta effettivamente data dal matching, e si verifica quanto una confidenza bassa predica un fallimento reale.

Nella configurazione $d = 3$, $p_{ct} = 10^{-2}$, in cui il matching sbaglia 5966 shot su 10^5 , il flag così costruito discrimina i fallimenti con $AUC = 0.941$. La Tabella E.3 ne

riporta il compromesso: abbassando la soglia si segnalano meno shot con precisione crescente, fino a individuare l'85.9% di errori veri segnalando appena lo 0.59% delle esecuzioni.

Soglia di confidenza	Shot segnalati	Precision	Recall
0.5	2.92%	0.664	0.324
0.3	1.54%	0.783	0.202
0.2	1.16%	0.825	0.160
0.1	0.59%	0.859	0.085

Tabella E.3: Capacità della rete di segnalare gli shot in cui il MWPM ha sbagliato ($d = 3$, $p_{ct} = 10^{-2}$, $AUC = 0.941$). La precisione cresce al restringersi della segnalazione: segnalando lo 0.59% degli shot, l'85.9% delle segnalazioni corrisponde a un fallimento reale.

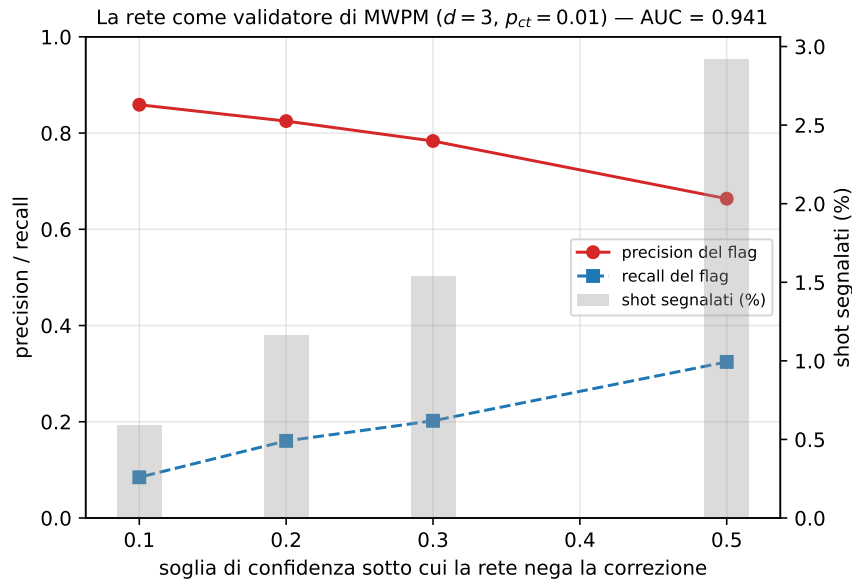


Figura E.3: Precision e recall del flag in funzione della soglia di confidenza, con la frazione di shot segnalati (barre, asse destro). Il flag non sostituisce la decodifica: la annota.

E.1.5 E4 — il decoder ibrido

L'esito di E3 suggerisce la costruzione che chiude la sezione. Se la rete riconosce i fallimenti del matching senza saper decodificare meglio di esso, allora il suo impiego corretto non è sostituirlo ma **correggerlo**. Si definisce quindi un decoder **ibrido** in

forma residuale:

$$\hat{y}_{\text{ibrido}} = \hat{y}_{\text{MWPM}} \oplus [\text{Pr}(\text{MWPM sbaglia} \mid \text{sindrome}) \geq \tau], \quad (\text{E.1})$$

dove $\oplus$ è la somma modulo 2 e la rete riceve in ingresso la sindrome e la decisione del matching. La soglia di ribaltamento τ è scelta su un insieme di **validazione separato** e poi applicata invariata all'insieme di test: sceglierla sul test costituirebbe una fuga di informazione, e su guadagni dell'ordine di pochi punti percentuali sarebbe un vizio sufficiente a invalidare il risultato. Fra le soglie candidate figura il valore che non ribalta mai, cosicché il criterio di validazione può sempre ricadere sul matching puro quando nessun ribaltamento conviene.

La formulazione ha una motivazione precisa, oltre alla convenienza pratica: alla rete non si chiede più di riprodurre da zero il calcolo di parità che il matching già esegue correttamente nella grande maggioranza dei casi, ma soltanto di modellarne il *residuo*, che è un evento raro e strutturato. È un compito più facile, ed è per questo che riesce dove la sostituzione fallisce.

d	p_{ct}	MWPM	rete sola	ibrido	ribalta	McNemar
3	0	0.00415	0.00409	0.00417	0.10%	$p = 0.67$
	5×10^{-3}	0.03089	0.02680	0.02607	1.19%	3×10^{-87}
	1×10^{-2}	0.05829	0.04917	0.04851	2.69%	1×10^{-156}
	2×10^{-2}	0.11144	0.09248	0.09183	5.38%	2×10^{-312}
	4×10^{-2}	0.20750	0.17615	0.17455	11.04%	$< 10^{-320}$
5	0	0.00165	0.01900	0.00165	0.00%	—
	5×10^{-3}	0.03561	0.06311	0.03521	0.32%	1×10^{-3}
	1×10^{-2}	0.08704	0.12874	0.08428	1.17%	3×10^{-30}
	2×10^{-2}	0.20231	0.25198	0.19662	2.88%	6×10^{-51}
	4×10^{-2}	0.37581	0.44182	0.37385	2.96%	4×10^{-7}
7	0	0.00055	0.06888	0.00055	0.00%	—
	5×10^{-3}	0.03707	0.24671	0.03707	0.00%	—
	1×10^{-2}	0.11319	0.36749	0.11318	0.00%	$p = 0.32$
	2×10^{-2}	0.29048	0.46219	0.29048	0.00%	—
	4×10^{-2}	0.46741	0.49980	0.46741	0.00%	—

Tabella E.4: Errore logico del decoder ibrido (E.1) confrontato con il MWPM e con la rete usata come sostituto. “Ribalta” è la frazione di shot in cui la rete inverte la decisione del matching. Il valore di significatività è quello del test di McNemar, appaiato sugli stessi campioni: poiché i due decoder differiscono *soltanto* sugli shot ribaltati, le coppie discordanti coincidono con essi e il test è esatto.

L’ibrido riduce l’errore logico in tutte e otto le configurazioni con crosstalk a

distanza 3 e 5, con guadagni del 18–21% nel primo caso e dell’1–3% nel secondo, tutti significativi al test di McNemar. Il dettaglio delle coppie discordanti rende l’effetto leggibile: a $d = 3$ e $p_{ct} = 4 \times 10^{-2}$ la rete rompe 7745 shot che il matching aveva risolto correttamente e ne aggiusta 14335, un rapporto di quasi due a uno; a $d = 5$ il bilancio si stringe (2769 contro 3160), da cui il guadagno più contenuto ma comunque netto.

Due proprietà meritano attenzione perché riguardano il comportamento del sistema fuori dal regime per cui è stato addestrato. La prima è che la frazione di ribaltamenti **scala con l’intensità del crosstalk** (dallo 0.10% all’11.04% a distanza 3): la rete non interviene a caso, ma in misura proporzionale a quanto il modello del matching è sbagliato. La seconda è il comportamento in assenza di crosstalk, dove il matching è già adeguato e non c’è nulla da correggere: a $d = 5$ la rete effettua **zero ribaltamenti** e l’ibrido coincide cifra per cifra con il MWPM, mentre a $d = 3$ ribalta lo 0.10% degli shot con effetto statisticamente nullo ($p = 0.67$). Il decoder ibrido *degrada con grazia*: riconosce di non avere nulla da aggiungere e si fa da parte. Va detto con esattezza che questa garanzia riguarda il criterio di validazione e non ogni singolo punto dell’insieme di test — il valore leggermente superiore a $d = 3$, $p_{ct} = 0$ ne è un esempio, entro il rumore statistico — sicché la formulazione corretta è che l’ibrido non risulta *mai peggiore del matching in modo statisticamente rilevabile*.

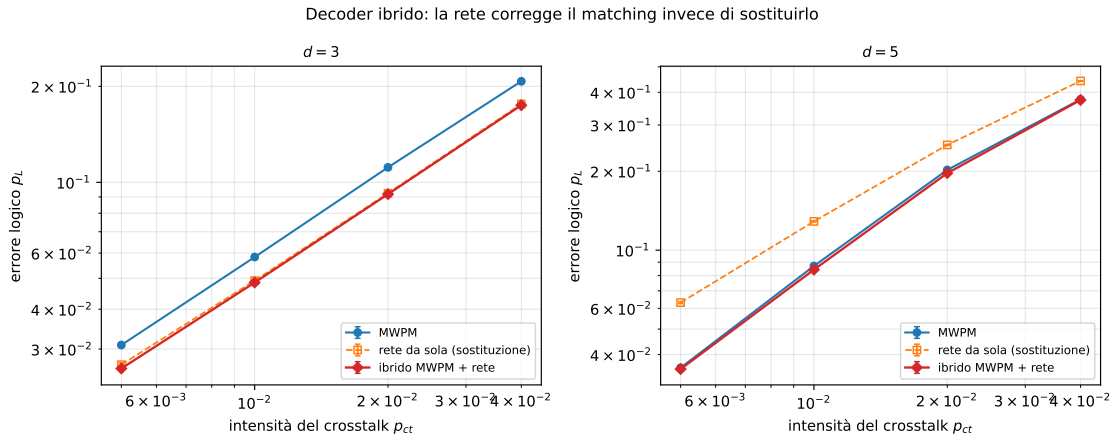


Figura E.4: Errore logico dei tre decoder in funzione dell’intensità del crosstalk. L’ibrido (rosso) è il migliore a entrambe le distanze; la rete usata come sostituto (arancio) domina il matching a $d = 3$ ma lo perde a $d = 5$, dove l’ibrido continua invece a guadagnare.

Il limite: il vantaggio non sopravvive alla distanza 7. Il dato più importante della Tabella E.4 è però l’ultimo blocco, e va enunciato senza attenuazioni. A distanza

7 l'ibrido **non ribalta alcuno shot** in nessuna delle cinque configurazioni: la soglia scelta in validazione è ogni volta quella corrispondente a “non ribaltare mai”, e il decoder ibrido coincide con il MWPM cifra per cifra. Il guadagno svanisce, e la progressione lungo le tre distanze è inequivocabile: 18–21% a $d = 3$, 1–3% a $d = 5$, esattamente zero a $d = 7$.

Due meccanismi possono spiegarlo, e vanno separati perché conducono a rimedi diversi. Il primo è la **scarsità di esempi**: l'evento da riconoscere è il fallimento del matching, la cui frequenza è p_L , e p_L decresce esponenzialmente con d . In assenza di crosstalk, a $d = 7$ si hanno $p_L = 5.5 \times 10^{-4}$ e 8×10^5 shot, ossia 264 esempi positivi nella partizione di addestramento, per un ingresso a 336 dimensioni. Il secondo è la **capacità rappresentativa**: la rete densa impiegata tratta i detector come dimensioni indipendenti, e ciò che distingue le due classi di errore su un reticolo grande è una correlazione a lungo raggio fra detector distanti.

Un primo indizio viene dal semplice conteggio. La spiegazione per scarsità è esatta nella configurazione senza crosstalk, dove i fallimenti del matching sono effettivamente poche centinaia; ma nelle altre quattro configurazioni a $d = 7$ la rete dispone da 1.8×10^4 a 2.2×10^5 esempi positivi — fino a duecentomila — e continua a non ribaltare alcuno shot. La spiegazione copre dunque una configurazione su cinque.

L'indizio non basta però a stabilire la causa, perché fra $d = 5$ (che funziona) e $d = 7$ (che non funziona) cambiano *due* cose insieme: la distanza del codice e il numero di detector. L'esperimento E9 le separa, sfruttando il fatto che il numero di detector è il prodotto fra gli stabilizzatori per ciclo e il numero di cicli, e può quindi essere variato a distanza fissa. Ne risulta un incrocio 2×2 le cui celle diagonali distinguono le due ipotesi, riportato nella Tabella E.5.

d	cicli	detector	$p_{ct} = 0.005$		$p_{ct} = 0.02$	
			es. pos.	residuo	es. pos.	residuo
7	3	144	5 933	si astiene	61 900	interviene
7	7	336	17 665	si astiene	139 275	si astiene
5	14	336	48 657	si astiene	188 582	si astiene
5	5	120	17 235	interviene	96 167	interviene

Tabella E.5: Esperimento E9: incrocio fra distanza del codice e numero di detector, ottenuto variando il numero di cicli di estrazione della sindrome. “Es. pos.” è il numero di fallimenti del MWPM disponibili all’addestramento su 4.8×10^5 shot; “residuo” indica se il criterio di validazione seleziona una soglia che ribalta almeno uno shot su diecimila. Le righe centrali sono le celle diagonali, quelle che distinguono le ipotesi: a distanza 5 con 336 detector e 1.9×10^5 esempi il residuo si astiene, a distanza 7 con 144 detector e tre volte meno esempi interviene. Il comportamento segue la colonna, non la riga.

L’esito è netto su entrambe le diagonali e va letto per colonne. A 336 detector il residuo si astiene sempre, a qualunque distanza e con qualunque numerosità — fino a 1.9×10^5 esempi positivi; a 120 e 144 detector interviene, anche a distanza 7 e anche con tre volte meno esempi. **L’ipotesi della distanza è confutata, quella della larghezza dell’ingresso è confermata.**

Una sfumatura completa l’enunciato e lo rende più preciso. A 144 detector con soli 5 933 esempi positivi il residuo si astiene: la numerosità è dunque un vincolo reale, al margine inferiore. Non è però il vincolo che opera nell’esperimento E4 a distanza 7, dove gli esempi erano 1.4×10^5 e la rete si asteneva comunque. La formulazione corretta è che disporre di abbastanza esempi è *necessario ma non sufficiente*, e che a parità di numerosità sufficiente è la larghezza dell’ingresso a decidere.

Che cosa significhi esattamente “si astiene” (E12). Il verbo va però disinnescato, perché astenersi è l’esito di una *selezione*: fra le soglie candidate figura quella corrispondente a “non ribaltare mai”, e che vinca in validazione significa che agire non conviene, non che non vi sia nulla da vedere. La distinzione non è oziosa, perché le due letture conducono a rimedi opposti — un’architettura diversa nella prima, una regola di decisione diversa nella seconda. L’esperimento E12 la risolve misurando l’AUC del residuo, che quantifica quanto la rete *ordini* i fallimenti del matching indipendentemente da qualunque soglia, e affiancando alla soglia scelta onestamente in validazione una soglia *oracolo*, la migliore sul test set stesso: inutilizzabile in

pratica, ma limite superiore assoluto di ciò che qualunque regola di decisione potrebbe ottenere.

detector	d	cicli	AUC	guadagno onesto	guadagno oracolo
24	3	3	0.912	1.2215	1.2233
120	5	5	0.769	1.0206	1.0215
144	7	3	0.788	1.0066	1.0066
336	7	7	0.621	1.0000	1.0000
336	5	14	0.573	1.0000	1.0003

Tabella E.6: Esperimento E12 a $p_{ct} = 2 \times 10^{-2}$. L’AUC è calcolata sul test set con bersaglio “il MWPM ha sbagliato”. Il guadagno oracolo usa la soglia migliore sul test set stesso e non è quindi realizzabile: serve unicamente a stabilire quanto del divario sia imputabile alla calibrazione. Ai livelli di crosstalk più bassi il quadro è il medesimo, con AUC sistematicamente più alte (0.958 a 24 detector, 0.771 a 336).

Il risultato smentisce entrambe le letture immediate. **La rete non è cieca:** a 336 detector l’AUC vale 0.57–0.77, ben sopra il caso, e la rete ordina dunque i fallimenti del matching assai meglio del lancio di una moneta. **Ma non si tratta neppure di calibrazione:** il guadagno oracolo è 1.0000, sicché nemmeno scegliendo la soglia sui dati di valutazione si otterrebbe alcunché.

La spiegazione sta nella forma stessa della correzione residuale, ed è quantitativa. Il ribaltamento è *simmetrico*: ribaltare uno shot che il matching aveva risolto costa esattamente quanto ribaltarne uno che aveva sbagliato rende. Perché l’operazione sia in attivo occorre dunque che fra gli shot ribaltati la **precisione superi il cinquanta per cento** — una soglia netta, non un ottimo graduale. L’AUC necessaria a raggiungerla dipende dal tasso di base: a 24 detector e crosstalk 5×10^{-3} il tasso di base è del 3.1% e un’AUC di 0.958 concentra i fallimenti abbastanza da superare la soglia, con un guadagno del 19%; a 336 detector, a parità di tasso di base, un’AUC di 0.771 richiederebbe un fattore di concentrazione quattordici volte superiore a quello che produce. Nessuna soglia porta la precisione sopra il cinquanta per cento, e il guadagno è allora *esattamente* 1.000 — non approssimativamente: è questa discontinuità a spiegare il salto brusco osservato lungo la Tabella E.5.

Ne discendono due conseguenze. La prima è che **l’astensione del criterio di validazione è corretta e non conservativa**: la soglia oracolo conferma che intervenire non converrebbe, e il metodo si comporta esattamente come dichiarato al momento della sua costruzione. La seconda riguarda il rimedio, e ne precisa il criterio di successo: un’architettura convoluzionale o a grafo è utile se e soltanto se innalza l’AUC quanto basta a riportare la precisione sopra la soglia del cinquanta per

cento, che è una condizione verificabile in anticipo su una rete addestrata — assai più informativa dell’osservazione che il modello “non regge”.

Questa è la delimitazione del risultato di M10, e conviene formularla in positivo. Il guadagno del decoder ibrido è supportato da un valore nominale estremamente piccolo del test di McNemar — a $d = 3$, $p < 10^{-320}$ — ma vive nel regime in cui la sindrome è stretta abbastanza da lasciare alla rete densa una discriminazione sufficiente a superare la soglia di precisione. Il valore p quantifica l’incompatibilità con l’ipotesi nulla nel modello del test, non la dimensione pratica dell’effetto, riportata separatamente tramite la riduzione di p_L . Poiché è proprio la crescita della distanza a costituire la strategia della correzione d’errore scalabile, il metodo così com’è non accompagna il codice nel regime che conta. Il rimedio indicato dalla diagnosi non è raccogliere più campioni — ne bastavano già — ma innalzare la discriminazione a parità di larghezza dell’ingresso, il che significa cambiare architettura: una rete convoluzionale o a grafo, che condivida i pesi fra regioni equivalenti del reticolo anziché trattare i 336 detector come dimensioni indipendenti, sfrutta la simmetria traslazionale del problema e rappresenta con pochi parametri le correlazioni locali che la rete densa deve stimare una per una. È l’estensione naturale, e la si indica fra gli sviluppi futuri (Capitolo 11).

E.1.6 E6 — e se il modello del decoder fosse semplicemente sbagliato?

Gli esperimenti fin qui condotti concedono al MWPM un vantaggio che l’hardware non offre: il detector error model che ne pesa gli archi è generato da Stim *dal circuito stesso*, ed è quindi una descrizione esatta del rumore. Un decoder in produzione lavora invece su un modello ricavato dalla calibrazione del dispositivo — misurata a intervalli, soggetta a deriva, inevitabilmente diversa dal rumore del momento. Se il vantaggio del modello appreso nascesse da questa discrepanza, sarebbe un vantaggio molto più generale di quello osservato con il crosstalk, perché la mis-calibrazione è la condizione ordinaria di ogni QPU.

L’esperimento la isola: il circuito viene campionato con il rumore reale, mentre il matching è costruito su un modello in cui l’errore di *misura* è sbagliato di un fattore $(1 + \delta)$ e quello di gate resta corretto.

Perché la perturbazione dev’essere sbilanciata. Un primo tentativo scalava tutte le probabilità di errore dello stesso fattore, e non produceva alcun effetto

misurabile: il costo rilevato era 1.000, 1.004 e 1.000 per δ pari a 0, +100% e -50%. La ragione è istruttiva e merita di essere esplicitata, perché delimita ciò che una calibrazione errata può e non può fare. Il MWPM minimizza il peso totale del matching, e il peso di un arco vale $\log \frac{1-p_e}{p_e} \approx -\log p_e$ per p_e piccolo: moltiplicare tutte le p_e per una costante trasla *tutti* i pesi della stessa quantità, e un matching di peso minimo è invariante rispetto a una traslazione uniforme. Ciò che rende il matching vulnerabile non è dunque sbagliare la *scala* del rumore, ma il suo *bilanciamento*: il rapporto fra archi temporali, che rappresentano errori di misura, e archi spaziali, che rappresentano errori sui dati. È questo rapporto che l'esperimento perturba.

d	δ	MWPM esatto	MWPM sbagliato	costo	rete	rete/MWPM
3	-80%	0.00407	0.00415	1.018	0.00525	0.79
	-50%	0.00435	0.00435	1.000	0.00493	0.88
	0	0.00427	0.00427	1.000	0.00558	0.77
	+100%	0.00387	0.00382	0.987	0.00443	0.86
	+300%	0.00388	0.00362	0.931	0.00429	0.84
	+900%	0.00385	0.00392	1.017	0.00463	0.85
5	-80%	0.00147	0.00153	1.034	0.02982	0.05
	-50%	0.00172	0.00173	1.010	0.02855	0.06
	0	0.00163	0.00163	1.000	0.02784	0.06
	+100%	0.00174	0.00175	1.005	0.02774	0.06
	+300%	0.00169	0.00167	0.985	0.02888	0.06
	+900%	0.00159	0.00191	1.199	0.03052	0.06

Tabella E.7: Errore logico del MWPM quando il modello che ne pesa gli archi attribuisce all'errore di misura il valore $p(1+\delta)$ anziché quello vero. La colonna “costo” è il rapporto fra l'errore logico con modello sbagliato e quello con modello esatto; “rete/MWPM” confronta il decoder appreso con il matching mis-calibrato — valori inferiori a 1 indicano che la rete è peggiore. Errore fisico reale $p = 3 \times 10^{-3}$, 4×10^5 campioni per configurazione.

L'esito è netto in entrambe le direzioni. Il MWPM si rivela **notevolmente robusto**: sbagliando l'errore di misura di un fattore cinque in difetto o dieci in eccesso, l'errore logico cambia tipicamente dell'1-3%, e in due configurazioni *migliora* leggermente. L'unico caso in cui la mis-calibrazione morde davvero è $d = 5$ con $\delta = +900\%$, dove il costo sale al 20%: sovrastimare di un ordine di grandezza l'errore di misura induce il matching a preferire spiegazioni temporali degli eventi rilevati anche quando la spiegazione corretta è spaziale. Ma occorre appunto un errore di calibrazione di un ordine di grandezza perché l'effetto sia visibile.

La rete, dal canto suo, **non batte il matching in nessuna delle dodici configurazioni**, neppure quelle in cui esso lavora sul modello sbagliato. A $d = 3$

resta indietro del 15–25%; a $d = 5$ è peggiore di un fattore venti, in coerenza con il crollo osservato nell’esperimento precedente.

Che cosa questo precisa del criterio. Il risultato è negativo, ma corregge un’imprecisione nel modo in cui il criterio della tesi era stato finora formulato. Si era detto che il modello appreso produce valore dove il metodo analitico è “cieco” rispetto a una parte dell’informazione; questo esperimento mostra che la cecità rilevante non è quella dei *parametri* ma quella della *struttura*. Un modello con i parametri sbagliati continua a rappresentare le stesse classi di errore, e il matching che ne deriva resta quasi ottimale perché la geometria del problema — quali eventi rilevati siano plausibilmente connessi — non cambia. Un modello che non contempla affatto una classe di errore, come il crosstalk fra qubit di dato adiacenti dell’esperimento E2, sbaglia invece la geometria stessa, e lì il margine esiste. La formulazione corretta del criterio è dunque: *l’apprendimento paga quando la classe di errore non è rappresentabile nella struttura del decoder, non quando è rappresentata con parametri imprecisi.*

Ne discende anche un’indicazione pratica di segno opposto a quella che ci si potrebbe attendere: investire in una calibrazione più accurata del modello di rumore, entro un ordine di grandezza, non produce guadagni apprezzabili sulla decodifica. È un’osservazione che vale la pena tenere presente perché la calibrazione periodica è un costo operativo reale nei dispositivi attuali.

E.1.7 E7 — e se si scegliesse semplicemente un decoder analitico migliore?

Resta l’obiezione più severa che si possa muovere a tutto l’impianto della sezione. Il MWPM è lo standard di fatto, ma non è il miglior decoder analitico disponibile: esso richiede che ogni meccanismo di errore sia decomponibile in archi di un grafo, e il crosstalk fra qubit di dato adiacenti viola precisamente questa ipotesi. Un decoder che non la richieda — **BP+OSD**, ossia *belief propagation* seguita da *ordered statistics decoding*, che opera direttamente sulla matrice di parità del detector error model [54, 55] — potrebbe catturare da solo il margine attribuito all’apprendimento. Se così fosse, il risultato di E4 non misurerebbe il valore di un modello appreso ma il costo di una scelta subottimale del decoder di riferimento.

Il confronto è condotto sugli stessi circuiti e con la stessa mis-informazione: entrambi i decoder analitici ricevono il DEM *nominale*, privo di crosstalk, che è

quanto un decoder reale avrebbe a disposizione. La significatività è valutata con il test di McNemar, appaiato sugli stessi shot.

d	p_{ct}	MWPM	BP+OSD	BP+OSD/MWPM	ibrido/MWPM	McNemar
3	0	0.00428	0.00374	1.146	0.993	1×10^{-9}
	5×10^{-3}	0.03193	0.03286	0.972	1.185	3×10^{-7}
	1×10^{-2}	0.05912	0.06034	0.980	1.202	5×10^{-7}
	2×10^{-2}	0.11176	0.11340	0.986	1.213	2×10^{-6}
	4×10^{-2}	0.20811	0.20826	0.999	1.189	$p = 0.77$
5	0	0.00192	0.00143	1.339	1.000	2×10^{-7}
	5×10^{-3}	0.03565	0.02807	1.270	1.011	2×10^{-113}
	1×10^{-2}	0.08638	0.07348	1.176	1.033	3×10^{-140}
	2×10^{-2}	0.19998	0.18288	1.094	1.029	6×10^{-108}
	4×10^{-2}	0.37561	0.36788	1.021	1.005	2×10^{-12}

Tabella E.8: Confronto fra il MWPM, il decoder BP+OSD e il decoder ibrido appreso, tutti sul medesimo modello di rumore nominale. Le due colonne di guadagno sono rapporti rispetto al MWPM: valori superiori a 1 indicano un errore logico minore. In grassetto, per ciascuna riga, il migliore fra i due approcci alternativi. Il valore di significatività si riferisce al confronto BP+OSD contro MWPM. Errore fisico $p = 3 \times 10^{-3}$, 2×10^5 campioni per configurazione.

Il primo dato da registrare è che **BP+OSD è effettivamente un decoder migliore**: sul rumore nominale, quello per cui il modello è corretto, riduce l'errore logico del 14.6% a distanza 3 e del 33.9% a distanza 5, con significatività schiacciante. Il vantaggio cresce con la distanza, come atteso: più il codice è grande, più numerose sono le configurazioni di errore degeneri che il matching, vincolato alla decomposizione in archi, non sa distinguere.

Il secondo dato è però quello che conta, ed è la struttura delle due colonne di guadagno letta lungo la colonna p_{ct} .

*Il guadagno di BP+OSD **decresce** al crescere del crosstalk; quello del decoder ibrido **cresce**.*

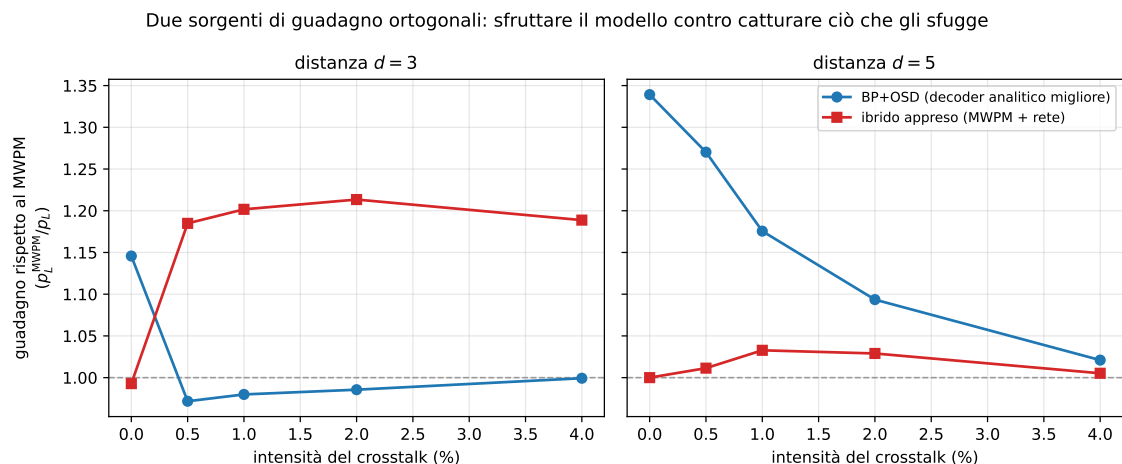


Figura E.5: Guadagno rispetto al MWPM dei due approcci alternativi, in funzione dell'intensità del crosstalk. La linea tratteggiata è il MWPM stesso. BP+OSD (blu) parte dal guadagno massimo, quando il modello di rumore che riceve è corretto, e lo perde man mano che il crosstalk lo rende incompleto; il decoder ibrido (rosso) parte da zero, perché a modello corretto non ha nulla da aggiungere, e guadagna in misura crescente. A distanza 3 le due curve si incrociano immediatamente; a distanza 5 il vantaggio iniziale di BP+OSD è così ampio da mantenerlo davanti anche eroso. Figura rigenerabile da `figure_src/gen_confronto_decoder.py`.

A distanza 5 il vantaggio di BP+OSD scende monotonamente da 1.339 a 1.021 mentre il crosstalk aumenta; a distanza 3 esso si annulla immediatamente, e il decoder diventa lievemente *peggiore* del matching. Specularmente, il guadagno dell'ibrido è nullo in assenza di crosstalk e sale fino a 1.213. I due metodi non competono sullo stesso asse: **BP+OSD sfrutta meglio il modello che riceve, il decoder appreso cattura ciò che al modello sfugge**. Il primo rende al massimo quando il modello è corretto e si degrada man mano che diventa incompleto — perché fidarsi più a fondo di una descrizione sbagliata è un danno, non un vantaggio; il secondo rende zero quando il modello è corretto, perché non c'è nulla da aggiungere, e cresce esattamente nella misura in cui il modello si allontana dal dispositivo.

Ne discende una conclusione più articolata di quella che l'esperimento si proponeva di verificare, e va enunciata riconoscendo entrambi i suoi lati. Da un lato l'obiezione è **in parte fondata**: a distanza 5, dove il vantaggio di partenza di BP+OSD è ampio, esso resta superiore all'ibrido in tutte le configurazioni, e adottare un decoder analitico migliore rende più che addestrare una rete. Chi disponga di un budget di ingegneria limitato dovrebbe spenderlo lì per primo, ed è una raccomandazione pratica che questa tesi non avrebbe potuto formulare senza l'esperimento. Dall'altro lato l'obiezione **non elimina il risultato**: a distanza 3 l'ibrido guadagna 18–21%

proprio dove BP+OSD perde.

Resta da chiarire che cosa significhi “agire su assi ortogonali”. L’inferenza immediata è che i due guadagni siano *sommabili*, e che costruire il residuo appreso sopra BP+OSD anziché sopra il matching debba produrre il prodotto dei due — ossia, a distanza 5 e crosstalk 5×10^{-3} , un guadagno di $1.289 \times 1.010 = 1.302$. È una previsione quantitativa, e l’esperimento successivo la mette alla prova anziché darla per buona.

E.1.8 E8 — le due sorgenti si sommano davvero? E10 — il pavimento comune

La previsione e il suo esito. L’esperimento E8 sostituisce il decoder di base della (E.1): il residuo appreso viene costruito sopra BP+OSD anziché sopra il matching, e i quattro bracci — MWPM, BP+OSD, ciascuno con e senza residuo — sono misurati *sui medesimi campioni*, il che rende ogni confronto appaiato ed elimina il disallineamento fra le campagne precedenti. Le due ipotesi in gioco erano dichiarate prima dell’esecuzione: quella *moltiplicativa*, per cui il combinato approssima il prodotto; e quella di *saturazione*, per cui il residuo trova sopra BP+OSD assai meno di quanto trovasse sopra il matching, perché una parte di ciò che recuperava era semplicemente il costo di un decoder analitico subottimale. La seconda è l’ipotesi sfavorevole al risultato di questa sezione, ed è quella che i dati sostengono.

d	p_{ct}	BP+OSD	MWPM+res	combinato	prodotto previsto
3	0	1.181	1.079	1.190	1.274
3	0.005	0.985	1.192	1.183	1.173
3	0.01	0.982	1.205	1.203	1.183
3	0.02	0.985	1.211	1.200	1.193
3	0.04	0.997	1.184	1.177	1.180
5	0	1.180	1.000	1.180	1.180
5	0.005	1.289	1.010	1.289	1.302
5	0.01	1.196	1.026	1.200	1.228
5	0.02	1.093	1.028	1.104	1.123
5	0.04	1.024	1.007	1.024	1.032

Tabella E.9: Esperimento E8. Tutti i guadagni sono rapporti $p_L^{\text{MWPM}}/p_L^{\text{metodo}}$ misurati sui medesimi 8×10^5 campioni per configurazione, 2×10^5 dei quali di test. “Combinato” è il residuo appreso costruito sopra BP+OSD; “prodotto previsto” è il valore che l’ipotesi moltiplicativa richiede. Il combinato non lo raggiunge mai quando entrambi i componenti hanno un guadagno reale, e coincide invece con il migliore dei due singoli.

Il confronto decisivo è nelle righe in cui *entrambi* i componenti guadagnano. A distanza 3 ve n’è una sola, quella senza crosstalk: il prodotto richiede 1.274, il combinato si ferma a 1.190, ossia esattamente il guadagno di BP+OSD da solo — e il residuo, sopra BP+OSD e senza crosstalk, ribalta lo 0.04% delle predizioni con $p = 0.58$, cioè non trova nulla. Nelle altre righe di quel blocco il “prodotto” è soddisfatto soltanto perché degenerare: con BP+OSD a 0.98 esso coincide per costruzione con il guadagno del residuo. A distanza 5, dove BP+OSD conserva un vantaggio ampio anche sotto crosstalk, tutte e cinque le righe sono informative e tutte e cinque smentiscono la previsione: lo scarto dal prodotto è negativo ovunque, da -0.007 a -0.027 , e il residuo migliora BP+OSD in modo statisticamente significativo in *una sola* configurazione su cinque, e ivi dell’uno per cento.

Il decoder combinato eguaglia sempre il migliore dei due singoli, mai il loro prodotto. Le due sorgenti di guadagno non sono sommabili: sono alternative.

Perché: esiste un pavimento comune. L’esperimento E10 individua la ragione. Sei bracci sono misurati sui medesimi campioni a distanza 3, in ordine crescente di informazione utilizzata: il decoder *banale* che predice sempre l’assenza di flip logico, la rete usata *da sola* senza alcun modello analitico, il MWPM, BP+OSD, e i due residui costruiti sui due decoder analitici.

p_{ct}	banale	MWPM	BP+OSD	rete sola	MWPM+res	BP+OSD+res	disp.
0	0.04863	0.00432	0.00367	0.00431	0.00405	0.00354	19.4%
0.005	0.08369	0.03106	0.03195	0.02704	0.02676	0.02677	1.0%
0.01	0.11611	0.05928	0.06035	0.04965	0.04885	0.04982	2.0%
0.02	0.17464	0.11169	0.11292	0.09222	0.09108	0.09156	1.2%
0.04	0.26492	0.20753	0.20819	0.17565	0.17413	0.17536	0.9%

Tabella E.10: Esperimento E10: errore logico dei sei bracci a distanza 3, tutti sugli stessi 8×10^5 campioni. La colonna “disp.” è la dispersione relativa fra i tre bracci di fondo — rete sola, MWPM+residuo, BP+OSD+residuo. In presenza di crosstalk essi convergono entro il 2% pur partendo da metodi radicalmente diversi; in sua assenza divergono del 19% e si ordinano per qualità intrinseca del decoder.

Il quadro si legge in due regimi, ed è la struttura stessa del risultato.

A *modello corretto* (prima riga) il pavimento non esiste: i metodi si ordinano per qualità, BP+OSD davanti a tutti con 3.54×10^{-3} e la rete sola ultima con 4.31×10^{-3} , e la dispersione fra i bracci di fondo è del 19%. Qui l’unica cosa che conta è quanto bene si sfrutti il modello, e il residuo appreso non ha nulla da aggiungere.

A *modello strutturalmente incompleto* (le altre quattro righe) i tre bracci di fondo collassano entro lo 0.9–2.0% l’uno dall’altro, mentre i due decoder analitici da cui provengono restano il 16–18% sopra. Vi arriva anche la rete usata da sola, che non dispone di alcun decoder analitico: il pavimento non è dunque una proprietà del metodo di decodifica, ma **dell’informazione contenuta nella sindrome**. Sotto crosstalk ciascuno di questi metodi la estrae per intero, e incontra lo stesso limite.

L’onestà richiede una precisazione sulla parola “coincidono”. Il test di McNemar fra la rete sola e il MWPM+residuo dà $p = 7.8 \times 10^{-4}$, 8.2×10^{-4} e 2.6×10^{-3} ai tre livelli superiori di crosstalk: con 2×10^5 campioni di test si risolve anche uno scarto dell’uno per cento, e quelle differenze residue sono *reali*. Ciò che si afferma è più debole e più preciso: i tre metodi atterrano entro l’1–2% l’uno dall’altro, e tale scarto è un ordine di grandezza inferiore al divario che separa tutti e tre dal decoder analitico da cui partono.

Il fallimento di E8 discende allora da E10 senza bisogno di ulteriori ipotesi: impilare i due metodi non porta sotto il pavimento, perché il pavimento non appartiene a nessuno dei due.

Il pavimento è del problema o del modello? (E11) Alla lettura appena data si può muovere un’obiezione che va presa sul serio, perché è la stessa che questa sezione muove agli altri risultati. I tre bracci che convergono contengono *tutti e tre la medesima rete*, un percettrone (128, 64) addestrato sugli stessi 4.8×10^5 campioni:

che convergano potrebbe significare soltanto “questo è quanto estrae questa rete”, e il pavimento sarebbe una proprietà del modello, non del problema. Per distinguere le due letture occorrono metodi più potenti di quel percettrone. L’esperimento E11 ne mette due alla prova, a distanza 3 e sui medesimi campioni.

Il primo è una rete con la stessa ricetta ma architettura (512, 256, 128), ossia circa quaranta volte i parametri. Il secondo è il **decoder empirico a massima verosimiglianza**: per ogni sindrome osservata in addestramento si predice l’osservabile di maggioranza, con ricaduta sul MWPM per le sindromi mai viste. Quest’ultimo non stima parametri — memorizza — e non ha dunque alcun limite di capacità.

p_{ct}	pavimento	rete 40×	residuo 40×	tabella empirica	copertura
0.005	0.02688	0.02687	0.02617	0.02765	98.1%
0.01	0.04935	0.04920	0.04850	0.05032	97.4%
0.02	0.09251	0.09222	0.09169	0.09499	95.6%
0.04	0.17602	0.17463	0.17377	0.18324	91.3%

Tabella E.11: Esperimento E11, distanza 3. “Pavimento” è il valore di E10 ricalcolato sugli stessi campioni. “Copertura” è la frazione di shot di test la cui sindrome era già stata osservata in addestramento, e misura quanto la tabella empirica sia effettivamente una tabella anziché un MWPM travestito.

L’obiezione è **respinta**: la rete con quaranta volte i parametri atterra sul pavimento entro lo 0.8%, e la tabella empirica — che di capacità ne ha infinita — risulta addirittura dal 2 al 4% *peggiore*. Il pavimento non è dunque il tetto dell’architettura scelta.

Sarebbe però scorretto concluderne che esso coincida con il limite informativo della sindrome, e la ragione sta nella tabella stessa. La sua copertura è alta (91–98%) ma la sua *profondità* non lo è: a crosstalk 4×10^{-2} le sindromi distinte osservate sono 63 180 su 4.8×10^5 campioni di addestramento, ossia circa sette osservazioni per sindrome, e un voto di maggioranza su sette campioni è in larga parte rumore. La tabella non fallisce perché l’informazione manchi, ma perché a questa numerosità non riesce a stimarla; essa mette dunque alla prova l’ipotesi della capacità — e la scarta — senza stabilire alcun limite superiore informativo. Nella stessa direzione va notato che il residuo costruito sulla rete grande scende dell’1–3% sotto il pavimento in tutti e quattro i punti, in modo piccolo ma sistematico: il pavimento non è perfettamente duro.

L’enunciato che i dati sostengono è pertanto il seguente, e va tenuto entro questi limiti: *il pavimento è robusto a un aumento di quaranta volte della capacità del modello*

e a un'alternativa non parametrica, e non è quindi un artefatto dell'architettura; se esso coincida con il limite informativo della sindrome resta una questione aperta, che solo una decodifica esatta a massima verosimiglianza potrebbe chiudere.

E.1.9 E5 — il decoder è trasferibile fra dispositivi?

Gli esperimenti precedenti addestrano e valutano il decoder sul *medesimo* profilo di rumore. È una scelta che lascia aperta l'obiezione più concreta che si possa muovere a un decoder appreso: se il modello funzionasse soltanto sul profilo su cui è stato addestrato, andrebbe riaddestrato per ogni esemplare di QPU e a ogni deriva di calibrazione, e il costo pratico ne annullerebbe il vantaggio. La domanda decisiva è dunque se la rete abbia appreso una *regola* o le idiosincrasie di un profilo.

La verifica consiste nell'addestrare su un'intensità di rumore correlato e valutare su un'altra, senza alcun riaddestramento. Il confronto ha due termini di riferimento: il MWPM, che non si addestra e resta quindi invariante, e la rete addestrata sullo stesso profilo di test, che rappresenta il limite superiore ottenibile conoscendo perfettamente il dominio.

Addestrata su	Valutata su p_{ct}			
	5×10^{-3}	10^{-2}	2×10^{-2}	4×10^{-2}
5×10^{-3}	<i>0.02804</i>	0.05000	0.09684	0.18749
10^{-2}	0.02800	<i>0.04912</i>	0.09492	0.18140
2×10^{-2}	0.02814	0.04915	<i>0.09286</i>	0.17800
4×10^{-2}	0.02841	0.04878	0.09312	<i>0.17643</i>
MWPM (nessun addestramento)	0.03232	0.05742	0.11172	0.20853

Tabella E.12: Errore logico dell'ibrido al variare del profilo di addestramento e di quello di valutazione. In corsivo la diagonale (addestramento e test sullo stesso profilo), in grassetto i due casi in cui una rete *trasferita* supera quella addestrata in casa. Tutte e dodici le configurazioni fuori diagonale battono comunque il MWPM.

L'esito è netto: **tutte e dodici** le reti trasferite mantengono il vantaggio sul decoder analitico, e il costo del trasferimento — il peggioramento medio rispetto alla rete addestrata sul profilo di test — è dell'1.6%. In due casi la rete trasferita risulta addirittura migliore di quella addestrata in casa, differenza che rientra nella variabilità statistica ma che conferma l'assenza di una dipendenza sistematica dal profilo di addestramento.

La regola appresa non è dunque il profilo di rumore, ma qualcosa di più generale: verosimilmente la struttura delle configurazioni di sindrome che il matching interpreta

male, la quale dipende dalla geometria del codice e dalla natura correlata dell'errore più che dalla sua intensità. Ne discende che il modello non richiede riaddestramento a ogni ricalibrazione del dispositivo — il che rimuove l'ostacolo pratico principale all'impiego di un decoder appreso, e distingue nettamente questo risultato da quello della prima fase, dove il classificatore richiedeva una fase di addestramento dedicata proprio per essere poi giudicato inutile.

E.1.10 Bilancio

I sette esperimenti convergono su una conclusione che è più informativa di un semplice confronto fra decoder, e che i due controlli finali hanno reso più precisa anziché più debole. **Sostituire** il decoder analitico con una rete non conviene: a parità di modello di rumore la rete pareggia a distanza 3 — al prezzo di 10^5 – 10^6 campioni — e perde al crescere della distanza. **Affiancarlo** sì: un decoder ibrido in cui la rete impara soltanto *quando ribaltare* la decisione del matching riduce l'errore logico del 18–21% a distanza 3 e dell'1–3% a distanza 5 in presenza di rumore correlato, e ricade sul matching quando il rumore è quello previsto.

Il guadagno compare dunque a due condizioni congiunte, e gli esperimenti E6 ed E7 permettono di enunciarle con precisione. La prima è che il rumore *esca dalla struttura* del modello del decoder — non che il modello sia mal calibrato: perturbandone i parametri fino a un ordine di grandezza l'errore logico varia dell'1–3%, e la rete non vince in nessuna delle dodici configurazioni. La seconda è che il reticolo resti *rappresentabile* dal modello appreso: a distanza 7, su 336 detector trattati come dimensioni indipendenti, l'ibrido si astiene dall'intervenire in tutte e cinque le configurazioni — anche là dove dispone di oltre duecentomila esempi positivi, sicché il limite è di architettura e non di numerosità (Tabella E.5).

A queste due condizioni se ne aggiunge una terza, emersa dal confronto con BP+OSD, che riguarda la scelta del termine di paragone. Il guadagno dell'ibrido va misurato contro il *miglior* decoder analitico disponibile, non contro quello standard: a distanza 5 BP+OSD recupera assai più margine dell'ibrido, e chi disponga di un budget di ingegneria limitato dovrebbe spenderlo lì per primo. Le due sorgenti di guadagno restano ortogonali — quella analitica decresce al crescere del rumore correlato, quella appresa cresce — ma non per questo sommabili: E8 ed E10 mostrano che entrambe conducono al medesimo pavimento, e che impilarle non porta al di sotto di esso. La costruzione corretta è dunque *sceglierne una in base al regime*, non cumularle: il miglior decoder analitico dove il modello è corretto, il residuo appreso

dove non lo è — e in quest’ultimo caso quale sia il decoder analitico sottostante cessa di contare.

Il perimetro va dichiarato. L’architettura impiegata è una rete densa di capacità modesta, deliberatamente scelta per continuità con lo strumento della prima fase sperimentale: i decoder allo stato dell’arte adottano architetture ricorrenti e convoluzionali che sfruttano la località del codice e la struttura temporale dei cicli di sindrome [10], e i risultati qui riportati non vanno letti come una stima del potenziale della decodifica appresa, ma come una verifica del *principio* su un’istanza trattabile. Le distanze esplorate arrivano a $d = 7$ — dove, come mostrato, il vantaggio si annulla — l’errore fisico è fissato a un solo valore sotto soglia, e il rumore correlato modellato resta di tipo Pauli: gli errori coerenti e il leakage discussi nella Sezione 9.5 costituiscono un asse ulteriore, sul quale ci si attende che il vantaggio del decoder appreso sia maggiore, non minore, poiché è lì che il modello analitico è più lontano dalla fisica.

Resta il fatto metodologico che collega questa sezione al Capitolo 7 e che il Capitolo 10 riprenderà: in entrambe le fasi sperimentali di questo lavoro il **complemento a basso costo ha battuto la sostituzione ambiziosa**. Nella prima, la ricerca multi-candidato TOP-4 ha battuto il classificatore che avrebbe dovuto filtrare l’output; nella seconda, una rete che si limita a segnalare quando il decoder sbaglia batte la rete che pretende di sostituirlo. L’apprendimento automatico non è né inutile né risolutivo: produce valore esattamente dove dispone di informazione che nessun metodo più semplice è in grado di sfruttare.

E.2 Compilazione Consapevole della Struttura: un’Esplorazione

Questa appendice esamina due verifiche esterne al confronto principale M1–M2: la sensibilità alla mappatura su una topologia eterogenea (M11/M11b) e il comportamento della ricerca TOP-4 nel proxy per gate (M13). Entrambe sono analisi esplorative. Il loro scopo è verificare se una metrica disponibile prima dell’esecuzione o una regola disponibile dopo la misura conservino potere informativo; non dimostrare prestazioni su una QPU reale.

E.3 Previsione e selezione non sono la stessa domanda

M11 usa la snapshot offline **FakeSherbrooke** distribuita con Qiskit IBM Runtime. Le proprietà `target.error` sono interpretate come *average gate infidelity* (AGI) complessiva e convertite in un canale depolarizzante con la stessa AGI. Il readout, esposto dalla snapshot come un solo errore medio, viene modellato simmetricamente. Le direzioni native ECR sono conservate e ogni ECR compilato deve avere una calibrazione esplicita; **rz** è virtuale.

Non viene aggiunto un secondo canale T_1/T_2 . L'AGI del gate è già una misura complessiva della sua non idealità e sommarvi il rilassamento ricavato dalla stessa snapshot produrrebbe un doppio conteggio non quantificato. Di conseguenza M11 non separa il contributo della decoerenza: studia l'eterogeneità della snapshot attraverso AGI e readout.

Il punteggio di calibrazione di un circuito compilato è

$$s = 1 - \prod_{g \in C} (1 - e_g), \quad (\text{E.2})$$

dove e_g è l'errore target dell'istruzione fisica o della misura. Un punteggio minore indica una fedeltà nominale maggiore. La correlazione fra s e successo risponde alla domanda *predittiva*; scegliere il layout con s minimo e valutarlo su dati non usati per la scelta risponde alla domanda *decisionale*. Le due non vanno confuse.

E.4 M11: pilota sui layout

Si campionano 60 sottografi connessi di 12 qubit mediante crescita casuale. Il campionamento non è uniforme sullo spazio dei sottografi: l'esperimento resta quindi esplorativo e le incertezze non descrivono l'intera topologia Sherbrooke. Su ciascun sottografo il circuito $N = 15$, $a = 7$ viene compilato in **sx/rz/x/ecr** con direzioni ECR native, livello di ottimizzazione 3 e seed 42.

Gli 8192 shot per layout sono divisi in otto batch con seed comuni. Quattro batch costituiscono la partizione di selezione e quattro l'holdout. Si confrontano due regole: layout con punteggio s minimo e layout con successo massimo sulla partizione di selezione; entrambe sono valutate soltanto sull'holdout. Questo evita di chiamare “migliore” il massimo osservato sugli stessi dati usati per stimarne la prestazione.

Voce	Contratto M11
backend	snapshot offline FakeSherbrooke, 127 qubit
layout richiesti	60 sottografi connessi, random-growth non uniforme
circuito	$N = 15$, $a = 7, 8$ qubit di conteggio
base / ottimizzazione	sx/rz/x/ecr , livello 3, seed 42
campionamento	8192 shot, 8 batch, split 4/4
inferenza	Spearman + bootstrap 2000 sui layout

Tabella E.13: Contratto preregistrato di M11. La snapshot è riproducibile tramite hash di calibrazione; non è una misura live di un dispositivo al momento dell'esecuzione.

Stima	Interpretazione ammessa
Spearman $s-P_{\text{holdout}}$	associazione di rango sul campione random-growth
P_{holdout} , minimo s	resa della regola basata solo sulla calibrazione
P_{holdout} , massimo train	resa di una selezione empirica su dati separati
differenza holdout	confronto esplorativo tra le due regole, non oracolo globale

Tabella E.14: La correlazione valuta capacità predittiva; il confronto holdout valuta una decisione. Nessuna riga autorizza un claim causale sull'hardware.

Dei 60 layout richiesti, 40 producono un modello di rumore valido. I 20 fallimenti non sono simulazioni con successo nullo: la snapshot assegna ad almeno un gate usato un'AGI pari a 1, valore fuori dal dominio della conversione depolarizzante adottata, e quei layout vengono pertanto esclusi. Fra i layout validi, P_{holdout} varia da 0,297 a 0,516, con mediana 0,431.

La correlazione di rango tra punteggio di calibrazione e successo holdout è $\rho_s = -0,184$, con IC bootstrap al 95% $[-0,456; 0,124]$ (2000 ricampionamenti). L'intervallo include ampiamente lo zero: sul campione random-growth il punteggio non mostra una capacità predittiva conclusiva. Il layout selezionato dal minimo punteggio (ID 35) ottiene $P_{\text{holdout}} = 0,451$; quello selezionato dal massimo successo sulla partizione train (ID 48) ottiene 0,516 sull'holdout. La differenza descrittiva è quindi +0,065, ma non è accompagnata da un intervallo dedicato e non va generalizzata oltre questo pilota.

La snapshot `fake_sherbrooke` è identificata nel JSON dal proprio hash SHA-256, il cui prefisso è 04b6a48f. Il risultato completo è nel file `results_M11_pilota_v2_20260826_222137.json`.

E.5 M11b: readout effettivo dopo il routing

Una seconda verifica genera variazione entro 12 sottografi. Per ciascuno vengono proposti cinque *initial layout*: registri di conteggio inizialmente assegnati ai qubit con readout nominale basso o alto, layout scelto dal transpiler e due permutazioni casuali. Queste etichette non sono trattamenti. Il routing può spostare i qubit virtuali, perciò l'esposizione analizzata è l'errore medio di readout dei qubit **effettivamente misurati nel circuito finale**.

Voce	Contratto M11b
sottografi	12, ciascuno con cinque strategie descrittive
base / ottimizzazione	sx/rz/x/ecr , livello 2, seed 42
shot	8192 per strategia, otto batch, holdout 50%
esposizione	readout medio sui qubit misurati dopo routing
controlli	effetti fissi di sottografo, numero ECR, profondità
stima primaria	Spearman parziale readout–successo
incertezza	cluster bootstrap sui sottografi analizzabili, 2000 ricampionamenti

Tabella E.15: L'unità indipendente del bootstrap è il sottografo, non la singola strategia annidata.

Etichetta iniziale	Uso nell'analisi
init-readout-basso	genera una permutazione iniziale; non garantisce il readout finale
init-readout-alto	genera variazione nella direzione opposta
transpiler	lascia la scelta iniziale al transpiler
casuale-1/2	due permutazioni riproducibili di confronto

Tabella E.16: Le etichette sono strumenti per produrre variazione entro sottografo. Il predittore usato nel modello è sempre la proprietà osservata dopo il routing.

Dei 12 sottografi richiesti, nove risultano analizzabili per tutte le cinque strategie, per un totale di 45 osservazioni holdout. Nei tre sottografi esclusi la conversione del modello di calibrazione incontra un'AGI pari a 1, fuori dal dominio della parametrizzazione depolarizzante: anche in questo caso le righe escluse non rappresentano esecuzioni con successo nullo.

Dopo la rimozione degli effetti fissi di sottografo e il controllo per numero di ECR e profondità, la correlazione parziale di Spearman fra errore medio di readout effettivo e successo è $\rho_s = -0,605$. L'IC al 95% ottenuto con 2000 ricampionamenti

cluster a livello di sottografo è $[-0, 826; -0, 190]$. Nel campione osservato, a maggiore errore di lettura corrisponde quindi un successo inferiore anche entro sottografo; l'intervallo non include lo zero. Il risultato resta associativo, non causale, sia per il numero ridotto di cluster sia per la possibile presenza di proprietà topologiche non misurate correlate con readout, conteggio ECR e profondità.

Il risultato canonico è `results_M11b_ruoli_v2_20260827_115109.json`; l'analisi indipendente è `analysis_M11b_v2_20260827_115119.json` e vincola il sorgente all'hash SHA-256 `1c4b5b4fad1e2289...f41c196b42152215`.

E.6 M13: TOP-4 sotto il proxy per gate

M13 usa lo stesso circuito, lo stesso manifest e la stessa definizione di p_L di M8. Per ciascun punto esegue 200 repliche da 1024 shot e riusa ogni istogramma per tre endpoint: massa di successo per singola misura, TOP-1 e TOP-4. Il successo TOP-K è binario per replica: almeno uno dei candidati selezionati conduce ai fattori.

Un pareggio al confine del quarto posto rende la selezione non identificata. Il runner salva quindi: (i) una scelta pseudo-casuale riproducibile; (ii) un limite inferiore, vero solo quando ogni scelta ammissibile contiene un esito utile; (iii) un limite superiore, vero quando almeno una scelta ammissibile può contenerlo. Il contrasto primario fra $p_L = 0$ e il massimo preregistrato usa un intervallo Newcombe; l'equivalenza richiede che l'intero intervallo cada entro $\pm 0, 10$. Una seconda verifica usa un involuppo Wilson simultaneo su tutte le risoluzioni dei pareggi.

p_L	per misura	TOP-1	TOP-4 seeded	identificato
0	0,7512	0,805	1,000	[1,000; 1,000]
10^{-4}	0,7459	0,730	1,000	[1,000; 1,000]
5×10^{-4}	0,7377	0,690	1,000	[1,000; 1,000]
10^{-3}	0,7229	0,620	1,000	[1,000; 1,000]
$1,7 \times 10^{-3}$	0,7080	0,620	1,000	[1,000; 1,000]
2×10^{-3}	0,7003	0,650	1,000	[1,000; 1,000]
5×10^{-3}	0,6396	0,505	1,000	[1,000; 1,000]
10^{-2}	0,5652	0,645	1,000	[1,000; 1,000]
2×10^{-2}	0,4528	0,695	1,000	[1,000; 1,000]
5×10^{-2}	0,2979	0,760	1,000	[1,000; 1,000]
10^{-1}	0,2506	0,535	0,915	[0,855; 0,940]
2×10^{-1}	0,2469	0,245	0,670	[0,535; 0,860]

Tabella E.17: Endpoint estratti dall'artefatto M13 schema 2 del 26 agosto 2026. La colonna "identificato" riporta i limiti inferiore e superiore generati dai pareggi al confine del TOP-4.

Il contrasto primario preregistrato confronta il TOP-4 seeded a $p_L = 0$ con quello al massimo valore della griglia, $p_L = 0, 2$. La differenza stimata è 0,330 e

l'IC Newcombe 95% è $[0, 266; 0, 398]$. La regola di equivalenza richiedeva l'intero intervallo entro $\pm 0, 10$; la regola di discriminazione richiedeva invece che il limite inferiore della diminuzione superasse $0, 10$. L'esito preregistrato è una diminuzione discriminante: il TOP-4 non è equivalente sul dominio completo della griglia.

Le analisi di sensibilità confermano che la gestione dei pareggi è materiale. Se tutti i pareggi sono risolti nel senso più sfavorevole, la diminuzione stimata è $0, 465$ con IC $[0, 395; 0, 534]$; nel senso più favorevole è $0, 140$ con IC $[0, 095; 0, 195]$, non sufficiente per superare il margine con la stessa regola di discriminazione. L'inviluppo simultaneo Bonferroni-Wilson su tutte le risoluzioni ammissibili è $[0, 059; 0, 553]$ e non supporta equivalenza per tutte le risoluzioni dei pareggi. Il risultato va quindi letto così: TOP-4 è estremamente efficace fino a $p_L = 0, 05$ su $N = 15$, ma degrada in modo statisticamente visibile agli errori più alti della griglia.

E.7 Conclusione e limiti

M11 e M11b non sono un benchmark su hardware. Il simulatore e il punteggio derivano dalla stessa snapshot; questa coerenza rende il confronto controllato, ma può favorire la metrica nominale e non include deriva, crosstalk, leakage o errori coerenti. Il campionamento random-growth non è uniforme. M11b aumenta il controllo confrontando strategie entro lo stesso sottografo, ma resta osservazionale: readout, numero ECR e profondità possono essere correlati con variabili topologiche non misurate.

M13 risponde a una domanda diversa. Una strategia TOP-4 può restare efficace anche quando la massa utile per misura è fortemente degradata, soprattutto nell'istanza $N = 15$ con pochi picchi strutturali. Perciò TOP-4 deve essere riportato insieme alla metrica per misura e all'incertezza dei pareggi; da solo non misura la fedeltà dello stato quantistico e non può essere estrapolato a moduli maggiori.

Bibliografia

- [1] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.
- [2] Ronald L. Rivest, Adi Shamir e Leonard Adleman. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”. In: *Communications of the ACM* 21.2 (1978), pp. 120–126. DOI: 10.1145/359340.359342.
- [3] John Preskill. “Quantum Computing in the NISQ Era and Beyond”. In: *Quantum* 2 (2018), p. 79. DOI: 10.22331/q-2018-08-06-79.
- [4] Unathi Skosana e Mark Tame. “Demonstration of Shor’s Factoring Algorithm for $N = 21$ on IBM Quantum Processors”. In: *Scientific Reports* 11 (2021), p. 16599. DOI: 10.1038/s41598-021-95973-w.
- [5] Craig Gidney e Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. In: *Quantum* 5 (2021), p. 433. DOI: 10.22331/q-2021-04-15-433.
- [6] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge, UK: Cambridge University Press, 2000. ISBN: 978-1107002173.
- [7] Andrew M. Steane. “Error Correcting Codes in Quantum Theory”. In: *Physical Review Letters* 77.5 (1996), pp. 793–797. DOI: 10.1103/PhysRevLett.77.793.
- [8] Austin G. Fowler et al. “Surface Codes: Towards Practical Large-Scale Quantum Computation”. In: *Physical Review A* 86.3 (2012), p. 032324. DOI: 10.1103/PhysRevA.86.032324.
- [9] Raul Conchello Vendrell et al. *Plaquette: A hardware-aware design platform for fault-tolerant quantum computers*. arXiv:2607.08767. 2026. arXiv: 2607.08767.

- [10] Johannes Bausch et al. “Learning High-Accuracy Error Decoding for Quantum Processors”. In: *Nature* 635 (2024). AlphaQubit: decoder neurale sviluppato da Google DeepMind e Google Quantum AI. Supera i decoder analitici classici su hardware Sycamore a 241 qubit, pp. 834–840. DOI: 10.1038/s41586-024-08148-8.
- [11] Richard P. Feynman. “Simulating Physics with Computers”. In: *International Journal of Theoretical Physics* 21.6–7 (1982), pp. 467–488. DOI: 10.1007/BF02650179.
- [12] David Deutsch. “Quantum Theory, the Church-Turing Principle and the Universal Quantum Computer”. In: *Proceedings of the Royal Society of London. Series A* 400.1818 (1985), pp. 97–117. DOI: 10.1098/rspa.1985.0070.
- [13] Wojciech Hubert Zurek. “Decoherence, einselection, and the quantum origins of the classical”. In: *Reviews of Modern Physics* 75.3 (2003), pp. 715–775. DOI: 10.1103/RevModPhys.75.715.
- [14] Fabrice Boudot et al. “Factoring RSA-250”. In: *Cryptology ePrint Archive* (2020). Report 2020/697, <https://eprint.iacr.org/2020/697>.
- [15] Stéphane Beauregard. “Circuit for Shor’s Algorithm Using $2n + 3$ Qubits”. In: *Quantum Information and Computation* 3.2 (2003). arXiv:quant-ph/0205095. Ottimizzazione del circuito originale di Shor: riduce il numero di qubit da $O(n)$ a $2n + 3$ tramite addizione modulare in-place, pp. 175–185.
- [16] William K. Wootters e Wojciech H. Zurek. “A Single Quantum Cannot be Cloned”. In: *Nature* 299 (1982), pp. 802–803. DOI: 10.1038/299802a0.
- [17] Mohan Sarovar et al. “Detecting crosstalk errors in quantum information processors”. In: *Quantum* 4 (2020), p. 321. DOI: 10.22331/q-2020-09-11-321.
- [18] Dorit Aharonov e Michael Ben-Or. “Fault-Tolerant Quantum Computation with Constant Error Rate”. In: *SIAM Journal on Computing* 38.4 (2008). Versione preliminare presentata a STOC 1997; preprint arXiv:quant-ph/9906129, pp. 1207–1282. DOI: 10.1137/S0097539799359385.
- [19] Peter W. Shor. “Scheme for Reducing Decoherence in Quantum Computer Memory”. In: *Physical Review A* 52.4 (1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493.

- [20] A. R. Calderbank e Peter W. Shor. “Good Quantum Error-Correcting Codes Exist”. In: *Physical Review A* 54.2 (1996), pp. 1098–1105. DOI: 10.1103/PhysRevA.54.1098.
- [21] Giacomo Torlai e Roger G. Melko. “Neural Decoder for Topological Codes”. In: *Physical Review Letters* 119.3 (2017), p. 030501. DOI: 10.1103/PhysRevLett.119.030501.
- [22] Qiskit contributors. *Qiskit: An Open-source Framework for Quantum Computing*. <https://qiskit.org>. 2023. DOI: 10.5281/zenodo.2573505.
- [23] Graychii. *Shor Algorithm Implementation*. <https://github.com/Graychii/Shor-Algorithm-Implementation>. Implementazione Python dell’algoritmo di Shor su Qiskit. 2023.
- [24] Craig Gidney. “Stim: a fast stabilizer circuit simulator”. In: *Quantum* 5 (2021), p. 497. DOI: 10.22331/q-2021-07-06-497.
- [25] Oscar Higgott. *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*. arXiv:2105.13082. 2021. arXiv: 2105.13082.
- [26] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [27] Joschka Roffe. “Quantum error correction: an introductory guide”. In: *Contemporary Physics* 60.3 (2019), pp. 226–245. DOI: 10.1080/00107514.2019.1667078.
- [28] Daniel Gottesman. “Stabilizer Codes and Quantum Error Correction”. Tesi di dott. California Institute of Technology, 1997. arXiv: quant-ph/9705052.
- [29] A. Yu. Kitaev. “Fault-tolerant quantum computation by anyons”. In: *Annals of Physics* 303.1 (2003), pp. 2–30. DOI: 10.1016/S0003-4916(02)00018-0.
- [30] Lieven M. K. Vandersypen et al. “Experimental Realization of Shor’s Quantum Factoring Algorithm Using Nuclear Magnetic Resonance”. In: *Nature* 414 (2001), pp. 883–887. DOI: 10.1038/414883a.
- [31] Thomas Monz et al. “Realization of a Scalable Shor Algorithm”. In: *Science* 351.6277 (2016), pp. 1068–1070. DOI: 10.1126/science.aad9480.
- [32] Aditya Singh, Aakash Khochare e Subhadeep Palit. *Practical Challenges in Executing Shor’s Algorithm on Existing Quantum Platforms*. 2025. arXiv: 2512.15330.

- [33] Oded Regev. *An Efficient Quantum Factoring Algorithm*. 2023. arXiv: 2308.06572.
- [34] IBM Quantum. *IBM Quantum Roadmap*. <https://www.ibm.com/roadmaps/quantum/>. Roadmap aggiornata a marzo 2026; obiettivi dichiarati soggetti a modifica. 2026.
- [35] Lov K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search”. In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1996, pp. 212–219. DOI: 10.1145/237814.237866.
- [36] Charles H. Bennett et al. “Strengths and Weaknesses of Quantum Computing”. In: *SIAM Journal on Computing* 26.5 (1997), pp. 1510–1523. DOI: 10.1137/S0097539796300933.
- [37] Christof Zalka. “Grover’s Quantum Searching Algorithm is Optimal”. In: *Physical Review A* 60.4 (1999), pp. 2746–2751. DOI: 10.1103/PhysRevA.60.2746.
- [38] Dewang Sun, Naifeng Zhang e Franz Franchetti. “Optimization and Performance Analysis of Shor’s Algorithm in Qiskit”. In: *IEEE High Performance Extreme Computing Conference (HPEC)*. Available at https://users.ece.cmu.edu/~franzf/papers/hpec_2023_Quantum_final.pdf. IEEE, 2023.
- [39] IBM Quantum. *Shor’s Algorithm – IBM Quantum Documentation*. <https://quantum.cloud.ibm.com/docs/en/tutorials/shors-algorithm>. Tutorial ufficiale IBM per l’implementazione dell’algoritmo di Shor con Qiskit Runtime su hardware reale (*ibm_marrakesh*). Include Dynamical Decoupling e Gate Twirling come tecniche di soppressione del rumore. 2024.
- [40] Google Quantum AI. *Cirq: A Python Framework for Writing, Manipulating, and Running Quantum Circuits*. <https://quantumai.google/cirq>. Framework open-source Google per processori superconduttori Sycamore. 2024.
- [41] Ville Bergholm et al. *PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations*. arXiv:1811.04968. Framework Xanadu orientato a circuiti variazionali e quantum machine learning. 2022.
- [42] Haoran Liao et al. *Machine Learning for Practical Quantum Error Mitigation*. 2023. arXiv: 2309.17368.
- [43] John S. Bell. “On the Einstein Podolsky Rosen Paradox”. In: *Physics* 1.3 (1964), pp. 195–200. DOI: 10.1103/PhysicsPhysiqueFizika.1.195.

- [44] Alain Aspect, Philippe Grangier e Gérard Roger. “Experimental Realization of Einstein-Podolsky-Rosen-Bohm Gedankenexperiment: A New Violation of Bell’s Inequalities”. In: *Physical Review Letters* 49.2 (1982), pp. 91–94. DOI: 10.1103/PhysRevLett.49.91.
- [45] Brian D. Josephson. “Possible New Effects in Superconductive Tunnelling”. In: *Physics Letters* 1.7 (1962), pp. 251–253. DOI: 10.1016/0031-9163(62)91369-0.
- [46] Jens Koch et al. “Charge-insensitive Qubit Design Derived from the Cooper Pair Box”. In: *Physical Review A* 76.4 (2007), p. 042319. DOI: 10.1103/PhysRevA.76.042319.
- [47] Arjen K. Lenstra e Hendrik W. Lenstra Jr. “The Development of the Number Field Sieve”. In: *Lecture Notes in Mathematics*. Vol. 1554. Springer-Verlag, 1993. DOI: 10.1007/BFb0091537.
- [48] National Institute of Standards and Technology. *What Is Post-Quantum Cryptography?* <https://www.nist.gov/cybersecurity-and-privacy/what-post-quantum-cryptography>. Aggiornato il 27 febbraio 2026; consultato il 26 agosto 2026. 2026.
- [49] National Security Agency. *Commercial National Security Algorithm Suite 2.0 (CNSA 2.0)*. https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF. Cybersecurity Advisory, settembre 2022. Fissa le scadenze di migrazione verso algoritmi resistenti ai computer quantistici entro il 2030–2035. 2022.
- [50] National Institute of Standards and Technology. *Post-Quantum Cryptography Standards: FIPS 203, 204, 205*. <https://csrc.nist.gov/pubs/fips/203/final>. FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA). Pubblicazione formale agosto 2024. 2024.
- [51] Apple Security Engineering and Architecture. *iMessage with PQ3: The new state of the art in quantum-secure messaging*. <https://security.apple.com/blog/imessage-pq3/>. Febbraio 2024. Primo protocollo di messaggistica con crittografia post-quantistica di livello 3, basato su ML-KEM per lo scambio di chiavi ibride. 2024.
- [52] Frank Arute et al. “Quantum Supremacy Using a Programmable Superconducting Processor”. In: *Nature* 574 (2019), pp. 505–510. DOI: 10.1038/s41586-019-1666-5.

- [53] Johannes Bausch, Andrew W. Senior et al. *Learning to Decode the Surface Code with a Recurrent, Transformer-Based Neural Network*. Decoder transformer per surface code; supera i decoder state-of-the-art su hardware Google Sycamore per distanze 3–11. 2023. arXiv: 2310.05900.
- [54] Pavel Panteleev e Gleb Kalachev. “Degenerate Quantum LDPC Codes With Good Finite Length Performance”. In: *Quantum* 5 (2021), p. 585. DOI: 10.22331/q-2021-11-22-585.
- [55] Joschka Roffe et al. “Decoding Across the Quantum Low-Density Parity-Check Code Landscape”. In: *Physical Review Research* 2.4 (2020), p. 043423. DOI: 10.1103/PhysRevResearch.2.043423.

Ringraziamenti

Desidero ringraziare in primo luogo il mio relatore, **Ing. Floriano Caprio**, per la disponibilità, la pazienza e la guida preziosa fornita durante l'intero percorso di sviluppo di questo lavoro. I suoi consigli e la sua competenza sono stati fondamentali per orientarmi in un campo così vasto e in continua evoluzione.

Ringrazio l'**Università Campus Bio-Medico di Roma** per il percorso formativo offerto, che mi ha fornito gli strumenti teorici e pratici per affrontare questa tesi.

Un ringraziamento speciale va alla mia **famiglia**, che ha rappresentato un punto fermo e una fonte inesauribile di sostegno morale durante gli anni di studio.

Ringrazio infine tutti i **colleghi e amici** che hanno condiviso con me questo percorso, rendendo più leggere le giornate di studio e ricerca.

Claudio Dragotta

Roma, 2026